《JavaInterview》第二版

它还有个名字叫**《面试小抄》**

必看

本项目介绍

关于更新

如何贡献

一、Java基础

1. Java语言有哪些特点?
2. Java和C++有什么关系，它们有什么区别?
3. JVM、JRE和JDK的关系是什么?
4. 什么是字节码?采用字节码的好处是什么?
5. Oracle JDK 和 OpenJDK 的区别是什么?
6. Java有哪些数据类型?
7. switch 是否能作用在 byte 上，是否能作用在 long 上，是否能作用在 String 上?
8. 访问修饰符public、private、protected、以及不写（默认）时的区别?
9. break ,continue ,return 的区别及作用?
10. final、finally、finalize的区别?
11. 为什么要用static关键字?
12. ”static”关键字是什么意思？Java中是否可以覆盖(override)一个private或者是static的方法?
13. 是否可以在static环境中访问非static变量?
14. static静态方法能不能引用非静态资源?
15. static静态方法里面能不能引用静态资源?
16. 非静态方法里面能不能引用静态资源?
17. java静态变量、代码块、和静态方法的执行顺序是什么?
18. 面向对象和面向过程的区别?
19. 讲讲面向对象三大特性
20. Java语言是如何实现多态的?
21. 重载（Overload）和重写（Override）的区别是什么?
22. 重载的方法能否根据返回值类型进行区分?
23. 构造器（constructor）是否可被重写（override）?
24. 抽象类和接口的区别是什么?
25. 抽象类能使用 final 修饰吗?
26. java 创建对象有哪几种方式?
27. 什么是不可变对象?好处是什么?

28. 能否创建一个包含可变对象的不可变对象?
29. 值传递和引用传递的区别的什么？为什么说Java中只有值传递?
30. == 和 equals 区别是什么?
31. 介绍下hashCode()?
32. 为什么要有 hashCode?
33. hashCode(),equals()两种方法是什么关系?
34. 为什么重写 equals 方法必须重写 hashcode 方法 ?
35. String,StringBuffer, StringBuilder 的区别是什么?
36. String为什么要设计成不可变的?
37. 字符型常量和字符串常量的区别?
38. 什么是字符串常量池?
39. String str="aaa"与 String str=new String("aaa")一样吗? `new String(“aaa”);` 创建了几个字符串对象?
40. String 是最基本的数据类型吗?
41. String有哪些特性?
42. 在使用 HashMap 的时候，用 String 做 key 有什么好处?
43. 包装类型是什么？基本类型和包装类型有什么区别?
44. 解释一下自动装箱和自动拆箱?
45. int 和 Integer 有什么区别?
46. 两个new生成的Integer变量的对比
47. Integer变量和int变量的对比
48. 非new生成的Integer变量和new Integer()生成变量的对比
49. 两个非new生成的Integer对象的对比
50. 什么是反射?
51. 反射机制的优缺点有哪些?
52. 如何获取反射中的Class对象?
53. Java反射API有几类?
54. 反射使用的步骤?
55. 为什么引入反射概念？反射机制的应用有哪些?
56. 反射机制的原理是什么?
57. Java中的泛型是什么 ?
58. 使用泛型的好处是什么?
60. 什么是泛型中的限定通配符和非限定通配符 ?
61. List<? extends T>和List <? super T>之间有什么区别 ?
63. 判断 `ArrayList<String>` 与 `ArrayList<Integer>` 是否相等?
64. Java序列化与反序列化是什么?
65. 为什么需要序列化与反序列化?
66. 序列化实现的方式有哪些?
67. 什么是serialVersionUID?
68. 为什么还要显示指定serialVersionUID的值?

69. serialVersionUID什么时候修改?
70. Java 序列化中如果有些字段不想进行序列化，怎么办?
71. 静态变量会被序列化吗?
72. Error 和 Exception 区别是什么?
73. 非受检查异常(运行时异常)和受检查异常(一般异常)区别是什么?
74. throw 和 throws 的区别是什么?
75. NoClassDefFoundError 和 ClassNotFoundException 区别?
76. Java常见异常有哪些?
77. try-catch-finally 中哪个部分可以省略?
78. try-catch-finally 中，如果 catch 中 return 了，finally 还会执行吗?
79. JVM 是如何处理异常的?
80. Java的IO 流分为几种?
81. 字节流如何转为字符流?
82. 字符流与字节流的区别?
83. 什么是阻塞IO？什么是非阻塞IO?
84. BIO、NIO、AIO的区别?
85. Java IO都有哪些设计模式?

二、Java集合

1. 常见的集合有哪些?
2. 线程安全的集合有哪些？线程不安全的呢?
3. Arraylist与 LinkedList 异同点?
4. ArrayList 与 Vector 区别?
5. 说一说ArrayList 的扩容机制?
6. Array 和 ArrayList 有什么区别？什么时候该应 Array 而不是 ArrayList 呢?
7. HashMap的底层数据结构是什么?
8. 解决hash冲突的办法有哪些？HashMap用的哪种?
9. 为什么在解决 hash 冲突的时候，不直接用红黑树？而选择先用链表，再转红黑树?
10. HashMap默认加载因子是多少？为什么是 0.75，不是 0.6 或者 0.8 ？
11. HashMap 中 key 的存储索引是怎么计算的?
12. HashMap 的put方法流程?
13. HashMap 的扩容方式?
14. 一般用什么作为HashMap的key?
15. HashMap为什么线程不安全?
16. ConcurrentHashMap 的实现原理是什么?
17. ConcurrentHashMap 的 put 方法执行逻辑是什么?
18. ConcurrentHashMap 的 get 方法是否要加锁，为什么?
19. get方法不需要加锁与volatile修饰的哈希桶有关吗?
20. ConcurrentHashMap 不支持 key 或者 value 为 null 的原因?
21. ConcurrentHashMap 的并发度是多少?
22. ConcurrentHashMap 迭代器是强一致性还是弱一致性?

23. JDK1.7与JDK1.8 中ConcurrentHashMap 的区别?
24. ConcurrentHashMap 和Hashtable的效率哪个更高？为什么?
25. 说一下Hashtable的锁机制 ?
26. 多线程下安全的操作 map还有其他方法吗?
27. HashSet 和 HashMap 区别?
28. Collection框架中实现比较要怎么做?
29. Iterator 和 ListIterator 有什么区别?
30. 讲一讲快速失败(fail-fast)和安全失败(fail-safe)

三、 JVM

1. 什么是JVM内存结构?
2. 什么是JVM内存模型?
3. heap 和stack 有什么区别?
4. 什么情况下会发生栈内存溢出?
5. 谈谈对 OOM 的认识？如何排查 OOM 的问题?
6. 谈谈 JVM 中的常量池?
7. 如何判断一个对象是否存活?
8. 强引用、软引用、弱引用、虚引用是什么，有什么区别?
9. 被引用的对象就一定能存活吗?
10. Java中的垃圾回收算法有哪些?
11. 有哪几种垃圾回收器，各自的优缺点是什么?
12. 详细说一下CMS的回收过程？CMS的问题是什么?
13. 详细说一下G1的回收过程?
14. JVM中一次完整的GC是什么样子的?
15. Minor GC 和 Full GC 有什么不同呢?
16. 介绍下空间分配担保原则?
17. 什么是类加载？类加载的过程?
18. 什么是类加载器，常见的类加载器有哪些?
19. 什么是双亲委派模型？为什么需要双亲委派模型?
20. 列举一些你知道的打破双亲委派机制的例子，为什么要打破?
21.说一下 JVM 调优的命令?

四、Java并发

多线程基础

1. 线程和进程有什么区别?
2. 创建线程的三种方式的对比?
3. 为什么要使用多线程呢?
4. 线程的状态流转
5. 什么是线程死锁?如何避免死锁?

死锁

死锁必须具备以下四个条件：

如何避免线程死锁?

6. 常见的对比
    Runnable VS Callable
    shutdown() VS shutdownNow()
    isTerminated() VS isShutdown()
7. sleep() 方法和 wait() 方法区别和共同点?
    区别
    相同
8.为什么我们调用 start() 方法时会执行 run() 方法，为什么我们不能直接调用 run() 方法
9. Thread类中的yield方法有什么作用?
10. 谈谈volatile的使用及其原理
11. 如何创建线程实例并运行?
12. 线程阻塞的三种情况
13. 线程死亡的三种方式
14. 为什么我们调用start()方法时会执行run()方法，为什么我们不能直接调用run()方法?
15. 守护线程是什么?
16. 了解Fork/Join框架吗?
17. CAS了解吗?
18. CAS有什么缺陷?
19. synchronized 和 volatile 的区别是什么?
20. synchronized 和 Lock 有什么区别?
21. synchronized 和 ReentrantLock 区别是什么?
22. synchronized的用法有哪些?
23. Synchronized的作用有哪些?
24. 说一下 synchronized 底层实现原理?
25. 多线程中 synchronized 锁升级的原理是什么?
26. synchronized 为什么是非公平锁？非公平体现在哪些地方?
27. JVM对synchronized的优化有哪些?
    1. 锁膨胀
    2.锁消除
    3. 锁粗化
    4. 自旋锁与自适应自旋锁
28. synchronized 锁能降级吗?
29. ThreadLocal是什么?
30. ThreadLocal的实现原理
31. 知道ThreadLocal 内存泄露问题吗?
32. 了解ReentrantLock吗?
33. ReadWriteLock是什么?

线程池专题

1. 为什么要用线程池?
2. 执行execute()方法和submit()方法的区别是什么呢?

3. 说下线程池核心参数?
4. 线程池执行任务的流程?
5. 常用的JAVA线程池有哪几种类型?
6. 线程池常用的阻塞队列有哪些?
7. 源码中线程池是怎么复用线程的?
8. 如何合理配置线程池参数?
9. Executor和Executors的区别?

AQS

1. 说一说什么是AQS?
2. AQS使用了哪些设计模式?
3. 了解AQS中同步队列的数据结构吗?
4. 了解AQS 对资源的共享方式吗?
5. AQS 组件了解吗?

Atomic 原子类

1. 介绍一下 Atomic 原子类
2. JUC 包中的原子类是哪4类?
3. 简单介绍一下 AtomicInteger 类的原理

End

五、MySQL

索引

1. 索引是什么?
2. 索引有哪些优缺点?
3. MySQL有哪几种索引类型?
4. 说一说索引的底层实现?
5. 为什么索引结构默认使用B+Tree，而不是B-Tree，Hash，二叉树，红黑树?
6. 讲一讲聚簇索引与非聚簇索引?
7. 非聚簇索引一定会回表查询吗?
8. 联合索引是什么？为什么需要注意联合索引中的顺序?
9. 讲一讲MySQL的最左前缀原则?
10. 讲一讲前缀索引?
11. 了解索引下推吗?
12. 怎么查看MySQL语句有没有用到索引?
13. 为什么官方建议使用自增长主键作为索引?
14. 如何创建索引?
15. 创建索引时需要注意什么?
16. 建索引的原则有哪些?
17. 使用索引查询一定能提高查询的性能吗?
18. 什么情况下不走索引（索引失效）?
    1、使用!= 或者 <> 导致索引失效
    2、类型不一致导致的索引失效

3、函数导致的索引失效
4、运算符导致的索引失效
5、OR引起的索引失效
6、模糊搜索导致的索引失效
7、NOT IN、NOT EXISTS导致索引失效

基础

1. 数据库的三范式是什么?
2. MySQL 支持哪些存储引擎?
3. 超键、候选键、主键、外键分别是什么?
4. SQL 约束有哪几种?
5. MySQL 中的 varchar 和 char 有什么区别?
6. MySQL中 in 和 exists 区别
7. drop、delete与truncate的区别
8. 什么是存储过程？有哪些优缺点?
9. MySQL 执行查询的过程

事务

1. 什么是数据库事务?
2. 介绍一下事务具有的四个特征
3. 说一下MySQL 的四种隔离级别
4. 什么是脏读？幻读？不可重复读?
5. 事务的实现原理
6. MySQL事务日志介绍下?
   redo log
   undo log
7. 什么是MySQL的 binlog?
8. 在事务中可以混合使用存储引擎吗?
9. MySQL中是如何实现事务隔离的?
10. 什么是 MVCC?
11. MVCC 的实现原理

锁

1. 为什么要加锁?
2. 按照锁的粒度分数据库锁有哪些?
3. 从锁的类别上分MySQL都有哪些锁呢?
4. 数据库的乐观锁和悲观锁是什么？怎么实现的?
5. InnoDB引擎的行锁是怎么实现的?
6. 什么是死锁？怎么解决?
7. 隔离级别与锁的关系
8. 优化锁方面的意见?

分库分表

1. 为什么要分库分表?

2. 用过哪些分库分表中间件？不同的分库分表中间件都有什么优点和缺点?

cobar

TDDL

atlas

sharding-jdbc

mycat

3. 如何对数据库如何进行垂直拆分或水平拆分的?

读写分离、主从同步（复制）

1. 什么是MySQL主从同步?
2. MySQL主从同步的目的？为什么要做主从同步?
3. 如何实现MySQL的读写分离?
4. MySQL主从复制流程和原理?
5. MySQL主从同步延时问题如何解决?

MySQL优化

1. 如何定位及优化SQL语句的性能问题?
2. 大表数据查询，怎么优化
3. 超大分页怎么处理?
4. 统计过慢查询吗？对慢查询都怎么优化过?
5. 如何优化查询过程中的数据访问
6. 如何优化关联查询
7. 数据库结构优化
8. MySQL数据库cpu飙升到500%的话他怎么处理?
9. 大表怎么优化?

End

六、Redis

概述

1. Redis是什么？简述它的优缺点?
2. Redis为什么这么快?
3. Redis相比Memcached有哪些优势?
4. 为什么要用 Redis 做缓存?
5. 为什么要用 Redis 而不用 map/guava 做缓存?
6. Redis的常用场景有哪些?
7. Redis的数据类型有哪些?

持久化

1. Redis持久化机制?
2. 如何选择合适的持久化方式
3. Redis持久化数据和缓存怎么做扩容?

过期键的删除策略、淘汰策略

1. Redis过期键的删除策略
2. Redis key的过期时间和永久有效分别怎么设置?

3. Redis内存淘汰策略

缓存异常

1. 如何保证缓存与数据库双写时的数据一致性?
2. 先删除缓存，后更新数据库

    答案一：延时双删

    答案二： 更新与读取操作进行异步串行化
3. 先更新数据库，后删除缓存
4. 什么是缓存击穿?
5. 什么是缓存穿透?
6. 什么是缓存雪崩?
7. 什么是缓存预热?
8. 什么是缓存降级?

线程模型

1. Redis为何选择单线程?
2. Redis真的是单线程?
3. Redis 6.0为何引入多线程?
4. Redis 6.0 采用多线程后，性能的提升效果如何?
5. 介绍下Redis的线程模型
6. Redis 6.0 多线程的实现机制?
7. Redis 6.0开启多线程后，是否会存在线程并发安全问题?
8. Redis 6.0 与 Memcached 多线程模型的对比

事务

1. Redis事务的概念
2. Redis事务的三个阶段
3. Redis事务相关命令
4. Redis事务支持隔离性吗?
5. Redis为什么不支持事务回滚?
6. Redis事务其他实现

主从、哨兵、集群

1. Redis常见使用方式有哪些?
2. 介绍下Redis单副本
3. 介绍下Redis多副本（主从）
4. 介绍下Redis Sentinel（哨兵）
5. 介绍下Redis Cluster
6. 介绍下Redis自研
7. Redis高可用方案具体怎么实施?

43. 了解主从复制的原理吗?

8. 由于主从延迟导致读取到过期数据怎么处理?
9. 主从复制的过程中如果因为网络原因停止复制了会怎么样?
10. Redis主从架构数据会丢失吗，为什么?

11. 如何解决主从架构数据丢失的问题?
12. Redis哨兵是怎么工作的?
13. 故障转移时会从剩下的slave选举一个新的master，被选举为master的标准是什么?
14. 同步配置的时候其他哨兵根据什么更新自己的配置呢?
15. 为什么Redis哨兵集群只有2个节点无法正常工作?
16. Redis cluster中是如何实现数据分布的？这种方式有什么优点?
17. Redis cluster节点间通信是什么机制?

分布式问题

1. 什么是分布式锁？为什么用分布式锁?
2. 常见的分布式锁有哪些解决方案?
3. Redis实现分布式锁
   分布式锁的三个核心要素
   上述分布式锁存在的问题
4. 了解RedLock吗?
5. RedLock的原理

其他

1. Redis如何做内存优化?
2. 如果现在有个读超高并发的系统，用Redis来抗住大部分读请求，你会怎么设计?

End

七、Spring

1. 使用Spring框架的好处是什么?
2. 什么是 Spring IOC 容器?
3. 什么是依赖注入？可以通过多少种方式完成依赖注入?
4. 区分 BeanFactory 和 ApplicationContext?
5. 区分构造函数注入和 setter 注入
6. spring 提供了哪些配置方式?
7. Spring 中的 bean 的作用域有哪些?
8. 如何理解IoC和DI?
9. 将一个类声明为Spring的 bean 的注解有哪些?
10. spring 支持几种 bean scope?
11. Spring 中的 bean 生命周期?
    创建过程:
    销毁过程:
    总结
12. 什么是 spring 的内部 bean?
13. 什么是 spring 装配?
14. 自动装配有什么局限?
15. Spring中出现同名bean怎么办?
16. Spring 怎么解决循环依赖问题?
17. Spring 中的单例 bean 的线程安全问题?

18. 什么是 AOP?
19. AOP 有哪些实现方式?
20. Spring AOP and AspectJ AOP 有什么区别?
21. Spring 框架中用到了哪些设计模式?
22. Spring 事务实现方式有哪些?
23. Spring框架的事务管理有哪些优点?
24. spring事务定义的传播规则
25. SpringMVC 工作原理了解吗?
26. 简单介绍 Spring MVC 的核心组件
27. @Controller 注解有什么用?
28. @RequestMapping 注解有什么用?
29. @RestController 和 @Controller 有什么区别?
30. @RequestMapping 和 @GetMapping 注解的不同之处在哪里?
31. @RequestParam 和 @PathVariable 两个注解的区别
32. 返回 JSON 格式使用什么注解?
33. 什么是springmvc拦截器以及如何使用它?
34. Spring MVC 和 Struts2 的异同?
35. REST 代表着什么?
36. 什么是安全的 REST 操作?
37. REST API 是无状态的吗?
38. REST安全吗? 你能做什么来保护它?
39. 为什么要用SpringBoot?
40. Spring Boot中如何实现对不同环境的属性配置文件的支持?
41. Spring Boot 的核心注解是哪个？它主要由哪几个注解组成的?
42. 你如何理解 Spring Boot 中的 Starters?
43. Spring Boot Starter 的工作原理是什么?
44. 保护 Spring Boot 应用有哪些方法?
45. Spring 、Spring Boot 和 Spring Cloud 的关系?

End

八、MyBatis

1. MyBatis是什么?
2. Mybaits的优缺点
3. 为什么说Mybatis是半自动ORM映射工具？它与全自动的区别在哪里?
4. Hibernate 和 MyBatis 的区别
5. JDBC编程有哪些不足之处，MyBatis是如何解决这些问题的?
6. MyBatis编程步骤是什么样的?
7. MyBatis与Hibernate有哪些不同?
8. Mybaits 的优点：
9. MyBatis 框架的缺点：
10. #{}和${}的区别?

11. 通常一个Xml映射文件，都会写一个Dao接口与之对应，那么这个Dao接口的工作原理是什么？Dao接口里的方法、参数不同时，方法能重载吗？

12. 在Mapper中如何传递多个参数？

13. Mybatis动态sql有什么用？执行原理是什么？有哪些动态sql？

14. xml映射文件中，不同的xml映射文件id是否可以重复？

15. Mybatis实现一对一有几种方式？具体是怎么操作的？

16. Mybatis实现一对多有几种方式？具体是怎么操作的？

17. Mybatis的一级、二级缓存

18. 使用MyBatis的Mapper接口调用时有哪些要求？

19. Mybatis动态sql是做什么的？都有哪些动态sql？

20. Mybatis的Xml映射文件中，不同的Xml映射文件，id是否可以重复？

九、MQ

概述

为什么使用MQ?

解耦

异步

削峰

消息队列的缺点

Kafka、ActiveMQ、RabbitMQ、RocketMQ 有什么优缺点?

RabbitMQ

1. RabbitMQ是什么?

2. RabbitMQ特点?

3. AMQP是什么?

4. AMQP的3层协议?

5. 说说Broker服务节点、Queue队列、Exchange交换器?

6. 如何保证消息的可靠性?

7. 生产者消息运转的流程?

8.消费者接收消息过程?

9. 生产者如何将消息可靠投递到RabbitMQ?

10. RabbitMQ如何将消息可靠投递到消费者?

11. 如何保证RabbitMQ消息队列的高可用?

RocketMQ

1. RocketMQ是什么?

2. RocketMQ由哪些角色组成，每个角色作用和特点是什么?

3. RocketMQ消费模式有几种?

4. RocketMQ消费消息是push还是pull?

追问：为什么要主动拉取消息而不使用事件监听方式?

5. broker如何处理拉取请求的?

6. 如何让RocketMQ保证消息的顺序消费?

7. RocketMQ如何保证消息不丢失?

7. rocketMQ的消息堆积如何处理?

追问：如果Consumer和Queue不对等，上线了多台也在短时间内无法消费完堆积的消息怎么办?

追问：堆积时间过长消息超时了?

追问：堆积的消息会不会进死信队列?

8. RocketMQ为什么自研nameserver而不用zk?

Kafka

1. Apache Kafka是什么?
2. Kafka 的设计是什么样的?
3. Kafka 如何保证高可用?
4. Kafka 消息是采用 Pull 模式，还是 Push 模式?
5. Kafka 与传统消息系统之间的区别
6. 什么是消费者组?
7. 在Kafka中，ZooKeeper的作用是什么?
8. 解释下Kafka中位移（offset）的作用
9. kafka 为什么那么快?
10. kafka producer发送数据，ack为0，1，-1分别是什么意思?
11. Kafka如何保证消息不丢失?

消费者异常导致的消息丢失

生产者数据传输导致的消息丢失

Kafka 导致的消息丢失

12. Kafka 如何保证消息的顺序性

如何保证消息在 Kafka 中顺序性

如何保证消费者处理消费的顺序性

13. Kafka中的ISR、AR代表什么？ISR的伸缩指什么?
14. 描述下 Kafka 中的领导者副本（Leader Replica）和追随者副本（Follower Replica）的区别
15. 分区Leader选举策略有几种?
16. Kafka的哪些场景中使用了零拷贝（Zero Copy）?
17. 为什么Kafka不支持读写分离?

十、计算机网络

1. 计算机网络的各层协议及作用?
2. TCP和UDP的区别?
3. UDP 和 TCP 对应的应用场景是什么?
4. 详细介绍一下 TCP 的三次握手机制?
5. 为什么需要三次握手，而不是两次?
6. 为什么要三次握手，而不是四次?
7. 什么是 SYN洪泛攻击？如何防范?
8. 三次握手连接阶段，最后一次ACK包丢失，会发生什么?
9. 详细介绍一下 TCP 的四次挥手过程?

10. 为什么连接的时候是三次握手，关闭的时候却是四次握手?
11. 为什么客户端的 TIME-WAIT 状态必须等待 2MSL ？
12. 如果已经建立了连接，但是客户端出现故障了怎么办?
13. TIME-WAIT 状态过多会产生什么后果？怎样处理?
14. TIME_WAIT 是服务器端的状态?还是客户端的状态?
15. TCP协议如何保证可靠性?
16. 详细讲一下TCP的滑动窗口?
17. 详细讲一下拥塞控制?
18. HTTP常见的状态码有哪些?
19. 状态码301和302的区别是什么?
20. HTTP 常用的请求方式?
21. GET请求和POST请求的区别?
22. 解释一下HTTP长连接和短连接?
23. HTTP请求报文和响应报文的格式?
24. HTTP1.0和HTTP1.1的区别?
25. HTTP1.1和 HTTP2.0的区别?
26. HTTP 与 HTTPS 的区别?
27. HTTPS 的优缺点?
28. 讲一讲HTTPS 的原理?
29. 在浏览器中输入www.baidu.com后执行的全部过程?
30. 什么是 Cookie 和 Session ？
31. Cookie 和 Session 是如何配合的呢?
32. Cookie和Session的区别?
33. 如何考虑分布式 Session 问题?
34. 什么是DDos攻击?
35. 什么是XSS攻击?
36. SQL注入是什么，如何避免SQL注入?
37. 负载均衡算法有哪些?

十一、操作系统

1. 进程和线程的区别?
2. 协程与线程的区别?
3. 并发和并行有什么区别?
4. 进程与线程的切换流程?
5. 为什么虚拟地址空间切换会比较耗时?
6. 进程间通信方式有哪些?
7. 进程间同步的方式有哪些?
8. 线程同步的方式有哪些?
9. 线程的分类?
10. 什么是临界区，如何解决冲突?
11. 什么是死锁？死锁产生的条件?

12. 进程调度策略有哪几种?
13. 进程有哪些状态?
14. 什么是分页?
15. 什么是分段?
16. 分页和分段有什区别?
17. 什么是交换空间?
18. 物理地址、逻辑地址、有效地址、线性地址、虚拟地址的区别?
19. 页面替换算法有哪些?
20. 什么是缓冲区溢出？有什么危害?
21. 什么是虚拟内存?
22. 虚拟内存的实现方式有哪些?
23. 讲一讲IO多路复用?
24. 硬链接和软链接有什么区别?
25. 中断的处理过程?
26. 中断和轮询有什么区别?
27. 什么是用户态和内核态?
28. 用户态和内核态是如何切换的?
29. Unix 常见的IO模型：
30. select、poll 和 epoll 之间的区别?

End

十二、 Dubbo

1.Dubbo是什么?
2. Dubbo和 Spring Cloud 有什么区别?
3. Dubbo核心组件有哪些?
4. Dubbo都支持什么协议，推荐用哪种?
5. Dubbo服务器注册与发现的流程?
6. Dubbo内置了哪几种服务容器?
7. Dubbo负载均衡的作用?
8. Dubbo有哪几种负载均衡策略，默认是哪种?
9. Dubbo服务之间的调用是阻塞的吗?
10. DubboMonitor 实现原理?
11. Dubbo有哪些注册中心?
12. Dubbo的集群容错方案有哪些?
13. Dubbo超时设置有哪些方式?
14. Dubbo用到哪些设计模式?

附：校招干货

1、简历如何写

简历模板

基本信息

教育背景

专业技能
项目/实习/工作经历
个人荣誉
自我评价
注意事项
End
2、大厂面试流程
一面
二面
项目本身
项目扩展
三面
HR面
参考

必看

本项目介绍

「Java面试小抄」一份通向理想互联网公司的面试汇总，包括 Java基础、Java并发、JVM、MySQL、Redis、Spring、MyBatis、Kafka、计算机操作系统、计算机网络、系统设计、分布式、Java 项目实战等。

现已将该面试总结开源到Github仓库上：https://github.com/cosen1024/Java-Interview，

国内Gitee：https://gitee.com/cosen1024/Java-Interview

库森将坚持维护这个开源项目 ，致力于打造最全Java面试宝典，欢迎各位star，第一时间了解最新面试题型。

关于更新

划重点， github仓库与库森公众号上的面试题是最新的！！！传到你手里的可能不知道是多少版之后的。

如果你发现你手中的PDF内容不全，**请扫描下方左侧二维码，关注公众号『程序员库森』**，回复关键字"**pdf**"即可领到最新的《面试小抄》PDF电子版本。

> 左侧是库森公众号，右侧是库森微信。

此外，我还创建了求职交流群，群内分享内推、面试等资源，一起来抱团取暖啊。扫描下方右侧二维码，备注【求职交流】，拉你进群~



第二版更新点：

- 更新Java多线程，包括多线程基础、synchronized连环问、线程池、AQS、原子类等。
- 更新Spring，包括Spring基础、bean、IOC、AOP、事务、Spring MVC、Spring Boot等。
- 更新MySQL，包括MySQL的基础、事务、锁、分库分表、读写分离、MySQl优化等。
- 更新Redis，包括Redis基础概念、持久化、删除策略、淘汰策略、缓存一致性、缓存击穿、缓存穿透、缓存雪崩、线程模型、事务、主从、哨兵、集群、分布式锁等。
- 更新操作系统
- 新增Mybatis
- 新增MQ，包括RabbitMQ、RocketMQ、Kafka
- 新增Dubbo

如何贡献

大家在阅读过程中，如果遇到错误的地方或者新的见解，都可以通过邮箱：cosen1024@163.com 与我交流！

也可以直接在Github上提交issue或pr，感谢大家的参与~

我还有一个**计算机书籍仓库**，仓库地址：https://github.com/cosen1024/awesome-cs-books，欢迎star和白嫖~

> 仓库包括几百本计算机优质电子书，并附带pdf下载链接，包括C，C++，Java，Python，Java，Linux，Go，数据结构与算法，人工智能，计算机基础，面试，设计模式，数据库，前端等。

一、Java基础

1. Java语言有哪些特点?

- 面向对象（封装，继承，多态）；
- 平台无关性，平台无关性的具体表现在于，Java 是“一次编写，到处运行（Write Once，Run any Where）”的语言，因此采用 Java 语言编写的程序具有很好的可移植性，而保证这一点的正是 Java 的虚拟机机制。在引入虚拟机之后，Java 语言在不同的平台上运行不需要重新编译。
- 可靠性、安全性；
- 支持多线程。C++ 语言没有内置的多线程机制，因此必须调用操作系统的多线程功能来进行多线程程序设计，而 Java 语言却提供了多线程支持；
- 支持网络编程并且很方便。Java 语言诞生本身就是为简化网络编程设计的，因此 Java 语言不仅支持网络编程而且很方便；
- 编译与解释并存；

2. Java和C++有什么关系，它们有什么区别?

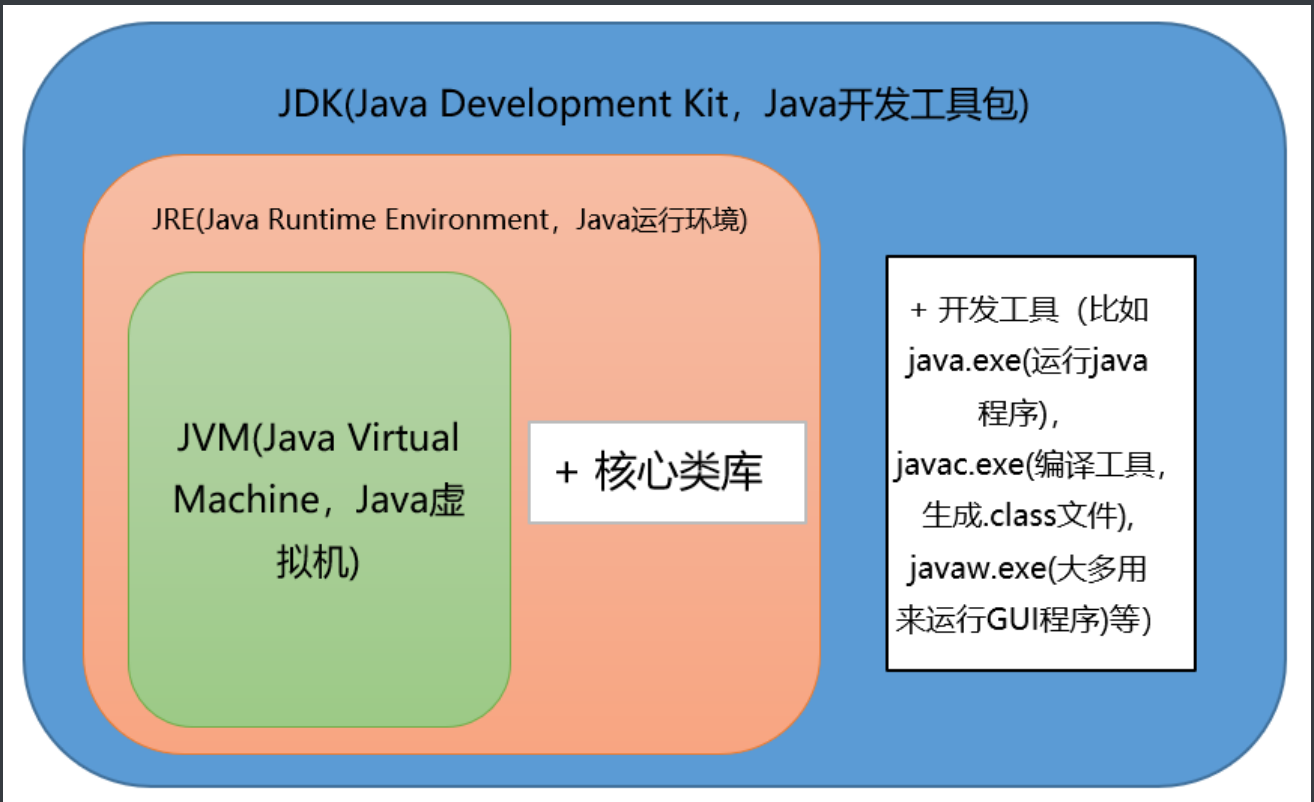
- 都是面向对象的语言，都支持封装、继承和多态；
- C++ 支持指针，而 Java 没有指针的概念；
- C++ 支持多继承，而 Java 不支持多重继承，但允许一个类实现多个接口；
- Java 自动进行无用内存回收操作，不再需要程序员进行手动删除，而 C++ 中必须由程序释放内存资源，这就增加了程序员的负担。
- Java 不支持操作符重载，操作符重载则被认为是 C++ 的突出特征；
- Java 是完全面向对象的语言，并且还取消了 C/C++ 中的结构和联合，使编译程序更加简洁；
- C 和 C++ 不支持字符串变量，在 C 和 C++ 程序中使用“Null”终止符代表字符串的结束。在 Java 中字符串是用类对象（String 和 StringBuffer）来实现的；
- goto 语句是 C 和 C++ 的“遗物”，Java 不提供 goto 语句，虽然 Java 指定 goto 作为关键字，但不支持它的使用，这使程序更简洁易读；

3. JVM、JRE和JDK的关系是什么?

JDK是（Java Development Kit）的缩写，它是功能齐全的 Java SDK。它拥有 JRE 所拥有的一切，还有编译器（javac）和工具（如 javadoc 和 jdb）。它能够创建和编译程序。

JRE是Java Runtime Environment缩写，它是运行已编译 Java 程序所需的所有内容的集合，包括 Java 虚拟机（JVM），Java 类库，java 命令和其他的一些基础构件。但是，它不能用于创建新程序。

JDK包含JRE，JRE包含JVM。

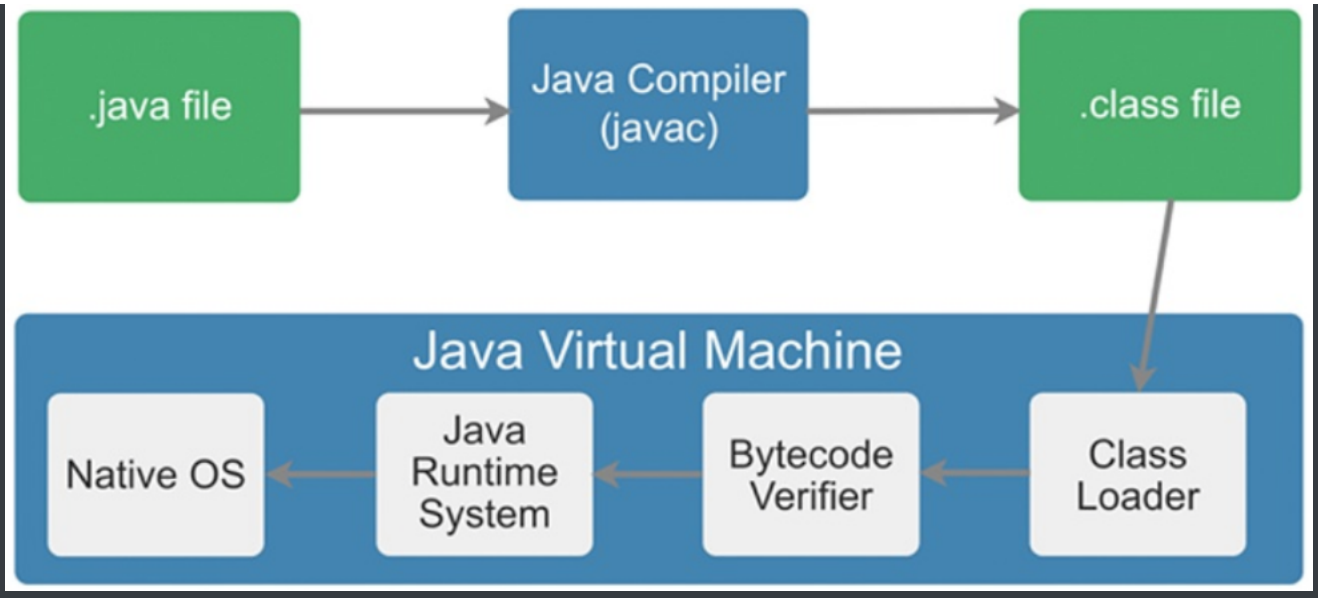


4. 什么是字节码?采用字节码的好处是什么?

> 这个问题，面试官可以扩展提问，Java 是编译执行的语言，还是解释执行的语言?

Java之所以可以“一次编译，到处运行”，一是因为JVM针对各种操作系统、平台都进行了定制，二是因为无论在什么平台，都可以编译生成固定格式的字节码（.class文件）供JVM使用。因此，也可以看出字节码对于Java生态的重要性。

之所以被称之为字节码，是因为字节码文件由十六进制值组成，而JVM以两个十六进制值为一组，即以字节为单位进行读取。在Java中一般是用javac命令编译源代码为字节码文件，一个.java文件从编译到运行的示例如图所示。



Java语言通过字节码的方式，在一定程度上解决了传统解释型语言执行效率低的问题，同时又保留了解释型语言可移植的特点。所以Java程序运行时比较高效，而且，由于字节码并不专对一种特定的机器，因此，Java程序无须重新编译便可在多种不同的计算机上运行。

5. Oracle JDK 和 OpenJDK 的区别是什么?

可能在看这个问题之前很多人和我一样并没有接触和使用过 OpenJDK 。下面通过我通过我收集到一些资料对你解答这个被很多人忽视的问题。

- Oracle JDK 版本将每三年发布一次，而 OpenJDK 版本每三个月发布一次；
- OpenJDK 是一个参考模型并且是完全开源的，而 Oracle JDK 是OpenJDK 的一个实现，并不是完全开源的；
- Oracle JDK 比 OpenJDK 更稳定。OpenJDK 和 Oracle JDK 的代码几乎相同，但 Oracle JDK 有更多的类和一些错误修复。因此，如果您想开发企业/商业软件，建议选择 Oracle JDK，因为它经过了彻底的测试和稳定。某些情况下，有些人提到在使用 OpenJDK 可能会遇到了许多应用程序崩溃的问题，但是，只需切换到 Oracle JDK 就可以解决问题；
- 在响应性和 JVM 性能方面，Oracle JDK 与 OpenJDK 相比提供了更好的性能；
- Oracle JDK 不会为即将发布的版本提供长期支持，用户每次都必须通过更新到最新版本获得支持来获取最新版本；
- Oracle JDK 根据二进制代码许可协议获得许可，而 OpenJDK 根据 GPLv2 许可获得许可。

6. Java有哪些数据类型?

Java 语言的数据类型分为两种：基本数据类型和引用数据类型。



基本数据类型包括 boolean（布尔型）、float（单精度浮点型）、char（字符型）、byte（字节型）、short（短整型）、int（整型）、long（长整型）和 double （双精度浮点型）共 8 种，如下图所示。

类型名称	关键字	占用内存	取值范围
字节型	byte	1 字节	-128~127
短整型	short	2 字节	-32768~32767
整型	int	4 字节	-2147483648~2147483647
长整型	long	8 字节	-9223372036854775808L~9223372036854775807L
单精度浮点型	float	4 字节	+/-3.4E+38F（6~7 个有效位）
双精度浮点型	double	8 字节	+/-1.8E+308 (15 个有效位)
字符型	char	2 字节	ISO 单一字符集
布尔型	boolean	1 字节	true 或 false

image-

引用数据类型建立在基本数据类型的基础上，包括数组、类和接口。引用数据类型是由用户自定义，用来限制其他数据的类型。另外，Java 语言中不支持 C++ 中的指针类型、结构类型、联合类型和枚举类型。

7. switch 是否能作用在 byte 上，是否能作用在 long 上，是否能作用在 String 上?

Java5 以前 switch(expr)中，expr 只能是 byte、short、char、int。

从 Java 5 开始，Java 中引入了枚举类型， expr 也可以是 enum 类型。

从 Java 7 开始，expr还可以是字符串(String)，但是长整型(long)在目前所有的版本中都是不可以的。

8. 访问修饰符public、private、protected、以及不写（默认）时的区别?

Java中，可以使用访问控制符来保护对类、变量、方法和构造方法的访问。Java 支持 4 种不同的访问权限。

- **default** (即默认，什么也不写）: 在同一包内可见，不使用任何修饰符。使用对象：类、接口、变量、方法。
- **private** : 在同一类内可见。使用对象：变量、方法。**注意：不能修饰类（外部类）**
- **public** : 对所有类可见。使用对象：类、接口、变量、方法
- **protected** : 对同一包内的类和所有子类可见。使用对象：变量、方法。**注意：不能修饰类（外部类）**。

修饰符	当前类	同一包内	子孙类(同一包)	子孙类(不同包)	其他包
public	Y	Y	Y	Y	Y
protected	Y	Y	Y	Y/N（说明）	N
default	Y	Y	Y	N	N
private	Y	N	N	N	N

9. break ,continue ,return 的区别及作用?

- break 跳出总上一层循环，不再执行循环(**结束当前的循环体**)
- continue 跳出本次循环，继续执行下次循环(**结束正在执行的循环 进入下一个循环条件**)
- return 程序返回，不再执行下面的代码(**结束当前的方法 直接返回**)

10. final、finally、finalize的区别?

final 用于修饰变量、方法和类。

- final 变量：被修饰的变量不可变，不可变分为 `引用不可变` 和 `对象不可变`，final 指的是 `引用不可变`，final 修饰的变量必须初始化，通常称被修饰的变量为 `常量`。
- final 方法：被修饰的方法不允许任何子类重写，子类可以使用该方法。
- final 类：被修饰的类不能被继承，所有方法不能被重写。

finally 作为异常处理的一部分，它只能在 `try/catch` 语句中，并且附带一个语句块表示这段语句最终一定被执行（无论是否抛出异常），经常被用在需要释放资源的情况下，`System.exit(0)` 可以阻断 finally 执行。

finalize 是在 `java.lang.Object` 里定义的方法，也就是说每一个对象都有这么个方法，这个方法在 `gc` 启动，该对象被回收的时候被调用。

一个对象的 finalize 方法只会被调用一次，finalize 被调用不一定会立即回收该对象，所以有可能调用 finalize 后，该对象又不需要被回收了，然后到了真正要被回收的时候，因为前面调用过一次，所以不会再次调用 finalize 了，进而产生问题，因此不推荐使用 finalize 方法。

11. 为什么要用static关键字?

通常来说，用new创建类的对象时，数据存储空间才被分配，方法才供外界调用。但有时我们只想为特定域分配单一存储空间，不考虑要创建多少对象或者说根本就不创建任何对象，再就是我们想在没有创建对象的情况下也想调用方法。在这两种情况下，static关键字，满足了我们的需求。

12. “static”关键字是什么意思？Java中是否可以覆盖(override)一个private或者是static的方法?

“static”关键字表明一个成员变量或者是成员方法可以在没有所属的类的实例变量的情况下被访问。

Java中static方法不能被覆盖，因为方法覆盖是基于运行时动态绑定的，而static方法是编译时静态绑定的。static方法跟类的任何实例都不相关，所以概念上不适用。

13. 是否可以在static环境中访问非static变量?

static变量在Java中是属于类的，它在所有的实例中的值是一样的。当类被Java虚拟机载入的时候，会对static变量进行初始化。如果你的代码尝试不用实例来访问非static的变量，编译器会报错，因为这些变量还没有被创建出来，还没有跟任何实例关联上。

14. static静态方法能不能引用非静态资源?

不能，new的时候才会产生的东西，对于初始化后就存在的静态资源来说，根本不认识它。

15. static静态方法里面能不能引用静态资源?

可以，因为都是类初始化的时候加载的，大家相互都认识。

16. 非静态方法里面能不能引用静态资源?

可以，非静态方法就是实例方法，那是new之后才产生的，那么属于类的内容它都认识。

17. java静态变量、代码块、和静态方法的执行顺序是什么?

基本上代码块分为三种：Static静态代码块、构造代码块、普通代码块

代码块执行顺序**静态代码块——> 构造代码块 ——> 构造函数——> 普通代码块**

继承中代码块执行顺序：**父类静态块——>子类静态块——>父类代码块——>父类构造器——>子类代码块——>子类构造器**

想要深入了解，可以参考这篇文章： https://juejin.cn/post/6844903986475040781

18. 面向对象和面向过程的区别?

面向过程:

- 优点：性能比面向对象高，因为类调用时需要实例化，开销比较大，比较消耗资源;比如单片机、嵌入式开发、Linux/Unix等一般采用面向过程开发，性能是最重要的因素。
- 缺点：没有面向对象易维护、易复用、易扩展。

面向对象:

- 优点：易维护、易复用、易扩展，由于面向对象有封装、继承、多态性的特性，可以设计出低耦合的系统，使系统更加灵活、更加易于维护。
- 缺点：性能比面向过程低。

19. 讲讲面向对象三大特性

- 封装。封装最好理解了。封装是面向对象的特征之一，是对象和类概念的主要特性。封装，也就是把客观事物封装成抽象的类，并且类可以把自己的数据和方法只让可信的类

或者对象操作，对不可信的进行信息隐藏。
- 继承。继承是指这样一种能力：它可以使用现有类的所有功能，并在无需重新编写原来的类的情况下对这些功能进行扩展。通过继承创建的新类称为“子类”或“派生类”，被继承的类称为“基类”、“父类”或“超类”。
- 多态性。它是指在父类中定义的属性和方法被子类继承之后，可以具有不同的数据类型或表现出不同的行为，这使得同一个属性或方法在父类及其各个子类中具有不同的含义。

20. Java语言是如何实现多态的?

本质上多态分两种：

> **1、编译时多态（又称静态多态）**
>
> **2、运行时多态（又称动态多态）**

重载（overload）就是编译时多态的一个例子，编译时多态在编译时就已经确定，运行的时候调用的是确定的方法。

我们通常所说的多态指的都是运行时多态，也就是编译时不确定究竟调用哪个具体方法，一直延迟到运行时才能确定。这也是为什么有时候多态方法又被称为延迟方法的原因。

Java实现多态有 3 个必要条件：继承、重写和向上转型。只有满足这 3 个条件，开发人员才能够在同一个继承结构中使用统一的逻辑实现代码处理不同的对象，从而执行不同的行为。

- 继承：在多态中必须存在有继承关系的子类和父类。
- 重写：子类对父类中某些方法进行重新定义，在调用这些方法时就会调用子类的方法。
- 向上转型：在多态中需要将子类的引用赋给父类对象，只有这样该引用才既能可以调用父类的方法，又能调用子类的方法。

Java多态的实现原理可看这篇文章：https://my.oschina.net/u/4432600/blog/4535042

21. 重载（Overload）和重写（Override）的区别是什么?

方法的重载和重写都是实现多态的方式，区别在于前者实现的是编译时的多态性，而后者实现的是运行时的多态性。

- 重写发生在子类与父类之间, 重写方法返回值和形参都不能改变，与方法返回值和访问修饰符无关，即重载的方法不能根据返回类型进行区分。**即外壳不变，核心重写！**

- 重载(overloading) 是在一个类里面，方法名字相同，而参数不同。返回类型可以相同也可以不同。每个重载的方法（或者构造函数）都必须有一个独一无二的参数类型列表。最常用的地方就是构造器的重载。



22. 重载的方法能否根据返回值类型进行区分?

不能根据返回值类型来区分重载的方法。因为调用时不指定类型信息，编译器不知道你要调用哪个函数。

```
float max(int a, int b);
int max(int a, int b);
```

当调用 `max(1,2)`；时无法确定调用的是哪个，单从这一点上来说，仅返回值类型不同的重载是不应该允许的。

23. 构造器（constructor）是否可被重写（override）?

构造器不能被继承，因此不能被重写，但可以被重载。每一个类必须有自己的构造函数，负责构造自己这部分的构造。子类不会覆盖父类的构造函数，相反必须一开始调用父类的构造函数。

24. 抽象类和接口的区别是什么?

语法层面上的区别：

- 抽象类可以提供成员方法的实现细节，而接口中只能存在public abstract 方法；
- 抽象类中的成员变量可以是各种类型的，而接口中的成员变量只能是public static final类型的；

- 接口中不能含有静态代码块以及静态方法，而抽象类可以有静态代码块和静态方法；
- 一个类只能继承一个抽象类，而一个类却可以实现多个接口。

设计层面上的区别：

- 抽象类是对一种事物的抽象，即对类抽象，而接口是对行为的抽象。抽象类是对整个类整体进行抽象，包括属性、行为，但是接口却是对类局部（行为）进行抽象。
- 设计层面不同，抽象类作为很多子类的父类，它是一种模板式设计。而接口是一种行为规范，它是一种辐射式设计。

想要深入了解，可以参考这篇文章： https://www.cnblogs.com/dolphin0520/p/3811437.html

25. 抽象类能使用 final 修饰吗?

不能，定义抽象类就是让其他类继承的，如果定义为 final 该类就不能被继承，这样彼此就会产生矛盾，所以 final 不能修饰抽象类

26. java 创建对象有哪几种方式?

java中提供了以下四种创建对象的方式:

- new创建新对象
- 通过反射机制
- 采用clone机制
- 通过序列化机制

前两者都需要显式地调用构造方法。对于clone机制,需要注意浅拷贝和深拷贝的区别，对于序列化机制需要明确其实现原理，在java中序列化可以通过实现Externalizable或者Serializable来实现。

27. 什么是不可变对象?好处是什么?

不可变对象指对象一旦被创建,状态就不能再改变,任何修改都会创建一个新的对象,如 String、Integer及其它包装类.不可变对象最大的好处是线程安全.

28. 能否创建一个包含可变对象的不可变对象?

当然可以,比如 `final Person[] persons = new Persion[]{}` . `persons` 是不可变对象的引用,但其数组中的Person实例却是可变的.这种情况下需要特别谨慎,不要共享可变对象的引用.这种情况下,如果数据需要变化时,就返回原对象的一个拷贝.

29. 值传递和引用传递的区别的什么？为什么说Java中只有值传递?

值传递：指的是在方法调用时，传递的参数是按值的拷贝传递，传递的是值的拷贝，也就是说传递后就互不相关了。

引用传递：指的是在方法调用时，传递的参数是按引用进行传递，其实传递的是引用的地址，也就是变量所对应的内存空间的地址。传递的是值的引用，也就是说传递前和传递后都指向同一个引用（也就是同一个内存空间）。

基本类型作为参数被传递时肯定是值传递；引用类型作为参数被传递时也是值传递，只不过“值”为对应的引用。

想要深入了解，可以参考这篇文章： http://www.itwanger.com/java/2019/11/26/java-yinyong-value.html

30. == 和 equals 区别是什么?

`==` 常用于相同的基本数据类型之间的比较，也可用于相同类型的对象之间的比较；

- 如果 `==` 比较的是基本数据类型，那么比较的是两个基本数据类型的值是否相等；
- 如果 `==` 是比较的两个对象，那么比较的是两个对象的引用，也就是判断两个对象是否指向了同一块内存区域；

equals方法主要用于两个对象之间，检测一个对象是否等于另一个对象

看一看Object类中equals方法的源码：

```
    public boolean equals(Object obj) {
            return (this == obj);
        }
```

它的作用也是**判断两个对象是否相等**，般有两种使用情况：

- 情况1，类没有覆盖equals()方法。则通过equals()比较该类的两个对象时，等价于通过“==”比较这两个对象。
- 情况2，类覆盖了equals()方法。一般，我们都覆盖equals()方法来两个对象的内容相等；若它们的内容相等，则返回true(即，认为这两个对象相等)。

java语言规范要求equals方法具有以下特性：

- 自反性。对于任意不为null的引用值x，x.equals(x)一定是true。
- 对称性）。对于任意不为null的引用值x和y，当且仅当x.equals(y)是true时，y.equals(x)也是true。
- 传递性。对于任意不为null的引用值x、y和z，如果x.equals(y)是true，同时y.equals(z)是true，那么x.equals(z)一定是true。
- 一致性。对于任意不为null的引用值x和y，如果用于equals比较的对象信息没有被修改的话，多次调用时x.equals(y)要么一致地返回true要么一致地返回false。
- 对于任意不为null的引用值x，x.equals(null)返回false。

31. 介绍下hashCode()?

hashCode() 的作用是获取哈希码，也称为散列码；它实际上是返回一个int整数。这个哈希码的作用是确定该对象在哈希表中的索引位置。hashCode() 定义在JDK的Object.java中，这就意味着Java中的任何类都包含有hashCode()函数。

散列表存储的是键值对(key-value)，它的特点是：能根据“键”快速的检索出对应的“值”。这其中就利用到了散列码！（可以快速找到所需要的对象）

32. 为什么要有 hashCode?

以“HashSet 如何检查重复”为例子来说明为什么要有 hashCode:

当你把对象加入 HashSet 时，HashSet 会先计算对象的 hashcode 值来判断对象加入的位置，同时也会与其他已经加入的对象的 hashcode 值作比较，如果没有相符的hashcode，HashSet会假设对象没有重复出现。

但是如果发现有相同 hashcode 值的对象，这时会调用 equals()方法来检查 hashcode 相等的对象是否真的相同。如果两者相同，HashSet 就不会让其加入操作成功。如果不同的话，就会重新散列到其他位置。这样我们就大大减少了 equals 的次数，相应就大大提高了执行速度。

33. hashCode(),equals()两种方法是什么关系?

Java 对于 `eqauls()` 方法和 `hashCode()` 方法是这样规定的：

- 同一对象上多次调用 hashCode() 方法，总是返回相同的整型值。
- 如果 a.equals(b)，则一定有 a.hashCode() 一定等于 b.hashCode()。
- 如果 !a.equals(b)，则 a.hashCode() 不一定等于 b.hashCode()。此时如果 a.hashCode() 总是不等于 b.hashCode()，会提高 hashtables 的性能。
- a.hashCode()==b.hashCode() 则 a.equals(b) 可真可假

- a.hashCode()！= b.hashCode() 则 a.equals(b) 为假。

上面结论简记：

- 如果两个对象 equals，Java 运行时环境会认为他们的 hashCode 一定相等。
- 如果两个对象不 equals，他们的 hashCode 有可能相等。
- 如果两个对象 hashCode 相等，他们不一定 equals。
- 如果两个对象 hashCode 不相等，他们一定不 equals

补充：关于 equals() 和 hashCode() 的重要规范

- 规范1：若重写 equals() 方法，有必要重写 hashcode()方法，确保通过 equals()方法判断结果为 true 的两个对象具备相等的 hashcode() 方法返回值。说得简单点就是：“如果两个对象相同，那么他们的 hashCode 应该相等”。不过请注意：这个只是规范，如果非要写一个类让 equals() 方法返回 true 而 hashCode() 方法返回两个不相等的值，编译和运行都是不会报错的。不过这样违反了 Java 规范，程序也就埋下了 BUG。
- 规范2：如果 equals() 方法返回 false，即两个对象“不相同”，并不要求对这两个对象调用 hashCode() 方法得到两个不相同的数。说的简单点就是：“如果两个对象不相同，他们的 hashCode 可能相同”。

34. 为什么重写 equals 方法必须重写 hashcode 方法？

断的时候先根据hashcode进行的判断，相同的情况下再根据equals()方法进行判断。如果只重写了equals方法，而不重写hashcode的方法，会造成hashcode的值不同，而equals()方法判断出来的结果为true。

在Java中的一些容器中，不允许有两个完全相同的对象，插入的时候，如果判断相同则会进行覆盖。这时候如果只重写了equals（）的方法，而不重写hashcode的方法，Object中hashcode是根据对象的存储地址转换而形成的一个哈希值。这时候就有可能因为没有重写hashcode方法，造成相同的对象散列到不同的位置而造成对象的不能覆盖的问题。

35. String,StringBuffer, StringBuilder 的区别是什么?

1.可变与不可变。String类中使用字符数组保存字符串，因为有“final”修饰符，所以string对象是不可变的。**对于已经存在的String对象的修改都是重新创建一个新的对象,然后把新的值保存进去.**

```
    private final char value[];
```

StringBuilder与StringBuffer都继承自AbstractStringBuilder类，在AbstractStringBuilder中也是使用字符数组保存字符串，这两种对象都是可变的。

```
char[] value;
```

2.是否线程安全。

String中的对象是不可变的，也就可以理解为常量，显然线程安全。

StringBuilder是非线程安全的。

StringBuffer对方法加了同步锁或者对调用的方法加了同步锁，所以是线程安全的。

```
@Override
public synchronized StringBuffer append(String str) {
    toStringCache = null;
    super.append(str);
    return this;
}
```

3.如果你只是在单线程中使用字符串缓冲区，那么StringBuilder的效率会更高些。值得注意的是StringBuilder是在JDK1.5版本中增加的。以前版本的JDK不能使用该类。

36. String为什么要设计成不可变的?

1.便于实现字符串池（String pool）

在Java中，由于会大量的使用String常量，如果每一次声明一个String都创建一个String对象，那将会造成极大的空间资源的浪费。Java提出了String pool的概念，在堆中开辟一块存储空间String pool，当初始化一个String变量时，如果该字符串已经存在了，就不会去创建一个新的字符串变量，而是会返回已经存在了的字符串的引用。

```
String a = "Hello world!";
String b = "Hello world!";
```

如果字符串是可变的，某一个字符串变量改变了其值，那么其指向的变量的值也会改变，String pool将不能够实现!

2.使多线程安全

在并发场景下，多个线程同时读一个资源，是安全的，不会引发竞争，但对资源进行写操作时是不安全的，不可变对象不能被写，所以保证了多线程的安全。

3.避免安全问题

在网络连接和数据库连接中字符串常常作为参数，例如，网络连接地址URL，文件路径path，反射机制所需要的String参数。其不可变性可以保证连接的安全性。如果字符串是可变的，黑客就有可能改变字符串指向对象的值，那么会引起很严重的安全问题。

4.加快字符串处理速度

由于String是不可变的，保证了hashcode的唯一性，于是在创建对象时其hashcode就可以放心的缓存了，不需要重新计算。这也就是Map喜欢将String作为Key的原因，处理速度要快过其它的键对象。所以HashMap中的键往往都使用String。

总体来说，String不可变的原因要包括 设计考虑，效率优化，以及安全性这三大方面。

37. 字符型常量和字符串常量的区别?

1. 形式上: 字符常量是单引号引起的一个字符，字符串常量是双引号引起的若干个字符;
2. 含义上: 字符常量相当于一个整型值(ASCII 值),可以参加表达式运算；字符串常量代表一个地址值(该字符串在内存中存放位置，相当于对象;
3. 占内存大小：字符常量只占2个字节；字符串常量占若干个字节(至少一个字符结束标志)(注意: char 在Java中占两个字节)。

38. 什么是字符串常量池?

java中常量池的概念主要有三个： `全局字符串常量池` ， `class文件常量池` ， `运行时常量池` 。我们现在所说的就是 `全局字符串常量池` ，对这个想弄明白的同学可以看这篇Java中几种常量池的区分。

jvm为了提升性能和减少内存开销，避免字符的重复创建，其维护了一块特殊的内存空间，即字符串池，当需要使用字符串时，先去字符串池中查看该字符串是否已经存在，如果存在，则可以直接使用，如果不存在，初始化，并将该字符串放入字符串常量池中。

字符串常量池的位置也是随着jdk版本的不同而位置不同。在jdk6中，常量池的位置在永久代（方法区）中，此时常量池中存储的是**对象**。在jdk7中，常量池的位置在堆中，此时，常量池存储的就是**引用**了。在jdk8中，永久代（方法区）被元空间取代了。

39. String str="aaa"与 String str=new String("aaa")一样吗? `new String("aaa");` 创建了几个字符串对象?

- 使用 `String a = "aaa";`，程序运行时会在常量池中查找"aaa"字符串，若没有，会将"aaa"字符串放进常量池，再将其地址赋给a；若有，将找到的"aaa"字符串的地址赋给a。
- 使用String b = new String("aaa");`，程序会在堆内存中开辟一片新空间存放新对象，同时会将"aaa"字符串放入常量池，相当于创建了两个对象，无论常量池中有没有"aaa"字符串，程序都会在堆内存中开辟一片新空间存放新对象。

具体分析，见以下代码：

```
 @Test
    public void test(){
        String s = new String("2");
        s.intern();
        String s2 = "2";
        System.out.println(s == s2);


        String s3 = new String("3") + new String("3");
        s3.intern();
        String s4 = "33";
        System.out.println(s3 == s4);
    }
```

运行结果：

```
jdk6
false
false

jdk7
false
true
```

这段代码在jdk6中输出是 `false false`，但是在jdk7中输出的是 `false true`。我们通过图来一行行解释。

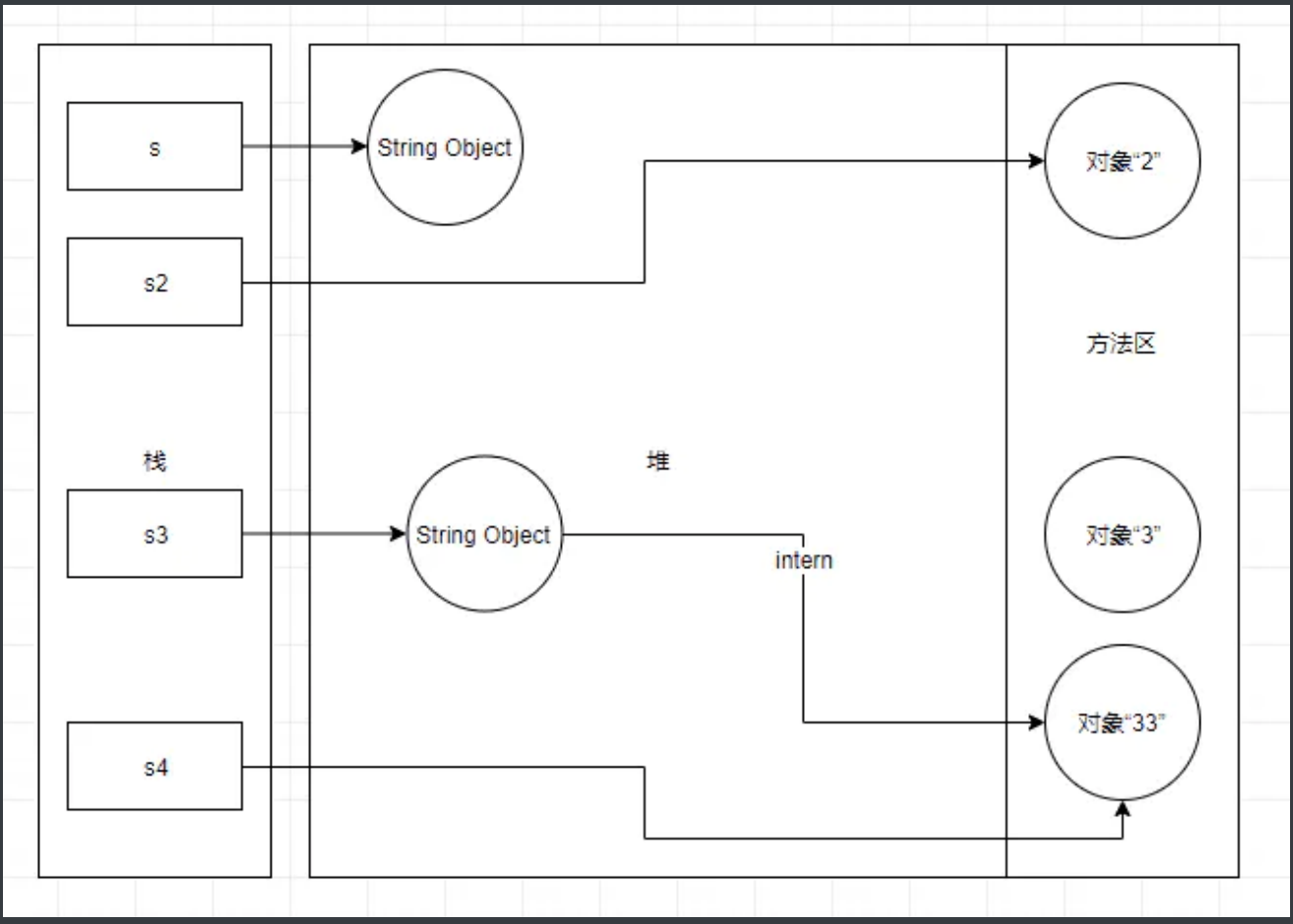
先来认识下intern()函数:

intern函数的作用是将对应的符号常量进入特殊处理，在JDK1.6以前 和 JDK1.7以后有不同的处理；

在JDK1.6中，intern的处理是 先判断字符串常量是否在字符串常量池中，如果存在直接返回该常量，如果没有找到，则将该字符串常量加入到字符串常量区，也就是在字符串常量区建立该常量；

在JDK1.7中，intern的处理是 先判断字符串常量是否在字符串常量池中，如果存在直接返回该常量，如果没有找到，说明该字符串常量在堆中，则处理是把堆区该对象的引用加入到字符串常量池中，以后别人拿到的是该字符串常量的引用，实际存在堆中

JDK1.6



`String s = new String("2");` 创建了两个对象，一个在堆中的StringObject对象，一个是在常量池中的“2”对象。
`s.intern();` 在常量池中寻找与s变量内容相同的对象，发现已经存在内容相同对象“2”，返回对象2的地址。

`String s2 = "2";` 使用字面量创建，在常量池寻找是否有相同内容的对象，发现有，返回对象"2"的地址。

`System.out.println(s == s2);` 从上面可以分析出，s变量和s2变量地址指向的是不同的对象，所以返回false

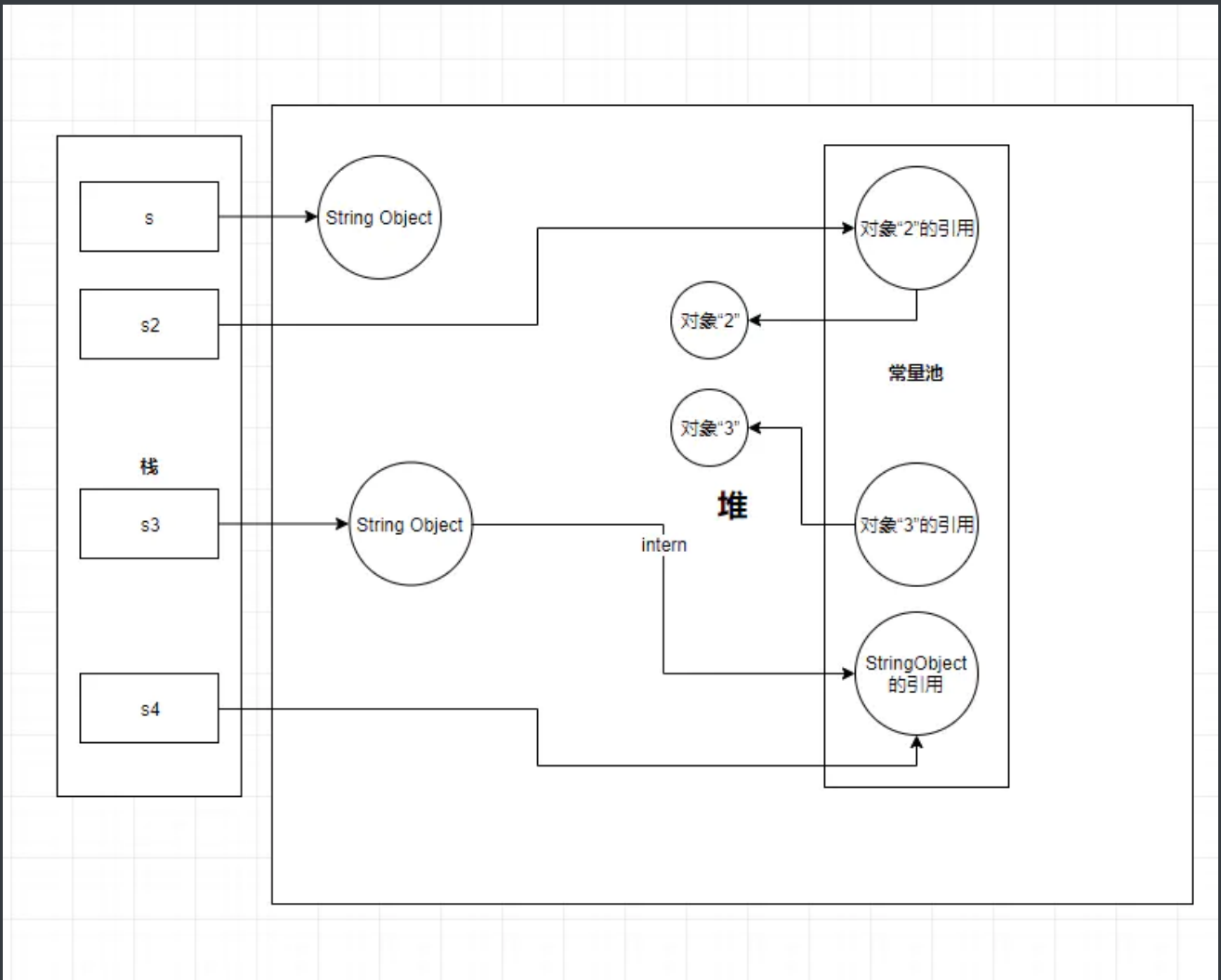
`String s3 = new String("3") + new String("3");` 创建了两个对象，一个在堆中的StringObject对象，一个是在常量池中的“3”对象。中间还有2个匿名的new String("3")我们不去讨论它们。

`s3.intern();` 在常量池中寻找与s3变量内容相同的对象，没有发现“33”对象，在常量池中创建“33”对象，返回“33”对象的地址。

`String s4 = "33";` 使用字面量创建，在常量池寻找是否有相同内容的对象，发现有，返回对象"33"的地址。

`System.out.println(s3 == s4);` 从上面可以分析出，s3变量和s4变量地址指向的是不同的对象，所以返回false

JDK1.7



`String s = new String("2");` 创建了两个对象，一个在堆中的StringObject对象，一个是在堆中的“2”对象，并在常量池中保存“2”对象的引用地址。
`s.intern();` 在常量池中寻找与s变量内容相同的对象，发现已经存在内容相同对象“2”，返回对象“2”的引用地址。
`String s2 = "2";` 使用字面量创建，在常量池寻找是否有相同内容的对象，发现有，返回对象“2”的引用地址。
`System.out.println(s == s2);` 从上面可以分析出，s变量和s2变量地址指向的是不同的对象，所以返回false

`String s3 = new String("3") + new String("3");` 创建了两个对象，一个在堆中的StringObject对象，一个是在堆中的“3”对象，并在常量池中保存“3”对象的引用地址。中间还有2个匿名的new String("3")我们不去讨论它们。
`s3.intern();` 在常量池中寻找与s3变量内容相同的对象，没有发现“33”对象，将s3对应的StringObject对象的地址保存到常量池中，返回StringObject对象的地址。
`String s4 = "33";` 使用字面量创建，在常量池寻找是否有相同内容的对象，发现有，返回其地址，也就是StringObject对象的引用地址。
`System.out.println(s3 == s4);` 从上面可以分析出，s3变量和s4变量地址指向的是相同的对象，所以返回true。

40. String 是最基本的数据类型吗?

不是。Java 中的基本数据类型只有 8 个： byte、short、int、long、float、double、char、boolean；除了基本类型（primitive type），剩下的都是引用类型（referencetype），Java 5 以后引入的枚举类型也算是一种比较特殊的引用类型。

41. String有哪些特性?

- 不变性：String 是只读字符串，是一个典型的 immutable 对象，对它进行任何操作，其实都是创建一个新的对象，再把引用指向该对象。不变模式的主要作用在于当一个对象需要被多线程共享并频繁访问时，可以保证数据的一致性；
- 常量池优化：String 对象创建之后，会在字符串常量池中进行缓存，如果下次创建同样的对象时，会直接返回缓存的引用；
- final：使用 final 来定义 String 类，表示 String 类不能被继承，提高了系统的安全性。

42. 在使用 HashMap 的时候，用 String 做 key 有什么好处?

HashMap 内部实现是通过 key 的 hashcode 来确定 value 的存储位置，因为字符串是不可变的，所以当创建字符串时，它的 hashcode 被缓存下来，不需要再次计算，所以相比于其他对象更快。

43. 包装类型是什么？基本类型和包装类型有什么区别?

Java 为每一个基本数据类型都引入了对应的包装类型（wrapper class），int 的包装类就是 Integer，从 Java 5 开始引入了自动装箱/拆箱机制，把基本类型转换成包装类型的过程叫做装箱（boxing）；反之，把包装类型转换成基本类型的过程叫做拆箱（unboxing），使得二者可以相互转换。

Java 为每个原始类型提供了包装类型：

原始类型: boolean，char，byte，short，int，long，float，double

包装类型：Boolean，Character，Byte，Short，Integer，Long，Float，Double

基本类型和包装类型的区别主要有以下 几点：

- **包装类型可以为 null，而基本类型不可以**。它使得包装类型可以应用于 POJO 中，而基本类型则不行。那为什么 POJO 的属性必须要用包装类型呢？《阿里巴巴 Java 开发手册》上有详细的说明， 数据库的查询结果可能是 null，如果使用基本类型的话，因为要自动拆箱（将包装类型转为基本类型，比如说把 Integer 对象转换成 int 值），就会抛出 `NullPointerException` 的异常。
- **包装类型可用于泛型，而基本类型不可以**。泛型不能使用基本类型，因为使用基本类型时会编译出错。

  ```
  List<int> list = new ArrayList<>(); // 提示 Syntax error, insert
  "Dimensions" to complete ReferenceType
  List<Integer> list = new ArrayList<>();
  ```

  因为泛型在编译时会进行类型擦除，最后只保留原始类型，而原始类型只能是 Object 类及其子类——基本类型是个特例。
- **基本类型比包装类型更高效**。基本类型在栈中直接存储的具体数值，而包装类型则存储的是堆中的引用。 很显然，相比较于基本类型而言，包装类型需要占用更多的内存空间。

44. 解释一下自动装箱和自动拆箱?

自动装箱：将基本数据类型重新转化为对象

```
    public class Test {
        public static void main(String[] args) {
            // 声明一个Integer对象，用到了自动的装箱：解析为:Integer num =
Integer.valueOf(9);
            Integer num = 9;
        }
    }
```

9是属于基本数据类型的，原则上它是不能直接赋值给一个对象Integer的。但jdk1.5 开始引入了自动装箱/拆箱机制，就可以进行这样的声明，自动将基本数据类型转化为对应的封装类型，成为一个对象以后就可以调用对象所声明的所有的方法。

自动拆箱：将对象重新转化为基本数据类型

```
    public class Test {
        public static void main(String[] args) {
            / /声明一个Integer对象
            Integer num = 9;

            // 进行计算时隐含的有自动拆箱
            System.out.print(num--);
        }
    }
```

因为**对象时不能直接进行运算的，而是要转化为基本数据类型后才能进行加减乘除。**

45. int 和 Integer 有什么区别?

- Integer是int的包装类；int是基本数据类型；
- Integer变量必须实例化后才能使用；int变量不需要；
- Integer实际是对象的引用，指向此new的Integer对象；int是直接存储数据值；
- Integer的默认值是null；int的默认值是0。

46. 两个new生成的Integer变量的对比

由于Integer变量实际上是对一个Integer对象的引用，所以两个通过new生成的Integer变量永远是不相等的（因为new生成的是两个对象，其内存地址不同）。

```
Integer i = new Integer(10000);
Integer j = new Integer(10000);
System.out.print(i == j); //false
```

47. Integer变量和int变量的对比

Integer变量和int变量比较时，只要两个变量的值是向等的，则结果为true（因为包装类Integer和基本数据类型int比较时，java会自动拆包装为int，然后进行比较，实际上就变为两个int变量的比较）

```
    int a = 10000;
    Integer b = new Integer(10000);
    Integer c=10000;
    System.out.println(a == b); // true
    System.out.println(a == c); // true
```

48. 非new生成的Integer变量和new Integer()生成变量的对比

非new生成的Integer变量和new Integer()生成的变量比较时，结果为false。（因为非new生成的Integer变量指向的是java常量池中的对象，而new Integer()生成的变量指向堆中新建的对象，两者在内存中的地址不同）

```
    Integer b = new Integer(10000);
    Integer c=10000;
    System.out.println(b == c); // false
```

49. 两个非new生成的Integer对象的对比

对于两个非new生成的Integer对象，进行比较时，如果两个变量的值在区间-128到127之间，则比较结果为true，如果两个变量的值不在此区间，则比较结果为false

```
Integer i = 100;
Integer j = 100;
System.out.print(i == j); //true

Integer i = 128;
Integer j = 128;
System.out.print(i == j); //false
```

当值在 -128 ~ 127之间时，java会进行自动装箱，然后会对值进行缓存，如果下次再有相同的值，会直接在缓存中取出使用。缓存是通过Integer的内部类IntegerCache来完成的。当值超出此范围，会在堆中new出一个对象来存储。

给一个Integer对象赋一个int值的时候，会调用Integer类的静态方法valueOf，源码如下：

```
public static Integer valueOf(String s, int radix) throws
NumberFormatException {
        return Integer.valueOf(parseInt(s,radix));
    }
```

```
/**
 *  (1) 在-128~127之内：静态常量池中cache数组是static final类型，cache数组对象会被存储于静态常量池中。
 * cache数组里面的元素却不是static final类型，而是cache[k] = new Integer(j++),
 * 那么这些元素是存储于堆中，只是cache数组对象存储的是指向了堆中的Integer对象（引用地址）
 *
 *  (2) 在-128~127 之外：新建一个 Integer对象，并返回。
 */
public static Integer valueOf(int i) {
        assert IntegerCache.high >= 127;
        if (i >= IntegerCache.low && i <= IntegerCache.high) {
            return IntegerCache.cache[i + (-IntegerCache.low)];
        }
        return new Integer(i);
    }
```

IntegerCache是Integer的内部类，源码如下：

```
    /**
     * 缓存支持自动装箱的对象标识语义 -128和127（含）。
     * 缓存在第一次使用时初始化。 缓存的大小可以由-XX：AutoBoxCacheMax = <size>选项控制。
     * 在VM初始化期间，java.lang.Integer.IntegerCache.high属性可以设置并保存在私有系统属性中
     */
    private static class IntegerCache {
        static final int low = -128;
        static final int high;
        static final Integer cache[];

        static {
            // high value may be configured by property
            int h = 127;
            String integerCacheHighPropValue =

 sun.misc.VM.getSavedProperty("java.lang.Integer.IntegerCache.high");
            if (integerCacheHighPropValue != null) {
                int i = parseInt(integerCacheHighPropValue);
                i = Math.max(i, 127);
                // Maximum array size is Integer.MAX_VALUE
                h = Math.min(i, Integer.MAX_VALUE - (-low) -1);
            }
            high = h;

            cache = new Integer[(high - low) + 1];
            int j = low;
            for(int k = 0; k < cache.length; k++) {
                cache[k] = new Integer(j++); // 创建一个对象
            }
        }

        private IntegerCache() {}
    }
```

50. 什么是反射?

反射是在运行状态中，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意一个方法和属性；这种动态获取的信息以及动态调用对象的方法的功能称为 Java 语言的反射机制。

51. 反射机制的优缺点有哪些?

优点：能够运行时动态获取类的实例，提高灵活性；可与动态编译结合 `Class.forName('com.mysql.jdbc.Driver.class');` ，加载MySQL的驱动类。

缺点：使用反射性能较低，需要解析字节码，将内存中的对象进行解析。其解决方案是：通过setAccessible(true)关闭JDK的安全检查来提升反射速度；多次创建一个类的实例时，有缓存会快很多；ReflflectASM工具类，通过字节码生成的方式加快反射速度。

52. 如何获取反射中的Class对象?

1. Class.forName(“类的路径”)；当你知道该类的全路径名时，你可以使用该方法获取 Class 类对象。

```
Class clz = Class.forName("java.lang.String");
```

2. 类名.class。这种方法只适合在编译前就知道操作的 Class。

```
Class clz = String.class;
```

3. 对象名.getClass()。

```
String str = new String("Hello");
Class clz = str.getClass();
```

4. 如果是基本类型的包装类，可以调用包装类的Type属性来获得该包装类的Class对象。

53. Java反射API有几类?

反射 API 用来生成 JVM 中的类、接口或则对象的信息。

- Class 类：反射的核心类，可以获取类的属性，方法等信息。
- Field 类：Java.lang.reflec 包中的类，表示类的成员变量，可以用来获取和设置类之中的属性值。
- Method 类：Java.lang.reflec 包中的类，表示类的方法，它可以用来获取类中的方法信息或者执行方法。

- Constructor 类：Java.lang.reflec 包中的类，表示类的构造方法。

54. 反射使用的步骤?

1. 获取想要操作的类的Class对象，这是反射的核心，通过Class对象我们可以任意调用类的方法。
2. 调用 Class 类中的方法，既就是反射的使用阶段。
3. 使用反射 API 来操作这些信息。

具体可以看下面的例子：

```
public class Apple {

    private int price;

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }

    public static void main(String[] args) throws Exception{
        //正常的调用
        Apple apple = new Apple();
        apple.setPrice(5);
        System.out.println("Apple Price:" + apple.getPrice());
        //使用反射调用
        Class clz = Class.forName("com.chenshuyi.api.Apple");
        Method setPriceMethod = clz.getMethod("setPrice", int.class);
        Constructor appleConstructor = clz.getConstructor();
        Object appleObj = appleConstructor.newInstance();
        setPriceMethod.invoke(appleObj, 14);
        Method getPriceMethod = clz.getMethod("getPrice");
        System.out.println("Apple Price:" +
getPriceMethod.invoke(appleObj));
    }
}
```

从代码中可以看到我们使用反射调用了 setPrice 方法，并传递了 14 的值。之后使用反射调用了 getPrice 方法，输出其价格。上面的代码整个的输出结果是：

```
Apple Price:5
Apple Price:14
```

从这个简单的例子可以看出，一般情况下我们使用反射获取一个对象的步骤：

- 获取类的 Class 对象实例

```
Class clz = Class.forName("com.zhenai.api.Apple");
```

- 根据 Class 对象实例获取 Constructor 对象

```
Constructor appleConstructor = clz.getConstructor();
```

- 使用 Constructor 对象的 newInstance 方法获取反射类对象

```
Object appleObj = appleConstructor.newInstance();
```

而如果要调用某一个方法，则需要经过下面的步骤：

- 获取方法的 Method 对象

```
Method setPriceMethod = clz.getMethod("setPrice", int.class);
```

- 利用 invoke 方法调用方法

```
setPriceMethod.invoke(appleObj, 14);
```

55. 为什么引入反射概念？反射机制的应用有哪些？

我们来看一下 Oracle 官方文档中对反射的描述：

从 Oracle 官方文档中可以看出，反射主要应用在以下几方面：

- 反射让开发人员可以通过外部类的全路径名创建对象，并使用这些类，实现一些扩展的功能。
- 反射让开发人员可以枚举出类的全部成员，包括构造函数、属性、方法。以帮助开发者写出正确的代码。
- 测试时可以利用反射 API 访问类的私有成员，以保证测试代码覆盖率。

也就是说，Oracle 希望开发者将反射作为一个工具，用来帮助程序员实现本不可能实现的功能。

举两个最常见使用反射的例子，来说明反射机制的强大之处：

第一种：**JDBC 的数据库的连接**

在JDBC 的操作中，如果要想进行数据库的连接，则必须按照以上的几步完成

1. 通过Class.forName()加载数据库的驱动程序 （通过反射加载，前提是引入相关了Jar包）；
2. 通过 DriverManager 类进行数据库的连接，连接的时候要输入数据库的连接地址、用户名、密码；
3. 通过Connection 接口接收连接。

```
public class ConnectionJDBC {

    /**
     * @param args
     */
    //驱动程序就是之前在classpath中配置的JDBC的驱动程序的JAR 包中
    public static final String DBDRIVER = "com.mysql.jdbc.Driver";
    //连接地址是由各个数据库生产商单独提供的，所以需要单独记住
    public static final String DBURL = "jdbc:mysql://localhost:3306/test";
    //连接数据库的用户名
    public static final String DBUSER = "root";
    //连接数据库的密码
    public static final String DBPASS = "";


    public static void main(String[] args) throws Exception {
        Connection con = null; //表示数据库的连接对象
        Class.forName(DBDRIVER); //1、使用CLASS 类加载驱动程序 ,反射机制的体现
```

```
            con = DriverManager.getConnection(DBURL,DBUSER,DBPASS); //2、连接数据
    库
            System.out.println(con);
            con.close(); // 3、关闭数据库
        }
```

第二种：**Spring 框架的使用，最经典的就是xml的配置模式。**

Spring 通过 XML 配置模式装载 Bean 的过程：

1. 将程序内所有 XML 或 Properties 配置文件加载入内存中；
2. Java类里面解析xml或properties里面的内容，得到对应实体类的字节码字符串以及相关的属性信息；
3. 使用反射机制，根据这个字符串获得某个类的Class实例；
4. 动态配置实例的属性。

Spring这样做的好处是：

- 不用每一次都要在代码里面去new或者做其他的事情；
- 以后要改的话直接改配置文件，代码维护起来就很方便了；
- 有时为了适应某些需求，Java类里面不一定能直接调用另外的方法，可以通过反射机制来实现。

模拟 Spring 加载 XML 配置文件：

```
    public class BeanFactory {
            private Map<String, Object> beanMap = new HashMap<String, Object>();
            /**
            * bean工厂的初始化.
            * @param xml xml配置文件
            */
            public void init(String xml) {
                    try {
                            //读取指定的配置文件
                            SAXReader reader = new SAXReader();
                            ClassLoader classLoader =
    Thread.currentThread().getContextClassLoader();
                            //从class目录下获取指定的xml文件
                            InputStream ins = classLoader.getResourceAsStream(xml);
```

```
                    Document doc = reader.read(ins);
                    Element root = doc.getRootElement();
                    Element foo;

                    //遍历bean
                    for (Iterator i = root.elementIterator("bean");
i.hasNext();) {
                        foo = (Element) i.next();
                        //获取bean的属性id和class
                        Attribute id = foo.attribute("id");
                        Attribute cls = foo.attribute("class");

                        //利用Java反射机制，通过class的名称获取Class对象
                        Class bean = Class.forName(cls.getText());

                        //获取对应class的信息
                        java.beans.BeanInfo info =
java.beans.Introspector.getBeanInfo(bean);
                        //获取其属性描述
                        java.beans.PropertyDescriptor pd[] =
info.getPropertyDescriptors();
                        //设置值的方法
                        Method mSet = null;
                        //创建一个对象
                        Object obj = bean.newInstance();

                        //遍历该bean的property属性
                        for (Iterator ite =
foo.elementIterator("property"); ite.hasNext();) {
                            Element foo2 = (Element) ite.next();
                            //获取该property的name属性
                            Attribute name = foo2.attribute("name");
                            String value = null;

                            //获取该property的子元素value的值
                            for(Iterator ite1 =
foo2.elementIterator("value"); ite1.hasNext();) {
                                Element node = (Element)
ite1.next();
                                value = node.getText();
```

```
                                        break;
                                }

                                for (int k = 0; k < pd.length; k++) {
                                        if
(pd[k].getName().equalsIgnoreCase(name.getText())) {
                                                mSet =
pd[k].getWriteMethod();
                                                //利用Java的反射极致调用对象
的某个set方法，并将值设置进去
                                                mSet.invoke(obj, value);
                                        }
                                }
                        }

                        //将对象放入beanMap中，其中key为id值，value为对象
                        beanMap.put(id.getText(), obj);
                }
        } catch (Exception e) {
                System.out.println(e.toString());
        }
    }

    //other codes
}
```

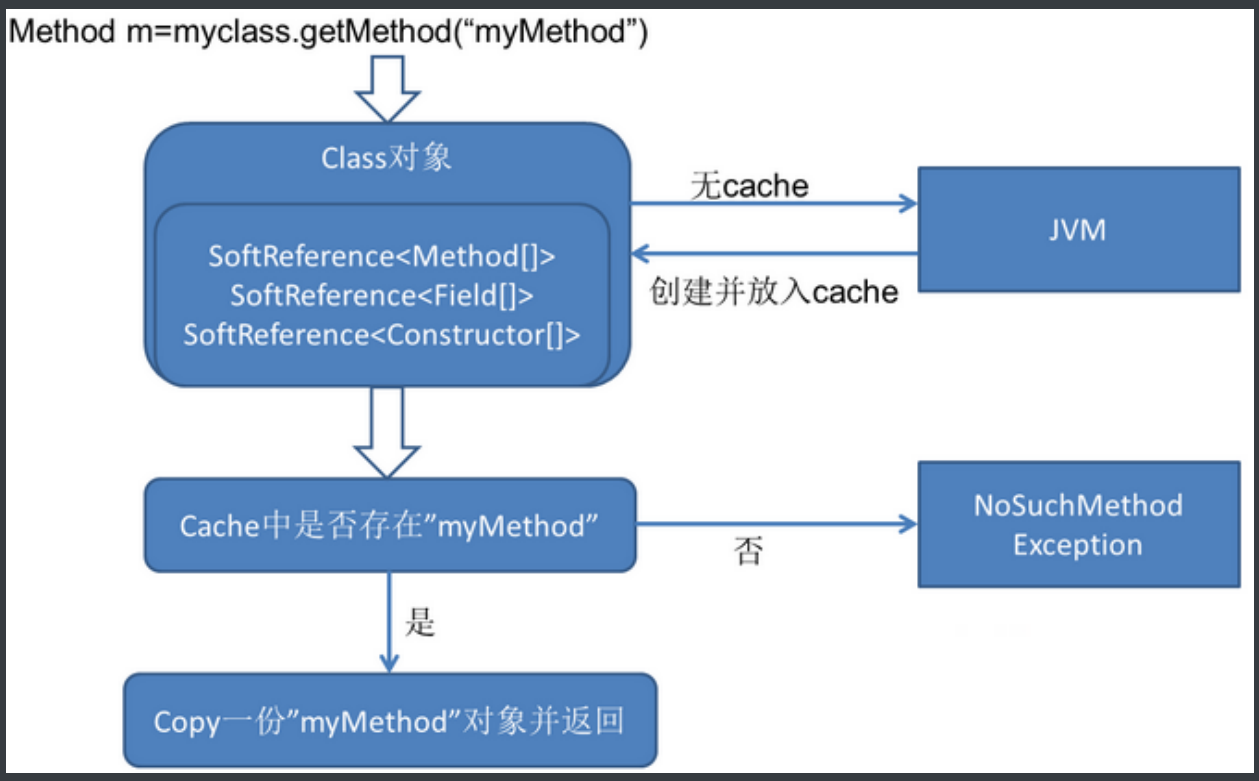
56. 反射机制的原理是什么?

```
Class actionClass=Class.forName("MyClass");
Object action=actionClass.newInstance();
Method method = actionClass.getMethod("myMethod",null);
method.invoke(action,null);
```

上面就是最常见的反射使用的例子，前两行实现了类的装载、链接和初始化（newInstance方法实际上也是使用反射调用了方法），后两行实现了从class对象中获取到method对象然后执行反射调用。

因反射原理较复杂，下面简要描述下流程，想要详细了解的小伙伴，可以看这篇文章：https://www.cnblogs.com/yougewe/p/10125073.html

1. 反射获取类实例 Class.forName()，并没有将实现留给了java,而是交给了jvm去加载！主要是先获取 ClassLoader, 然后调用 native 方法，获取信息，加载类则是回调 java.lang.ClassLoader。最后，jvm又会回调 ClassLoader 进类加载！
2. newInstance() 主要做了三件事：
   - 权限检测，如果不通过直接抛出异常；
   - 查找无参构造器，并将其缓存起来；
   - 调用具体方法的无参构造方法，生成实例并返回。
3. 获取Method对象，



上面的Class对象是在加载类时由JVM构造的，JVM为每个类管理一个独一无二的Class对象，这份Class对象里维护着该类的所有Method，Field，Constructor的cache，这份cache也可以被称作根对象。

每次getMethod获取到的Method对象都持有对根对象的引用，因为一些重量级的Method的成员变量（主要是MethodAccessor），我们不希望每次创建Method对象都要重新初始化，于是所有代表同一个方法的Method对象都共享着根对象的MethodAccessor，每一次创建都会调用根对象的copy方法复制一份：

```
    Method copy() {

            Method res = new Method(clazz, name, parameterTypes, returnType,

                                    exceptionTypes, modifiers, slot, signature,
```

```
                                    annotations, parameterAnnotations,
    annotationDefault);

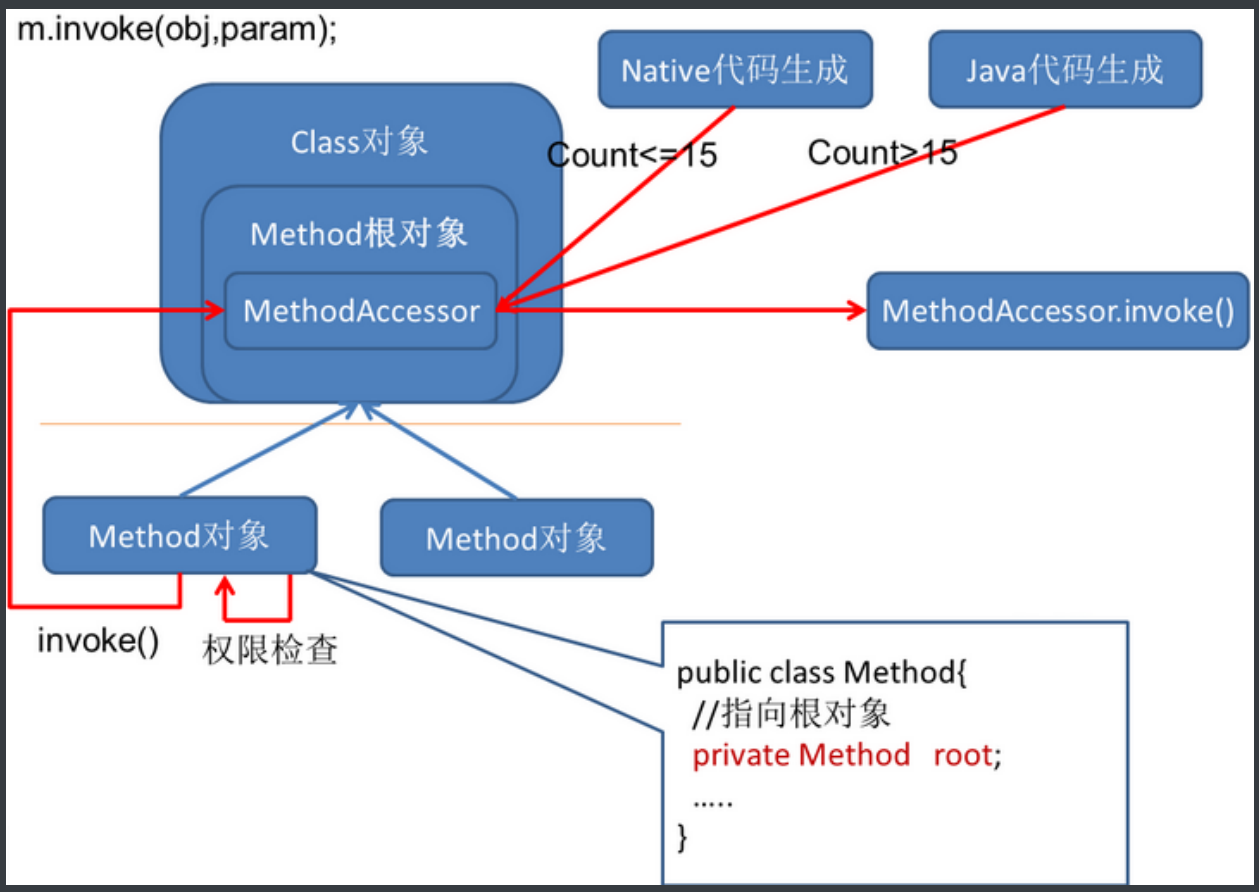
            res.root = this;

            res.methodAccessor = methodAccessor;

            return res;

        }
```

4. 调用invoke()方法。调用invoke方法的流程如下：



调用Method.invoke之后，会直接去调MethodAccessor.invoke。MethodAccessor就是上面提到的所有同名method共享的一个实例，由ReflectionFactory创建。

创建机制采用了一种名为inflation的方式（JDK1.4之后）：如果该方法的累计调用次数<=15，会创建出NativeMethodAccessorImpl，它的实现就是直接调用native方法实现反射；如果该方法的累计调用次数>15，会由java代码创建出字节码组装而成的MethodAccessorImpl。（是否采用inflation和15这个数字都可以在jvm参数中调整）

以调用MyClass.myMethod(String s)为例，生成出的MethodAccessorImpl字节码翻译成Java代码大致如下：

```
public class GeneratedMethodAccessor1 extends MethodAccessorImpl {
    public Object invoke(Object obj, Object[] args)  throws Exception {
        try {
            MyClass target = (MyClass) obj;
            String arg0 = (String) args[0];
            target.myMethod(arg0);
        } catch (Throwable t) {
            throw new InvocationTargetException(t);
        }
    }
}
```

57. Java中的泛型是什么？

泛型是 JDK1.5 的一个新特性，**泛型就是将类型参数化，其在编译时才确定具体的参数。**这种参数类型可以用在类、接口和方法的创建中，分别称为泛型类、泛型接口、泛型方法。

58. 使用泛型的好处是什么?

远在 JDK 1.4 版本的时候，那时候是没有泛型的概念的，如果使用 Object 来实现通用、不同类型的处理，有这么两个缺点：

1. 每次使用时都需要强制转换成想要的类型
2. 在编译时编译器并不知道类型转换是否正常，运行时才知道，不安全。

如这个例子：

```
List list = new ArrayList();
list.add("www.cnblogs.com");
list.add(23);
String name = (String)list.get(0);
String number = (String)list.get(1);    //ClassCastException
```

上面的代码在运行时会发生强制类型转换异常。这是因为我们在存入的时候，第二个是一个 Integer 类型，但是取出来的时候却将其强制转换为 String 类型了。Sun 公司为了使 Java 语言更加安全，减少运行时异常的发生。于是在 JDK 1.5 之后推出了泛型的概念。

根据《Java 编程思想》中的描述，泛型出现的动机在于：有许多原因促成了泛型的出现，而最引人注意的一个原因，就是**为了创建容器类**。

使用泛型的好处有以下几点：

1. 类型安全
    - 泛型的主要目标是提高 Java 程序的类型安全
    - 编译时期就可以检查出因 Java 类型不正确导致的 ClassCastException 异常
    - 符合越早出错代价越小原则
2. 消除强制类型转换
    - 泛型的一个附带好处是，使用时直接得到目标类型，消除许多强制类型转换
    - 所得即所需，这使得代码更加可读，并且减少了出错机会
3. 潜在的性能收益
    - 由于泛型的实现方式，支持泛型（几乎）不需要 JVM 或类文件更改
    - 所有工作都在编译器中完成
    - 编译器生成的代码跟不使用泛型（和强制类型转换）时所写的代码几乎一致，只是更能确保类型安全而已

59

泛型是一种语法糖，泛型这种语法糖的基本原理是类型擦除。Java中的泛型基本上都是在编译器这个层次来实现的，也就是说：**泛型只存在于编译阶段，而不存在于运行阶段。**在编译后的 class 文件中，是没有泛型这个概念的。

类型擦除：使用泛型的时候加上的类型参数，编译器在编译的时候去掉类型参数。

例如：

```
public class Caculate<T> {
    private T num;
}
```

我们定义了一个泛型类，定义了一个属性成员，该成员的类型是一个泛型类型，这个 T 具体是什么类型，我们也不知道，它只是用于限定类型的。反编译一下这个 Caculate 类：

```
public class Caculate{
    public Caculate(){}
    private Object num;
}
```

发现编译器擦除 Caculate 类后面的两个尖括号，并且将 num 的类型定义为 Object 类型。

那么是不是所有的泛型类型都以 Object 进行擦除呢？大部分情况下，泛型类型都会以 Object 进行替换，而有一种情况则不是。那就是使用到了extends和super语法的有界类型，如：

```
public class Caculate<T extends String> {
    private T num;
}
```

这种情况的泛型类型，num 会被替换为 String 而不再是 Object。这是一个类型限定的语法，它限定 T 是 String 或者 String 的子类，也就是你构建 Caculate 实例的时候只能限定 T 为 String 或者 String 的子类，所以无论你限定 T 为什么类型，String 都是父类，不会出现类型不匹配的问题，于是可以使用 String 进行类型擦除。

实际上编译器会正常的将使用泛型的地方编译并进行类型擦除，然后返回实例。但是除此之外的是，如果构建泛型实例时使用了泛型语法，那么编译器将标记该实例并关注该实例后续所有方法的调用，每次调用前都进行安全检查，非指定类型的方法都不能调用成功。

实际上编译器不仅关注一个泛型方法的调用，它还会为某些返回值为限定的泛型类型的方法进行强制类型转换，由于类型擦除，返回值为泛型类型的方法都会擦除成 Object 类型，当这些方法被调用后，编译器会额外插入一行 checkcast 指令用于强制类型转换。这一个过程就叫做『泛型翻译』。

6o. 什么是泛型中的限定通配符和非限定通配符？

限定通配符对类型进行了限制。有两种限定通配符，一种是<? extends T>它通过确保类型必须是T的子类来设定类型的上界，另一种是<? super T>它通过确保类型必须是T的父类来设定类型的下界。泛型类型必须用限定内的类型来进行初始化，否则会导致编译错误。

非限定通配符 **？** ,可以用任意类型来替代。如 `List<?>` 的意思是这个集合是一个可以持有任意类型的集合，它可以是 `List<A>` ，也可以是 `List<B>` ,或者 `List<C>` 等等。

61. List<? extends T>和List <? super T>之间有什么区别？

这两个List的声明都是限定通配符的例子，List<? extends T>可以接受任何继承自T的类型的List，而List<? super T>可以接受任何T的父类构成的List。例如List<? extends Number>可以接受List或List。

62

不可以。真这样做的话会导致编译错误。因为List可以存储任何类型的对象包括String, Integer等等，而List却只能用来存储String。

```
List<Object> objectList;
List<String> stringList;
objectList = stringList;  //compilation error incompatible types
```

63. 判断 ArrayList<String> 与 ArrayList<Integer> 是否相等?

```
ArrayList<String> a = new ArrayList<String>();
ArrayList<Integer> b = new ArrayList<Integer>();
Class c1 = a.getClass();
Class c2 = b.getClass();
System.out.println(c1 == c2);
```

输出的结果是 true。因为无论对于 ArrayList 还是 ArrayList，它们的 Class 类型都是一直的，都是 ArrayList.class。

那它们声明时指定的 String 和 Integer 到底体现在哪里呢?

答案是体现在类编译的时候。当 JVM 进行类编译时，会进行泛型检查，如果一个集合被声明为 String 类型，那么它往该集合存取数据的时候就会对数据进行判断，从而避免存入或取出错误的数据。

补充: **Array中可以用泛型吗?**

不可以。这也是为什么 Joshua Bloch 在 《Effective Java》一书中建议使用 List 来代替 Array，因为 List 可以提供编译期的类型安全保证，而 Array 却不能。

64. Java序列化与反序列化是什么?

Java序列化是指把Java对象转换为字节序列的过程，而Java反序列化是指把字节序列恢复为Java对象的过程：

- **序列化：**序列化是把对象转换成有序字节流，以便在网络上传输或者保存在本地文件中。核心作用是对象状态的保存与重建。我们都知道，Java对象是保存在JVM的堆内存中的，也就是说，如果JVM堆不存在了，那么对象也就跟着消失了。

  而序列化提供了一种方案，可以让你在即使JVM停机的情况下也能把对象保存下来的方案。就像我们平时用的U盘一样。把Java对象序列化成可存储或传输的形式（如二进制流），比如保存在文件中。这样，当再次需要这个对象的时候，从文件中读取出二进制流，再从二进制流中反序列化出对象。
- **反序列化：**客户端从文件中或网络上获得序列化后的对象字节流，根据字节流中所保存的对象状态及描述信息，通过反序列化重建对象。

65. 为什么需要序列化与反序列化?

简要描述：**对内存中的对象进行持久化或网络传输, 这个时候都需要序列化和反序列化**

深入描述：

1. **对象序列化可以实现分布式对象。**

主要应用例如：RMI(即远程调用Remote Method Invocation)要利用对象序列化运行远程主机上的服务，就像在本地机上运行对象时一样。

2. **java对象序列化不仅保留一个对象的数据，而且递归保存对象引用的每个对象的数据。**

可以将整个对象层次写入字节流中，可以保存在文件中或在网络连接上传递。利用对象序列化可以进行对象的"深复制"，即复制对象本身及引用的对象本身。序列化一个对象可能得到整个对象序列。

3. **序列化可以将内存中的类写入文件或数据库中。**

比如：将某个类序列化后存为文件，下次读取时只需将文件中的数据反序列化就可以将原先的类还原到内存中。也可以将类序列化为流数据进行传输。

总的来说就是将一个已经实例化的类转成文件存储，下次需要实例化的时候只要反序列化即可将类实例化到内存中并保留序列化时类中的所有变量和状态。

4. **对象、文件、数据，有许多不同的格式，很难统一传输和保存。**

序列化以后就都是字节流了，无论原来是什么东西，都能变成一样的东西，就可以进行通用的格式传输或保存，传输结束以后，要再次使用，就进行反序列化还原，这样对象还是对象，文件还是文件。

66. 序列化实现的方式有哪些?

实现**Serializable**接口或者**Externalizable**接口。

Serializable接口

类通过实现 `java.io.Serializable` 接口以启用其序列化功能。可序列化类的所有子类型本身都是可序列化的。**序列化接口没有方法或字段，仅用于标识可序列化的语义。**

如以下例子：

```
import java.io.Serializable;

public class User implements Serializable {
   private String name;
   private int age;
   public String getName() {
       return name;
   }
   public void setName(String name) {
       this.name = name;
   }

   @Override
   public String toString() {
       return "User{" +
               "name='" + name +
               '}';
   }
}
```

通过下面的代码进行序列化及反序列化：

```
public class SerializableDemo {
```

```
    public static void main(String[] args) {
        //Initializes The Object
        User user = new User();
        user.setName("cosen");
        System.out.println(user);

        //Write Obj to File
        try (FileOutputStream fos = new FileOutputStream("tempFile");
ObjectOutputStream oos = new ObjectOutputStream(
            fos)) {
            oos.writeObject(user);
        } catch (IOException e) {
            e.printStackTrace();
        }

        //Read Obj from File
        File file = new File("tempFile");
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(file))) {
            User newUser = (User)ois.readObject();
            System.out.println(newUser);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}

//OutPut:
//User{name='cosen'}
//User{name='cosen'}
```

Externalizable接口

`Externalizable` 继承自 `Serializable`，该接口中定义了两个抽象方法：`writeExternal()` 与 `readExternal()`。

当使用 `Externalizable` 接口来进行序列化与反序列化的时候需要开发人员重写 `writeExternal()` 与 `readExternal()` 方法。否则所有变量的值都会变成默认值。

```
public class User implements Externalizable {

    private String name;
    private int age;

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeObject(name);
    }
    public void readExternal(ObjectInput in) throws IOException,
ClassNotFoundException {
        name = (String) in.readObject();
    }

    @Override
    public String toString() {
        return "User{" +
                "name='" + name +
                '}';
    }
}
```

通过下面的代码进行序列化及反序列化：

```
public class ExternalizableDemo1 {

  public static void main(String[] args) {
      //Write Obj to file
      User user = new User();
      user.setName("cosen");
      try(ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("tempFile"))){
          oos.writeObject(user);
      } catch (IOException e) {
```

```
            e.printStackTrace();
        }

        //Read Obj from file
        File file = new File("tempFile");
        try(ObjectInputStream ois =  new ObjectInputStream(new
    FileInputStream(file))){
            User newInstance = (User) ois.readObject();
            //output
            System.out.println(newInstance);
        } catch (IOException | ClassNotFoundException e ) {
            e.printStackTrace();
        }
      }
    }

    //OutPut:
    //User{name='cosen'}
```

两种序列化的对比

实现Serializable接口	实现Externalizable接口
系统自动存储必要的信息	程序员决定存储哪些信息
Java内建支持，易于实现，只需要实现该接口即可，无需任何代码支持	必须实现接口内的两个方法
性能略差	性能略好

67. 什么是serialVersionUID?

serialVersionUID 用来表明类的不同版本间的兼容性

Java的序列化机制是通过在运行时判断类的serialVersionUID来验证版本一致性的。在进行反序列化时，JVM会把传来的字节流中的serialVersionUID与本地相应实体（类）的serialVersionUID进行比较，如果相同就认为是一致的，可以进行反序列化，否则就会出现序列化版本不一致的异常。

68. 为什么还要显示指定serialVersionUID的值?

如果不显示指定serialVersionUID, JVM在序列化时会根据属性自动生成一个serialVersionUID, 然后与属性一起序列化, 再进行持久化或网络传输. 在反序列化时, JVM会再根据属性自动生成一个新版serialVersionUID, 然后将这个新版serialVersionUID与序列化时生成的旧版serialVersionUID进行比较, 如果相同则反序列化成功, 否则报错.

如果显示指定了, JVM在序列化和反序列化时仍然都会生成一个serialVersionUID, 但值为我们显示指定的值, 这样在反序列化时新旧版本的serialVersionUID就一致了.

在实际开发中, 不显示指定serialVersionUID的情况会导致什么问题? 如果我们的类写完后不再修改, 那当然不会有问题, 但这在实际开发中是不可能的, 我们的类会不断迭代, 一旦类被修改了, 那旧对象反序列化就会报错. 所以在实际开发中, 我们都会显示指定一个serialVersionUID, 值是多少无所谓, 只要不变就行。

69. serialVersionUID什么时候修改?

《阿里巴巴Java开发手册》中有以下规定：

> 10. 【强制】序列化类新增属性时，请不要修改 serialVersionUID 字段，避免反序列失败；如果完全不兼容升级，避免反序列化混乱，那么请修改 serialVersionUID 值。
> 说明：注意 serialVersionUID 不一致会抛出序列化运行时异常。

想要深入了解的小伙伴，可以看这篇文章：https://juejin.cn/post/6844903746682486791

70. Java 序列化中如果有些字段不想进行序列化，怎么办?

对于不想进行序列化的变量，使用 transient 关键字修饰。

`transient` 关键字的作用是控制变量的序列化，在变量声明前加上该关键字，可以阻止该变量被序列化到文件中，在被反序列化后， `transient` 变量的值被设为初始值，如 int 型的是 0，对象型的是 null。transient 只能修饰变量，不能修饰类和方法。

71. 静态变量会被序列化吗?

不会。因为序列化是针对对象而言的, 而静态变量优先于对象存在, 随着类的加载而加载, 所以不会被序列化.

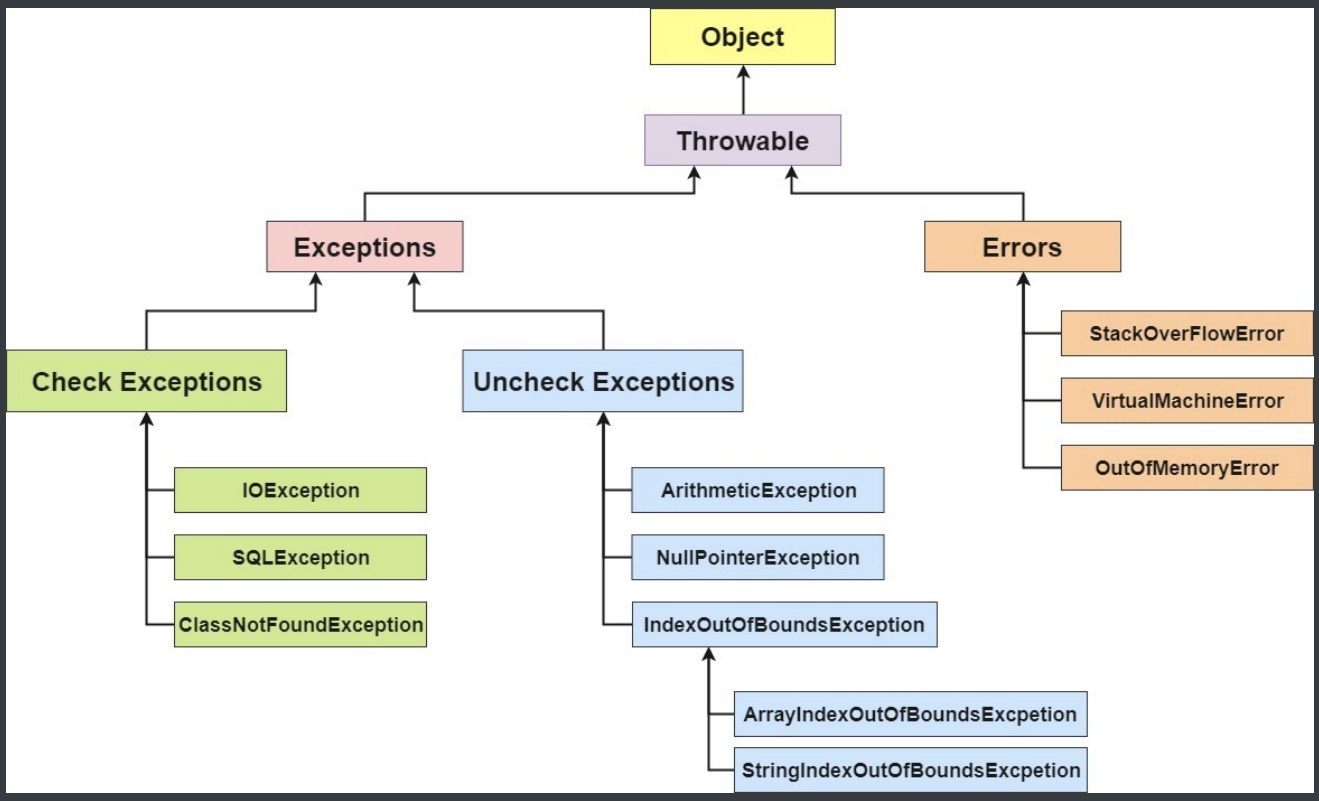
看到这个结论, 是不是有人会问, serialVersionUID也被static修饰, 为什么serialVersionUID会被序列化? 其实serialVersionUID属性并没有被序列化, JVM在序列化对象时会自动生成一个serialVersionUID, 然后将我们显示指定的serialVersionUID属性值赋给自动生成的serialVersionUID。

72. Error 和 Exception 区别是什么?

Java 中，所有的异常都有一个共同的祖先 `java.lang` 包中的 `Throwable` 类。 `Throwable` 类有两个重要的子类 `Exception` （异常）和 `Error` （错误）。

`Exception` 和 `Error` 二者都是 Java 异常处理的重要子类，各自都包含大量子类。

- **`Exception`** :程序本身可以处理的异常，可以通过 `catch` 来进行捕获，通常遇到这种错误，应对其进行处理，使应用程序可以继续正常运行。 `Exception` 又可以分为运行时异常(RuntimeException, 又叫非受检查异常)和非运行时异常(又叫受检查异常) 。
- **`Error`** ： `Error` 属于程序无法处理的错误 ， 我们没办法通过 `catch` 来进行捕获 。例如，系统崩溃，内存不足，堆栈溢出等，编译器不会对这类错误进行检测，一旦这类错误发生，通常应用程序会被终止，仅靠应用程序本身无法恢复。



73. 非受检查异常(运行时异常)和受检查异常(一般异常)区别是什么?

非受检查异常：包括 `RuntimeException` 类及其子类，表示 JVM 在运行期间可能出现的异常。 Java 编译器不会检查运行时异常。例如： `NullPointException(空指针)`、`NumberFormatException（字符串转换为数字）` 、 `IndexOutOfBoundsException(数组越界)`、`ClassCastException(类转换异常)`、 `ArrayStoreException(数据存储异常，操作数组时类型不一致)` 等。

受检查异常：是Exception 中除 `RuntimeException` 及其子类之外的异常。 Java 编译器会检查受检查异常。常见的受检查异常有： IO 相关的异常、 `ClassNotFoundException` 、`SQLException` 等。

非受检查异常和受检查异常之间的区别：是否强制要求调用者必须处理此异常，如果强制要求调用者必须进行处理，那么就使用受检查异常，否则就选择非受检查异常。

74. throw 和 throws 的区别是什么?

Java 中的异常处理除了包括捕获异常和处理异常之外，还包括声明异常和抛出异常，可以通过 throws 关键字在方法上声明该方法要抛出的异常，或者在方法内部通过 throw 抛出异常对象。

throws 关键字和 throw 关键字在使用上的几点区别如下：

- throw 关键字用在方法内部，只能用于抛出一种异常，用来抛出方法或代码块中的异常，受查异常和非受查异常都可以被抛出。
- throws 关键字用在方法声明上，可以抛出多个异常，用来标识该方法可能抛出的异常列表。一个方法用 throws 标识了可能抛出的异常列表，调用该方法的方法中必须包含可处理异常的代码，否则也要在方法签名中用 throws 关键字声明相应的异常。

举例如下：

throw 关键字：

```
    public static void main(String[] args) {
            String s = "abc";
            if(s.equals("abc")) {
                throw new NumberFormatException();
            } else {
                System.out.println(s);
            }
            //function();
    }
```

throws 关键字:

```
    public static void function() throws NumberFormatException{
            String s = "abc";
            System.out.println(Double.parseDouble(s));
        }

        public static void main(String[] args) {
            try {
                function();
            } catch (NumberFormatException e) {
                System.err.println("非数据类型不能转换。");
                //e.printStackTrace();
            }
    }
```

75. NoClassDefFoundError 和 ClassNotFoundException 区别?

NoClassDefFoundError 是一个 Error 类型的异常，是由 JVM 引起的，不应该尝试捕获这个异常。引起该异常的原因是 JVM 或 ClassLoader 尝试加载某类时在内存中找不到该类的定义，该动作发生在运行期间，即编译时该类存在，但是在运行时却找不到了，可能是变异后被删除了等原因导致。

ClassNotFoundException 是一个受检查异常，需要显式地使用 try-catch 对其进行捕获和处理，或在方法签名中用 throws 关键字进行声明。当使用 Class.forName, ClassLoader.loadClass 或 ClassLoader.findSystemClass 动态加载类到内存的时候，通过传入的类路径参数没有找到该类，就会抛出该异常；另一种抛出该异常的可能原因是某个类已经由一个类加载器加载至内存中，另一个加载器又尝试去加载它。

76. Java常见异常有哪些?

- java.lang.IllegalAccessError：违法访问错误。当一个应用试图访问、修改某个类的域（Field）或者调用其方法，但是又违反域或方法的可见性声明，则抛出该异常。
- java.lang.InstantiationError：实例化错误。当一个应用试图通过Java的new操作符构造一个抽象类或者接口时抛出该异常.
- java.lang.OutOfMemoryError：内存不足错误。当可用内存不足以让Java虚拟机分配给一个对象时抛出该错误。
- java.lang.StackOverflowError：堆栈溢出错误。当一个应用递归调用的层次太深而导致堆栈溢出或者陷入死循环时抛出该错误。
- java.lang.ClassCastException：类造型异常。假设有类A和B（A不是B的父类或子类），O是A的实例，那么当强制将O构造为类B的实例时抛出该异常。该异常经常被称为强制类型转换异常。
- java.lang.ClassNotFoundException：找不到类异常。当应用试图根据字符串形式的类名构造类，而在遍历CLASSPAH之后找不到对应名称的class文件时，抛出该异常。
- java.lang.ArithmeticException：算术条件异常。譬如：整数除零等。
- java.lang.ArrayIndexOutOfBoundsException：数组索引越界异常。当对数组的索引值为负数或大于等于数组大小时抛出。
- java.lang.IndexOutOfBoundsException：索引越界异常。当访问某个序列的索引值小于0或大于等于序列大小时，抛出该异常。
- java.lang.InstantiationException：实例化异常。当试图通过newInstance()方法创建某个类的实例，而该类是一个抽象类或接口时，抛出该异常。
- java.lang.NoSuchFieldException：属性不存在异常。当访问某个类的不存在的属性时抛出该异常。
- java.lang.NoSuchMethodException：方法不存在异常。当访问某个类的不存在的方法时抛出该异常。
- java.lang.NullPointerException：空指针异常。当应用试图在要求使用对象的地方使用了null时，抛出该异常。譬如：调用null对象的实例方法、访问null对象的属性、计算null对象的长度、使用throw语句抛出null等等。
- java.lang.NumberFormatException：数字格式异常。当试图将一个String转换为指定的数字类型，而该字符串确不满足数字类型要求的格式时，抛出该异常。
- java.lang.StringIndexOutOfBoundsException：字符串索引越界异常。当使用索引值访问某个字符串中的字符，而该索引值小于0或大于等于序列大小时，抛出该异常。

77. try-catch-finally 中哪个部分可以省略?

catch 可以省略。更为严格的说法其实是：try只适合处理运行时异常，try+catch适合处理运行时异常+普通异常。也就是说，如果你只用try去处理普通异常却不加以catch处理，编译是通不过的，因为编译器硬性规定，普通异常如果选择捕获，则必须用catch显示声明以便进一步处理。而运行时异常在编译时没有如此规定，所以catch可以省略，你加上catch编译器也觉

得无可厚非。

理论上，编译器看任何代码都不顺眼，都觉得可能有潜在的问题，所以你即使对所有代码加上try，代码在运行期时也只不过是在正常运行的基础上加一层皮。但是你一旦对一段代码加上try，就等于显示地承诺编译器，对这段代码可能抛出的异常进行捕获而非向上抛出处理。如果是普通异常，编译器要求必须用catch捕获以便进一步处理；如果运行时异常，捕获然后丢弃并且+finally扫尾处理，或者加上catch捕获以便进一步处理。

至于加上finally，则是在不管有没捕获异常，都要进行的“扫尾”处理。

78. try-catch-finally 中，如果 catch 中 return 了，finally 还会执行吗?

会执行，在 return 前执行。

在 finally 中改变返回值的做法是不好的，因为如果存在 finally 代码块，try中的 return 语句不会立马返回调用者，而是记录下返回值待 finally 代码块执行完毕之后再向调用者返回其值，然后如果在 finally 中修改了返回值，就会返回修改后的值。显然，在 finally 中返回或者修改返回值会对程序造成很大的困扰，Java 中也可以通过提升编译器的语法检查级别来产生警告或错误。

代码示例1：

```
public static int getInt() {
    int a = 10;
    try {
        System.out.println(a / 0);
        a = 20;
    } catch (ArithmeticException e) {
        a = 30;
        return a;
        /*
         * return a 在程序执行到这一步的时候，这里不是return a 而是 return 30;这个返回路径就形成了
         * 但是呢，它发现后面还有finally，所以继续执行finally的内容，a=40
         * 再次回到以前的路径,继续走return 30，形成返回路径之后，这里的a就不是a变量了，而是常量30
         */
    } finally {
        a = 40;
    }
```

```
        return a;
    }

    //执行结果：30
```

代码示例2：

```
    public static int getInt() {
        int a = 10;
        try {
            System.out.println(a / 0);
            a = 20;
        } catch (ArithmeticException e) {
            a = 30;
            return a;
        } finally {
            a = 40;
            //如果这样，就又重新形成了一条返回路径，由于只能通过1个return返回，所以这
    里直接返回40
            return a;
        }

    }

    // 执行结果：40
```

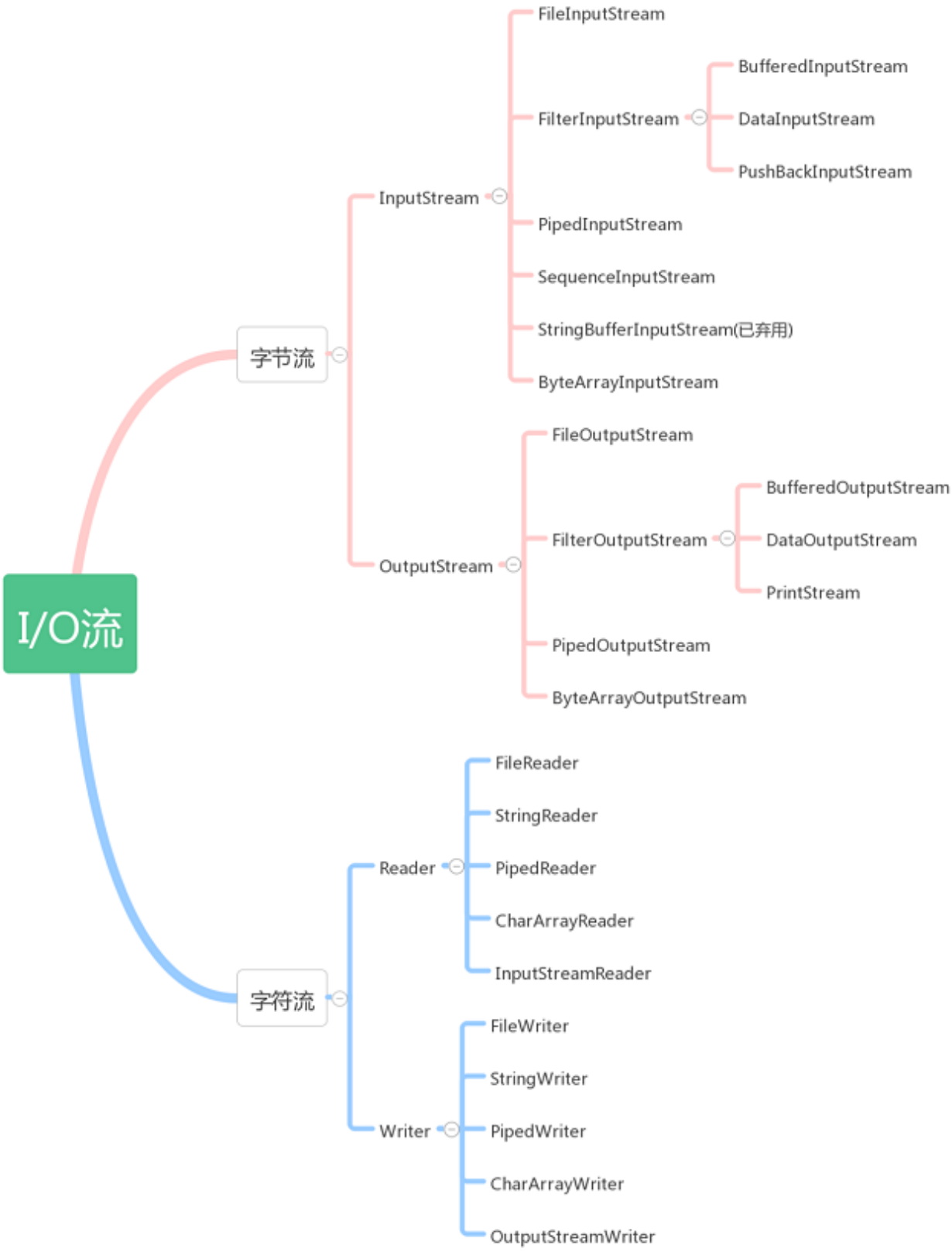
79. JVM 是如何处理异常的？

在一个方法中如果发生异常，这个方法会创建一个异常对象，并转交给 JVM，该异常对象包含异常名称，异常描述以及异常发生时应用程序的状态。创建异常对象并转交给 JVM 的过程称为抛出异常。可能有一系列的方法调用，最终才进入抛出异常的方法，这一系列方法调用的有序列表叫做调用栈。

JVM 会顺着调用栈去查找看是否有可以处理异常的代码，如果有，则调用异常处理代码。当 JVM 发现可以处理异常的代码时，会把发生的异常传递给它。如果 JVM 没有找到可以处理该异常的代码块，JVM 就会将该异常转交给默认的异常处理器（默认处理器为 JVM 的一部分），默认异常处理器打印出异常信息并终止应用程序。
想要深入了解的小伙伴可以看这篇文章：https://www.cnblogs.com/qdhxhz/p/10765839.html

8o. Java的IO 流分为几种?

- 按照流的方向：输入流（inputStream）和输出流（outputStream）；
- 按照实现功能分：节点流（可以从或向一个特定的地方读写数据，如 FileReader）和处理流（是对一个已存在的流的连接和封装，通过所封装的流的功能调用实现数据读写，BufferedReader）；
- 按照处理数据的单位： 字节流和字符流。分别由四个抽象类来表示（每种流包括输入和输出两种所以一共四个）:InputStream，OutputStream，Reader，Writer。Java中其他多种多样变化的流均是由它们派生出来的。



81. 字节流如何转为字符流?

字节输入流转字符输入流通过 InputStreamReader 实现，该类的构造函数可以传入 InputStream 对象。

字节输出流转字符输出流通过 OutputStreamWriter 实现，该类的构造函数可以传入 OutputStream 对象。

82. 字符流与字节流的区别?

- 读写的时候字节流是按字节读写，字符流按字符读写。
- 字节流适合所有类型文件的数据传输，因为计算机字节（Byte）是电脑中表示信息含义的最小单位。字符流只能够处理纯文本数据，其他类型数据不行，但是字符流处理文本要比字节流处理文本要方便。
- 在读写文件需要对内容按行处理，比如比较特定字符，处理某一行数据的时候一般会选择字符流。
- 只是读写文件，和文件内容无关时，一般选择字节流。

83. 什么是阻塞IO？什么是非阻塞IO?

IO操作包括：对硬盘的读写、对socket的读写以及外设的读写。

当用户线程发起一个IO请求操作（本文以读请求操作为例），内核会去查看要读取的数据是否就绪，对于阻塞IO来说，如果数据没有就绪，则会一直在那等待，直到数据就绪；对于非阻塞IO来说，如果数据没有就绪，则会返回一个标志信息告知用户线程当前要读的数据没有就绪。当数据就绪之后，便将数据拷贝到用户线程，这样才完成了一个完整的IO读请求操作，也就是说一个完整的IO读请求操作包括两个阶段：

1）查看数据是否就绪；

2）进行数据拷贝（内核将数据拷贝到用户线程）。

那么阻塞（blocking IO）和非阻塞（non-blocking IO）的区别就在于第一个阶段，如果数据没有就绪，在查看数据是否就绪的过程中是一直等待，还是直接返回一个标志信息。

Java中传统的IO都是阻塞IO，比如通过socket来读数据，调用read()方法之后，如果数据没有就绪，当前线程就会一直阻塞在read方法调用那里，直到有数据才返回；而如果是非阻塞IO的话，当数据没有就绪，read()方法应该返回一个标志信息，告知当前线程数据没有就绪，而不是一直在那里等待。

深入了解可看这篇文章：https://mp.weixin.qq.com/s/p5qM2UJ1uIWyongfVpRbCg

84. BIO、NIO、AIO的区别?

- BIO：同步并阻塞，在服务器中实现的模式为**一个连接一个线程**。也就是说，客户端有连接请求的时候，服务器就需要启动一个线程进行处理，如果这个连接不做任何事情会造成不必要的线程开销，当然这也可以通过线程池机制改善。BIO**一般适用于连接数目小且固定的架构**，这种方式对于服务器资源要求比较高，而且并发局限于应用中，是JDK1.4之前的唯一选择，但好在程序直观简单，易理解。
- NIO：同步并非阻塞，在服务器中实现的模式为**一个请求一个线程**，也就是说，客户端发送的连接请求都会注册到多路复用器上，多路复用器轮询到有连接IO请求时才会启动一个线程进行处理。**NIO一般适用于连接数目多且连接比较短（轻操作）的架构**，并发局限于应用中，编程比较复杂，从JDK1.4开始支持。
- AIO：异步并非阻塞，在服务器中实现的模式为**一个有效请求一个线程**，也就是说，客户端的IO请求都是通过操作系统先完成之后，再通知服务器应用去启动线程进行处理。AIO一般适用于连接数目多且连接比较长（重操作）的架构，充分调用操作系统参与并发操作，编程比较复杂，从JDK1.7开始支持。

85. Java IO都有哪些设计模式?

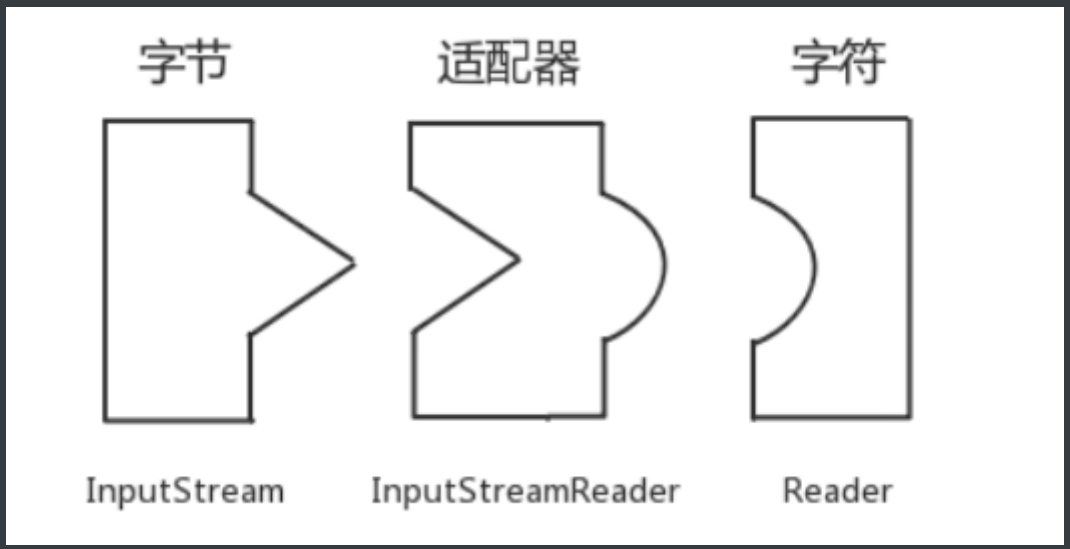
使用了**适配器模式**和**装饰器模式**

适配器模式:

```
Reader reader = new INputStreamReader(inputStream);
```

把一个类的接口变换成客户端所期待的另一种接口，从而使原本因接口不匹配而无法在一起工作的两个类能够在一起工作

- **类适配器**：Adapter类（适配器）继承Adaptee类（源角色）实现Target接口（目标角色）
- **对象适配器**：Adapter类（适配器）持有Adaptee类（源角色）对象实例，实现Target接口（目标角色）

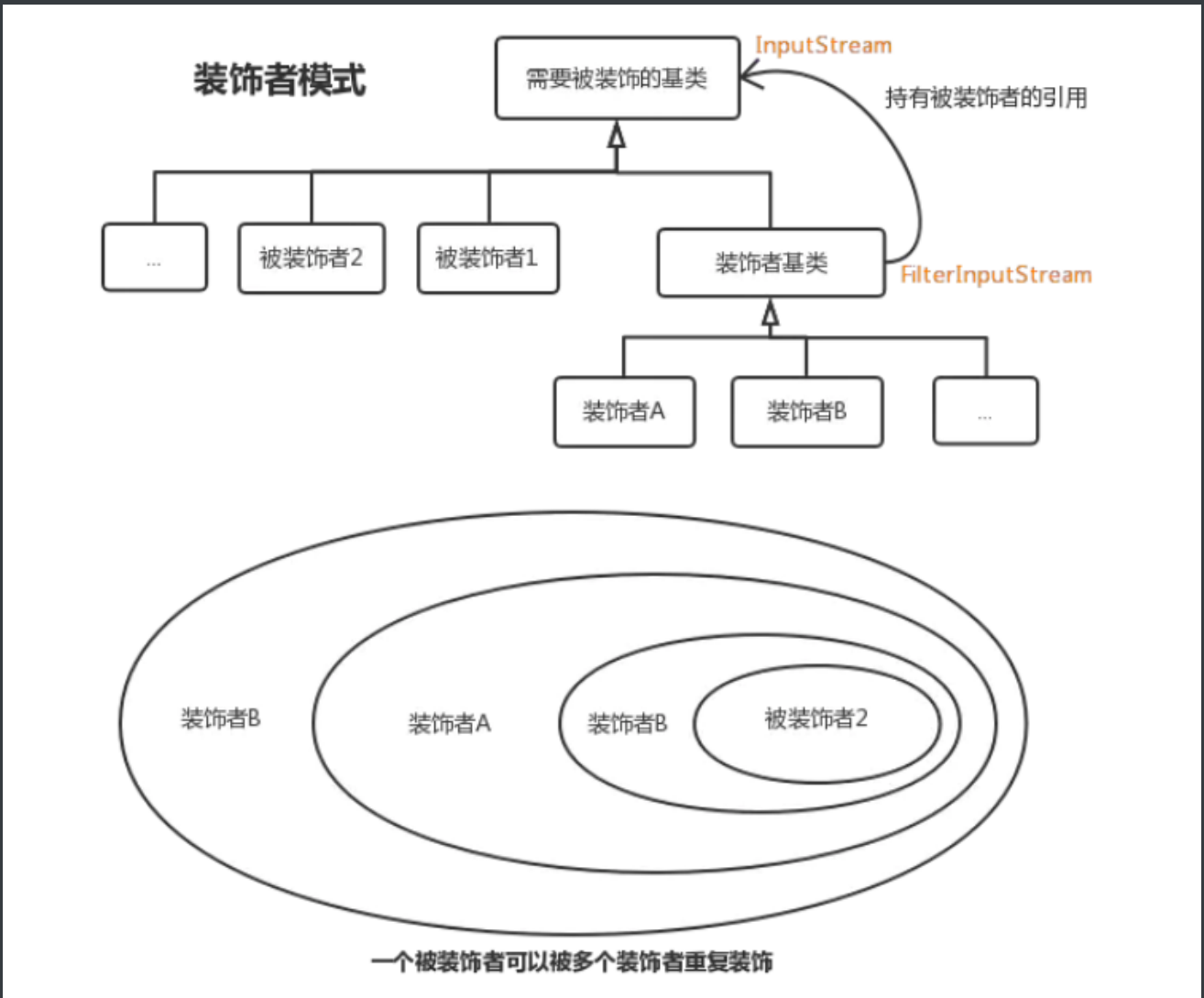


装饰器模式:

```
new BufferedInputStream(new FileInputStream(inputStream));
```

一种动态地往一个类中添加新的行为的设计模式。就功能而言，装饰器模式相比生成子类更为灵活，这样可以给某个对象而不是整个类添加一些功能。

- ConcreteComponent（具体对象）和Decorator（抽象装饰器）实现相同的Conponent（接口）并且Decorator（抽象装饰器）里面持有Conponent（接口）对象，可以传递请求。
- ConcreteComponent（具体装饰器）覆盖Decorator（抽象装饰器）的方法并用super进行调用，传递请求。



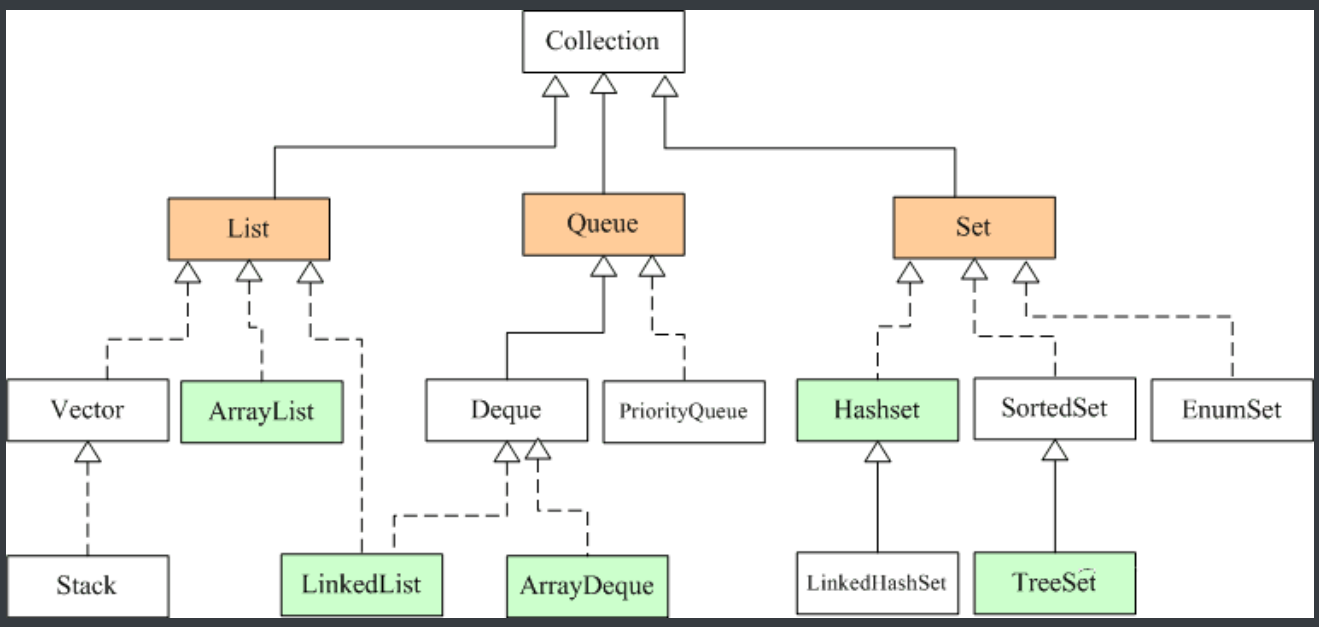
二、Java集合

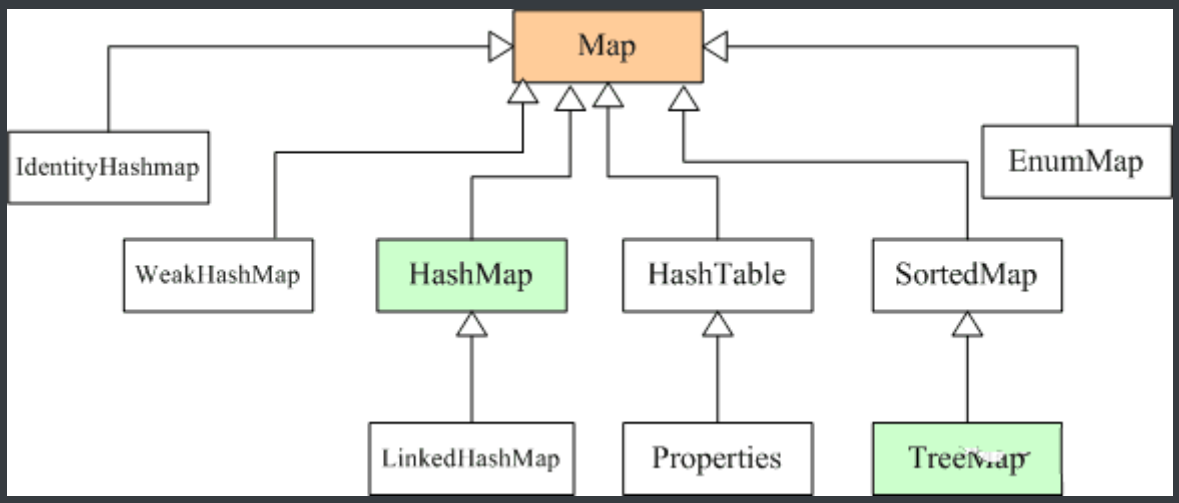
1. 常见的集合有哪些?

Java集合类主要由两个根接口**Collection**和**Map**派生出来的，Collection派生出了三个子接口：List、Set、Queue（Java5新增的队列），因此Java集合大致也可分成List、Set、Queue、Map四种接口体系。

注意：Collection是一个接口，Collections是一个工具类，Map不是Collection的子接口。

Java集合框架图如下：





图中，List代表了有序可重复集合，可直接根据元素的索引来访问；Set代表无序不可重复集合，只能根据元素本身来访问；Queue是队列集合。

Map代表的是存储key-value对的集合，可根据元素的key来访问value。

上图中淡绿色背景覆盖的是集合体系中常用的实现类，分别是ArrayList、LinkedList、ArrayQueue、HashSet、TreeSet、HashMap、TreeMap等实现类。

2. 线程安全的集合有哪些？线程不安全的呢?

线程安全的：

- Hashtable：比HashMap多了个线程安全。
- ConcurrentHashMap:是一种高效但是线程安全的集合。
- Vector：比Arraylist多了个同步化机制。
- Stack：栈，也是线程安全的，继承于Vector。

线性不安全的：

- HashMap
- Arraylist
- LinkedList
- HashSet
- TreeSet
- TreeMap

3. Arraylist与 LinkedList 异同点?

- **是否保证线程安全：** ArrayList 和 LinkedList 都是不同步的，也就是不保证线程安全；
- **底层数据结构：** Arraylist 底层使用的是Object数组；LinkedList 底层使用的是双向循环链表数据结构；
- **插入和删除是否受元素位置的影响： ArrayList 采用数组存储，所以插入和删除元素的时间复杂度受元素位置的影响。** 比如：执行`add(E e)`方法的时候， ArrayList 会默认在将指定的元素追加到此列表的末尾，这种情况时间复杂度就是O(1)。但是如果要在指定位置 i 插入和删除元素的话（`add(int index, E element)`）时间复杂度就为 O(n-i)。因为在进行上述操作的时候集合中第 i 和第 i 个元素之后的(n-i)个元素都要执行向后位/向前移一位的操作。 **LinkedList 采用链表存储，所以插入，删除元素时间复杂度不受元素位置的影响，都是近似 O（1）而数组为近似 O（n）。**
- **是否支持快速随机访问：** LinkedList 不支持高效的随机元素访问，而ArrayList 实现了 RandmoAccess 接口，所以有随机访问功能。快速随机访问就是通过元素的序号快速获取元素对象(对应于`get(int index)`方法)。
- **内存空间占用：** ArrayList的空 间浪费主要体现在在list列表的结尾会预留一定的容量空间，而LinkedList的空间花费则体现在它的每一个元素都需要消耗比ArrayList更多的空间（因为要存放直接后继和直接前驱以及数据）。

4. ArrayList 与 Vector 区别?

- Vector是线程安全的，ArrayList不是线程安全的。其中，Vector在关键性的方法前面都加了synchronized关键字，来保证线程的安全性。如果有多个线程会访问到集合，那最好是使用 Vector，因为不需要我们自己再去考虑和编写线程安全的代码。
- ArrayList在底层数组不够用时在原来的基础上扩展0.5倍，Vector是扩展1倍，这样ArrayList就有利于节约内存空间。

5. 说一说ArrayList 的扩容机制?

ArrayList扩容的本质就是计算出新的扩容数组的size后实例化，并将原有数组内容复制到新数组中去。**默认情况下，新的容量会是原容量的1.5倍。**

以JDK1.8为例说明:

```
public boolean add(E e) {
    //判断是否可以容纳e，若能，则直接添加在末尾；若不能，则进行扩容，然后再把e添加在末尾
    ensureCapacityInternal(size + 1);  // Increments modCount!!
    //将e添加到数组末尾
    elementData[size++] = e;
    return true;
    }

// 每次在add()一个元素时，arraylist都需要对这个list的容量进行一个判断。通过ensureCapacityInternal()方法确保当前ArrayList维护的数组具有存储新元素的能力，经过处理之后将元素存储在数组elementData的尾部

private void ensureCapacityInternal(int minCapacity) {
      ensureExplicitCapacity(calculateCapacity(elementData, minCapacity));
}

private static int calculateCapacity(Object[] elementData, int minCapacity)
{
        //如果传入的是个空数组则最小容量取默认容量与minCapacity之间的最大值
        if (elementData == DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
            return Math.max(DEFAULT_CAPACITY, minCapacity);
        }
        return minCapacity;
    }

  private void ensureExplicitCapacity(int minCapacity) {
```

```
            modCount++;
            // 若ArrayList已有的存储能力满足最低存储要求，则返回add直接添加元素；如果最低要求的存储能力>ArrayList已有的存储能力，这就表示ArrayList的存储能力不足，因此需要调用 grow();方法进行扩容
            if (minCapacity - elementData.length > 0)
                grow(minCapacity);
        }


    private void grow(int minCapacity) {
            // 获取elementData数组的内存空间长度
            int oldCapacity = elementData.length;
            // 扩容至原来的1.5倍
            int newCapacity = oldCapacity + (oldCapacity >> 1);
            //校验容量是否够
            if (newCapacity - minCapacity < 0)
                newCapacity = minCapacity;
            //若预设值大于默认的最大值，检查是否溢出
            if (newCapacity - MAX_ARRAY_SIZE > 0)
                newCapacity = hugeCapacity(minCapacity);
            // 调用Arrays.copyOf方法将elementData数组指向新的内存空间
             //并将elementData的数据复制到新的内存空间
            elementData = Arrays.copyOf(elementData, newCapacity);
        }
```

6. Array 和 ArrayList 有什么区别？什么时候该应 Array 而不是 ArrayList 呢?

- Array 可以包含基本类型和对象类型，ArrayList 只能包含对象类型。
- Array 大小是固定的，ArrayList 的大小是动态变化的。
- ArrayList 提供了更多的方法和特性，比如：addAll()，removeAll()，iterator() 等等。
-

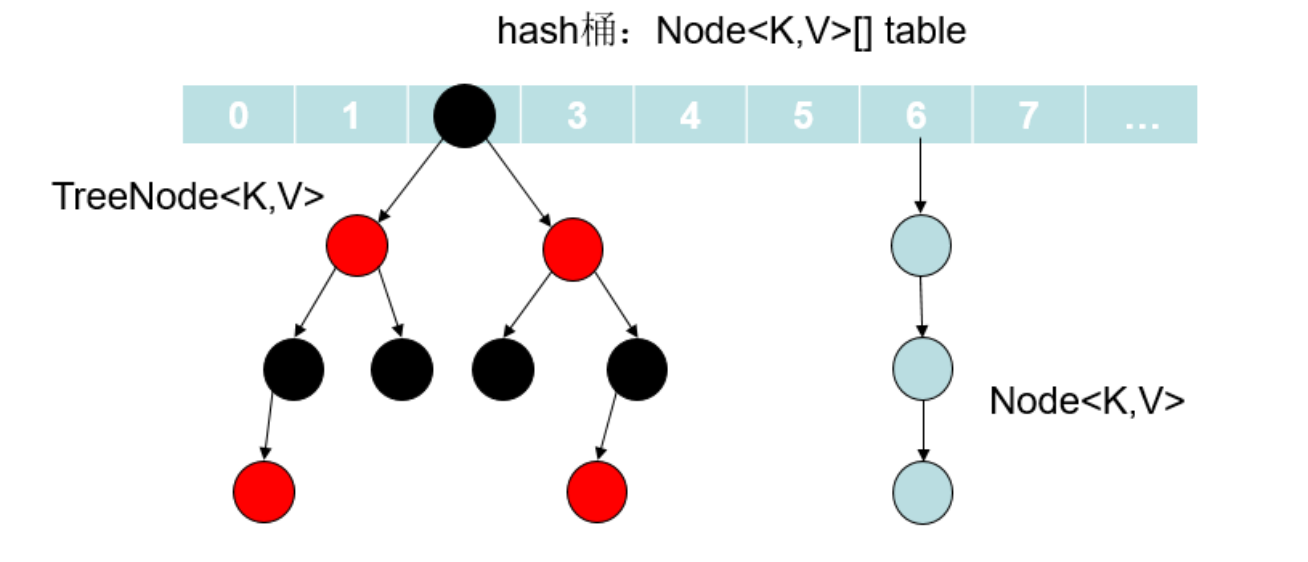
7. HashMap的底层数据结构是什么?

在JDK1.7 和JDK1.8 中有所差别：

在JDK1.7 中，由“数组+链表”组成，数组是 HashMap 的主体，链表则是主要为了解决哈希冲突而存在的。

在JDK1.8 中，由“数组+链表+红黑树”组成。当链表过长，则会严重影响 HashMap 的性能，红黑树搜索时间复杂度是 O(logn)，而链表是糟糕的 O(n)。因此，JDK1.8 对数据结构做了进一步的优化，引入了红黑树，链表和红黑树在达到一定条件会进行转换：

- 当链表超过 8 且数据总量超过 64 才会转红黑树。
- 将链表转换成红黑树前会判断，如果当前数组的长度小于 64，那么会选择先进行数组扩容，而不是转换为红黑树，以减少搜索时间。



8. 解决hash冲突的办法有哪些？HashMap用的哪种?

解决Hash冲突方法有:开放定址法、再哈希法、链地址法（拉链法）、建立公共溢出区。HashMap中采用的是 链地址法 。

- 开放定址法也称为 再散列法 ，基本思想就是，如果 p=H(key) 出现冲突时，则以 p 为基础，再次hash， p1=H(p) ,如果p1再次出现冲突，则以p1为基础，以此类推，直到找到一个不冲突的哈希地址 pi 。 因此开放定址法所需要的hash表的长度要大于等于所需要存放的元素，而且因为存在再次hash，所以 只能在删除的节点上做标记，而不能真正删除节点。
- 再哈希法(双重散列，多重散列)，提供多个不同的hash函数，当 R1=H1(key1) 发生冲突时，再计算 R2=H2(key1) ，直到没有冲突为止。 这样做虽然不易产生堆集，但增加了计算的时间。
- 链地址法(拉链法)，将哈希值相同的元素构成一个同义词的单链表,并将单链表的头指针存放在哈希表的第i个单元中，查找、插入和删除主要在同义词链表中进行。链表法适用于经常进行插入和删除的情况。
- 建立公共溢出区，将哈希表分为公共表和溢出表，当溢出发生时，将所有溢出数据统一放到溢出区。

9. 为什么在解决 hash 冲突的时候，不直接用红黑树？而选择先用链表，再转红黑树?

因为红黑树需要进行左旋，右旋，变色这些操作来保持平衡，而单链表不需要。当元素小于 8 个的时候，此时做查询操作，链表结构已经能保证查询性能。当元素大于 8 个的时候， 红黑树搜索时间复杂度是 O(logn)，而链表是 O(n)，此时需要红黑树来加快查询速度，但是新增节点的效率变慢了。

因此，如果一开始就用红黑树结构，元素太少，新增效率又比较慢，无疑这是浪费性能的。

10. HashMap默认加载因子是多少？为什么是0.75，不是0.6或者0.8？

回答这个问题前，我们来先看下HashMap的默认构造函数：

```
    int threshold;             // 容纳键值对的最大值
    final float loadFactor;    // 负载因子
    int modCount;
    int size;
```

Node[] table的初始化长度length(默认值是16)，Load factor为负载因子(默认值是0.75)，threshold是HashMap所能容纳键值对的最大值。threshold = length * Load factor。也就是说，在数组定义好长度之后，负载因子越大，所能容纳的键值对个数越多。

默认的loadFactor是0.75，0.75是对空间和时间效率的一个平衡选择，一般不要修改，除非在时间和空间比较特殊的情况下：

- 如果内存空间很多而又对时间效率要求很高，可以降低负载因子Load factor的值 。
- 相反，如果内存空间紧张而对时间效率要求不高，可以增加负载因子loadFactor的值，这个值可以大于1。

我们来追溯下作者在源码中的注释（JDK1.7）：

```
As a general rule, the default load factor (.75) offers a good tradeoff
between time and space costs. Higher values decrease the space overhead but
increase the lookup cost (reflected in most of the operations of the HashMap
class, including get and put). The expected number of entries in the map and
its load factor should be taken into account when setting its initial
capacity, so as to minimize the number of rehash operations. If the initial
capacity is greater than the maximum number of entries divided by the load
factor, no rehash operations will ever occur.
```

翻译过来大概的意思是：作为一般规则，默认负载因子（0.75）在时间和空间成本上提供了很好的折衷。较高的值会降低空间开销，但提高查找成本（体现在大多数的HashMap类的操作，包括get和put）。设置初始大小时，应该考虑预计的entry数在map及其负载系数，并且尽量减少rehash操作的次数。如果初始容量大于最大条目数除以负载因子，rehash操作将不会发生。

11. HashMap 中 key 的存储索引是怎么计算的?

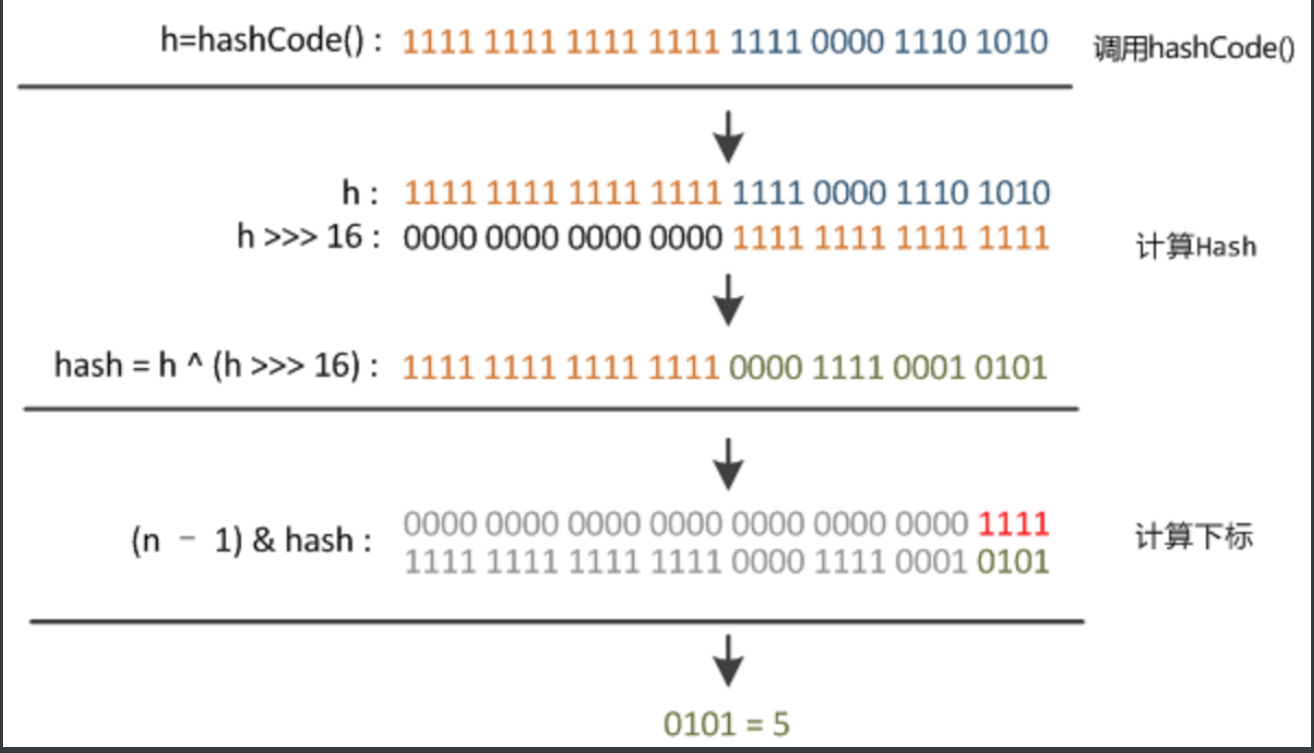
首先根据key的值计算出hashcode的值，然后根据hashcode计算出hash值，最后通过hash&（length-1）计算得到存储的位置。看看源码的实现：

```
// jdk1.7
方法一：
static int hash(int h) {
    int h = hashSeed;
        if (0 != h && k instanceof String) {
            return sun.misc.Hashing.stringHash32((String) k);
        }

    h ^= k.hashCode(); // 为第一步：取hashCode值
    h ^= (h >>> 20) ^ (h >>> 12);
    return h ^ (h >>> 7) ^ (h >>> 4);
}
方法二：
static int indexFor(int h, int length) {  //jdk1.7的源码，jdk1.8没有这个方法，但实现原理一样
     return h & (length-1);  //第三步：取模运算
}
```

```
// jdk1.8
static final int hash(Object key) {
     int h;
     return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
    /*
     h = key.hashCode() 为第一步：取hashCode值
     h ^ (h >>> 16)  为第二步：高位参与运算
    */
}
```

这里的 Hash 算法本质上就是三步：**取key的 hashCode 值、根据 hashcode 计算出hash 值、通过取模计算下标**。其中，JDK1.7和1.8的不同之处，就在于第二步。我们来看下详细过程，以JDK1.8为例，n为table的长度。

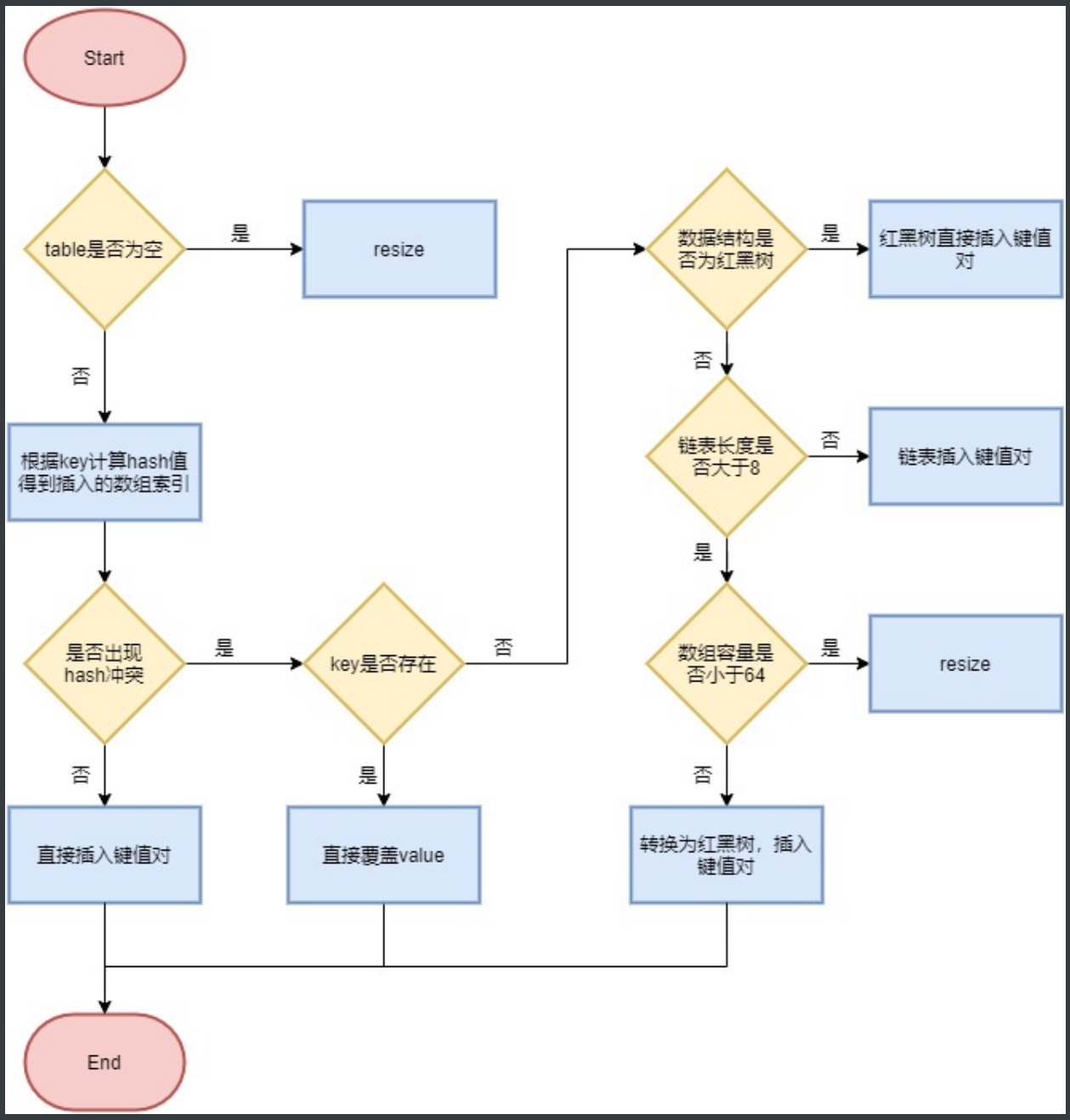


12. HashMap 的put方法流程?

以JDK1.8为例，简要流程如下：

1. 首先根据 key 的值计算 hash 值，找到该元素在数组中存储的下标；
2. 如果数组是空的，则调用 resize 进行初始化；
3. 如果没有哈希冲突直接放在对应的数组下标里；
4. 如果冲突了，且 key 已经存在，就覆盖掉 value；

5. 如果冲突后，发现该节点是红黑树，就将这个节点挂在树上；
6. 如果冲突后是链表，判断该链表是否大于 8 ，如果大于 8 并且数组容量小于 64，就进行扩容；如果链表节点大于 8 并且数组的容量大于 64，则将这个结构转换为红黑树；否则，链表插入键值对，若 key 存在，就覆盖掉 value。



13. HashMap 的扩容方式?

HashMap 在容量超过负载因子所定义的容量之后，就会扩容。Java 里的数组是无法自动扩容的，方法是将 HashMap 的大小扩大为原来数组的两倍，并将原来的对象放入新的数组中。

那扩容的具体步骤是什么？让我们看看源码。

先来看下JDK1.7 的代码

```
void resize(int newCapacity) {   //传入新的容量
        Entry[] oldTable = table;    //引用扩容前的Entry数组
        int oldCapacity = oldTable.length;
        if (oldCapacity == MAXIMUM_CAPACITY) {  //扩容前的数组大小如果已经达到最大(2^30)了
            threshold = Integer.MAX_VALUE; //修改阈值为int的最大值(2^31-1)，这样以后就不会扩容了
            return;
        }

        Entry[] newTable = new Entry[newCapacity];  //初始化一个新的Entry数组
        transfer(newTable);                         //！！将数据转移到新的Entry数组里
        table = newTable;                           //HashMap的table属性引用新的Entry数组
        threshold = (int)(newCapacity * loadFactor);//修改阈值
    }
```

这里就是使用一个容量更大的数组来代替已有的容量小的数组，transfer()方法将原有Entry数组的元素拷贝到新的Entry数组里。

```
void transfer(Entry[] newTable) {
        Entry[] src = table;                    //src引用了旧的Entry数组
        int newCapacity = newTable.length;
        for (int j = 0; j < src.length; j++) { //遍历旧的Entry数组
            Entry<K,V> e = src[j];              //取得旧Entry数组的每个元素
            if (e != null) {
                src[j] = null;//释放旧Entry数组的对象引用（for循环后，旧的Entry数组不再引用任何对象）
                do {
                    Entry<K,V> next = e.next;
                    int i = indexFor(e.hash, newCapacity); //！！重新计算每个元素在数组中的位置
                    e.next = newTable[i]; //标记[1]
                    newTable[i] = e;      //将元素放在数组上
                    e = next;             //访问下一个Entry链上的元素
                } while (e != null);
            }
        }
```

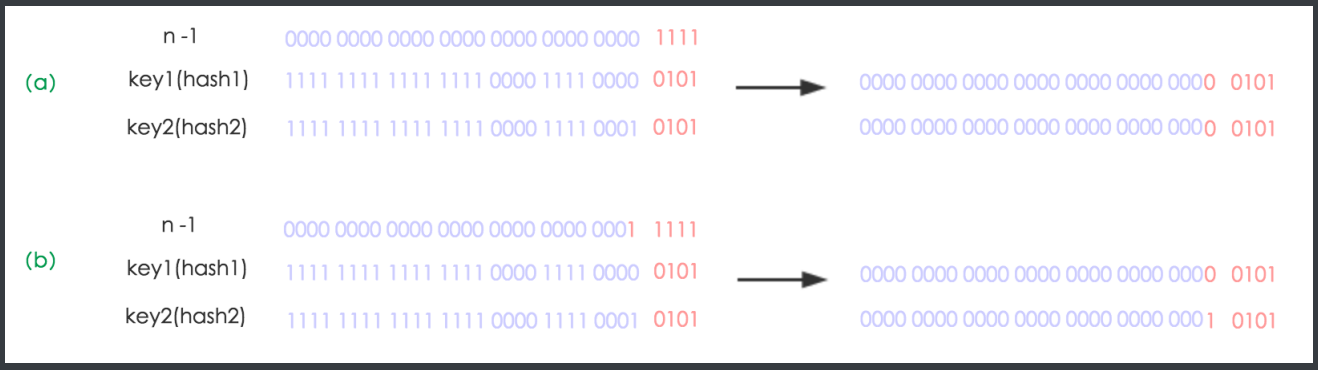
```
        }
```

newTable[i] 的引用赋给了 e.next ，也就是使用了单链表的头插入方式，同一位置上新元素总会被放在链表的头部位置；这样先放在一个索引上的元素终会被放到 Entry 链的尾部(如果发生了 hash 冲突的话）。

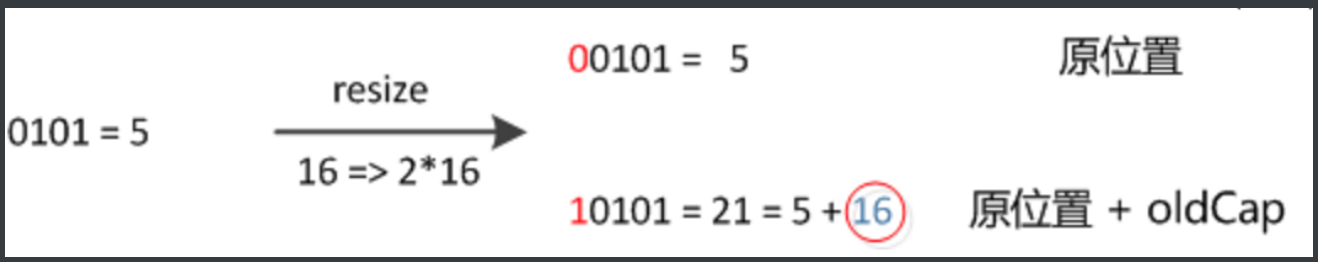
JDK1.8做了两处优化

1. resize 之后，元素的位置在原来的位置，或者原来的位置 +oldCap (原来哈希表的长度）。不需要像 JDK1.7 的实现那样重新计算hash ，只需要看看原来的 hash 值新增的那个bit是1还是0就好了，是0的话索引没变，是1的话索引变成“原索引 + oldCap ”。这个设计非常的巧妙，省去了重新计算 hash 值的时间。

   如下图所示，n 为 table 的长度，图（a）表示扩容前的 key1 和 key2 两种 key 确定索引位置的示例，图（b）表示扩容后 key1 和key2 两种 key 确定索引位置的示例，其中 hash1 是 key1 对应的哈希与高位运算结果。



元素在重新计算 hash 之后，因为 n 变为 2倍，那么 n-1 的 mask 范围在高位多 1 bit(红色)，因此新的index就会发生这样的变化：



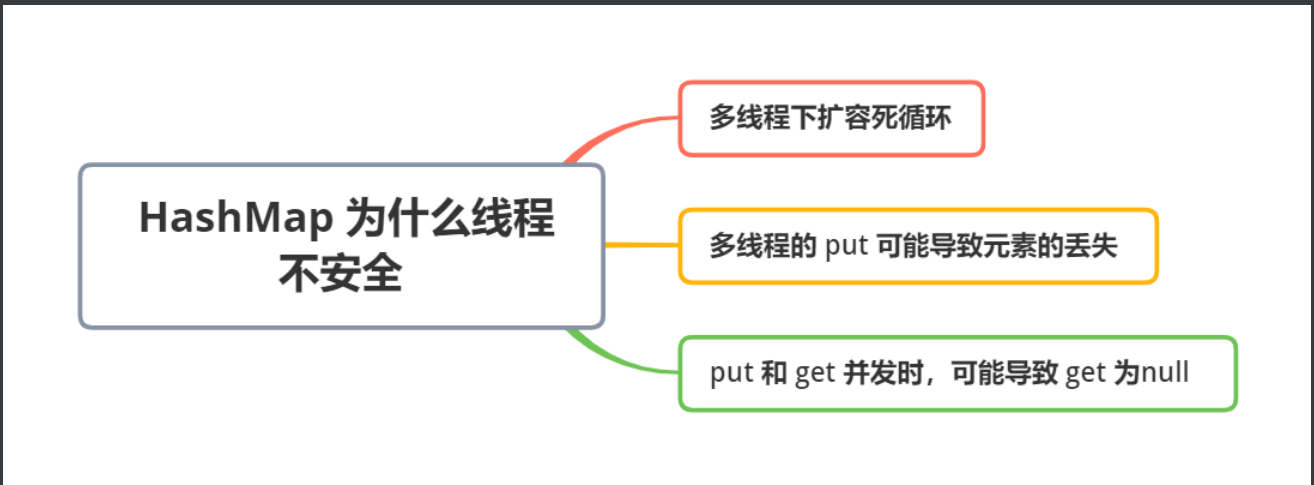
2. JDK1.7 中 rehash 的时候，旧链表迁移新链表的时候，如果在新表的数组索引位置相同，则链表元素会倒置（头插法）。JDK1.8 不会倒置，使用尾插法。

14. 一般用什么作为HashMap的key?

一般用Integer、String 这种不可变类当 HashMap 当 key，而且 String 最为常用。

- 因为字符串是不可变的，所以在它创建的时候 hashcode 就被缓存了，不需要重新计算。这就是 HashMap 中的键往往都使用字符串的原因。
- 因为获取对象的时候要用到 equals() 和 hashCode() 方法，那么键对象正确的重写这两个方法是非常重要的,这些类已经很规范的重写了 hashCode() 以及 equals() 方法。

15. HashMap为什么线程不安全?



- 多线程下扩容死循环。JDK1.7中的 HashMap 使用头插法插入元素，在多线程的环境下，扩容的时候有可能导致环形链表的出现，形成死循环。因此，JDK1.8使用尾插法插入元素，在扩容时会保持链表元素原本的顺序，不会出现环形链表的问题。
- 多线程的put可能导致元素的丢失。多线程同时执行 put 操作，如果计算出来的索引位置是相同的，那会造成前一个 key 被后一个 key 覆盖，从而导致元素的丢失。此问题在 JDK 1.7和 JDK 1.8 中都存在。
- put和get并发时，可能导致get为null。线程1执行put时，因为元素个数超出threshold而导致rehash，线程2此时执行get，有可能导致这个问题。此问题在JDK 1.7和 JDK 1.8 中都存在。

具体分析可见我的这篇文章：面试官：HashMap 为什么线程不安全？

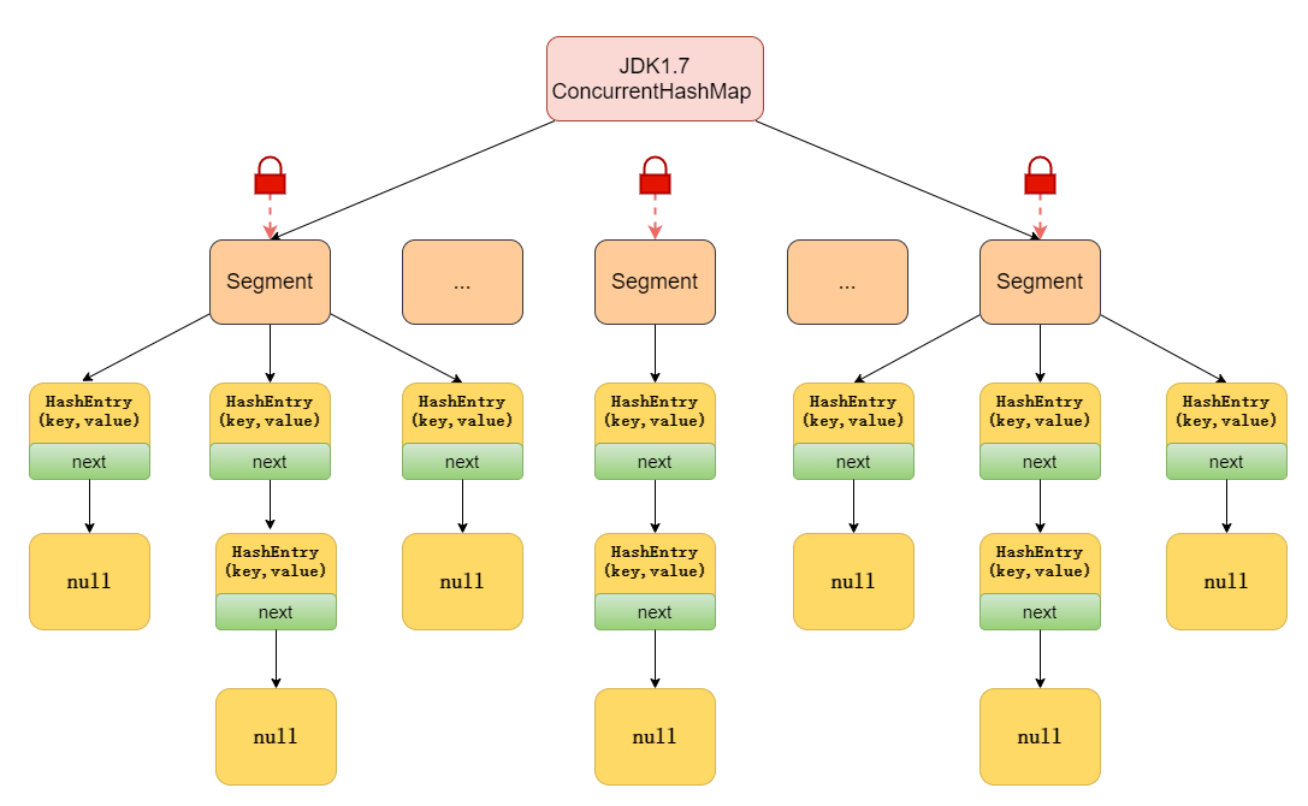
16. ConcurrentHashMap 的实现原理是什么?

ConcurrentHashMap 在 JDK1.7 和 JDK1.8 的实现方式是不同的。

先来看下JDK1.7

JDK1.7中的ConcurrentHashMap 是由 `Segment` 数组结构和 `HashEntry` 数组结构组成，即 ConcurrentHashMap 把哈希桶切分成小数组（Segment ），每个小数组有 n 个 HashEntry 组成。

其中，Segment 继承了 ReentrantLock，所以 Segment 是一种可重入锁，扮演锁的角色；HashEntry 用于存储键值对数据。

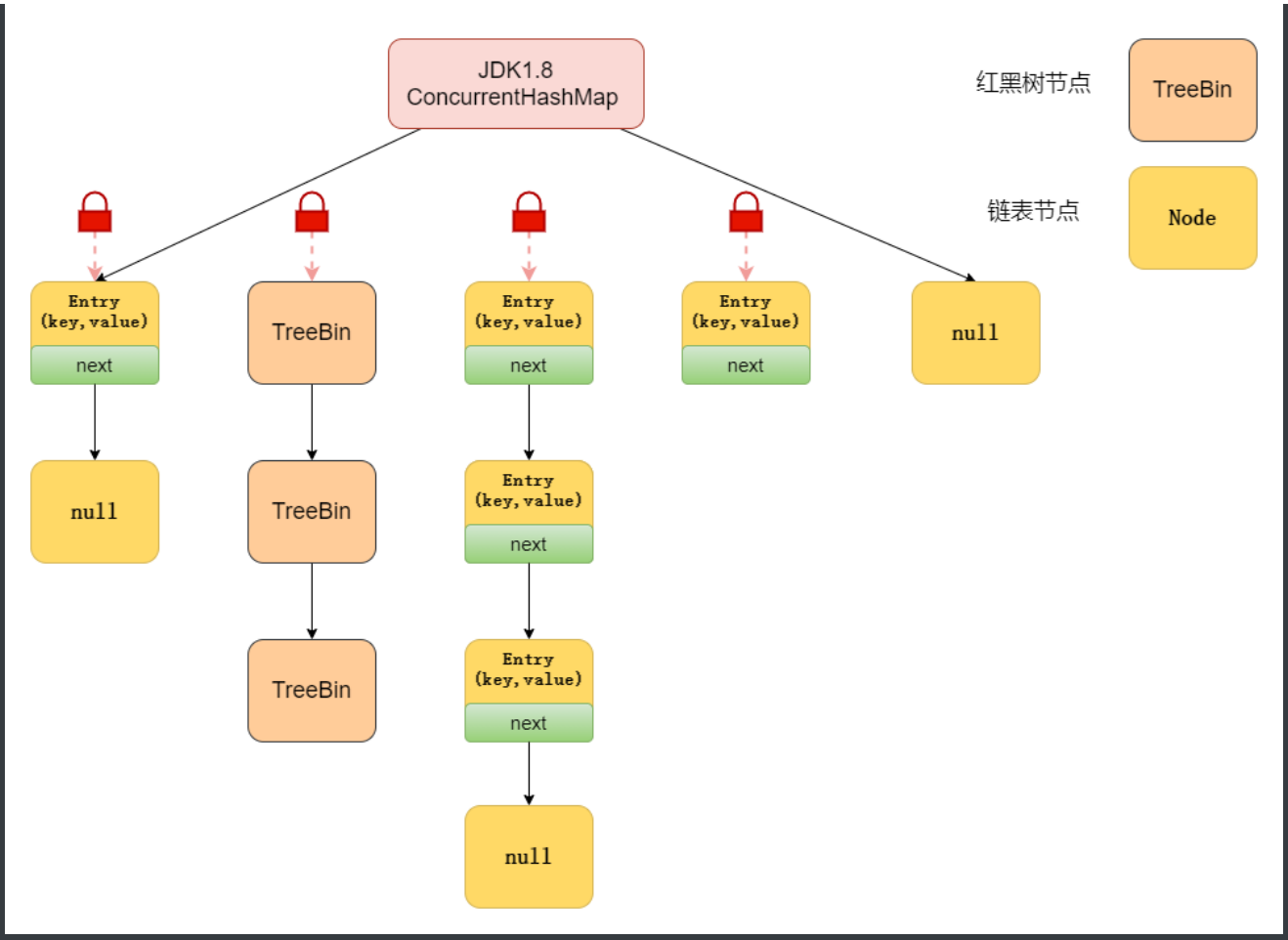


首先将数据分为一段一段的存储，然后给每一段数据配一把锁，当一个线程占用锁访问其中一个段数据时，其他段的数据也能被其他线程访问，能够实现真正的并发访问。

再来看下JDK1.8

在数据结构上， JDK1.8 中的ConcurrentHashMap 选择了与 HashMap 相同的**数组+链表+红黑树**结构；在锁的实现上，抛弃了原有的 Segment 分段锁，采用 `CAS + synchronized` 实现更加低粒度的锁。

将锁的级别控制在了更细粒度的哈希桶元素级别，也就是说只需要锁住这个链表头结点（红黑树的根节点），就不会影响其他的哈希桶元素的读写，大大提高了并发度。



17. ConcurrentHashMap 的 put 方法执行逻辑是什么?

先来看JDK1.7

首先，会尝试获取锁，如果获取失败，利用自旋获取锁；如果自旋重试的次数超过 64 次，则改为阻塞获取锁。

获取到锁后：

1. 将当前 Segment 中的 table 通过 key 的 hashcode 定位到 HashEntry。
2. 遍历该 HashEntry，如果不为空则判断传入的 key 和当前遍历的 key 是否相等，相等则覆盖旧的 value。
3. 不为空则需要新建一个 HashEntry 并加入到 Segment 中，同时会先判断是否需要扩容。
4. 释放 Segment 的锁。

再来看JDK1.8

大致可以分为以下步骤：

1. 根据 key 计算出 hash值。
2. 判断是否需要进行初始化。

3. 定位到 Node，拿到首节点 f，判断首节点 f：
    - 如果为 null ，则通过cas的方式尝试添加。
    - 如果为 `f.hash = MOVED = -1` ，说明其他线程在扩容，参与一起扩容。
    - 如果都不满足 ， synchronized 锁住 f 节点，判断是链表还是红黑树，遍历插入。
4. 当在链表长度达到8的时候，数组扩容或者将链表转换为红黑树。

源码分析可看这篇文章：面试 ConcurrentHashMap ，看这一篇就够了！

18. ConcurrentHashMap 的 get 方法是否要加锁，为什么?

get 方法不需要加锁。因为 Node 的元素 val 和指针 next 是用 volatile 修饰的，在多线程环境下线程A修改结点的val或者新增节点的时候是对线程B可见的。

这也是它比其他并发集合比如 Hashtable、用 Collections.synchronizedMap()包装的 HashMap 安全效率高的原因之一。

```
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;
    final K key;
    //可以看到这些都用了volatile修饰
    volatile V val;
    volatile Node<K,V> next;
}
```

19. get方法不需要加锁与volatile修饰的哈希桶有关吗?

没有关系。哈希桶 `table` 用volatile修饰主要是保证在数组扩容的时候保证可见性。

```
static final class Segment<K,V> extends ReentrantLock implements
Serializable {

    // 存放数据的桶
    transient volatile HashEntry<K,V>[] table;
```

20. ConcurrentHashMap 不支持 key 或者 value 为 null 的原因?

我们先来说value 为什么不能为 null ， 因为 `ConcurrentHashMap` 是用于多线程的 ， 如果 `map.get(key)` 得到了 null ， 无法判断，是映射的value是 null ， 还是没有找到对应的key而为 null ， 这就有了二义性。

而用于单线程状态的 `HashMap` 却可以用 `containsKey(key)` 去判断到底是否包含了这个 null 。

我们用**反证法**来推理：

假设ConcurrentHashMap 允许存放值为 null 的value，这时有A、B两个线程，线程A调用 ConcurrentHashMap .get(key)方法，返回为 null ， 我们不知道这个 null 是没有映射的 null ，还是存的值就是 null 。

假设此时，返回为 null 的真实情况是没有找到对应的key。那么，我们可以用 ConcurrentHashMap .containsKey(key)来验证我们的假设是否成立，我们期望的结果是返回 false。

但是在我们调用ConcurrentHashMap .get(key)方法之后，containsKey方法之前，线程B执行了ConcurrentHashMap .put(key, null)的操作。那么我们调用containsKey方法返回的就是 true了，这就与我们的假设的真实情况不符合了，这就有了二义性。

至于ConcurrentHashMap 中的key为什么也不能为 null 的问题，源码就是这样写的，哈哈。如果面试官不满意，就回答因为作者Doug不喜欢 null ， 所以在设计之初就不允许了 null 的key存在。想要深入了解的小伙伴，可以看这篇文章这道面试题我真不知道面试官想要的回答是什么

21. ConcurrentHashMap 的并发度是多少?

在JDK1.7中，并发度默认是16，这个值可以在构造函数中设置。如果自己设置了并发度，ConcurrentHashMap 会使用大于等于该值的最小的2的幂指数作为实际并发度，也就是比如你设置的值是17，那么实际并发度是32。

22. ConcurrentHashMap 迭代器是强一致性还是弱一致性?

与HashMap迭代器是强一致性不同，ConcurrentHashMap 迭代器是弱一致性。

ConcurrentHashMap 的迭代器创建后，就会按照哈希表结构遍历每个元素，但在遍历过程中，内部元素可能会发生变化，如果变化发生在已遍历过的部分，迭代器就不会反映出来，而如果变化发生在未遍历过的部分，迭代器就会发现并反映出来，这就是弱一致性。

这样迭代器线程可以使用原来老的数据，而写线程也可以并发的完成改变，更重要的，这保证了多个线程并发执行的连续性和扩展性，是性能提升的关键。想要深入了解的小伙伴，可以看这篇文章为什么ConcurrentHashMap 是弱一致的

23. JDK1.7与JDK1.8 中ConcurrentHashMap 的区别?

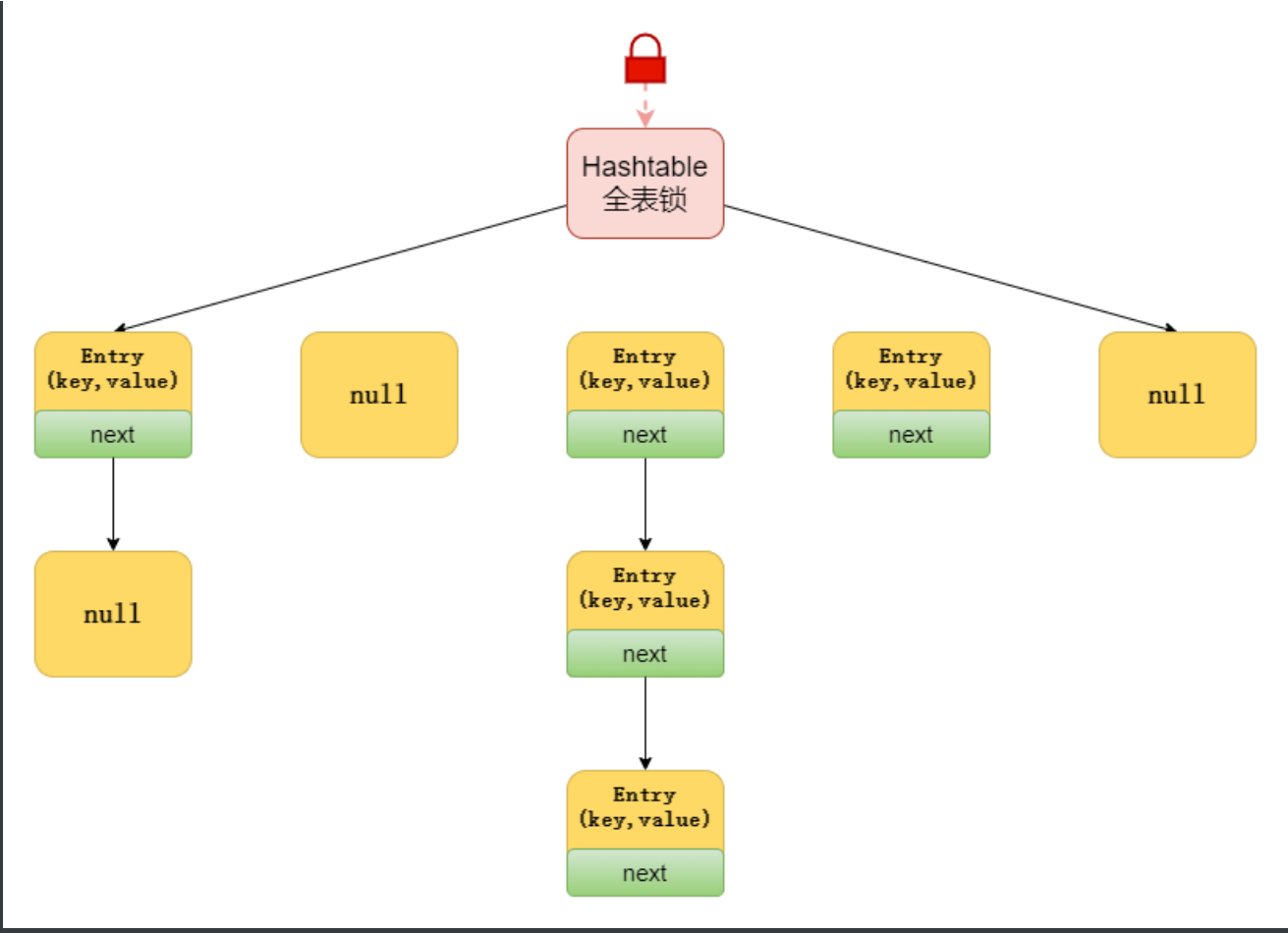
- 数据结构：取消了Segment分段锁的数据结构，取而代之的是数组+链表+红黑树的结构。
- 保证线程安全机制：JDK1.7采用Segment的分段锁机制实现线程安全，其中segment继承自ReentrantLock。JDK1.8 采用CAS+Synchronized保证线程安全。
- 锁的粒度：原来是对需要进行数据操作的Segment加锁，现调整为对每个数组元素加锁（Node）。
- 链表转化为红黑树:定位结点的hash算法简化会带来弊端,Hash冲突加剧,因此在链表节点数量大于8时，会将链表转化为红黑树进行存储。
- 查询时间复杂度：从原来的遍历链表O(n)，变成遍历红黑树O(logN)。

24. ConcurrentHashMap 和Hashtable的效率哪个更高？为什么?

ConcurrentHashMap 的效率要高于Hashtable，因为Hashtable给整个哈希表加了一把大锁从而实现线程安全。而ConcurrentHashMap 的锁粒度更低，在JDK1.7中采用分段锁实现线程安全，在JDK1.8 中采用 `CAS+Synchronized` 实现线程安全。

25. 说一下Hashtable的锁机制？

Hashtable是使用Synchronized来实现线程安全的，给整个哈希表加了一把大锁，多线程访问时候，只要有一个线程访问或操作该对象，那其他线程只能阻塞等待需要的锁被释放，在竞争激烈的多线程场景中性能就会非常差!



26. 多线程下安全的操作 map还有其他方法吗?

还可以使用 `Collections.synchronizedMap` 方法，对方法进行加同步锁

```
private static class SynchronizedMap<K,V>
        implements Map<K,V>, Serializable {
        private static final long serialVersionUID = 1978198479659022715L;

        private final Map<K,V> m;     // Backing Map
        final Object      mutex;        // Object on which to synchronize

        SynchronizedMap(Map<K,V> m) {
            this.m = Objects.requireNon null (m);
            mutex = this;
        }

        SynchronizedMap(Map<K,V> m, Object mutex) {
            this.m = m;
            this.mutex = mutex;
        }
```

```
        // 省略部分代码
        }
```

如果传入的是 HashMap 对象，其实也是对 HashMap 做的方法做了一层包装，里面使用对象锁来保证多线程场景下，线程安全，本质也是对 HashMap 进行全表锁。**在竞争激烈的多线程环境下性能依然也非常差，不推荐使用！**

27. HashSet 和 HashMap 区别?

HashMap	HashSet
实现了Map接口	实现Set接口
存储键值对	仅存储对象
调用put（）向map中添加元素	调用add（）方法向Set中添加元素
HashMap使用键（Key）计算Hashcode	HashSet使用成员对象来计算hashcode值，对于两个对象来说hashcode可能相同，所以equals()方法用来判断对象的相等性，如果两个对象不同的话，那么返回false
HashMap相对于HashSet较快，因为它是使用唯一的键获取对象	HashSet较HashMap来说比较慢

补充HashSet的实现：HashSet的底层其实就是HashMap，只不过我们**HashSet是实现了Set接口并且把数据作为K值，而V值一直使用一个相同的虚值来保存**。如源码所示：

```
public boolean add(E e) {
    return map.put(e, PRESENT)==null;// 调用HashMap的put方法,PRESENT是一个至始至终都相同的虚值
}
```

由于HashMap的K值本身就不允许重复，并且在HashMap中如果K/V相同时，会用新的V覆盖掉旧的V，然后返回旧的V，那么在HashSet中执行这一句话始终会返回一个false，导致插入失败，这样就保证了数据的不可重复性。

28. Collection框架中实现比较要怎么做?

第一种，实体类实现Comparable接口，并实现 compareTo(T t) 方法，称为内部比较器。

第二种，创建一个外部比较器，这个外部比较器要实现Comparator接口的 compare(T t1, T t2)方法。

29. Iterator和ListIterator有什么区别?

- 遍历。使用Iterator，可以遍历所有集合，如Map，List，Set；但只能在向前方向上遍历集合中的元素。

使用ListIterator，只能遍历List实现的对象，但可以向前和向后遍历集合中的元素。

- 添加元素。Iterator无法向集合中添加元素；而，ListIteror可以向集合添加元素。
- 修改元素。Iterator无法修改集合中的元素；而，ListIterator可以使用set()修改集合中的元素。
- 索引。Iterator无法获取集合中元素的索引；而，使用ListIterator，可以获取集合中元素的索引。

30. 讲一讲快速失败(fail-fast)和安全失败(fail-safe)

快速失败（fail—fast）

- 在用迭代器遍历一个集合对象时，如果遍历过程中对集合对象的内容进行了修改（增加、删除、修改），则会抛出Concurrent Modification Exception。
- 原理：迭代器在遍历时直接访问集合中的内容，并且在遍历过程中使用一个modCount 变量。集合在被遍历期间如果内容发生变化，就会改变modCount的值。每当迭代器使用hashNext()/next()遍历下一个元素之前，都会检测modCount变量是否为expectedmodCount值，是的话就返回遍历；否则抛出异常，终止遍历。
- 注意：这里异常的抛出条件是检测到 modCount！=expectedmodCount 这个条件。如果集合发生变化时修改modCount值刚好又设置为了expectedmodCount值，则异常不会抛出。因此，不能依赖于这个异常是否抛出而进行并发操作的编程，这个异常只建议用于检测并发修改的bug。
- 场景：java.util包下的集合类都是快速失败的，不能在多线程下发生并发修改（迭代过程中被修改），比如HashMap、ArrayList 这些集合类。

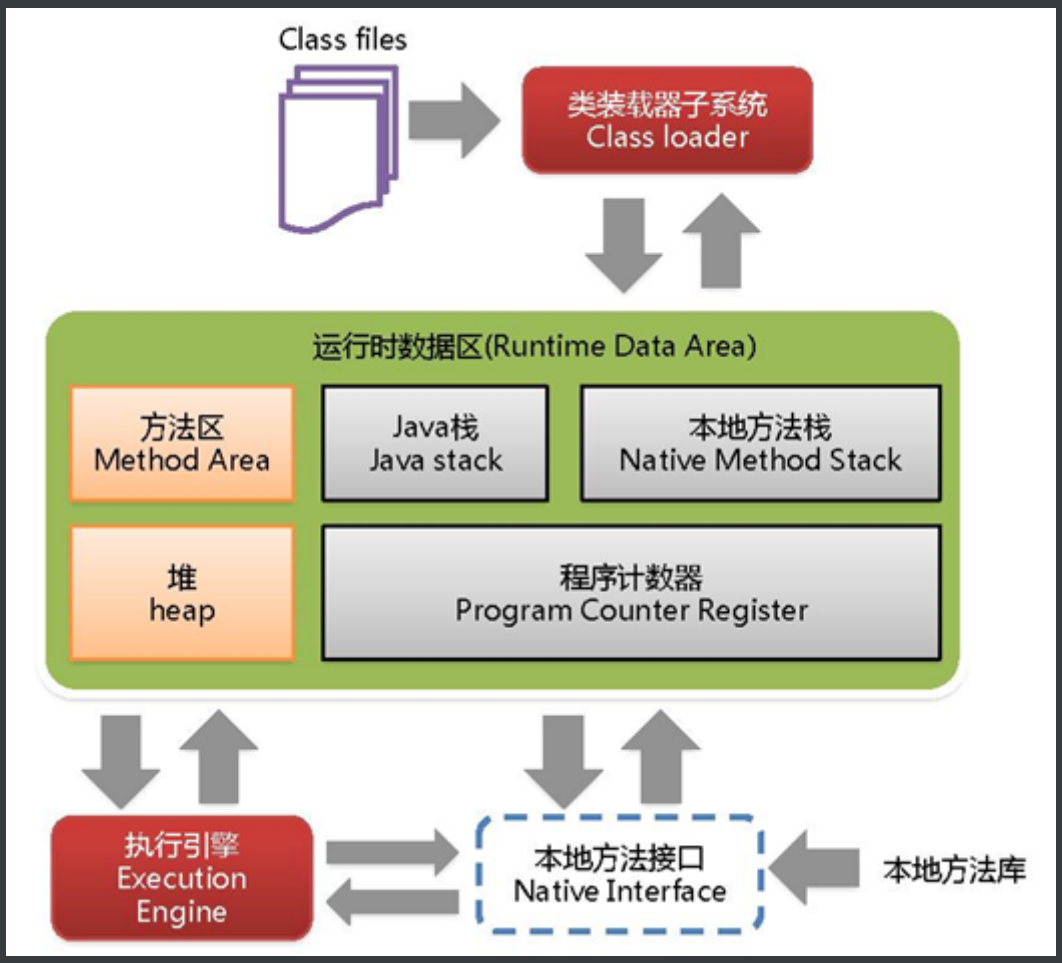
安全失败（fail—safe）

- 采用安全失败机制的集合容器，在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，在拷贝的集合上进行遍历。
- 原理：由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到，所以不会触发Concurrent Modification Exception。
- 缺点：基于拷贝内容的优点是避免了Concurrent Modification Exception，但同样地，迭代器并不能访问到修改后的内容，即：迭代器遍历的是开始遍历那一刻拿到的集合拷贝，在遍历期间原集合发生的修改迭代器是不知道的。

- 场景：java.util.concurrent包下的容器都是安全失败，可以在多线程下并发使用，并发修改，比如：ConcurrentHashMap。

三、 JVM

1. 什么是JVM内存结构?



jvm将虚拟机分为5大区域，程序计数器、虚拟机栈、本地方法栈、java堆、方法区；

- 程序计数器：线程私有的，是一块很小的内存空间，作为当前线程的行号指示器，用于记录当前虚拟机正在执行的线程指令地址；
- 虚拟机栈：线程私有的，每个方法执行的时候都会创建一个栈帧，用于存储局部变量表、操作数、动态链接和方法返回等信息，当线程请求的栈深度超过了虚拟机允许的最大深度时，就会抛出StackOverFlowError；
- 本地方法栈：线程私有的，保存的是native方法的信息，当一个jvm创建的线程调用native方法后，jvm不会在虚拟机栈中为该线程创建栈帧，而是简单的动态链接并直接调用该方法；

- 堆：java堆是所有线程共享的一块内存，几乎所有对象的实例和数组都要在堆上分配内存，因此该区域经常发生垃圾回收的操作；
- 方法区：存放已被加载的类信息、常量、静态变量、即时编译器编译后的代码数据。即永久代，在jdk1.8中不存在方法区了，被元数据区替代了，原方法区被分成两部分；1：加载的类信息，2：运行时常量池；加载的类信息被保存在元数据区中，运行时常量池保存在堆中；

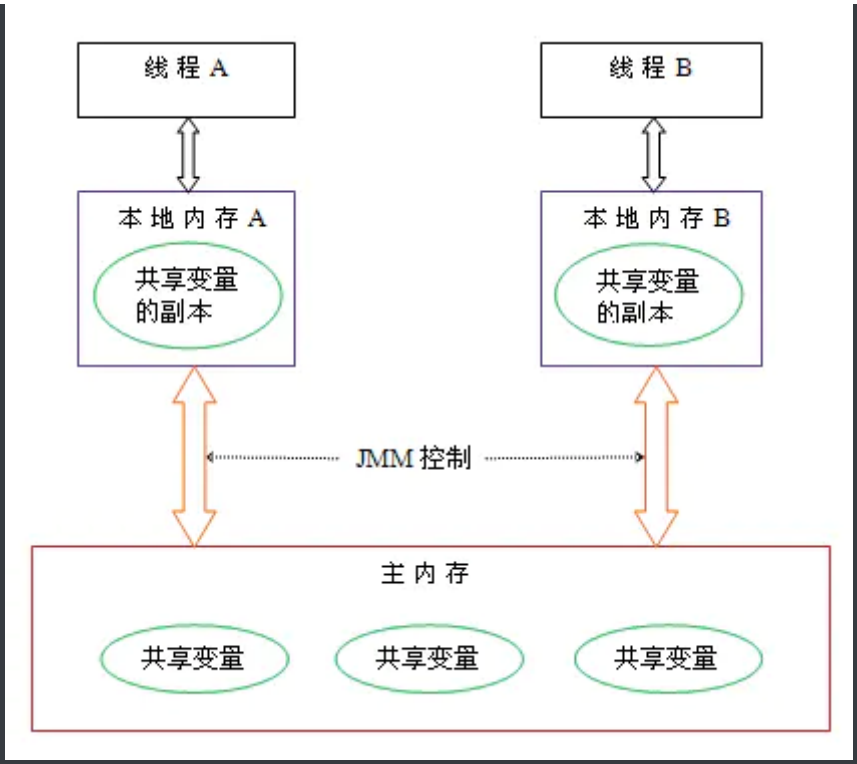
2. 什么是JVM内存模型?

Java 内存模型（下文简称 **JMM**）就是在底层处理器内存模型的基础上，定义自己的多线程语义。它明确指定了一组排序规则，来保证线程间的可见性。

这一组规则被称为 **Happens-Before**, JMM 规定，要想保证 B 操作能够看到 A 操作的结果（无论它们是否在同一个线程），那么 A 和 B 之间必须满足 **Happens-Before 关系**:

- **单线程规则**：一个线程中的每个动作都 happens-before 该线程中后续的每个动作
- **监视器锁定规则**：监听器的**解锁**动作 happens-before 后续对这个监听器的**锁定**动作
- **volatile 变量规则**：对 volatile 字段的写入动作 happens-before 后续对这个字段的每个读取动作
- **线程 start 规则**：线程 **start()** 方法的执行 happens-before 一个启动线程内的任意动作
- **线程 join 规则**：一个线程内的所有动作 happens-before 任意其他线程在该线程 **join()** 成功返回之前
- **传递性**：如果 A happens-before B, 且 B happens-before C, 那么 A happens-before C

怎么理解 happens-before 呢？如果按字面意思，比如第二个规则，线程（不管是不是同一个）的解锁动作发生在锁定之前？这明显不对。happens-before 也是为了保证可见性，比如那个解锁和加锁的动作，可以这样理解，线程1释放锁退出同步块，线程2加锁进入同步块，那么线程2就能看见线程1对共享对象修改的结果。



Java 提供了几种语言结构，包括 *volatile*, *final* 和 *synchronized*, 它们旨在帮助程序员向**编译器**描述程序的并发要求，其中：

- **volatile** - 保证**可见性**和**有序性**
- **synchronized** - 保证**可见性**和**有序性**; 通过**管程（Monitor）*保证一组动作的*原子性**
- **final** - 通过禁止**在构造函数初始化**和**给 final 字段赋值**这两个动作的重排序，保证**可见性**（如果 **this 引用逃逸**就不好说可见性了）

编译器在遇到这些关键字时，会插入相应的内存屏障，保证语义的正确性。

有一点需要**注意**的是，**synchronized 不保证**同步块内的代码禁止重排序，因为它通过锁保证同一时刻只有**一个线程**访问同步块（或临界区），也就是说同步块的代码只需满足 **as-if-serial** 语义 - 只要单线程的执行结果不改变，可以进行重排序。

所以说，Java 内存模型描述的是多线程对共享内存修改后彼此之间的可见性，另外，还确保正确同步的 Java 代码可以在不同体系结构的处理器上正确运行。

3. heap 和stack 有什么区别?

（1）申请方式

stack:由系统自动分配。例如，声明在函数中一个局部变量 int b; 系统自动在栈中为 b 开辟空间

heap:需要程序员自己申请，并指明大小，在 c 中 malloc 函数，对于Java 需要手动 new Object()的形式开辟

（2）申请后系统的响应

stack：只要栈的剩余空间大于所申请空间，系统将为程序提供内存，否则将报异常提示栈溢出。

heap：首先应该知道操作系统有一个记录空闲内存地址的链表，当系统收到程序的申请时，会遍历该链表，寻找第一个空间大于所申请空间的堆结点，然后将该结点从空闲结点链表中删除，并将该结点的空间分配给程序。另外，由于找到的堆结点的大小不一定正好等于申请的大小，系统会自动的将多余的那部分重新放入空闲链表中。

（3）申请大小的限制

stack：栈是向低地址扩展的数据结构，是一块连续的内存的区域。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的，在 WINDOWS 下，栈的大小是 2M（默认值也取决于虚拟内存的大小），如果申请的空间超过栈的剩余空间时，将提示 overflow。因此，能从栈获得的空间较小。

heap：堆是向高地址扩展的数据结构，是不连续的内存区域。这是由于系统是用链表来存储的空闲内存地址的， 自然是不连续的，而链表的遍历方向是由低地址向高地址。堆的大小受限于计算机系统中有效的虚拟内存。由此可见， 堆获得的空间比较灵活，也比较大。

（4）申请效率的比较

stack：由系统自动分配，速度较快。但程序员是无法控制的。

heap：由 new 分配的内存，一般速度比较慢，而且容易产生内存碎片,不过用起来最方便。

（5）heap和stack中的存储内容

stack：在函数调用时，第一个进栈的是主函数中后的下一条指令（函数调用语句的下一条可执行语句）的地址， 然后是函数的各个参数，在大多数的 C 编译器中，参数是由右往左入栈的，然后是函数中的局部变量。注意静态变量是不入栈的。

当本次函数调用结束后，局部变量先出栈，然后是参数，最后栈顶指针指向最开始存的地址，也就是主函数中的下一条指令，程序由该点继续运行。

heap：一般是在堆的头部用一个字节存放堆的大小。堆中的具体内容有程序员安排。

4. 什么情况下会发生栈内存溢出?

1、栈是线程私有的，栈的生命周期和线程一样，每个方法在执行的时候就会创建一个栈帧，它包含局部变量表、操作数栈、动态链接、方法出口等信息，局部变量表又包括基本数据类型和对象的引用；
2、当线程请求的栈深度超过了虚拟机允许的最大深度时，会抛出StackOverFlowError异常，方法递归调用肯可能会出现该问题；
3、调整参数-xss去调整jvm栈的大小

5. 谈谈对 OOM 的认识？如何排查 OOM 的问题?

除了程序计数器，其他内存区域都有 OOM 的风险。

- 栈一般经常会发生 StackOverflowError，比如 32 位的 windows 系统单进程限制 2G 内存，无限创建线程就会发生栈的 OOM
- Java 8 常量池移到堆中，溢出会出 java.lang.OutOfMemoryError: Java heap space，设置最大元空间大小参数无效；
- 堆内存溢出，报错同上，这种比较好理解，GC 之后无法在堆中申请内存创建对象就会报错；
- 方法区 OOM，经常会遇到的是动态生成大量的类、jsp 等；
- 直接内存 OOM，涉及到 -XX:MaxDirectMemorySize 参数和 Unsafe 对象对内存的申请。

排查 OOM 的方法：

- 增加两个参数 -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heapdump.hprof，当 OOM 发生时自动 dump 堆内存信息到指定目录；
- 同时 jstat 查看监控 JVM 的内存和 GC 情况，先观察问题大概出在什么区域；
- 使用 MAT 工具载入到 dump 文件，分析大对象的占用情况，比如 HashMap 做缓存未清理，时间长了就会内存溢出，可以把改为弱引用 。

6. 谈谈 JVM 中的常量池?

JVM常量池主要分为**Class文件常量池、运行时常量池，全局字符串常量池，以及基本类型包装类对象常量池**。

- **Class文件常量池**。class文件是一组以字节为单位的二进制数据流，在java代码的编译期间，我们编写的java文件就被编译为.class文件格式的二进制数据存放在磁盘中，其中就包括class文件常量池。

- **运行时常量池**：运行时常量池相对于class常量池一大特征就是具有动态性，java规范并不要求常量只能在运行时才产生，也就是说运行时常量池的内容并不全部来自class常量池，在运行时可以通过代码生成常量并将其放入运行时常量池中，这种特性被用的最多的就是String.intern()。
- **全局字符串常量池**：字符串常量池是JVM所维护的一个字符串实例的引用表，在HotSpot VM中，它是一个叫做StringTable的全局表。在字符串常量池中维护的是字符串实例的引用，底层C++实现就是一个Hashtable。这些被维护的引用所指的字符串实例，被称作”被驻留的字符串”或”interned string”或通常所说的”进入了字符串常量池的字符串”。
- 基本类型包装类对象常量池：java中基本类型的包装类的大部分都实现了常量池技术，这些类是Byte,Short,Integer,Long,Character,Boolean,另外两种浮点数类型的包装类则没有实现。另外上面这5种整型的包装类也只是在对应值小于等于127时才可使用对象池，也即对象不负责创建和管理大于127的这些类的对象。

7. 如何判断一个对象是否存活?

判断一个对象是否存活，分为两种算法1：引用计数法；2：可达性分析算法；

引用计数法:
给每一个对象设置一个引用计数器，当有一个地方引用该对象的时候，引用计数器就+1，引用失效时，引用计数器就-1；当引用计数器为0的时候，就说明这个对象没有被引用，也就是垃圾对象，等待回收；
缺点：无法解决循环引用的问题，当A引用B，B也引用A的时候，此时AB对象的引用都不为0，此时也就无法垃圾回收，所以一般主流虚拟机都不采用这个方法；

可达性分析法
从一个被称为GC Roots的对象向下搜索，如果一个对象到GC Roots没有任何引用链相连接时，说明此对象不可用，在java中可以作为GC Roots的对象有以下几种：

- 虚拟机栈中引用的对象
- 方法区类静态属性引用的变量
- 方法区常量池引用的对象
- 本地方法栈JNI引用的对象

但一个对象满足上述条件的时候，不会马上被回收，还需要进行两次标记；第一次标记：判断当前对象是否有finalize()方法并且该方法没有被执行过，若不存在则标记为垃圾对象，等待回收；若有的话，则进行第二次标记；第二次标记将当前对象放入F-Queue队列，并生成一个finalize线程去执行该方法，虚拟机不保证该方法一定会被执行，这是因为如果线程执行缓慢或进入了死锁，会导致回收系统的崩溃；如果执行了finalize方法之后仍然没有与GC Roots有直接或者间接的引用，则该对象会被回收；

8. 强引用、软引用、弱引用、虚引用是什么，有什么区别?

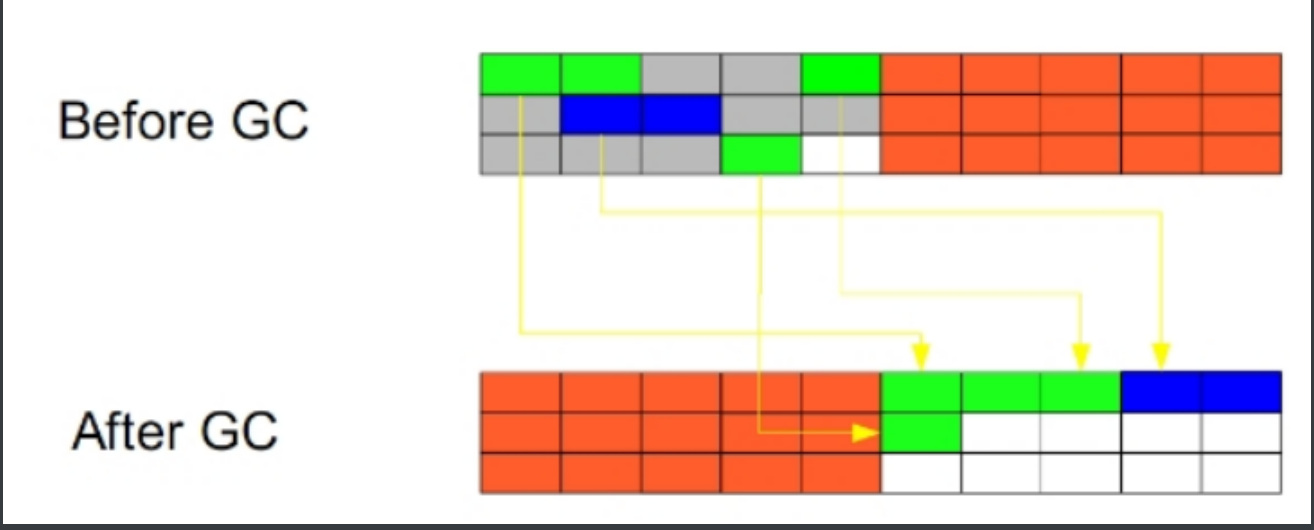
- 强引用，就是普通的对象引用关系，如 String s = new String("ConstXiong")
- 软引用，用于维护一些可有可无的对象。只有在内存不足时，系统则会回收软引用对象，如果回收了软引用对象之后仍然没有足够的内存，才会抛出内存溢出异常。SoftReference 实现
- 弱引用，相比软引用来说，要更加无用一些，它拥有更短的生命周期，当 JVM 进行垃圾回收时，无论内存是否充足，都会回收被弱引用关联的对象。WeakReference 实现
- 虚引用是一种形同虚设的引用，在现实场景中用的不是很多，它主要用来跟踪对象被垃圾回收的活动。PhantomReference 实现

9. 被引用的对象就一定能存活吗?

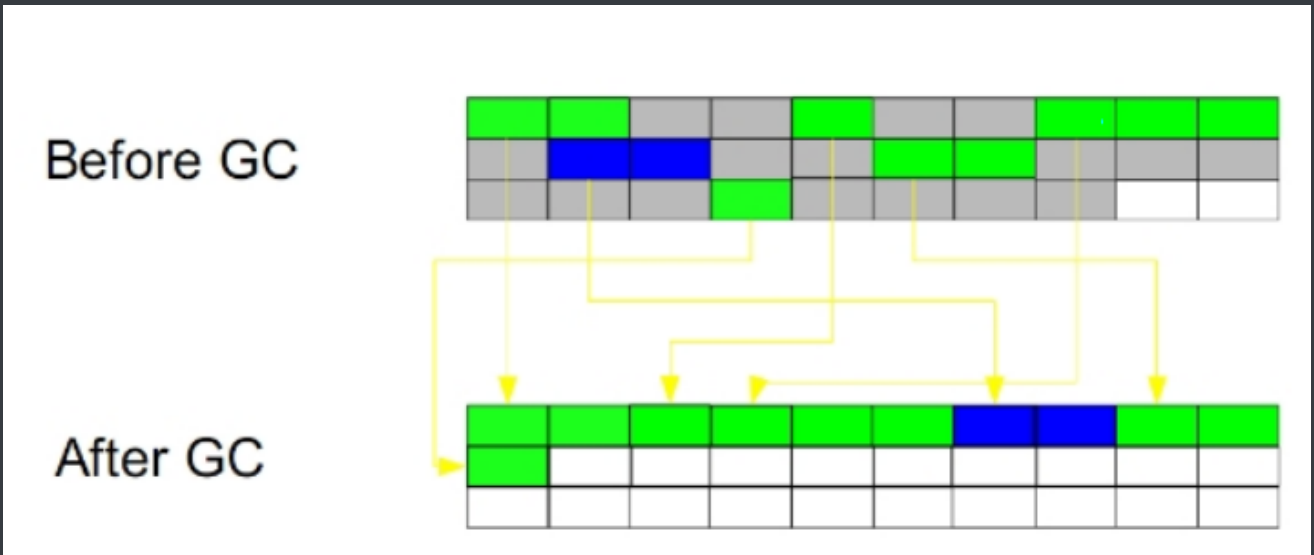
> 不一定，看 Reference 类型，弱引用在 GC 时会被回收，软引用在内存不足的时候，即 OOM 前会被回收，但如果没有在 Reference Chain 中的对象就一定会被回收。

10. Java中的垃圾回收算法有哪些?

java中有四种垃圾回收算法，分别是标记清除法、标记整理法、复制算法、分代收集算法；
标记清除法:
第一步：利用可达性去遍历内存，把存活对象和垃圾对象进行标记；
第二步：在遍历一遍，将所有标记的对象回收掉；
特点：效率不行，标记和清除的效率都不高；标记和清除后会产生大量的不连续的空间分片，可能会导致之后程序运行的时候需分配大对象而找不到连续分片而不得不触发一次GC；



标记整理法:
第一步：利用可达性去遍历内存，把存活对象和垃圾对象进行标记；
第二步：将所有的存活的对象向一段移动，将端边界以外的对象都回收掉；
特点：适用于存活对象多，垃圾少的情况；需要整理的过程，无空间碎片产生；



复制算法:
将内存按照容量大小分为大小相等的两块，每次只使用一块，当一块使用完了，就将还存活的对象移到另一块上，然后在把使用过的内存空间移除；
特点：不会产生空间碎片；内存使用率极低；

分代收集算法:
根据内存对象的存活周期不同，将内存划分成几块，java虚拟机一般将内存分成新生代和老生代，在新生代中，有大量对象死去和少量对象存活，所以采用复制算法，只需要付出少量存活对象的复制成本就可以完成收集；老年代中因为对象的存活率极高，没有额外的空间对他进行分配担保，所以采用标记清理或者标记整理算法进行回收；

对比

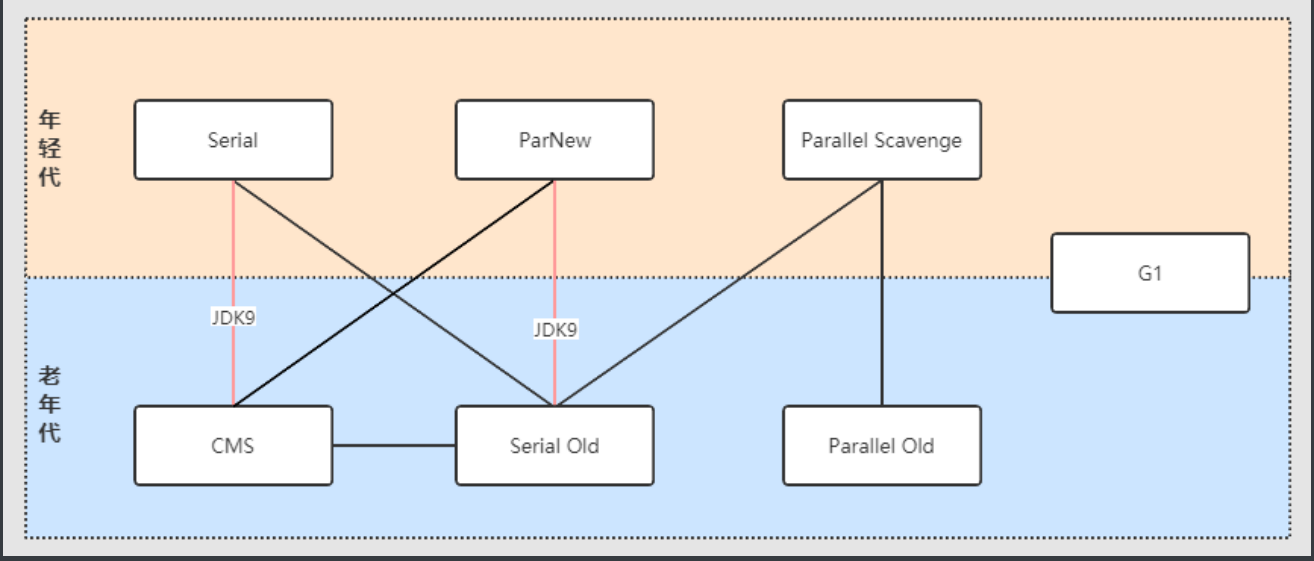
回收算法类型	优点	缺点	适用区域
标记-清除算法（Mark-Sweep）	不需要移动对象，简单高效	标记、清除过程效率低，产生内存碎片	老年代
标记-复制算法（Copying）	清理速度快，没有内存碎片产生	内存使用率低，有可能产生频繁复制的问题	年轻代
标记-整理算法（Mark-Compact）	简单高效、没有内存碎片产生 综了合前两种算法的优点	仍然需要移动局部对象	老年代

11. 有哪几种垃圾回收器，各自的优缺点是什么?

垃圾回收器主要分为以下几种：Serial、ParNew、Parallel Scavenge、Serial Old、Parallel Old、CMS、G1；

- Serial:单线程的收集器，收集垃圾时，必须stop the world，使用复制算法。它的最大特点是在进行垃圾回收时，需要对所有正在执行的线程暂停（stop the world），对于有些应用是难以接受的，但是如果应用的实时性要求不是那么高，只要停顿的时间控制在N毫秒之内，大多数应用还是可以接受的，是client级别的默认GC方式。
- ParNew:Serial收集器的多线程版本，也需要stop the world，复制算
- Parallel Scavenge:新生代收集器，复制算法的收集器，并发的多线程收集器，目标是达到一个可控的吞吐量，和ParNew的最大区别是GC自动调节策略；虚拟机会根据系统的运行状态收集性能监控信息，动态设置这些参数，以提供最优停顿时间和最高的吞吐量；
- Serial Old:Serial收集器的老年代版本，单线程收集器，使用标记整理算法。
- Parallel Old：是Parallel Scavenge收集器的老年代版本，使用多线程，标记-整理算法。
- CMS:是一种以获得最短回收停顿时间为目标的收集器，标记清除算法，运作过程：初始标记，并发标记，重新标记，并发清除，收集结束会产生大量空间碎片；
- G1:标记整理算法实现，运作流程主要包括以下：初始标记，并发标记，最终标记，筛选回收。不会产生空间碎片，可以精确地控制停顿；G1将整个堆分为大小相等的多个Region（区域），G1跟踪每个区域的垃圾大小，在后台维护一个优先级列表，每次根据允许的收集时间，优先回收价值最大的区域，已达到在有限时间内获取尽可能高的回收效率；

垃圾回收器间的配合使用图：



各个垃圾回收器对比：

垃圾回收器类型	使用的回收算法	特点	设置参数	适用区域
Serial	复制算法	单线程回收器，简单高效，停顿时间长。服务端程序几乎不用，一般用于客户端程序。	-XX:+UseSerialGC（年轻代：Serial，老年代：Serial Old）	年轻代
ParNew	复制算法	多线程回收器，降低了停顿时间，但增加了线程上下文切换的消耗。	-XX:+UseParNewGC（年轻代：ParNew，老年代：Serial Old，JDK9生效） -XX:ParallelGCThreads=4（设置并发线程数，默认等于CPU核心数）	年轻代
Parallel Scavenge	复制算法	多线程回收器，追求高吞吐量，高效利用CPU。可以控制最大垃圾回收停顿时间。	-XX:+UseParallelGC（年轻代：Parallel Scavenge，老年代：Parallel Old） -XX:ParallelGCThreads=4（设置并发线程数，默认等于CPU核心数） -XX:MaxGCPauseMillis=200（设置最大GC停顿时间） -XX:GCTimeRatio=99（设置吞吐量） -XX: +UseAdaptiveSizePolicy（启用自适应调解策略，默认开启）	年轻代
Serial Old	标记-整理算法	单线程回收器，简单高效，停顿时间长，主要用于客户端程序。或者与其它回收器配合使用。		老年代
Parallel Old	标记-整理算法	多线程回收器，追求高吞吐量。	-XX:+UseParallelOldGC（年轻代：Parallel Scavenge，老年代：Parallel Old）	老年代
CMS	标记-清除算法	并发回收器，可以与用户线程同时进行。高并发，低停顿，追求最短回收停顿时间，CPU占用比较高，响应时间快，停顿时间短。需要 Serial Old 来避免并发失败的风险。	-XX:+UseConcMarkSweepGC（年轻代：ParNew，老年代：CMS，Serial Old 备用） -XX:CMSInitiatingOccupancyFraction=92（老年代使用空间超过这个比例后触发CMS回收） -XX:+UseCMSInitiatingOccupancyOnly（让JVM使用设定的阀值） -XX:CMSWaitDuration=2000（CMS GC间隔时间，默认2000毫秒） -XX:+UseCMSCompactAtFullCollection（在必须 Full GC 时整理内存碎片，默认开启） -XX:CMSFullGCsBeforeCompaction=0（多少次 Full GC 后整理内存碎片） -XX:CMSBootstrapOccupancy=50（默认50%）	老年代
G1	标记-整理算法 + 复制算法	并发回收器，可以与用户线程同时进行。基于Region的内存布局形式，高并发、低停顿，可控的回收停顿时间。	-XX:+UseG1GC（年轻代：G1，老年代：G1） -XX:MaxGCPauseMillis=200（设置最大GC停顿时间，默认200毫秒） -XX:G1HeapRegionSize=1（设置每个 Region 的大小，1M~32M，且为2的N次冥） -XX:InitiatingHeapOccupancyPercent（老年代超过这个比例后触发MixedGC，默认45%） -XX:G1HeapWastePercent（空闲Region比例，超过这个阀值后停止MixedGC，默认5%） -XX:G1MixedGCLiveThresholdPercent（Region存活对象小于这个比例才会被回收，默认85%） -XX:G1NewSizePercent（新生代占用初始的堆内存比例，默认5%） -XX:G1MaxNewSizePercent（新生代占用最大的堆内存比例，默认60%） -XX:ParallelGCThreads=4（设置并发线程数，默认等于CPU核心数）	年轻代、老年代

12. 详细说一下CMS的回收过程？CMS的问题是什么？

CMS(Concurrent Mark Sweep，并发标记清除) 收集器是以获取最短回收停顿时间为目标的收集器（追求低停顿），它在垃圾收集时使得用户线程和 GC 线程并发执行，因此在垃圾收集过程中用户也不会感到明显的卡顿。

从名字就可以知道，CMS是基于“标记-清除”算法实现的。CMS 回收过程分为以下四步：

1. 初始标记 （CMS initial mark)：主要是标记 GC Root 开始的下级（注：仅下一级）对象，这个过程会 STW，但是跟 GC Root 直接关联的下级对象不会很多，因此这个过程其实很快。
2. 并发标记 (CMS concurrent mark)：根据上一步的结果，继续向下标识所有关联的对象，直到这条链上的最尽头。这个过程是多线程的，虽然耗时理论上会比较长，但是其它工作线程并不会阻塞，没有 STW。
3. 重新标记（CMS remark）：顾名思义，就是要再标记一次。为啥还要再标记一次？因为第 2 步并没有阻塞其它工作线程，其它线程在标识过程中，很有可能会产生新的垃圾。
4. 并发清除（CMS concurrent sweep）：清除阶段是清理删除掉标记阶段判断的已经死亡的对象，由于不需要移动存活对象，所以这个阶段也是可以与用户线程同时并发进行的。

CMS 的问题：

1. 并发回收导致CPU资源紧张：

在并发阶段，它虽然不会导致用户线程停顿，但却会因为占用了一部分线程而导致应用程序变慢，降低程序总吞吐量。CMS默认启动的回收线程数是：（CPU核数 + 3）/ 4，当CPU核数不足四个时，CMS对用户程序的影响就可能变得很大。

2. 无法清理浮动垃圾：

在CMS的并发标记和并发清理阶段，用户线程还在继续运行，就还会伴随有新的垃圾对象不断产生，但这一部分垃圾对象是出现在标记过程结束以后，CMS无法在当次收集中处理掉它们，只好留到下一次垃圾收集时再清理掉。这一部分垃圾称为“浮动垃圾”。

3. 并发失败（Concurrent Mode Failure）：

由于在垃圾回收阶段用户线程还在并发运行，那就还需要预留足够的内存空间提供给用户线程使用，因此CMS不能像其他回收器那样等到老年代几乎完全被填满了再进行回收，必须预留一部分空间供并发回收时的程序运行使用。默认情况下，当老年代使用了 92% 的空间后就会触发 CMS 垃圾回收，这个值可以通过 -XX: CMSInitiatingOccupancyFraction 参数来设置。

这里会有一个风险：要是CMS运行期间预留的内存无法满足程序分配新对象的需要，就会出现一次“并发失败”（Concurrent Mode Failure），这时候虚拟机将不得不启动后备预案：Stop The World，临时启用 Serial Old 来重新进行老年代的垃圾回收，这样一来停顿时间就很长了。

4.内存碎片问题：

CMS是一款基于“标记-清除”算法实现的回收器，这意味着回收结束时会有内存碎片产生。内存碎片过多时，将会给大对象分配带来麻烦，往往会出现老年代还有很多剩余空间，但就是无法找到足够大的连续空间来分配当前对象，而不得不提前触发一次 Full GC 的情况。

为了解决这个问题，CMS收集器提供了一个 -XX:+UseCMSCompactAtFullCollection 开关参数（默认开启），用于在 Full GC 时开启内存碎片的合并整理过程，由于这个内存整理必须移动存活对象，是无法并发的，这样停顿时间就会变长。还有另外一个参数 -XX:CMSFullGCsBeforeCompaction，这个参数的作用是要求CMS在执行过若干次不整理空间的 Full GC 之后，下一次进入 Full GC 前会先进行碎片整理（默认值为0，表示每次进入 Full GC 时都进行碎片整理）。

13. 详细说一下G1的回收过程?

G1（Garbage First）回收器采用面向局部收集的设计思路和基于Region的内存布局形式，是一款主要面向服务端应用的垃圾回收器。G1设计初衷就是替换 CMS，成为一种全功能收集器。G1 在JDK9 之后成为服务端模式下的默认垃圾回收器，取代了 Parallel Scavenge 加 Parallel Old 的默认组合，而 CMS 被声明为不推荐使用的垃圾回收器。G1从整体来看是基于标记-整理 算法实现的回收器，但从局部（两个Region之间）上看又是基于 标记-复制 算法实现的。

G1 回收过程，G1 回收器的运作过程大致可分为四个步骤：

1. 初始标记（会STW）：仅仅只是标记一下 GC Roots 能直接关联到的对象，并且修改TAMS指针的值，让下一阶段用户线程并发运行时，能正确地在可用的Region中分配新对象。这个阶段需要停顿线程，但耗时很短，而且是借用进行Minor GC的时候同步完成的，所以G1收集器在这个阶段实际并没有额外的停顿。
2. 并发标记：从 GC Roots 开始对堆中对象进行可达性分析，递归扫描整个堆里的对象图，找出要回收的对象，这阶段耗时较长，但可与用户程序并发执行。当对象图扫描完成以后，还要重新处理在并发时有引用变动的对象。
3. 最终标记（会STW）：对用户线程做短暂的暂停，处理并发阶段结束后仍有引用变动的对象。
4. 清理阶段（会STW）：更新Region的统计数据，对各个Region的回收价值和成本进行排序，根据用户所期望的停顿时间来制定回收计划，可以自由选择任意多个Region构成回收集，然后把决定回收的那一部分Region的存活对象复制到空的Region中，再清理掉整个旧Region的全部空间。这里的操作涉及存活对象的移动，必须暂停用户线程，由多条回收器线程并行完成的。

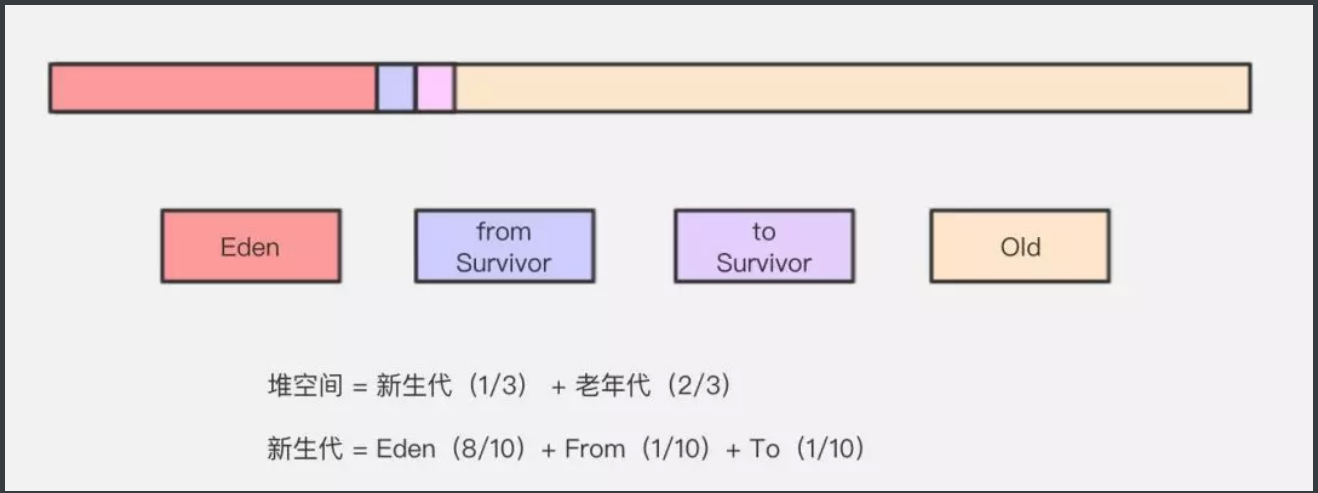
14. JVM中一次完整的GC是什么样子的?

先描述一下Java堆内存划分。

在 Java 中，堆被划分成两个不同的区域：新生代 (Young)、老年代 (Old)，新生代默认占总空间的 1/3，老年代默认占 2/3。
新生代有 3 个分区：Eden、To Survivor、From Survivor，它们的默认占比是 8:1:1。

新生代的垃圾回收（又称Minor GC）后只有少量对象存活，所以选用复制算法，只需要少量的复制成本就可以完成回收。

老年代的垃圾回收（又称Major GC）通常使用“标记-清理”或“标记-整理”算法。



再描述它们之间转化流程：

- 对象优先在Eden分配。当 eden 区没有足够空间进行分配时，虚拟机将发起一次 Minor GC。
  - 在 Eden 区执行了第一次 GC 之后，存活的对象会被移动到其中一个 Survivor 分区；
  - Eden 区再次 GC，这时会采用复制算法，将 Eden 和 from 区一起清理，存活的对象会被复制到 to 区；
  - 移动一次，对象年龄加 1，对象年龄大于一定阀值会直接移动到老年代。GC年龄的阀值可以通过参数 -XX:MaxTenuringThreshold 设置，默认为 15；
  - 动态对象年龄判定：Survivor 区相同年龄所有对象大小的总和 > (Survivor 区内存大小 * 这个目标使用率)时，大于或等于该年龄的对象直接进入老年代。其中这个使用率通过 -XX:TargetSurvivorRatio 指定，默认为 50%；
  - Survivor 区内存不足会发生担保分配，超过指定大小的对象可以直接进入老年代。
- 大对象直接进入老年代，大对象就是需要大量连续内存空间的对象（比如：字符串、数组），为了避免为大对象分配内存时由于分配担保机制带来的复制而降低效率。
- 老年代满了而**无法容纳更多的对象**，Minor GC 之后通常就会进行Full GC，Full GC 清理整个内存堆 – **包括年轻代和老年代**。

15. Minor GC 和 Full GC 有什么不同呢?

Minor GC：只收集新生代的GC。

Full GC: 收集整个堆，包括 新生代，老年代，永久代(在 JDK 1.8及以后，永久代被移除，换为metaspace 元空间)等所有部分的模式。

Minor GC触发条件：当Eden区满时，触发Minor GC。

Full GC触发条件：

- 通过Minor GC后进入老年代的平均大小大于老年代的可用内存。如果发现统计数据说之前Minor GC的平均晋升大小比目前old gen剩余的空间大，则不会触发Minor GC而是转为触发full GC。
- 老年代空间不够分配新的内存（或永久代空间不足，但只是JDK1.7有的，这也是用元空间来取代永久代的原因，可以减少Full GC的频率，减少GC负担，提升其效率）。
- 由Eden区、From Space区向To Space区复制时，对象大小大于To Space可用内存，则把该对象转存到老年代，且老年代的可用内存小于该对象大小。
- 调用System.gc时，系统建议执行Full GC，但是不必然执行。

16. 介绍下空间分配担保原则?

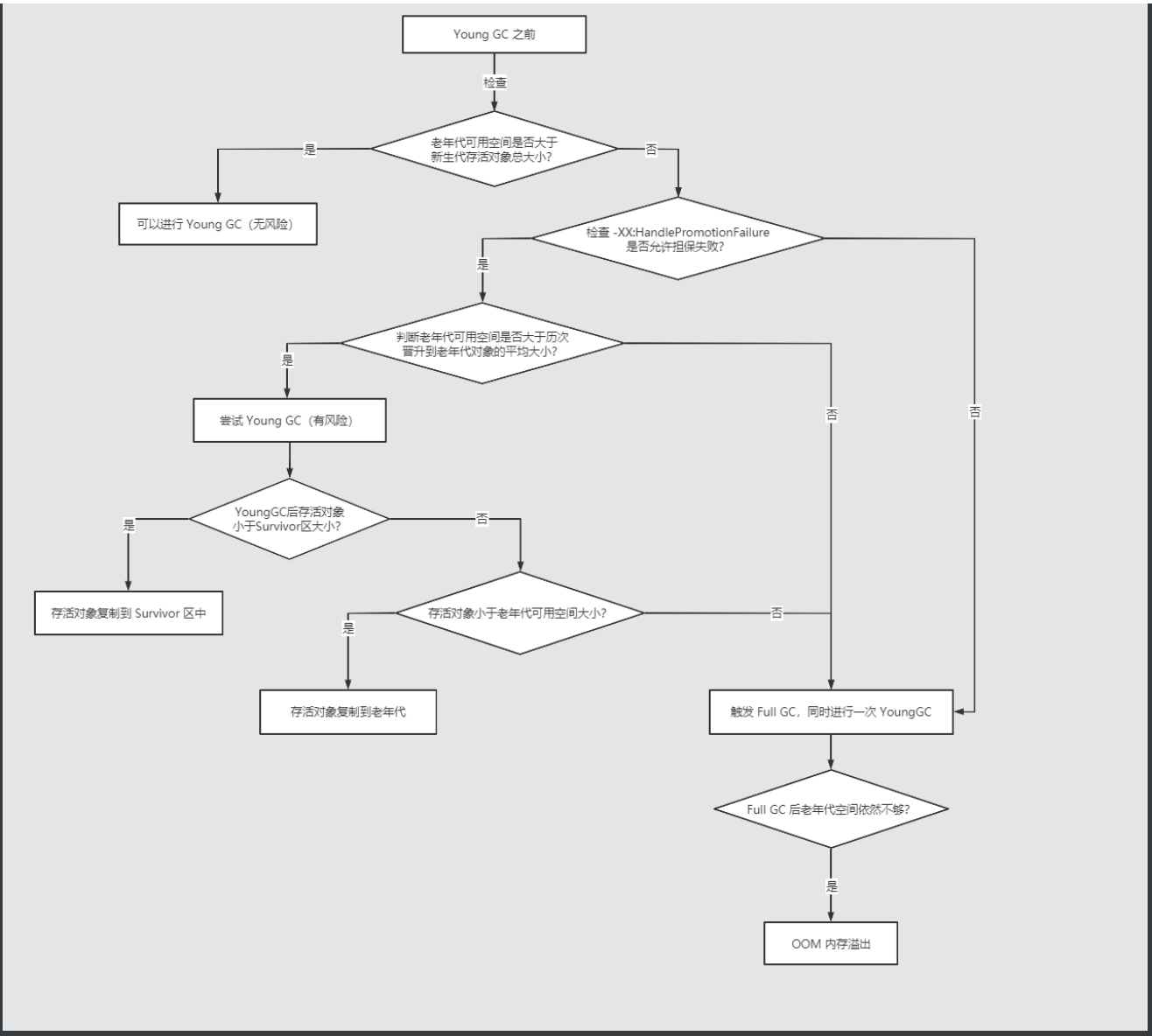
如果YougGC时新生代有大量对象存活下来，而 survivor 区放不下了，这时必须转移到老年代中，但这时发现老年代也放不下这些对象了，那怎么处理呢？其实JVM有一个老年代空间分配担保机制来保证对象能够进入老年代。

在执行每次 YoungGC 之前，JVM会先检查老年代最大可用连续空间是否大于新生代所有对象的总大小。因为在极端情况下，可能新生代 YoungGC 后，所有对象都存活下来了，而 survivor 区又放不下，那可能所有对象都要进入老年代了。这个时候如果老年代的可用连续空间是大于新生代所有对象的总大小的，那就可以放心进行 YoungGC。但如果老年代的内存大小是小于新生代对象总大小的，那就有可能老年代空间不够放入新生代所有存活对象，这个时候JVM就会先检查 -XX:HandlePromotionFailure 参数是否允许担保失败，如果允许，就会判断老年代最大可用连续空间是否大于历次晋升到老年代对象的平均大小，如果大于，将尝试进行一次YoungGC，尽快这次YoungGC是有风险的。如果小于，或者 -XX:HandlePromotionFailure 参数不允许担保失败，这时就会进行一次 Full GC。

在允许担保失败并尝试进行YoungGC后，可能会出现三种情况：

- ① YoungGC后，存活对象小于survivor大小，此时存活对象进入survivor区中
- ② YoungGC后，存活对象大于survivor大小，但是小于老年大可用空间大小，此时直接进入老年代。
- ③ YoungGC后，存活对象大于survivor大小，也大于老年大可用空间大小，老年代也放不下这些对象了，此时就会发生“Handle Promotion Failure”，就触发了 Full GC。如果 Full GC后，老年代还是没有足够的空间，此时就会发生OOM内存溢出了。

通过下图来了解空间分配担保原则：



17. 什么是类加载？类加载的过程？

虚拟机把描述类的数据加载到内存里面，并对数据进行校验、解析和初始化，最终变成可以被虚拟机直接使用的class对象；

类的整个生命周期包括：加载（Loading）、验证（Verification）、准备(Preparation)、解析(Resolution)、初始化(Initialization)、使用(Using)和卸载(Unloading)7个阶段。其中准备、验证、解析3个部分统称为连接（Linking）。如图所示：



加载、验证、准备、初始化和卸载这5个阶段的顺序是确定的，类的加载过程必须按照这种顺序按部就班地开始，而解析阶段则不一定：它在某些情况下可以在初始化阶段之后再开始，这是为了支持Java语言的运行时绑定（也称为动态绑定或晚期绑定）

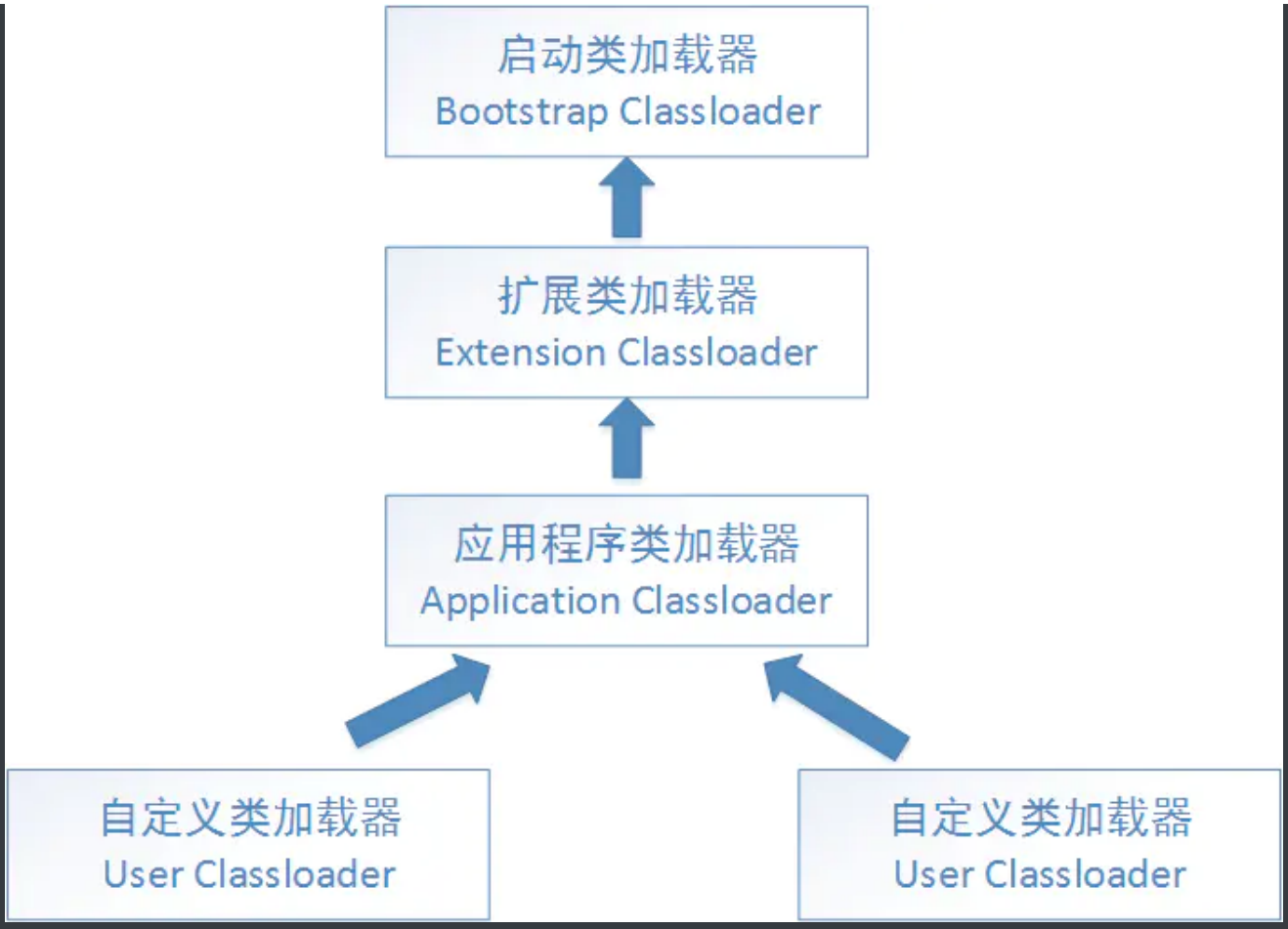
类加载过程如下：

- 加载，加载分为三步：
  1、通过类的全限定性类名获取该类的二进制流；
  2、将该二进制流的静态存储结构转为方法区的运行时数据结构；
  3、在堆中为该类生成一个class对象；
- 验证：验证该class文件中的字节流信息复合虚拟机的要求，不会威胁到jvm的安全；
- 准备：为class对象的静态变量分配内存，初始化其初始值；
- 解析：该阶段主要完成符号引用转化成直接引用；
- 初始化：到了初始化阶段，才开始执行类中定义的java代码；初始化阶段是调用类构造器的过程；

18. 什么是类加载器，常见的类加载器有哪些?

类加载器是指：通过一个类的全限定性类名获取该类的二进制字节流叫做类加载器；类加载器分为以下四种：

- 启动类加载器（BootStrapClassLoader）：用来加载java核心类库，无法被java程序直接引用；
- 扩展类加载器（Extension ClassLoader）：用来加载java的扩展库，java的虚拟机实现会提供一个扩展库目录，该类加载器在扩展库目录里面查找并加载java类；
- 系统类加载器（AppClassLoader）：它根据java的类路径来加载类，一般来说，java应用的类都是通过它来加载的；
- 自定义类加载器：由java语言实现，继承自ClassLoader；



19. 什么是双亲委派模型？为什么需要双亲委派模型？

当一个类加载器收到一个类加载的请求，他首先不会尝试自己去加载，而是将这个请求委派给父类加载器去加载，只有父类加载器在自己的搜索范围类查找不到给类时，子加载器才会尝试自己去加载该类；

为了防止内存中出现多个相同的字节码；因为如果没有双亲委派的话，用户就可以自己定义一个java.lang.String类，那么就无法保证类的唯一性。

补充：**那怎么打破双亲委派模型**？

自定义类加载器，继承ClassLoader类，重写loadClass方法和findClass方法。

20. 列举一些你知道的打破双亲委派机制的例子，为什么要打破？

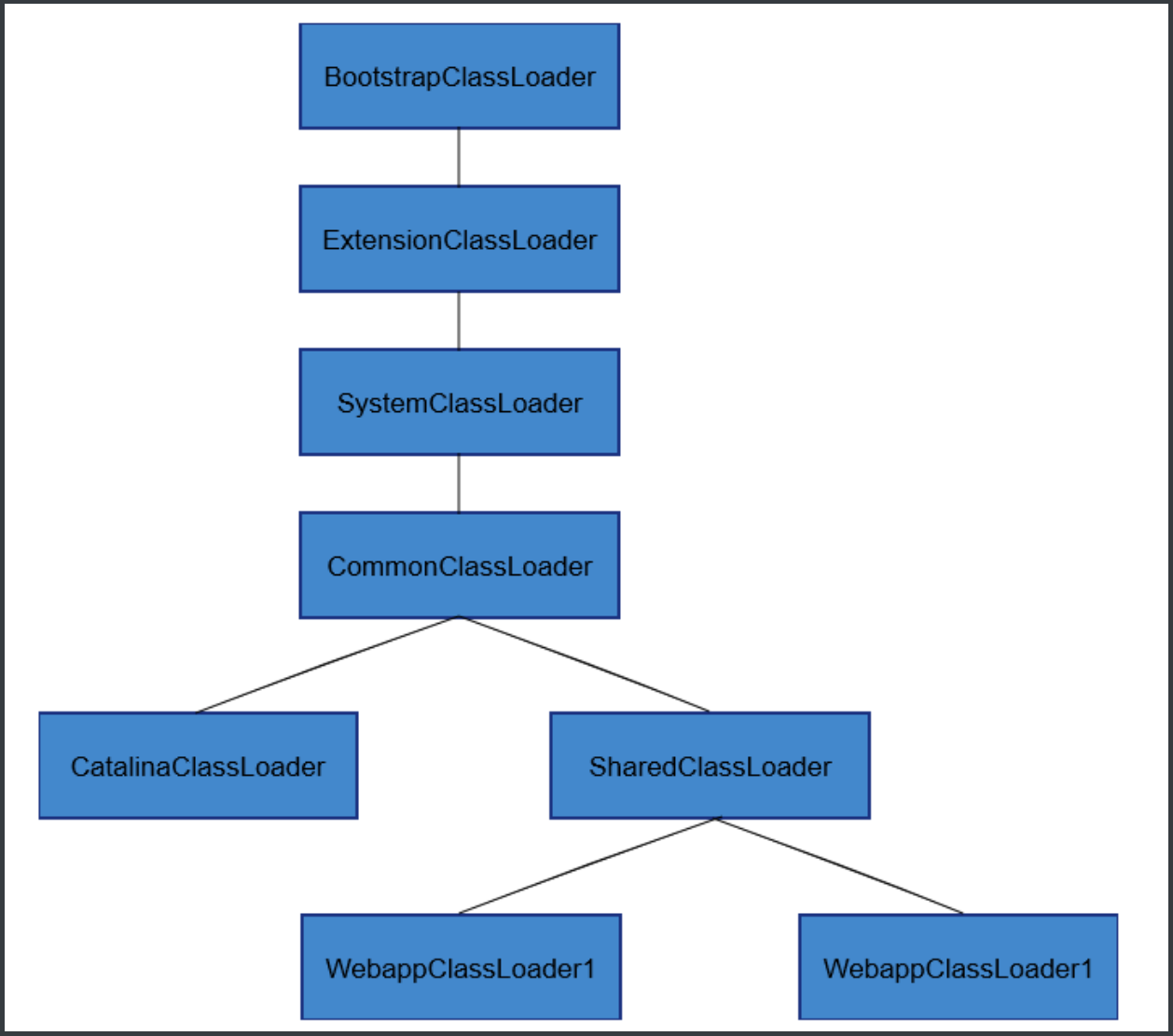
- JNDI 通过引入线程上下文类加载器，可以在 Thread.setContextClassLoader 方法设置，默认是应用程序类加载器，来加载 SPI 的代码。有了线程上下文类加载器，就可以完成父类加载器请求子类加载器完成类加载的行为。打破的原因，是为了 JNDI 服务的类加载器是启动器类加载，为了完成高级类加载器请求子类加载器（即上文中的线程上下文加载器）加载类。

- Tomcat，应用的类加载器优先自行加载应用目录下的 class，并不是先委派给父加载器，加载不了才委派给父加载器。

  tomcat之所以造了一堆自己的classloader，大致是出于下面三类目的：

  - 对于各个 `webapp` 中的 `class` 和 `lib`，需要相互隔离，不能出现一个应用中加载的类库会影响另一个应用的情况，而对于许多应用，需要有共享的lib以便不浪费资源。
  - 与 `jvm` 一样的安全性问题。使用单独的 `classloader` 去装载 `tomcat` 自身的类库，以免其他恶意或无意的破坏；
  - 热部署。

  tomcat类加载器如下图：

- OSGi，实现模块化热部署，为每个模块都自定义了类加载器，需要更换模块时，模块与类加载器一起更换。其类加载的过程中，有平级的类加载器加载行为。打破的原因是为了实现模块热替换。
- JDK 9，Extension ClassLoader 被 Platform ClassLoader 取代，当平台及应用程序类加载器收到类加载请求，在委派给父加载器加载前，要先判断该类是否能够归属到某一个系统模块中，如果可以找到这样的归属关系，就要优先委派给负责那个模块的加载器完成加载。打破的原因，是为了添加模块化的特性。

21.说一下 JVM 调优的命令?

- jps：JVM Process Status Tool,显示指定系统内所有的HotSpot虚拟机进程。
- jstat：jstat(JVM statistics Monitoring)是用于监视虚拟机运行时状态信息的命令，它可以显示出虚拟机进程中的类装载、内存、垃圾收集、JIT编译等运行数据。
- jmap：jmap(JVM Memory Map)命令用于生成heap dump文件，如果不使用这个命令，还阔以使用-XX:+HeapDumpOnOutOfMemoryError参数来让虚拟机出现OOM的时候·自动生成dump文件。
  jmap不仅能生成dump文件，还阔以查询finalize执行队列、Java堆和永久代的详细信息，如当前使用率、当前使用的是哪种收集器等。
- jhat：jhat(JVM Heap Analysis Tool)命令是与jmap搭配使用，用来分析jmap生成的dump，jhat内置了一个微型的HTTP/HTML服务器，生成dump的分析结果后，可以在浏览器中查看。在此要注意，一般不会直接在服务器上进行分析，因为jhat是一个耗时并且耗费硬件资源的过程，一般把服务器生成的dump文件复制到本地或其他机器上进行分析。
- jstack：jstack用于生成java虚拟机当前时刻的线程快照。jstack来查看各个线程的调用堆栈，就可以知道没有响应的线程到底在后台做什么事情，或者等待什么资源。 如果java程序崩溃生成core文件，jstack工具可以用来获得core文件的java stack和native stack的信息，从而可以轻松地知道java程序是如何崩溃和在程序何处发生问题。

四、Java并发

多线程基础

1. 线程和进程有什么区别?

线程具有许多传统进程所具有的特征，故又称为轻型进程(Light—Weight Process)或进程元；而把传统的进程称为重型进程(Heavy—Weight Process)，它相当于只有一个线程的任务。在引入了线程的操作系统中，通常一个进程都有若干个线程，至少包含一个线程。

根本区别：进程是操作系统资源分配的基本单位，而线程是处理器任务调度和执行的基本单位

资源开销：每个进程都有独立的代码和数据空间（程序上下文），程序之间的切换会有较大的开销；线程可以看做轻量级的进程，同一类线程共享代码和数据空间，每个线程都有自己独立的运行栈和程序计数器（PC），线程之间切换的开销小。

包含关系：如果一个进程内有多个线程，则执行过程不是一条线的，而是多条线（线程）共同完成的；线程是进程的一部分，所以线程也被称为轻权进程或者轻量级进程。

内存分配：同一进程的线程共享本进程的地址空间和资源，而进程之间的地址空间和资源是相互独立的

影响关系：一个进程崩溃后，在保护模式下不会对其他进程产生影响，但是一个线程崩溃整个进程都死掉。所以多进程要比多线程健壮。

执行过程：每个独立的进程有程序运行的入口. 顺序执行序列和程序出口。但是线程不能独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制，两者均可并发执行

2. 创建线程的三种方式的对比?

1）采用实现Runnable. Callable接口的方式创建多线程。

优势是:

线程类只是实现了Runnable接口或Callable接口，还可以继承其他类。

在这种方式下，多个线程可以共享同一个target对象，所以非常适合多个相同线程来处理同一份资源的情况，从而可以将CPU. 代码和数据分开，形成清晰的模型，较好地体现了面向对象的思想。

劣势是:

编程稍微复杂，如果要访问当前线程，则必须使用Thread.currentThread()方法。

2）使用继承Thread类的方式创建多线程

优势是:

编写简单，如果需要访问当前线程，则无需使用Thread.currentThread()方法，直接使用this即可获得当前线程。

劣势是:

线程类已经继承了Thread类，所以不能再继承其他父类。

3）Runnable和Callable的区别

- Callable规定（重写）的方法是call()，Runnable规定（重写）的方法是run()。
- Callable的任务执行后可返回值，而Runnable的任务是不能返回值的。
- Call方法可以抛出异常，run方法不可以。
- 运行Callable任务可以拿到一个Future对象，表示异步计算的结果。它提供了检查计算是否完成的方法，以等待计算的完成，并检索计算的结果。通过Future对象可以了解任务执行情况，可取消任务的执行，还可获取执行结果。

3. 为什么要使用多线程呢?

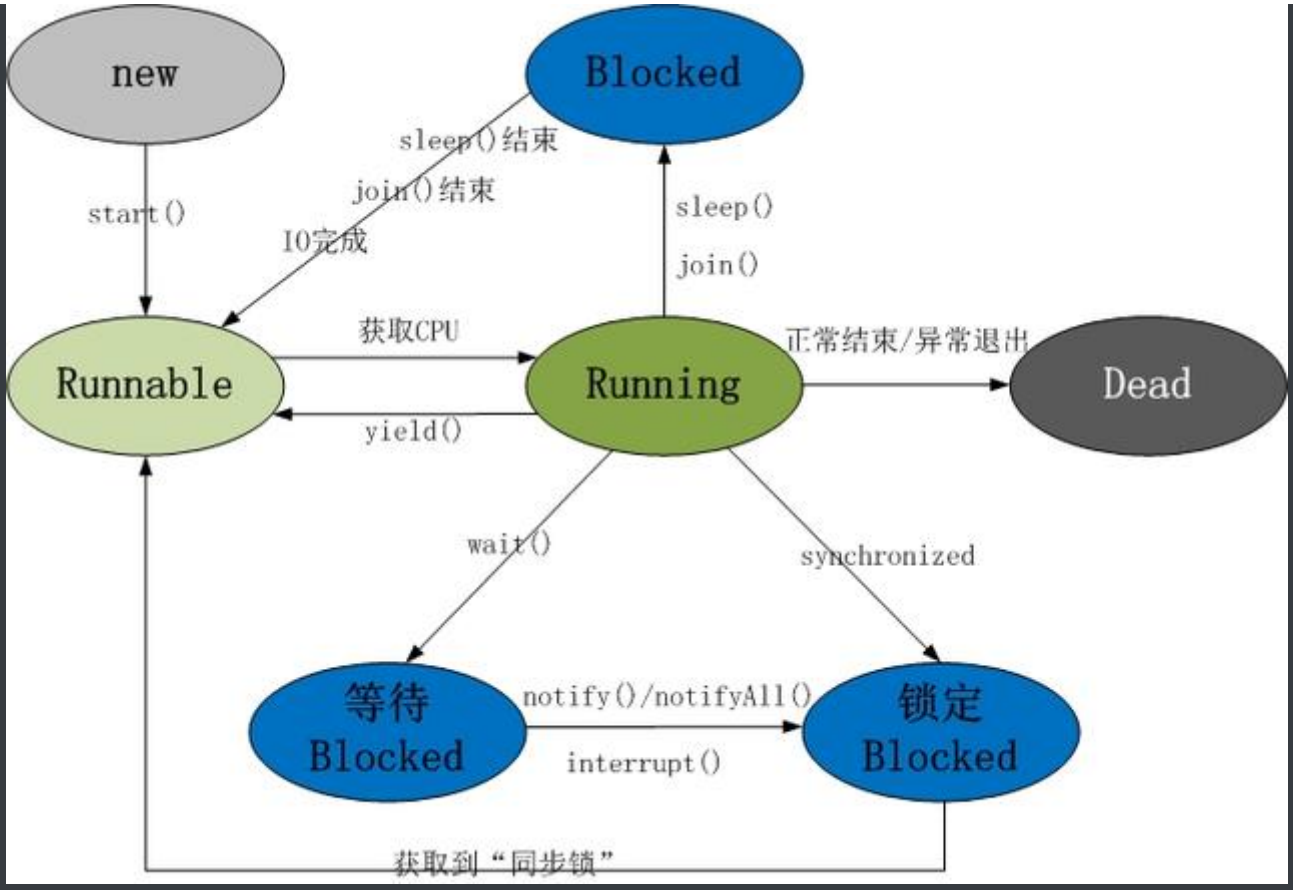
- 从计算机底层来说： 线程可以比作是轻量级的进程，是程序执行的最小单位,**线程间的切换和调度的成本远远小于进程**。另外，多核 CPU 时代意味着多个线程可以同时运行，这减少了线程上下文切换的开销。
- 从当代互联网发展趋势来说： 现在的系统动不动就要求百万级甚至千万级的并发量，而**多线程并发编程正是开发高并发系统的基础**，利用好多线程机制可以大大提高系统整体的并发能力以及性能。

从计算机底层来说：

- 单核时代： **在单核时代多线程主要是为了提高 CPU 和 IO 设备的综合利用率**。举个例子：当只有一个线程的时候会导致 CPU 计算时，IO 设备空闲；进行 IO 操作时，CPU 空闲。我们可以简单地说这两者的利用率目前都是 50%左右。但是当有两个线程的时候就不一样了，当一个线程执行 CPU 计算时，另外一个线程可以进行 IO 操作，这样两个的利用率就可以在理想情况下达到 100%了。
- 多核时代:**多核时代多线程主要是为了提高 CPU 利用率**。举个例子：假如我们要计算一个复杂的任务，我们只用一个线程的话，CPU 只会一个 CPU 核心被利用到，而创建多个线程就可以让多个 CPU 核心被利用到，这样就提高了 CPU 的利用率。

4. 线程的状态流转

线程的生命周期及五种基本状态：



Java线程具有五中基本状态

1）新建状态（New）：当线程对象对创建后，即进入了新建状态，如：Thread t = new MyThread();

2）就绪状态（Runnable）：当调用线程对象的start()方法（t.start();），线程即进入就绪状态。处于就绪状态的线程，只是说明此线程已经做好了准备，随时等待CPU调度执行，并不是说执行了t.start()此线程立即就会执行；

3）运行状态（Running）：当CPU开始调度处于就绪状态的线程时，此时线程才得以真正执行，即进入到运行状态。注：就 绪状态是进入到运行状态的唯一入口，也就是说，线程要想进入运行状态执行，首先必须处于就绪状态中；

4）阻塞状态（Blocked）：处于运行状态中的线程由于某种原因，暂时放弃对CPU的使用权，停止执行，此时进入阻塞状态，直到其进入到就绪状态，才 有机会再次被CPU调用以进入到运行状态。根据阻塞产生的原因不同，阻塞状态又可以分为三种：

1.等待阻塞：运行状态中的线程执行wait()方法，使本线程进入到等待阻塞状态；

2.同步阻塞 — 线程在获取synchronized同步锁失败(因为锁被其它线程所占用)，它会进入同步阻塞状态；

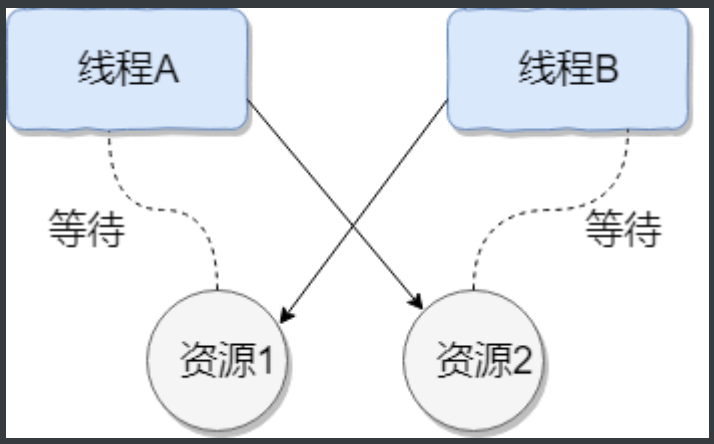
3.其他阻塞 — 通过调用线程的sleep()或join()或发出了I/O请求时，线程会进入到阻塞状态。当sleep()状态超时. join()等待线程终止或者超时. 或者I/O处理完毕时，线程重新转入就绪状态。

5）死亡状态（Dead）：线程执行完了或者因异常退出了run()方法，该线程结束生命周期。

5.什么是线程死锁?如何避免死锁?

死锁

- 多个线程同时被阻塞，它们中的一个或者全部都在等待某个资源被释放。由于线程被无限期地阻塞，因此程序不可能正常终止。



死锁必须具备以下四个条件：

- 互斥条件：该资源任意一个时刻只由一个线程占用。
- 请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。
- 不剥夺条件:线程已获得的资源在未使用完之前不能被其他线程强行剥夺，只有自己使用完毕后才释放资源。
- 循环等待条件:若干进程之间形成一种头尾相接的循环等待资源关系。

如何避免线程死锁?

只要破坏产生死锁的四个条件中的其中一个就可以了

- 破坏互斥条件
  这个条件我们没有办法破坏，因为我们用锁本来就是想让他们互斥的（临界资源需要互斥访问）
- 破坏请求与保持条件
  一次性申请所有的资源。
- 破坏不剥夺条件
  占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。

- 破坏循环等待条件
  靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏循环等待条件。
- 锁排序法：（必须回答出来的点）
  指定获取锁的顺序，比如某个线程只有获得A锁和B锁，才能对某资源进行操作，在多线程条件下，如何避免死锁?
  通过指定锁的获取顺序，比如规定，只有获得A锁的线程才有资格获取B锁，按顺序获取锁就可以避免死锁。这通常被认为是解决死锁很好的一种方法。
- 使用显式锁中的ReentrantLock.try(long,TimeUnit)来申请锁

6. 常见的对比

Runnable VS Callable

- Callable仅在 Java 1.5 中引入,目的就是为了来处理Runnable不支持的用例。Callable 接口可以返回结果或抛出检查异常
- Runnable 接口不会返回结果或抛出检查异常，
- 如果任务不需要返回结果或抛出异常推荐使用 Runnable接口，这样代码看起来会更加简洁
- 工具类 Executors 可以实现 Runnable 对象和 Callable 对象之间的相互转换。（Executors.callable（Runnable task）或 Executors.callable（Runnable task，Object resule））

shutdown() VS shutdownNow()

- shutdown（） :关闭线程池，线程池的状态变为 SHUTDOWN。线程池不再接受新任务了，但是队列里的任务得执行完毕。
- shutdownNow（） :关闭线程池，线程的状态变为 STOP。线程池会终止当前正在运行的任务，并停止处理排队的任务并返回正在等待执行的 List。
  shutdownNow的原理是遍历线程池中的工作线程，然后逐个调用线程的interrupt方法来中断线程，所以无法响应中断的任务可能永远无法终

isTerminated() VS isShutdown()

- isShutDown 当调用 shutdown() 方法后返回为 true。
- isTerminated 当调用 shutdown() 方法后，并且所有提交的任务完成后返回为 true

7. sleep() 方法和 wait() 方法区别和共同点?

区别

- sleep方法：是Thread类的静态方法，当前线程将睡眠n毫秒，线程进入阻塞状态。当睡眠时间到了，会解除阻塞，进入可运行状态，等待CPU的到来。睡眠不释放锁（如果有的话）。
- wait方法：是Object的方法，必须与synchronized关键字一起使用，线程进入阻塞状态，当notify或者notifyall被调用后，会解除阻塞。但是，只有重新占用互斥锁之后才会进入可运行状态。睡眠时，会释放互斥锁。
- sleep 方法没有释放锁，而 wait 方法释放了锁 。
- sleep 通常被用于暂停执行Wait 通常被用于线程间交互/通信
- sleep() 方法执行完成后，线程会自动苏醒。或者可以使用 wait(long timeout)超时后线程会自动苏醒。wait() 方法被调用后，线程不会自动苏醒，需要别的线程调用同一个对象上的 notify() 或者 notifyAll() 方法

相同

- 两者都可以暂停线程的执行。

8.为什么我们调用 start() 方法时会执行 run() 方法，为什么我们不能直接调用 run() 方法

- new 一个 Thread，线程进入了新建状态; 调用start() 会执行线程的相应准备工作，然后自动执行 run() 方法的内容，（调用 start() 方法，会启动一个线程并使线程进入了就绪状态，当分配到时间片后就可以开始运行了。）这是真正的多线程工作。
- 直接执行 run() 方法，会把 run 方法当成一个 main 线程下的普通方法去执行，并不会在某个线程中执行它，所以这并不是多线程工作。
  调用 start 方法方可启动线程并使线程进入就绪状态，而 run 方法只是 thread 的一个普通方法调用，还是在主线程里执行。

9. Thread类中的yield方法有什么作用?

Yield方法可以暂停当前正在执行的线程对象，让其它有相同优先级的线程执行。它是一个静态方法而且只保证当前线程放弃CPU占用而不能保证使其它线程一定能占用CPU，执行yield()的线程有可能在进入到暂停状态后马上又被执行。

10. 谈谈volatile的使用及其原理

volatile的两层语义：

1. volatile保证变量对所有线程的可见性：当volatile变量被修改，新值对所有线程会立即更新。或者理解为多线程环境下使用volatile修饰的变量的值一定是最新的。
2. jdk1.5以后volatile完全避免了指令重排优化，实现了有序性。

volatile的原理:

获取JIT（即时Java编译器，把字节码解释为机器语言发送给处理器）的汇编代码，发现volatile多加了lock addl指令，这个操作相当于一个内存屏障，使得lock指令后的指令不能重排序到内存屏障前的位置。这也是为什么JDK1.5以后可以使用双锁检测实现单例模式。

lock前缀的另一层意义是使得本线程工作内存中的volatile变量值立即写入到主内存中，并且使得其他线程共享的该volatile变量无效化，这样其他线程必须重新从主内存中读取变量值。

具体原理见这篇文章：https://www.javazhiyin.com/61019.html

11. 如何创建线程实例并运行?

`Thread` 类本质上是实现 `Runnable` 接口的一个实例，代表一个线程的实例。创建线程实例一般有两种方法：

1. 创建 Thread 的子类并重写 `run()`

复制代码

```
public class MyThread extends Thread {
    @Override
    public void run(){
        System.out.println("MyThread running");
    }
}
```

`run()` 方在调用 `start()` 方法后被执行，而且一旦线程启动后 `start()` 方法后就会立即返回，而不是等到 `run()` 方法执行完毕后再返回。

```
MyThread myThread = new MyThread();
myThread.start();
```

2. 实现 Runnable 接口

```
public class MyRunnable implements Runnable{
    @Override
    public void run(){
        System.out.println("MyRunnable running");
    }
}
```

在新建类时实现 `Runnable` 接口，然后在 `Thread` 类的构造函数中传入 `MyRunnable` 的实例对象，最后执行 `start()` 方法即可；

```
Thread thread = new Thread(new MyRunnable());
thread.start();
```

12. 线程阻塞的三种情况

当线程因为某种原因放弃 CPU 使用权后，即让出了 CPU 时间片，暂时就会停止运行，知道线程进入可运行状态（ `Runnable` ），才有机会再次获得 CPU 时间片转入 `RUNNING` 状态。一般来讲，阻塞的情况可以分为如下三种：

1. **等待阻塞（Object.wait -> 等待队列）**

`RUNNING` 状态的线程执行 `Object.wait()` 方法后，JVM 会将线程放入等待序列（waitting queue）；

2. **同步阻塞（lock -> 锁池）**

`RUNNING` 状态的线程在获取对象的同步锁时，若该 **同步锁被其他线程占用，则 JVM 将该线程放入锁池（lock pool）中**；

3. **其他阻塞（sleep/join）**

`RUNNING` 状态的线程执行 `Thread.sleep(long ms)` 或 `Thread.join()` 方法，或发出 I/O 请求时，JVM 会将该线程置为阻塞状态。当 `sleep()` 状态超时， `join()` 等待线程终止或超时. 或者 I/O 处理完毕时，线程重新转入可运行状态（ `RUNNABLE` ）；

13. 线程死亡的三种方式

1. **正常结束**

`run()` 或者 `call()` 方法执行完成后，线程正常结束；

2. **异常结束**

线程抛出一个未捕获的 `Exception` 或 `Error`，导致线程异常结束；

3. **调用 stop()**

直接调用线程的 `stop()` 方法来结束该线程，但是一般不推荐使用该种方式，**因为该方法通常容易导致死锁**；

14. 为什么我们调用start()方法时会执行run()方法，为什么我们不能直接调用run()方法?

JVM执行start方法，会另起一条线程执行thread的run方法，这才起到多线程的效果~

如果直接调用Thread的run()方法，其方法还是运行在主线程中，没有起到多线程效果。

15. 守护线程是什么?

守护线程是运行在后台的一种特殊进程。它独立于控制终端并且周期性地执行某种任务或等待处理某些发生的事件。在 Java 中垃圾回收线程就是特殊的守护线程。

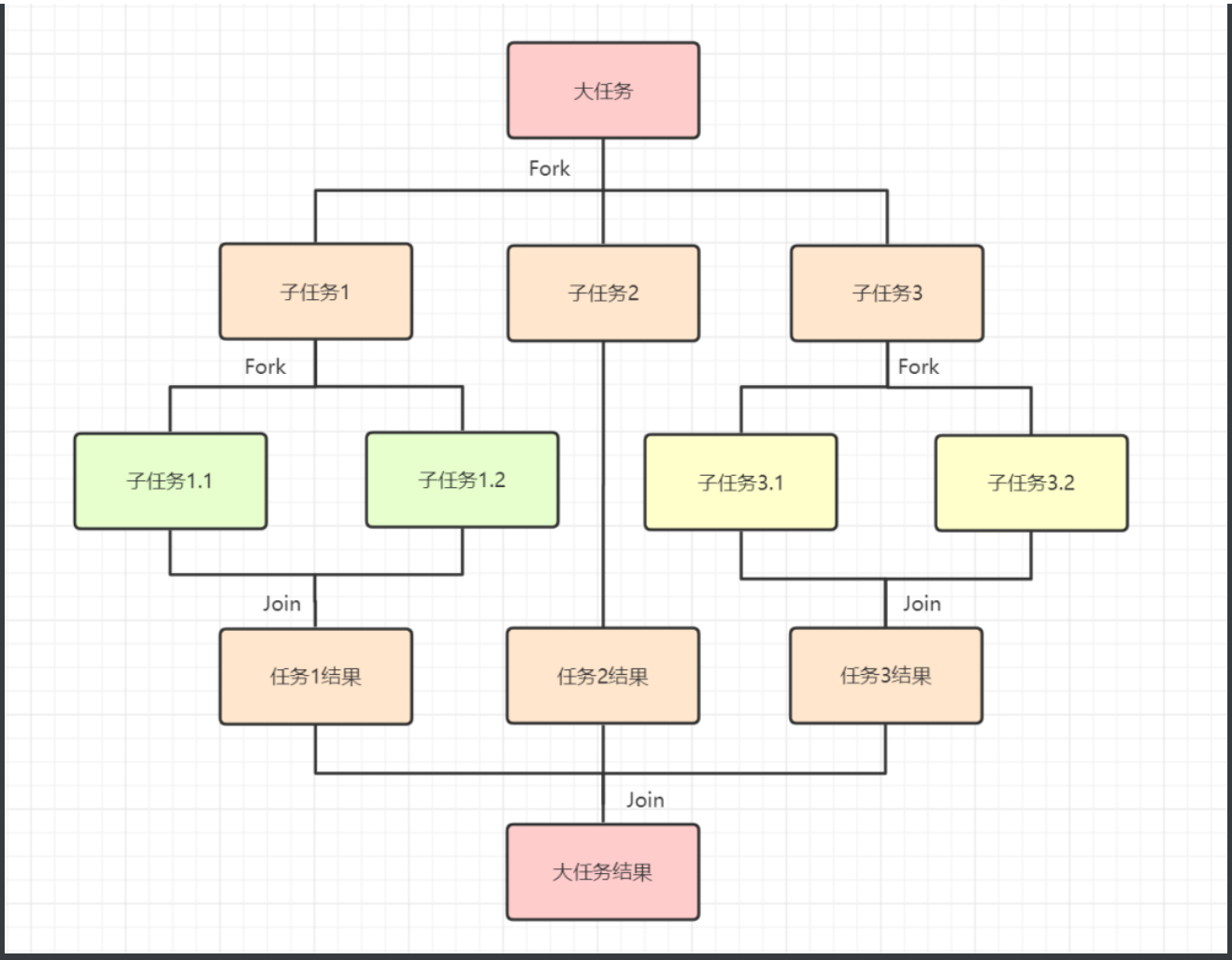
16. 了解Fork/Join框架吗?

Fork/Join框架是Java7提供的一个用于并行执行任务的框架，是一个把大任务分割成若干个小任务，最终汇总每个小任务结果后得到大任务结果的框架。

Fork/Join框架需要理解两个点，**「分而治之」**和**「工作窃取算法」**。

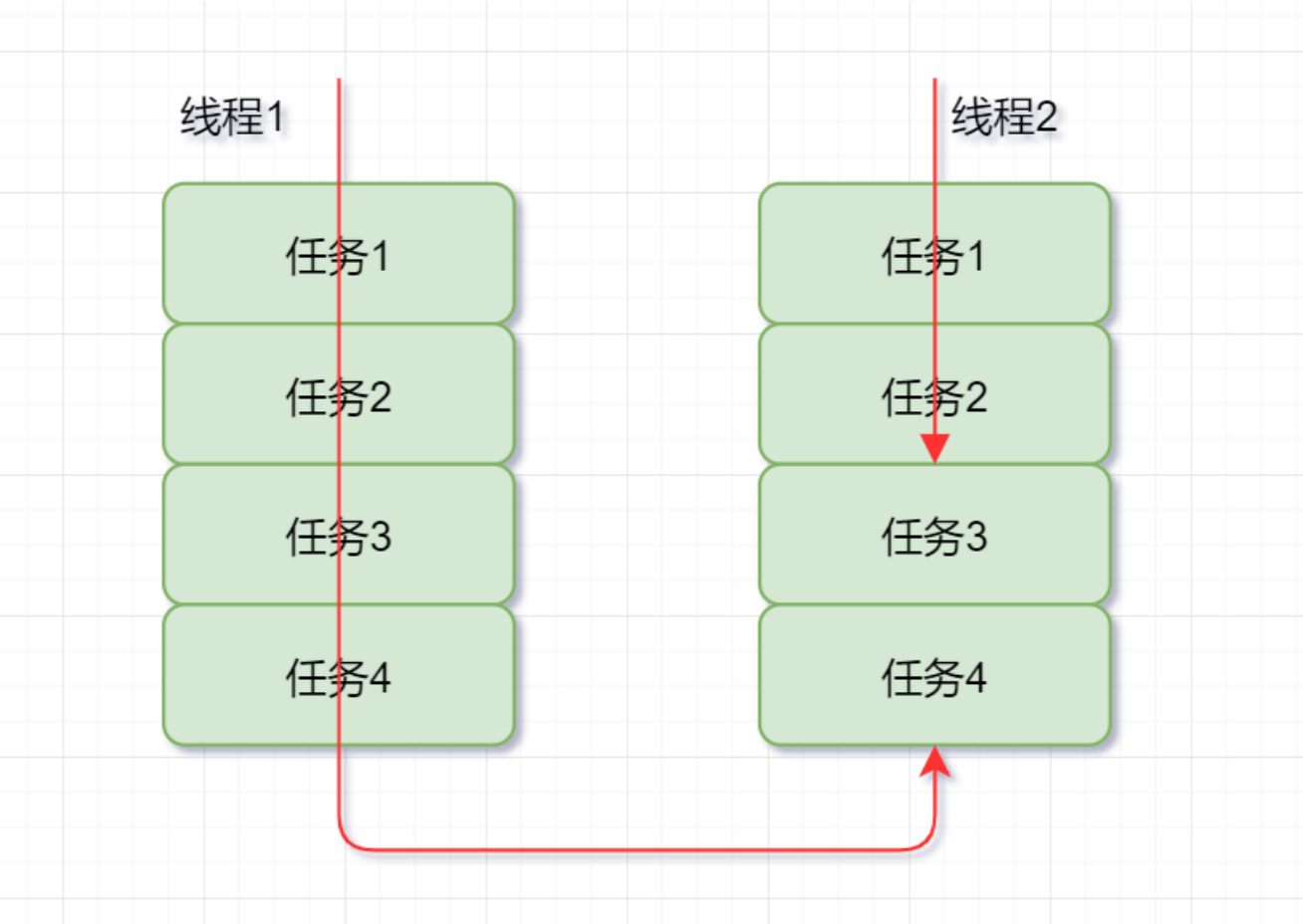
「分而治之」

以上Fork/Join框架的定义，就是分而治之思想的体现啦



「工作窃取算法」

把大任务拆分成小任务，放到不同队列执行，交由不同的线程分别执行时。有的线程优先把自己负责的任务执行完了，其他线程还在慢慢悠悠处理自己的任务，这时候为了充分提高效率，就需要工作盗窃算法啦~



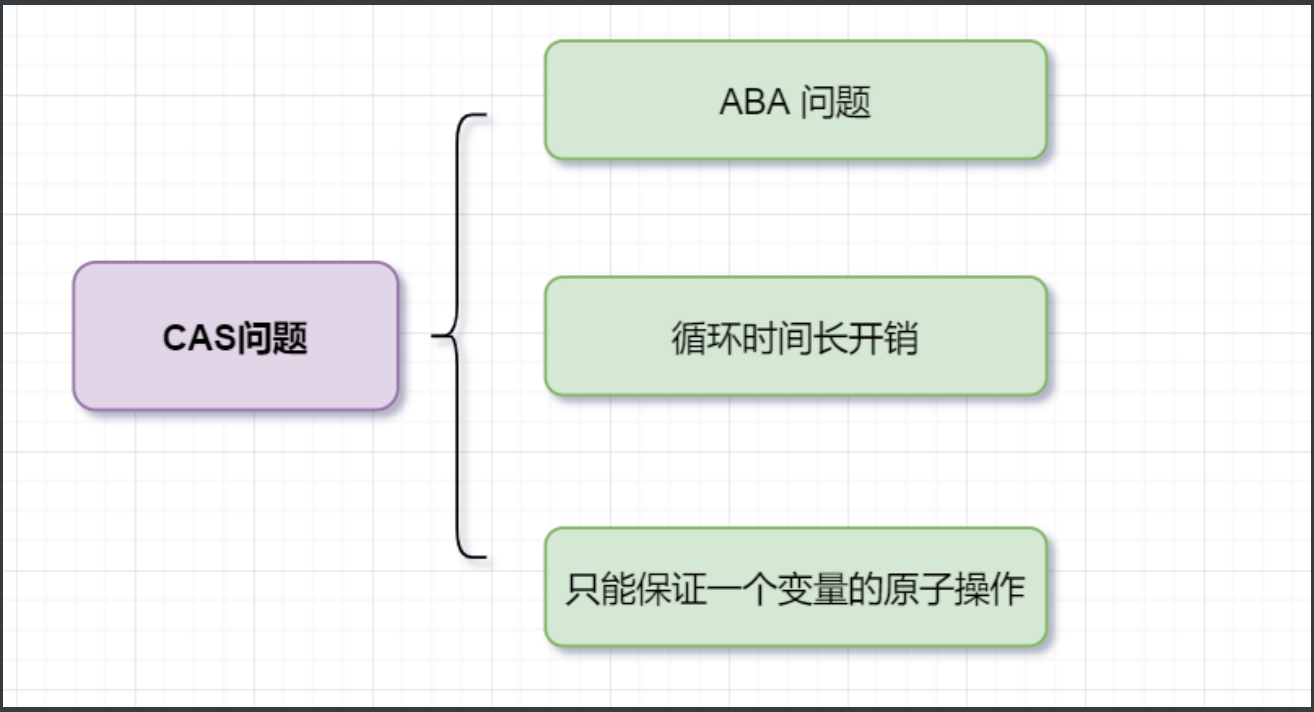
工作盗窃算法就是，**「某个线程从其他队列中窃取任务进行执行的过程」**。一般就是指做得快的线程（盗窃线程）抢慢的线程的任务来做，同时为了减少锁竞争，通常使用双端队列，即快线程和慢线程各在一端。

17. CAS了解吗?

- CAS：全称 `Compare and swap`，即**比较并交换**，它是一条 **CPU 同步原语**。是一种硬件对并发的支持，针对多处理器操作而设计的一种特殊指令，用于管理对共享数据的并发访问。
- CAS 是一种无锁的非阻塞算法的实现。
- CAS 包含了 3 个操作数：
- - 需要读写的内存值 V
  - 旧的预期值 A
  - 要修改的更新值 B
- 当且仅当 V 的值等于 A 时，CAS 通过原子方式用新值 B 来更新 V 的 值，否则不会执行任何操作（他的功能是判断内存某个位置的值是否为预期值，如果是则更改为新的值，这个过程是原子的。）

CAS 并发原语体现在 Java 语言中的 `sum.misc.Unsafe` 类中的各个方法。调用 Unsafe 类中的 CAS 方法， JVM 会帮助我们实现出 CAS 汇编指令。这是一种完全依赖于硬件的功能，通过它实现了原子操作。再次强调，由于 CAS是一种系统原语，**原语属于操作系统用于范畴，是由若干条指令组成的，用于完成某个功能的一个过程，并且原语的执行必须是连续的，在执行过程中不允许被中断**，CAS 是一条 CPU 的原子指令，不会造成数据不一致问题。

18. CAS有什么缺陷?



1. ABA 问题

并发环境下，假设初始条件是A，去修改数据时，发现是A就会执行修改。但是看到的虽然是A，中间可能发生了A变B，B又变回A的情况。此时A已经非彼A，数据即使成功修改，也可能有问题。

可以通过AtomicStampedReference**解决ABA问题**，它，一个带有标记的原子引用类，通过控制变量值的版本来保证CAS的正确性。

2. 循环时间长开销

自旋CAS，如果一直循环执行，一直不成功，会给CPU带来非常大的执行开销。

很多时候，CAS思想体现，是有个自旋次数的，就是为了避开这个耗时问题~

3. 只能保证一个变量的原子操作。

CAS 保证的是对一个变量执行操作的原子性，如果对多个变量操作时，CAS 目前无法直接保证操作的原子性的。

可以通过这两个方式解决这个问题：

- 使用互斥锁来保证原子性；
- 将多个变量封装成对象，通过AtomicReference来保证原子性。

19. synchronized 和 volatile 的区别是什么?

`volatile` 解决的是内存可见性问题，会使得所有对 `volatile` 变量的读写都直接写入主存，即 **保证了变量的可见性**。

`synchronized` 解决的事执行控制的问题，它会阻止其他线程获取当前对象的监控锁，这样一来就让当前对象中被 `synchronized` 关键字保护的代码块无法被其他线程访问，也就是无法并发执行。而且， `synchronized` 还会创建一个 **内存屏障**，内存屏障指令保证了所有 CPU 操作结果都会直接刷到主存中，从而 **保证操作的内存可见性**，同时也使得这个锁的线程的所有操作都 `happens-before` 于随后获得这个锁的线程的操作。

两者的区别主要有如下：

1. volatile 本质是在告诉 JVM 当前变量在寄存器（工作内存）中的值是不确定的，需要从主存中读取； synchronized 则是锁定当前变量，只有当前线程可以访问该变量，其他线程被阻塞住。
2. volatile **仅能使用在变量级别**；synchronized 则可以使用在 **变量. 方法. 和类级别的**
3. volatile 仅能实现变量的修改可见性，**不能保证原子性**；而synchronized 则可以 **保证变量的修改可见性和原子性**
4. volatile **不会造成线程的阻塞**；synchronized **可能会造成线程的阻塞**。
5. volatile 标记的变量不会被编译器优化；synchronized 标记的变量可以被编译器优化。

20. synchronized 和 Lock 有什么区别?

- synchronized 可以给类. 方法. 代码块加锁；而 lock 只能给代码块加锁。
- synchronized 不需要手动获取锁和释放锁，使用简单，发生异常会自动释放锁，不会造成死锁；而 lock 需要自己加锁和释放锁，如果使用不当没有 unLock()去释放锁就会造成死锁。
- 通过 Lock 可以知道有没有成功获取锁，而 synchronized 却无法办到。

21. synchronized 和 ReentrantLock 区别是什么?

1.两者都是可重入锁

可重入锁：重入锁，也叫做递归锁，可重入锁指的是在一个线程中可以多次获取同一把锁，比如：
一个线程在执行一个带锁的方法，该方法中又调用了另一个需要相同锁的方法，则该线程可以直接执行调用的方法，而无需重新获得锁，
两者都是同一个线程每进入一次，锁的计数器都自增1，所以要等到锁的计数器下降为0时才能释放锁。

2.synchronized 依赖于 JVM 而 ReentrantLock 依赖于 API

- synchronized 是依赖于 JVM 实现的，前面我们也讲到了 虚拟机团队在 JDK1.6 为 synchronized 关键字进行了很多优化，但是这些优化都是在虚拟机层面实现的
- ReentrantLock 是 JDK 层面实现的（也就是 API 层面，需要 lock() 和 unlock() 方法配合 try/finally 语句块来完成）

3.ReentrantLock 比 synchronized 增加了一些高级功能

相比synchronized，ReentrantLock增加了一些高级功能。主要来说主要有三点：①等待可中断；②可实现公平锁；③可实现选择性通知（锁可以绑定多个条件）

- 等待可中断.通过lock.lockInterruptibly()来实现这个机制。也就是说正在等待的线程可以选择放弃等待，改为处理其他事情。
- ReentrantLock可以指定是公平锁还是非公平锁。而synchronized只能是非公平锁。所谓的公平锁就是先等待的线程先获得锁。 ReentrantLock默认情况是非公平的，可以通过ReentrantLock类的ReentrantLock(boolean fair)构造方法来制定是否是公平的。
- ReentrantLock类线程对象可以注册在指定的Condition中，从而可以有选择性的进行线程通知，在调度线程上更加灵活。 在使用notify()/notifyAll()方法进行通知时，被通知的线程是由 JVM 选择的，**用ReentrantLock类结合Condition实例可以实现“选择性通知”**

4.使用选择

- 除非需要使用 ReentrantLock 的高级功能，否则优先使用 synchronized。
- synchronized 是 JVM 实现的一种锁机制，JVM 原生地支持它，而 ReentrantLock 不是所有的 JDK 版本都支持。并且使用 synchronized 不用担心没有释放锁而导致死锁问题，因为 JVM 会确保锁的释放

22. synchronized的用法有哪些?

- 修饰普通方法:作用于当前对象实例，进入同步代码前要获得当前对象实例的锁

- 修饰静态方法:作用于当前类，进入同步代码前要获得当前类对象的锁,synchronized 关键字加到 static 静态方法和 synchronized(class)代码块上都是是给 Class 类上锁
- 修饰代码块:指定加锁对象，对给定对象加锁，进入同步代码库前要获得给定对象的锁

特别注意：

①如果一个线程A调用一个实例对象的非静态 synchronized 方法，而线程B需要调用这个实例对象所属类的静态 synchronized 方法，是允许的，不会发生互斥现象，因为访问静态 synchronized 方法占用的锁是当前类的锁

②尽量不要使用 synchronized(String s) ,因为JVM中，字符串常量池具有缓冲功能

23. Synchronized的作用有哪些?

1. 原子性：确保线程互斥的访问同步代码；
2. 可见性：保证共享变量的修改能够及时可见，其实是通过Java内存模型中的 “**对一个变量unlock操作之前，必须要同步到主内存中；如果对一个变量进行lock操作，则将会清空工作内存中此变量的值，在执行引擎使用此变量前，需要重新从主内存中load操作或assign操作初始化变量值**” 来保证的；
3. 有序性：有效解决重排序问题，即 “一个unlock操作先行发生(happen-before)于后面对同一个锁的lock操作”。

24. 说一下 synchronized 底层实现原理?

synchronized 同步代码块的实现是通过 monitorenter 和 monitorexit 指令，其中 monitorenter 指令指向同步代码块的开始位置，monitorexit 指令则指明同步代码块的结束位置。当执行 monitorenter 指令时，线程试图获取锁也就是获取 monitor(monitor对象存在于每个Java对象的对象头中，synchronized 锁便是通过这种方式获取锁的，也是为什么Java中任意对象可以作为锁的原因) 的持有权。

其内部包含一个计数器，当计数器为0则可以成功获取，获取后将锁计数器设为1也就是加1。相应的在执行 monitorexit 指令后，将锁计数器设为0，表明锁被释放。如果获取对象锁失败，那当前线程就要阻塞等待，直到锁被另外一个线程释放为止

synchronized 修饰的方法并没有 monitorenter 指令和 monitorexit 指令，取得代之的确实是 ACC_SYNCHRONIZED 标识，该标识指明了该方法是一个同步方法，JVM 通过该 ACC_SYNCHRONIZED 访问标志来辨别一个方法是否声明为同步方法，从而执行相应的同步调用。

25. 多线程中 synchronized 锁升级的原理是什么?

synchronized 锁升级原理：在锁对象的对象头里面有一个 threadid 字段，在第一次访问的时候 threadid 为空，jvm 让其持有偏向锁，并将 threadid 设置为其线程 id，再次进入的时候会先判断 threadid 是否与其线程 id 一致，如果一致则可以直接使用此对象，如果不一致，则升级偏向锁为轻量级锁，通过自旋循环一定次数来获取锁，执行一定次数之后，如果还没有正常获取到要使用的对象，此时就会把锁从轻量级升级为重量级锁，此过程就构成了 synchronized 锁的升级。

锁的升级的目的：锁升级是为了减低了锁带来的性能消耗。在 Java 6 之后优化 synchronized 的实现方式，使用了偏向锁升级为轻量级锁再升级到重量级锁的方式，从而减低了锁带来的性能消耗。

26. synchronized为什么是非公平锁？非公平体现在哪些地方？

synchronized 的非公平其实在源码中应该有不少地方，因为设计者就没按公平锁来设计，核心有以下几个点：

1）当持有锁的线程释放锁时，该线程会执行以下两个重要操作：

1. 先将锁的持有者 owner 属性赋值为 null
2. 唤醒等待链表中的一个线程（假定继承者）。

在1和2之间，如果有其他线程刚好在尝试获取锁（例如自旋），则可以马上获取到锁。

2）当线程尝试获取锁失败，进入阻塞时，放入链表的顺序，和最终被唤醒的顺序是不一致的，也就是说你先进入链表，不代表你就会先被唤醒。

27. JVM对synchronized的优化有哪些？

从最近几个jdk版本中可以看出，Java的开发团队一直在对synchronized优化，其中最大的一次优化就是在jdk6的时候，新增了两个锁状态，通过锁消除、锁粗化、自旋锁等方法使用各种场景，给synchronized性能带来了很大的提升。

1. 锁膨胀

上面讲到锁有四种状态，并且会因实际情况进行膨胀升级，其膨胀方向是：**无锁——>偏向锁——>轻量级锁——>重量级锁**，并且膨胀方向不可逆。

偏向锁

一句话总结它的作用：**减少统一线程获取锁的代价。**在大多数情况下，锁不存在多线程竞争，总是由同一线程多次获得，那么此时就是偏向锁。

核心思想：

如果一个线程获得了锁，那么锁就进入偏向模式，此时 `Mark Word` 的结构也就变为偏向锁结构，**当该线程再次请求锁时，无需再做任何同步操作，即获取锁的过程只需要检查** `**Mark Word**` **的锁标记位为偏向锁以及当前线程ID等于** `**Mark Word**` **的ThreadID即可**，这样就省去了大量有关锁申请的操作。

轻量级锁

轻量级锁是由偏向锁升级而来，当存在第二个线程申请同一个锁对象时，偏向锁就会立即升级为轻量级锁。注意这里的第二个线程只是申请锁，不存在两个线程同时竞争锁，可以是一前一后地交替执行同步块。

重量级锁

重量级锁是由轻量级锁升级而来，当**同一时间**有多个线程竞争锁时，锁就会被升级成重量级锁，此时其申请锁带来的开销也就变大。

重量级锁一般使用场景会在追求吞吐量，同步块或者同步方法执行时间较长的场景。

2.锁消除

消除锁是虚拟机另外一种锁的优化，这种优化更彻底，在JIT编译时，对运行上下文进行扫描，去除不可能存在竞争的锁。比如下面代码的method1和method2的执行效率是一样的，因为object锁是私有变量，不存在所得竞争关系。

```java
public class Test4 {

    public void method1() {
        Object object = new Object();
        synchronized (object) {
            //  执行同步代码
            System.out.println("hello world.");
        }
    }

    //  优化后的方法，和上面method1执行效率一样
    public void method2() {
        Object object = new Object();
        System.out.println("hello world.");
    }
}
```

3. 锁粗化

锁粗化是虚拟机对另一种极端情况的优化处理，通过扩大锁的范围，避免反复加锁和释放锁。比如下面method3经过锁粗化优化之后就和method4执行效率一样了。

```java
public void method3() {
    for (int i = 0; i < 10000; ++i) {
        synchronized (Test4.class) {
            System.out.println("hello world.");
        }
    }
}

//  锁粗化，与上面一样
public void method4() {
    synchronized (Test4.class) {
        for (int i = 0; i < 10000; ++i) {
            System.out.println("hello world.");
        }
    }
}
```

4. 自旋锁与自适应自旋锁

轻量级锁失败后，虚拟机为了避免线程真实地在操作系统层面挂起，还会进行一项称为自旋锁的优化手段。

自旋锁：许多情况下，共享数据的锁定状态持续时间较短，切换线程不值得，通过让线程执行循环等待锁的释放，不让出CPU。如果得到锁，就顺利进入临界区。如果还不能获得锁，那就会将线程在操作系统层面挂起，这就是自旋锁的优化方式。但是它也存在缺点：如果锁被其他线程长时间占用，一直不释放CPU，会带来许多的性能开销。

自适应自旋锁：这种相当于是对上面自旋锁优化方式的进一步优化，它的自旋的次数不再固定，其自旋的次数由前一次在同一个锁上的自旋时间及锁的拥有者的状态来决定，这就解决了自旋锁带来的缺点。

为什么要引入偏向锁和轻量级锁？为什么重量级锁开销大？

重量级锁底层依赖于系统的同步函数来实现，在 linux 中使用 pthread_mutex_t（互斥锁）来实现。

这些底层的同步函数操作会涉及到：操作系统用户态和内核态的切换、进程的上下文切换，而这些操作都是比较耗时的，因此重量级锁操作的开销比较大。

而在很多情况下，可能获取锁时只有一个线程，或者是多个线程交替获取锁，在这种情况下，使用重量级锁就不划算了，因此引入了偏向锁和轻量级锁来降低没有并发竞争时的锁开销。

28. synchronized 锁能降级吗？

可以的。

具体的触发时机：在全局安全点（safepoint）中，执行清理任务的时候会触发尝试降级锁。

当锁降级时，主要进行了以下操作：

1）恢复锁对象的 markword 对象头；

2）重置 ObjectMonitor，然后将该 ObjectMonitor 放入全局空闲列表，等待后续使用。

29. ThreadLocal是什么？

ThreadLocal，即线程本地变量。如果你创建了一个ThreadLocal变量，那么访问这个变量的每个线程都会有这个变量的一个本地拷贝，多个线程操作这个变量的时候，实际是操作自己本地内存里面的变量，从而起到线程隔离的作用，避免了线程安全问题。

```
//创建一个ThreadLocal变量
static ThreadLocal<String> localVariable = new ThreadLocal<>();
```

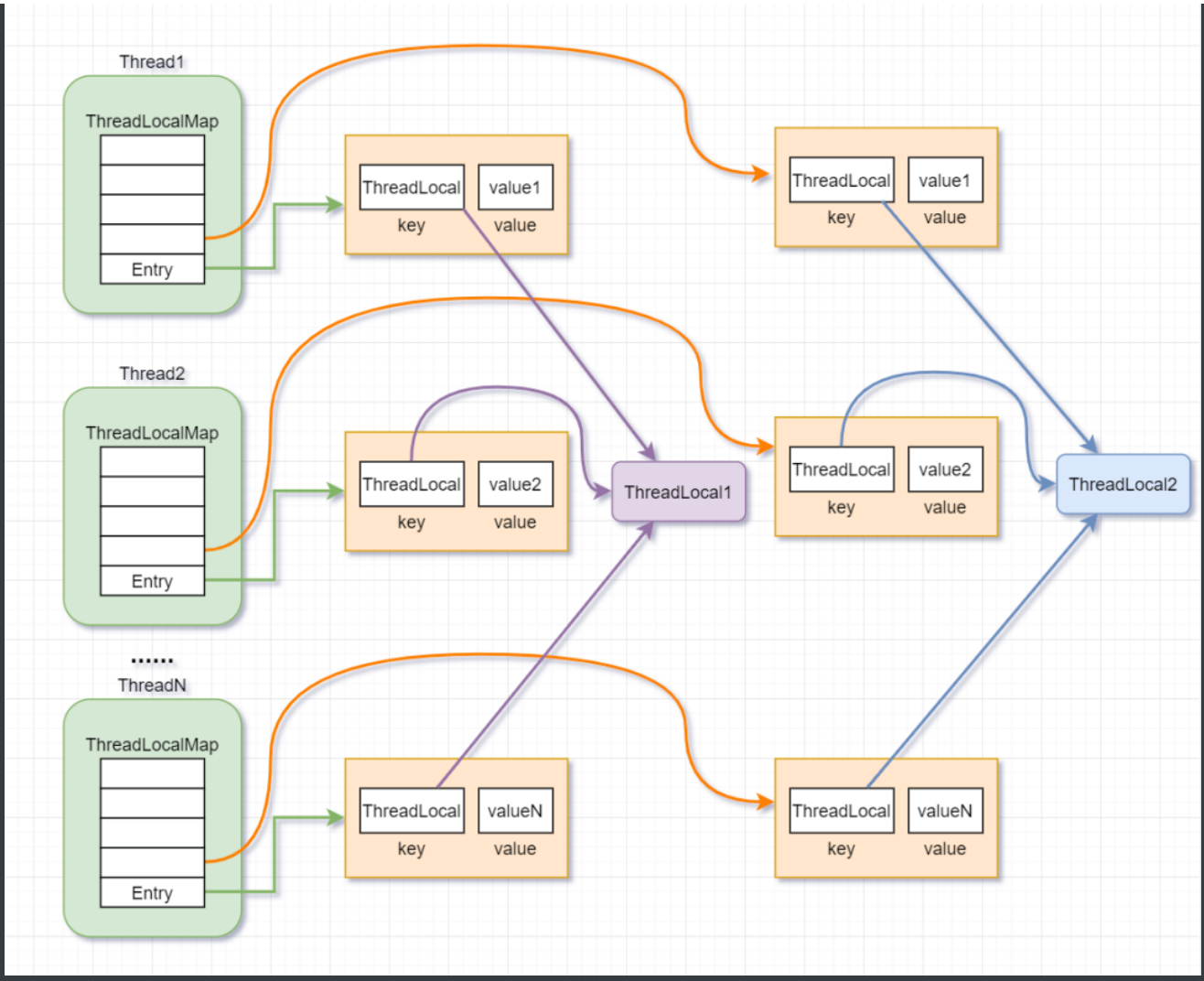
ThreadLocal的应用场景有

- 数据库连接池
- 会话管理中使用

30. ThreadLocal的实现原理

- Thread类有一个类型为ThreadLocal.ThreadLocalMap的实例变量threadLocals，即每个线程都有一个属于自己的ThreadLocalMap。
- ThreadLocalMap内部维护着Entry数组，每个Entry代表一个完整的对象，key是ThreadLocal本身，value是ThreadLocal的泛型值。
- 每个线程在往ThreadLocal里设置值的时候，都是往自己的ThreadLocalMap里存，读也是以某个ThreadLocal作为引用，在自己的map里找对应的key，从而实现了线程隔离。

ThreadLocal内存结构图：

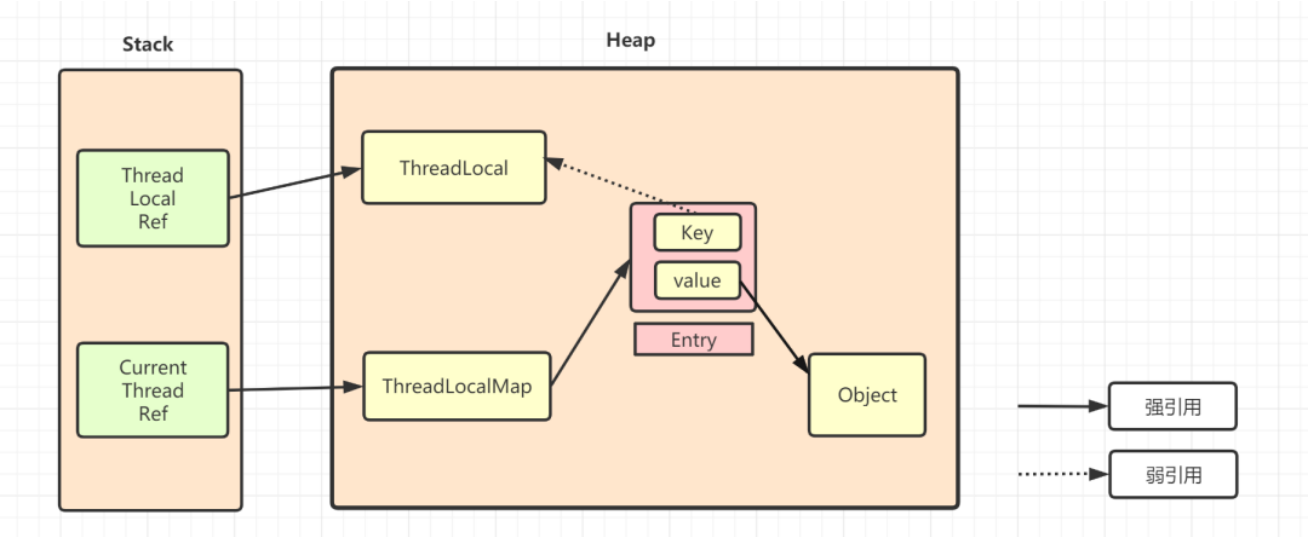


由结构图是可以看出：

- Thread对象中持有一个ThreadLocal.ThreadLocalMap的成员变量。
- ThreadLocalMap内部维护了Entry数组，每个Entry代表一个完整的对象，key是ThreadLocal本身，value是ThreadLocal的泛型值。

31. 知道ThreadLocal 内存泄露问题吗?

先看看一下的TreadLocal的引用示意图哈，



ThreadLocalMap中使用的 key 为 ThreadLocal 的弱引用，如下：

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
}
```

> 弱引用：只要垃圾回收机制一运行，不管JVM的内存空间是否充足，都会回收该对象占用的内存。

弱引用比较容易被回收。因此，如果ThreadLocal（ThreadLocalMap的Key）被垃圾回收器回收了，但是因为ThreadLocalMap生命周期和Thread是一样的，它这时候如果不被回收，就会出现这种情况：ThreadLocalMap的key没了，value还在，这就会**「造成了内存泄漏问题」**。

如何**「解决内存泄漏问题」**？使用完ThreadLocal后，及时调用remove()方法释放内存空间。

32. 了解ReentrantLock吗?

ReetrantLock是一个可重入的独占锁，主要有两个特性，一个是支持公平锁和非公平锁，一个是可重入。
ReetrantLock实现依赖于AQS(AbstractQueuedSynchronizer)。

ReetrantLock主要依靠AQS维护一个阻塞队列，多个线程对加锁时，失败则会进入阻塞队列。等待唤醒，重新尝试加锁。

33. ReadWriteLock是什么?

首先ReentrantLock某些时候有局限，如果使用ReentrantLock，可能本身是为了防止线程A在写数据、线程B在读数据造成的数据不一致，但这样，如果线程C在读数据、线程D也在读数据，读数据是不会改变数据的，没有必要加锁，但是还是加锁了，降低了程序的性能。

因为这个，才诞生了读写锁ReadWriteLock。ReadWriteLock是一个读写锁接口，ReentrantReadWriteLock是ReadWriteLock接口的一个具体实现，实现了读写的分离，读锁是共享的，写锁是独占的，读和读之间不会互斥，读和写、写和读、写和写之间才会互斥，提升了读写的性能

线程池专题

1. 为什么要用线程池?

线程池提供了一种限制和管理资源（包括执行一个任务）。 每个线程池还维护一些基本统计信息，例如已完成任务的数量。

使用线程池的好处：

- **降低资源消耗。** 通过重复利用已创建的线程降低线程创建和销毁造成的消耗。
- **提高响应速度。** 当任务到达时，任务可以不需要的等到线程创建就能立即执行。
- **提高线程的可管理性。** 线程是稀缺资源，如果无限制的创建，不仅会消耗系统资源，还会降低系统的稳定性，使用线程池可以进行统一的分配，调优和监控。

2. 执行execute()方法和submit()方法的区别是什么呢?

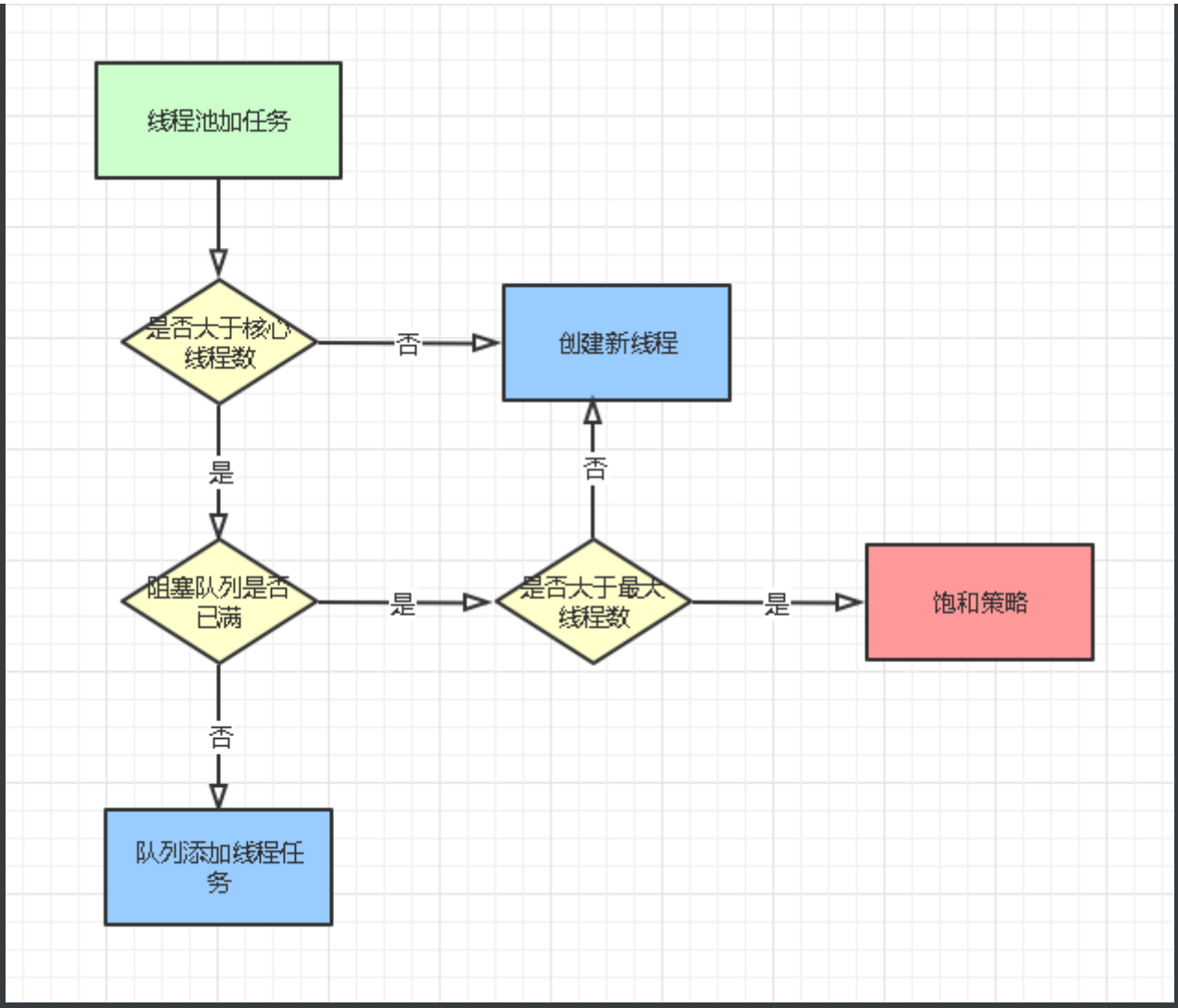
- **`execute()` 方法用于提交不需要返回值的任务，所以无法判断任务是否被线程池执行成功与否；**

- **submit()方法用于提交需要返回值的任务。线程池会返回一个future类型的对象，通过这个future对象可以判断任务是否执行成功**，并且可以通过future的get()方法来获取返回值，get()方法会阻塞当前线程直到任务完成，而使用 `get (long timeout, TimeUnit unit)` 方法则会阻塞当前线程一段时间后立即返回，这时候有可能任务没有执行完。

3. 说下线程池核心参数?

- corePoolSize： 核心线程大小。线程池一直运行，核心线程就不会停止。
- maximumPoolSize： 线程池最大线程数量。非核心线程数量=maximumPoolSize-corePoolSize
- keepAliveTime： 非核心线程的心跳时间。如果非核心线程在keepAliveTime内没有运行任务，非核心线程会消亡。
- workQueue： 阻塞队列。ArrayBlockingQueue，LinkedBlockingQueue等，用来存放线程任务。
- defaultHandler： 饱和策略。ThreadPoolExecutor类中一共有4种饱和策略。通过实现**RejectedExecutionHandler**接口。
  - AbortPolicy： 线程任务丢弃报错。默认饱和策略。
  - DiscardPolicy： 线程任务直接丢弃不报错。
  - DiscardOldestPolicy： 将workQueue**队首任务丢弃**，将最新线程任务重新加入队列执行。
  - CallerRunsPolicy： 线程池之外的线程直接调用run方法执行。
- ThreadFactory： 线程工厂。新建线程工厂。

4. 线程池执行任务的流程?



1. 线程池执行execute/submit方法向线程池添加任务，当任务小于核心线程数corePoolSize，线程池中可以创建新的线程。
2. 当任务大于核心线程数corePoolSize，就向阻塞队列添加任务。
3. 如果阻塞队列已满，需要通过比较参数maximumPoolSize，在线程池创建新的线程，当线程数量大于maximumPoolSize，说明当前设置线程池中线程已经处理不了了，就会执行饱和策略。

5. 常用的JAVA线程池有哪几种类型?

1、newCachedThreadPool

创建一个可缓存线程池，如果线程池长度超过处理需要，可灵活回收空闲线程，若无可回收，则新建线程。

这种类型的线程池特点是：

工作线程的创建数量几乎没有限制(其实也有限制的,数目为Interger. MAX_VALUE), 这样可灵活的往线程池中添加线程。

如果长时间没有往线程池中提交任务，即如果工作线程空闲了指定的时间(默认为1分钟)，则该工作线程将自动终止。终止后，如果你又提交了新的任务，则线程池重新创建一个工作线程。

在使用CachedThreadPool时，一定要注意控制任务的数量，否则，由于大量线程同时运行，很有会造成系统OOM。

2、newFixedThreadPool

创建一个指定工作线程数量的线程池。每当提交一个任务就创建一个工作线程，如果工作线程数量达到线程池初始的最大数，则将提交的任务存入到池队列中。

FixedThreadPool是一个典型且优秀的线程池，它具有线程池提高程序效率和节省创建线程时所耗的开销的优点。但是，在线程池空闲时，即线程池中没有可运行任务时，它不会释放工作线程，还会占用一定的系统资源。

3、newSingleThreadExecutor

创建一个单线程化的Executor，即只创建唯一的工作者线程来执行任务，它只会用唯一的工作线程来执行任务，保证所有任务按照指定顺序(FIFO, LIFO, 优先级)执行。如果这个线程异常结束，会有另一个取代它，保证顺序执行。单工作线程最大的特点是可保证顺序地执行各个任务，并且在任意给定的时间不会有多个线程是活动的。

4、newScheduleThreadPool

创建一个定长的线程池，而且支持定时的以及周期性的任务执行，支持定时及周期性任务执行。

6. 线程池常用的阻塞队列有哪些?

FixedThreadPool	LinkedBlockingQueue
SingleThreadExecutor	LinkedBlockingQueue
CachedThreadPool	SynchronousQueue
ScheduledThreadPool	DelayedWorkQueue
SingleThreadScheduledExecutor	DelayedWorkQueue

https://blog.csdn.net/a904364908

表格左侧是线程池，右侧为它们对应的阻塞队列，可以看到 5 种线程池对应了 3 种阻塞队列

1. LinkedBlockingQueue

   对于 FixedThreadPool 和 SingleThreadExector 而言，它们使用的阻塞队列是容量为 Integer.MAX_VALUE 的 LinkedBlockingQueue，可以认为是无界队列。由于 FixedThreadPool 线程池的线程数是固定的，所以没有办法增加特别多的线程来处理任务，这时就需要 LinkedBlockingQueue 这样一个没有容量限制的阻塞队列来存放任务。

   这里需要注意，由于线程池的任务队列永远不会放满，所以线程池只会创建核心线程数量的线程，所以此时的最大线程数对线程池来说没有意义，因为并不会触发生成多于核心线程数的线程。

2. SynchronousQueue

   第二种阻塞队列是 SynchronousQueue，对应的线程池是 CachedThreadPool。线程池 CachedThreadPool 的最大线程数是 Integer 的最大值，可以理解为线程数是可以无限扩展的。CachedThreadPool 和上一种线程池 FixedThreadPool 的情况恰恰相反，FixedThreadPool 的情况是阻塞队列的容量是无限的，而这里 CachedThreadPool 是线程数可以无限扩展，所以 CachedThreadPool 线程池并不需要一个任务队列来存储任务，因为一旦有任务被提交就直接转发给线程或者创建新线程来执行，而不需要另外保存它们。

   我们自己创建使用 SynchronousQueue 的线程池时，如果不希望任务被拒绝，那么就需要注意设置最大线程数要尽可能大一些，以免发生任务数大于最大线程数时，没办法把任务放到队列中也没有足够线程来执行任务的情况。

3. DelayedWorkQueue

   第三种阻塞队列是DelayedWorkQueue，它对应的线程池分别是 ScheduledThreadPool 和 SingleThreadScheduledExecutor，这两种线程池的最大特点就是可以延迟执行任务，比如说一定时间后执行任务或是每隔一定的时间执行一次任务。

DelayedWorkQueue 的特点是内部元素并不是按照放入的时间排序，而是会按照延迟的时间长短对任务进行排序，内部采用的是“堆”的数据结构。之所以线程池 ScheduledThreadPool 和 SingleThreadScheduledExecutor 选择 DelayedWorkQueue，是因为它们本身正是基于时间执行任务的，而延迟队列正好可以把任务按时间进行排序，方便任务的执行。

7. 源码中线程池是怎么复用线程的?

源码中ThreadPoolExecutor中有个内置对象Worker，每个worker都是一个线程，worker线程数量和参数有关，每个worker会while死循环从阻塞队列中取数据，**通过置换worker中Runnable对象，运行其run方法起到线程置换的效果**，这样做的好处是避免多线程频繁线程切换，提高程序运行性能。

8. 如何合理配置线程池参数?

自定义线程池就需要我们自己配置最大线程数 maximumPoolSize ，为了高效的并发运行，这时需要看我们的业务是IO密集型还是CPU密集型。

CPU密集型
CPU密集的意思是该任务需要最大的运算，而没有阻塞，CPU一直全速运行。CPU密集任务只有在真正的多核CPU上才能得到加速(通过多线程)。而在单核CPU上，无论你开几个模拟的多线程该任务都不可能得到加速，因为CPU总的运算能力就那么多。

IO密集型
IO密集型，即该任务需要大量的IO，即大量的阻塞。在单线程上运行IO密集型的任务会导致大量的CPU运算能力浪费在等待。所以在IO密集型任务中使用多线程可以大大的加速程序运行，即使在单核CPU上这种加速主要就是利用了被浪费掉的阻塞时间。

IO 密集型时，大部分线程都阻塞，故需要多配制线程数。公式为：

```
CPU核数*2
CPU核数/(1-阻塞系数) 阻塞系数在0.8~0.9之间
查看CPU核数:
System.out.println(Runtime.getRuntime().availableProcessors());
```

当以上都不适用时，选用动态化线程池，看美团技术团队的实践：https://tech.meituan.com/2020/04/02/java-pooling-pratice-in-meituan.html

9. Executor和Executors的区别?

Executors 工具类的不同方法按照我们的需求创建了不同的线程池，来满足业务的需求。

Executor 接口对象能执行我们的线程任务。ExecutorService接口继承了Executor接口并进行了扩展，提供了更多的方法我们能获得任务执行的状态并且可以获取任务的返回值。

使用ThreadPoolExecutor 可以创建自定义线程池。Future 表示异步计算的结果，他提供了检查计算是否完成的方法，以等待计算的完成，并可以使用get()方法获取计算的结果。

AQS

1. 说一说什么是AQS?

1. AQS 是一个锁框架，它定义了锁的实现机制，并开放出扩展的地方，让子类去实现，比如我们在 lock 的时候，AQS 开放出 state 字段，让子类可以根据 state 字段来决定是否能够获得锁，对于获取不到锁的线程 AQS 会自动进行管理，无需子类锁关心，这就是 lock 时锁的内部机制，封装的很好，又暴露出子类锁需要扩展的地方；
2. AQS 底层是由同步队列 + 条件队列联手组成，同步队列管理着获取不到锁的线程的排队和释放，条件队列是在一定场景下，对同步队列的补充，比如获得锁的线程从空队列中拿数据，肯定是拿不到数据的，这时候条件队列就会管理该线程，使该线程阻塞；
3. AQS 围绕两个队列，提供了四大场景，分别是：获得锁、释放锁、条件队列的阻塞，条件队列的唤醒，分别对应着 AQS 架构图中的四种颜色的线的走向。

2. AQS使用了哪些设计模式?

AQS同步器的设计是基于模板方法模式的，如果需要自定义同步器一般的方式是这样（模板方法模式很经典的一个应用）：

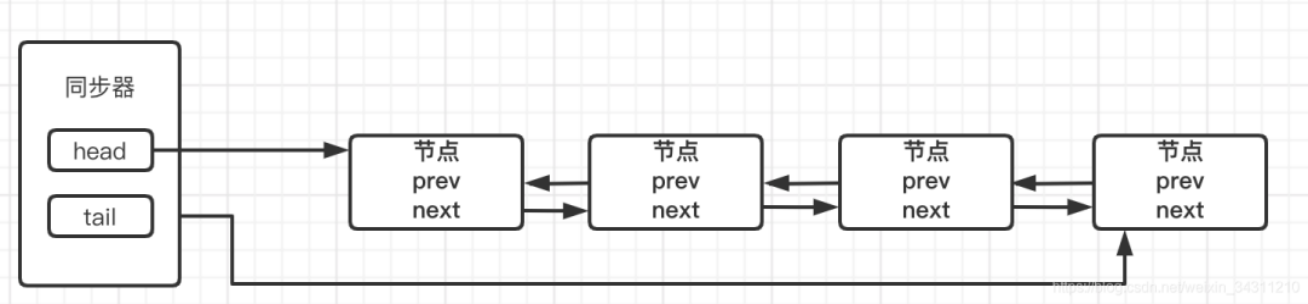
1. 使用者继承AbstractQueuedSynchronizer并重写指定的方法。（这些重写方法很简单，无非是对于共享资源state的获取和释放）
2. 将AQS组合在自定义同步组件的实现中，并调用其模板方法，而这些模板方法会调用使用者重写的方法。

这和我们以往通过实现接口的方式有很大区别，这是模板方法模式很经典的一个运用。

AQS使用了模板方法模式，自定义同步器时需要重写下面几个AQS提供的模板方法：

```
isHeldExclusively()//该线程是否正在独占资源。只有用到condition才需要去实现它。
tryAcquire(int)//独占方式。尝试获取资源，成功则返回true，失败则返回false。
tryRelease(int)//独占方式。尝试释放资源，成功则返回true，失败则返回false。
tryAcquireShared(int)//共享方式。尝试获取资源。负数表示失败；0表示成功，但没有剩余可用资源；正数表示成功，且有剩余资源。
tryReleaseShared(int)//共享方式。尝试释放资源，成功则返回true，失败则返回false。
```

3. 了解AQS中同步队列的数据结构吗?



- 当前线程获取同步状态失败，同步器将当前线程机等待状态等信息构造成一个Node节点加入队列，放在队尾，同步器重新设置尾节点
- 加入队列后，会阻塞当前线程
- 同步状态被释放并且同步器重新设置首节点，同步器唤醒等待队列中第一个节点，让其再次获取同步状态

4. 了解AQS对资源的共享方式吗?

AQS定义两种资源共享方式

- Exclusive

  （独占）：只有一个线程能执行，如ReentrantLock。又可分为公平锁和非公平锁：

  - 公平锁：按照线程在队列中的排队顺序，先到者先拿到锁
  - 非公平锁：当线程要获取锁时，无视队列顺序直接去抢锁，谁抢到就是谁的
- **Share**（共享）：多个线程可同时执行，如Semaphore/CountDownLatch。Semaphore、CountDownLatCh、 CyclicBarrier、ReadWriteLock 我们都会在后面讲到。

ReentrantReadWriteLock 可以看成是组合式，因为ReentrantReadWriteLock也就是读写锁允许多个线程同时对某一资源进行读。

不同的自定义同步器争用共享资源的方式也不同。自定义同步器在实现时只需要实现共享资源 state 的获取与释放方式即可，至于具体线程等待队列的维护（如获取资源失败入队/唤醒出队等），AQS已经在顶层实现好了。

5. AQS 组件了解吗?

- **Semaphore(信号量)-允许多个线程同时访问：** synchronized 和 ReentrantLock 都是一次只允许一个线程访问某个资源，Semaphore(信号量)可以指定多个线程同时访问某个资源。
- **CountDownLatch （倒计时器）：** CountDownLatch是一个同步工具类，用来协调多个线程之间的同步。这个工具通常用来控制线程等待，它可以让某一个线程等待直到倒计时结束，再开始执行。
- **CyclicBarrier(循环栅栏)：** CyclicBarrier 和 CountDownLatch 非常类似，它也可以实现线程间的技术等待，但是它的功能比 CountDownLatch 更加复杂和强大。主要应用场景和 CountDownLatch 类似。CyclicBarrier 的字面意思是可循环使用（Cyclic）的屏障（Barrier）。它要做的事情是，让一组线程到达一个屏障（也可以叫同步点）时被阻塞，直到最后一个线程到达屏障时，屏障才会开门，所有被屏障拦截的线程才会继续干活。CyclicBarrier默认的构造方法是 CyclicBarrier(int parties)，其参数表示屏障拦截的线程数量，每个线程调用await方法告诉 CyclicBarrier 我已经到达了屏障，然后当前线程被阻塞。

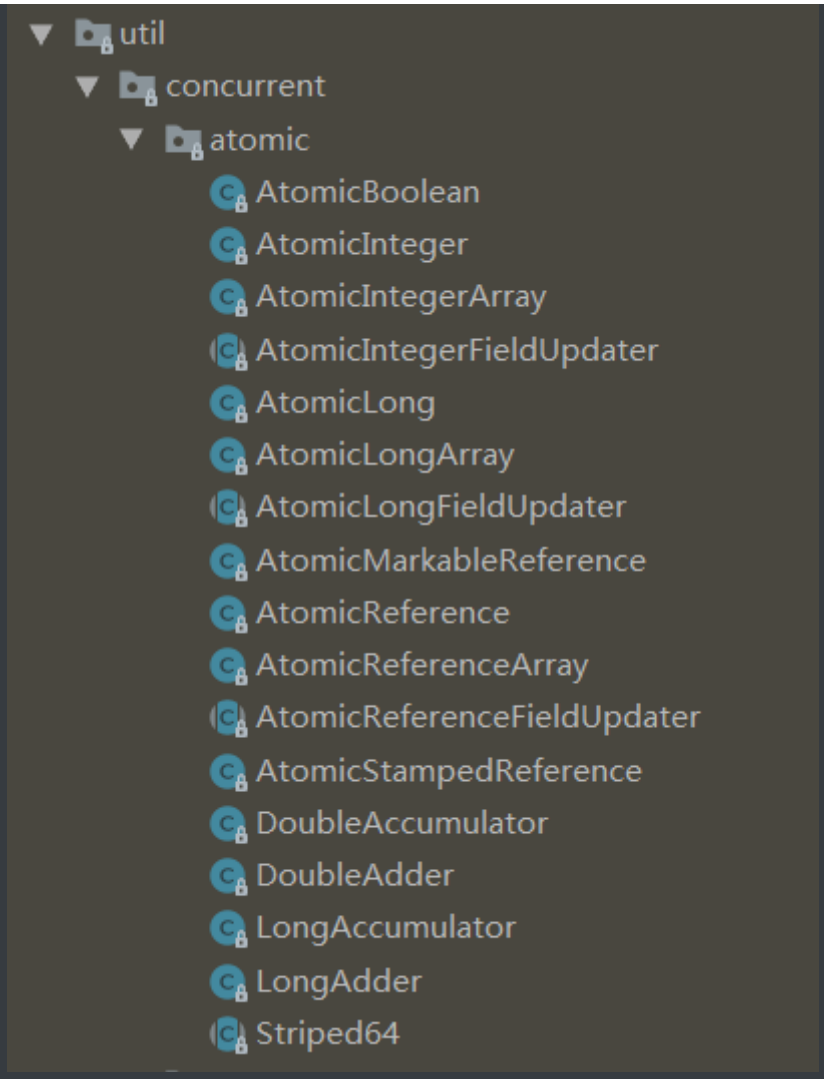
Atomic 原子类

1. 介绍一下 Atomic 原子类

Atomic 是指一个操作是不可中断的。即使是在多个线程一起执行的时候，一个操作一旦开始，就不会被其他线程干扰。

所以，所谓原子类说简单点就是具有原子 / 原子操作特征的类。

并发包 java.util.concurrent 的原子类都存放在 java.util.concurrent.atomic 下：



2. JUC包中的原子类是哪4类?

基本类型
使用原子的方式更新基本类型:

- AtomicInteger： 整型原子类
- AtomicLong: 长整型原子类
- AtomicBoolean: 布尔型原子类

数组类型
使用原子的方式更新数组里的某个元素:

- AtomicIntegerArray: 整型数组原子类
- AtomicLongArray: 长整型数组原子类
- AtomicReferenceArray: 引用类型数组原子类

引用类型
使用原子的方式更新引用类型:

- AtomicReference： 引用类型原子类
- AtomicStampedReference： 原子更新带有版本号的引用类型。该类将整型数值与引用关联起来，可用于解决原子的更新数据和数据的版本号，可以解决使用 CAS 进行原子更新时可能出现的 ABA 问题。
- AtomicMarkableReference： 原子更新带有标记位的引用类型。**对象属性修改类型**
- AtomicIntegerFieldUpdater： 原子更新整型字段的更新器
- AtomicLongFieldUpdater： 原子更新长整型字段的更新器
- AtomicMarkableReference： 原子更新带有标记位的引用类型

3. 简单介绍一下 AtomicInteger 类的原理

AtomicInteger 类主要利用 CAS和 volatile 和 native 方法来保证原子操作，从而避免 synchronized 的高开销，执行效率大为提升。

`AtomicInteger` 类的部分源码：

```
    // 更新操作时提供“比较并替换”的作用
    private static final Unsafe unsafe = Unsafe.getUnsafe();

    private static final long valueOffset;

    static {
        try{
            valueOffset =
  unsafe.objectFieldOffset(AutomicInteger.class.getDeclaredField("value"));
        }catch(Exception ex){
            throw new Error(ex);
        }
    }

    private volatile int value;
```

End

以上是Java并发-基础的常见面试题，Java并发其他部分正在整理，敬请期待。

若有更新，将第一时间同步公众号【程序员库森】，扫描下方左侧二维码关注！

此外，我还创建了求职交流群，群内分享内推、面试等资源，一起来抱团取暖啊。扫描下方右侧二维码，备注【加群】，拉你进群~



五、MySQL

索引

1. 索引是什么?

索引是一种特殊的文件(InnoDB数据表上的索引是表空间的一个组成部分)，它们包含着对数据表里所有记录的引用指针。

索引是一种数据结构。数据库索引，是数据库管理系统中一个排序的数据结构，以协助快速查询、更新数据库表中数据。索引的实现通常使用B树及其变种B+树。更通俗的说，索引就相当于目录。为了方便查找书中的内容，通过对内容建立索引形成目录。而且索引是一个文件，它是要占据物理空间的。

MySQL索引的建立对于MySQL的高效运行是很重要的，索引可以大大提高MySQL的检索速度。比如我们在查字典的时候，前面都有检索的拼音和偏旁、笔画等，然后找到对应字典页码，这样然后就打开字典的页数就可以知道我们要搜索的某一个key的全部值的信息了。

2. 索引有哪些优缺点?

索引的优点

- 可以大大加快数据的检索速度，这也是创建索引的最主要的原因。
- 通过使用索引，可以在查询的过程中，使用优化隐藏器，提高系统的性能。

索引的缺点

- 时间方面：创建索引和维护索引要耗费时间，具体地，当对表中的数据进行增加、删除和修改的时候，索引也要动态的维护，会降低增/改/删的执行效率；
- 空间方面：索引需要占物理空间。

3. MySQL有哪几种索引类型?

1、从存储结构上来划分：BTree索引（B-Tree或B+Tree索引），Hash索引，full-index全文索引，R-Tree索引。这里所描述的是索引存储时保存的形式，

2、从应用层次来分：普通索引，唯一索引，复合索引。

- 普通索引：即一个索引只包含单个列，一个表可以有多个单列索引
- 唯一索引：索引列的值必须唯一，但允许有空值
- 复合索引：多列值组成一个索引，专门用于组合搜索，其效率大于索引合并
- 聚簇索引(聚集索引)：并不是一种单独的索引类型，而是一种数据存储方式。具体细节取决于不同的实现，InnoDB的聚簇索引其实就是在同一个结构中保存了B-Tree索引(技术上来说是B+Tree)和数据行。
- 非聚簇索引： 不是聚簇索引，就是非聚簇索引

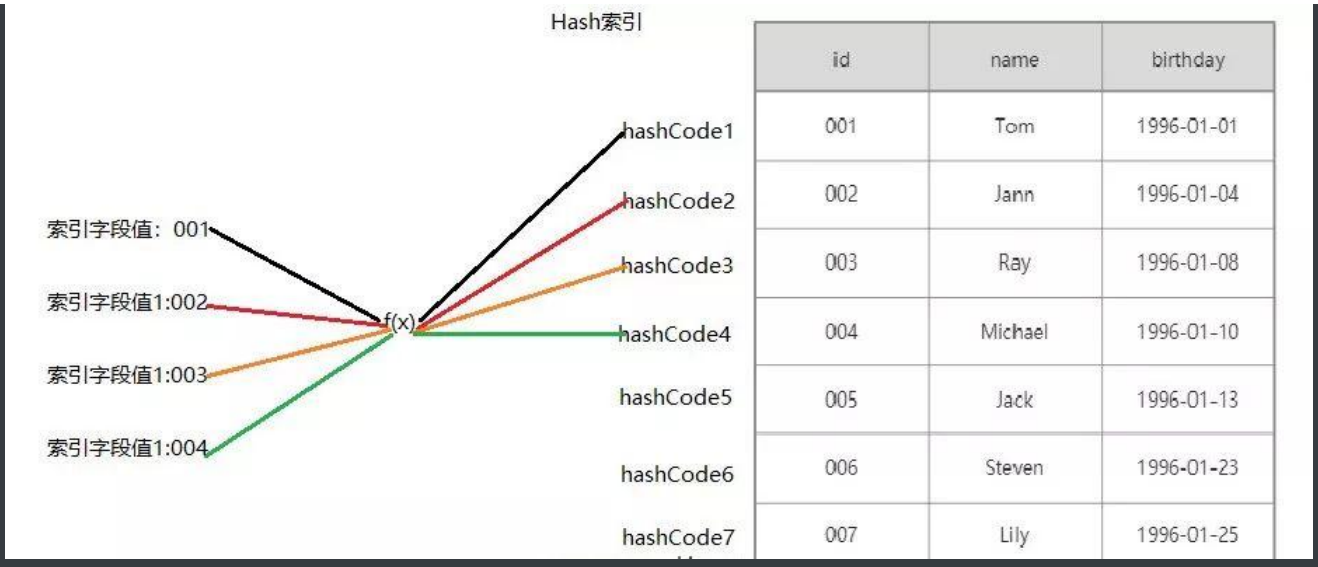
3、根据中数据的物理顺序与键值的逻辑（索引）顺序关系： 聚集索引，非聚集索引。

4. 说一说索引的底层实现?

Hash索引

基于哈希表实现，只有精确匹配索引所有列的查询才有效，对于每一行数据，存储引擎都会对所有的索引列计算一个哈希码（hash code），并且Hash索引将所有的哈希码存储在索引中，同时在索引表中保存指向每个数据行的指针。

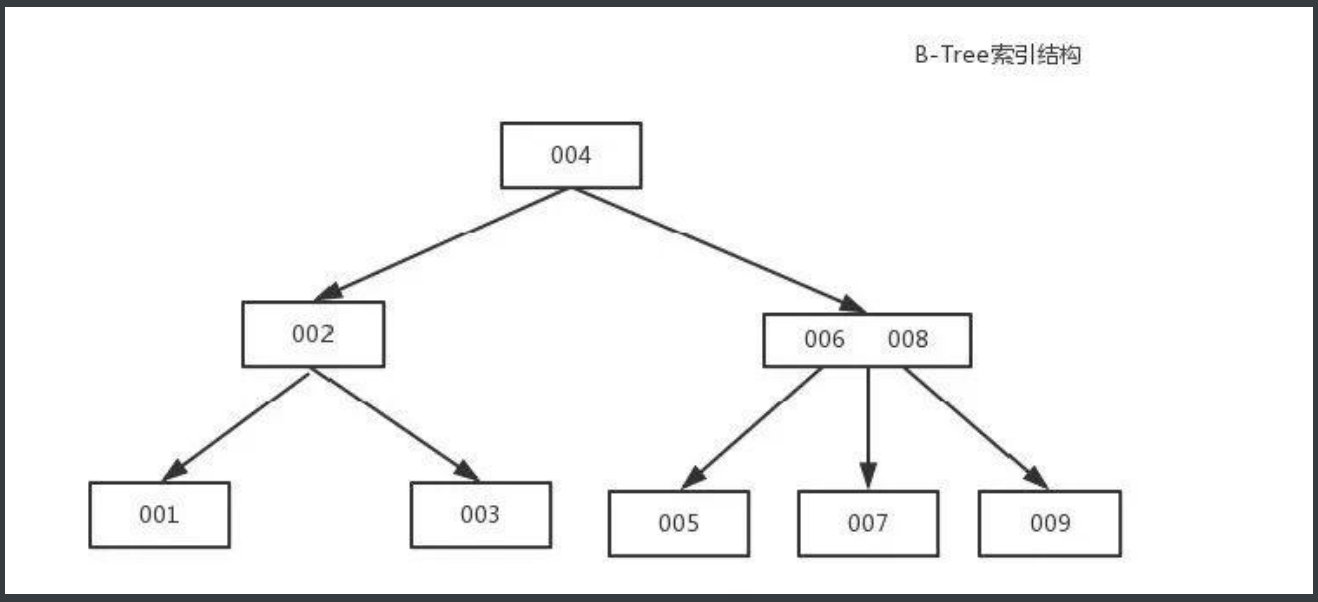
> 图片来源：https://www.javazhiyin.com/40232.html



id	name	birthday
001	Tom	1996-01-01
002	Jann	1996-01-04
003	Ray	1996-01-08
004	Michael	1996-01-10
005	Jack	1996-01-13
006	Steven	1996-01-23
007	Lily	1996-01-25

B-Tree索引（MySQL使用B+Tree）

B-Tree能加快数据的访问速度，因为存储引擎不再需要进行全表扫描来获取数据，数据分布在各个节点之中。



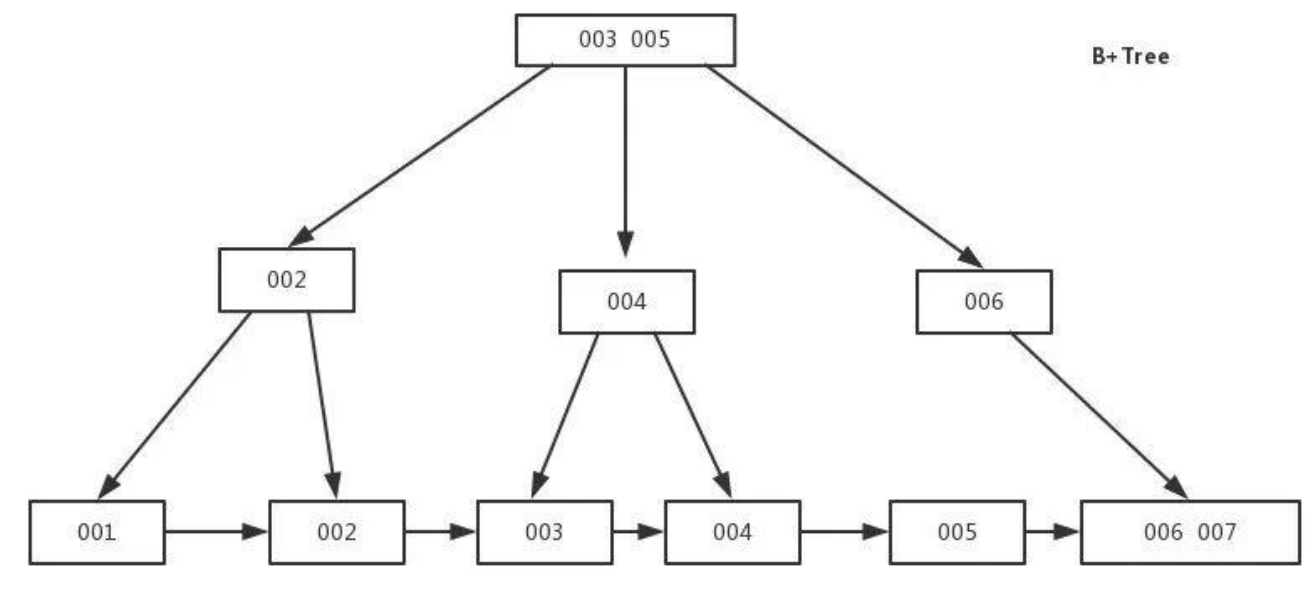
B+Tree索引

是B-Tree的改进版本，同时也是数据库索引索引所采用的存储结构。数据都在叶子节点上，并且增加了顺序访问指针，每个叶子节点都指向相邻的叶子节点的地址。相比B-Tree来说，进行范围查找时只需要查找两个节点，进行遍历即可。而B-Tree需要获取所有节点，相比之下B+Tree效率更高。

B+tree性质：

- n棵子tree的节点包含n个关键字，不用来保存数据而是保存数据的索引。

- 所有的叶子结点中包含了全部关键字的信息，及指向含这些关键字记录的指针，且叶子结点本身依关键字的大小自小而大顺序链接。
- 所有的非终端结点可以看成是索引部分，结点中仅含其子树中的最大（或最小）关键字。
- B+ 树中，数据对象的插入和删除仅在叶节点上进行。
- B+树有2个头指针，一个是树的根节点，一个是最小关键码的叶节点。



5. 为什么索引结构默认使用B+Tree，而不是B-Tree，Hash，二叉树，红黑树?

B-tree： 从两个方面来回答

- B+树的磁盘读写代价更低：B+树的内部节点并没有指向关键字具体信息的指针，因此其内部节点相对B(B-)树更小，如果把所有同一内部节点的关键字存放在同一盘块中，那么盘块所能容纳的关键字数量也越多，一次性读入内存的需要查找的关键字也就越多，相对 IO读写次数就降低 了。
- 由于B+树的数据都存储在叶子结点中，分支结点均为索引，方便扫库，只需要扫一遍叶子结点即可，但是B树因为其分支结点同样存储着数据，我们要找到具体的数据，需要进行一次中序遍历按序来扫，所以B+树更加适合在 区间查询 的情况，所以通常B+树用于数据库索引。

Hash：

- 虽然可以快速定位，但是没有顺序，IO复杂度高；

- 基于Hash表实现，只有Memory存储引擎显式支持哈希索引；
- 适合**等值查询**，如=、in()、<=>，不支持范围查询；

- 因为不是按照索引值顺序存储的，就不能像B+Tree索引一样利用索引完成排序；
- Hash索引在查询等值时非常快；
- 因为Hash索引始终索引的**所有列的全部内容**，所以不支持部分索引列的匹配查找；
- 如果有大量重复键值得情况下，哈希索引的效率会很低，因为存在哈希碰撞问题 。

二叉树： 树的高度不均匀，不能自平衡，查找效率跟数据有关（树的高度），并且IO代价高。

红黑树： 树的高度随着数据量增加而增加，IO代价高。

6.讲一讲聚簇索引与非聚簇索引?

在 InnoDB 里，索引B+ Tree的叶子节点存储了整行数据的是主键索引，也被称之为聚簇索引，即将数据存储与索引放到了一块，找到索引也就找到了数据。

而索引B+ Tree的叶子节点存储了主键的值的是非主键索引，也被称之为非聚簇索引、二级索引。

聚簇索引与非聚簇索引的区别：

- 非聚集索引与聚集索引的区别在于非聚集索引的叶子节点不存储表中的数据，而是存储该列对应的主键（行号）
- 对于InnoDB来说，想要查找数据我们还需要根据主键再去聚集索引中进行查找，这个再根据聚集索引查找数据的过程，我们称为**回表**。第一次索引一般是顺序IO，回表的操作属于随机IO。需要回表的次数越多，即随机IO次数越多，我们就越倾向于使用全表扫描。
- 通常情况下， 主键索引（聚簇索引）查询只会查一次，而非主键索引（非聚簇索引）需要回表查询多次。当然，如果是覆盖索引的话，查一次即可
- 注意：MyISAM无论主键索引还是二级索引都是非聚簇索引，而InnoDB的主键索引是聚簇索引，二级索引是非聚簇索引。我们自己建的索引基本都是非聚簇索引。

7.非聚簇索引一定会回表查询吗?

不一定，这涉及到查询语句所要求的字段是否全部命中了索引，如果全部命中了索引，那么就不必再进行回表查询。一个索引包含（覆盖）所有需要查询字段的值，被称之为"覆盖索引"。

举个简单的例子，假设我们在员工表的年龄上建立了索引，那么当进行 `select score from student where score > 90` 的查询时，在索引的叶子节点上，已经包含了score 信息，不会再次进行回表查询。

8. 联合索引是什么？为什么需要注意联合索引中的顺序?

MySQL可以使用多个字段同时建立一个索引，叫做联合索引。在联合索引中，如果想要命中索引，需要按照建立索引时的字段顺序挨个使用，否则无法命中索引。

具体原因为:

MySQL使用索引时需要索引有序，假设现在建立了"name，age，school"的联合索引，那么索引的排序为: 先按照name排序，如果name相同，则按照age排序，如果age的值也相等，则按照school进行排序。

当进行查询时，此时索引仅仅按照name严格有序，因此必须首先使用name字段进行等值查询，之后对于匹配到的列而言，其按照age字段严格有序，此时可以使用age字段用做索引查找，以此类推。因此在建立联合索引的时候应该注意索引列的顺序，一般情况下，将查询需求频繁或者字段选择性高的列放在前面。此外可以根据特例的查询或者表结构进行单独的调整。

9. 讲一讲MySQL的最左前缀原则?

最左前缀原则就是最左优先，在创建多列索引时，要根据业务需求，where子句中使用最频繁的一列放在最左边。
mysql会一直向右匹配直到遇到范围查询(>、<、between、like)就停止匹配，比如a = 1 and b = 2 and c > 3 and d = 4 如果建立(a,b,c,d)顺序的索引，d是用不到索引的，如果建立(a,b,d,c)的索引则都可以用到，a,b,d的顺序可以任意调整。

=和in可以乱序，比如a = 1 and b = 2 and c = 3 建立(a,b,c)索引可以任意顺序，mysql的查询优化器会帮你优化成索引可以识别的形式。

10. 讲一讲前缀索引?

因为可能我们索引的字段非常长，这既占内存空间，也不利于维护。所以我们就想，如果只把很长字段的前面的公共部分作为一个索引，就会产生超级加倍的效果。但是，我们需要注意，order by不支持前缀索引 。

流程是:

先计算完整列的选择性：`select count(distinct col_1)/count(1) from table_1`

再计算不同前缀长度的选择性：`select count(distinct left(col_1,4))/count(1) from table_1`

找到最优长度之后，创建前缀索引：`create index idx_front on table_1 (col_1(4))`

11. 了解索引下推吗?

MySQL 5.6引入了索引下推优化。默认开启，使用SET optimizer_switch = ‘index_condition_pushdown=off’;可以将其关闭。

- 有了索引下推优化，可以在**减少回表次数**
- 在InnoDB中只针对二级索引有效

官方文档中给的例子和解释如下：

在 people_table中有一个二级索引(zipcode，lastname，address)，查询是SELECT * FROM people WHERE zipcode=’95054′ AND lastname LIKE ‘%etrunia%’ AND address LIKE ‘%Main Street%’;

- 如果没有使用索引下推技术，则MySQL会通过zipcode=’95054’从存储引擎中查询对应的数据，返回到MySQL服务端，然后MySQL服务端基于lastname LIKE ‘%etrunia%’ and address LIKE ‘%Main Street%’来判断数据是否符合条件
- 如果使用了索引下推技术，则MYSQL首先会返回符合zipcode=’95054’的索引，然后根据lastname LIKE ‘%etrunia%’ and address LIKE ‘%Main Street%’来判断索引是否符合条件。如果符合条件，则根据该索引来定位对应的数据，如果不符合，则直接reject掉。

12. 怎么查看MySQL语句有没有用到索引?

通过explain，如以下例子：

```
EXPLAIN SELECT * FROM employees.titles WHERE emp_no='10001' AND title='Senior Engineer' AND from_date='1986-06-26';
```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	titles	null	const	PRIMARY	PRIMARY	59

- id：在一个大的查询语句中每个**SELECT**关键字都对应一个唯一的id，如explain select * from s1 where id = (select id from s1 where name = 'egon1');第一个select的id是1，第二个select的id是2。有时候会出现两个select，但是id却都是1，这是因为优化器把子查询变成了连接查询 。

- select_type：select关键字对应的那个查询的类型，如SIMPLE,PRIMARY,SUBQUERY,DEPENDENT,SNION 。
- table：每个查询对应的表名 。
- type： `type` 字段比较重要, 它提供了判断查询是否高效的重要依据依据. 通过 `type` 字段, 我们判断此次查询是 `全表扫描` 还是 `索引扫描` 等。如const(主键索引或者唯一二级索引进行等值匹配的情况下),ref(普通的二级索引列与常量进行等值匹配),index(扫描全表索引的覆盖索引) 。

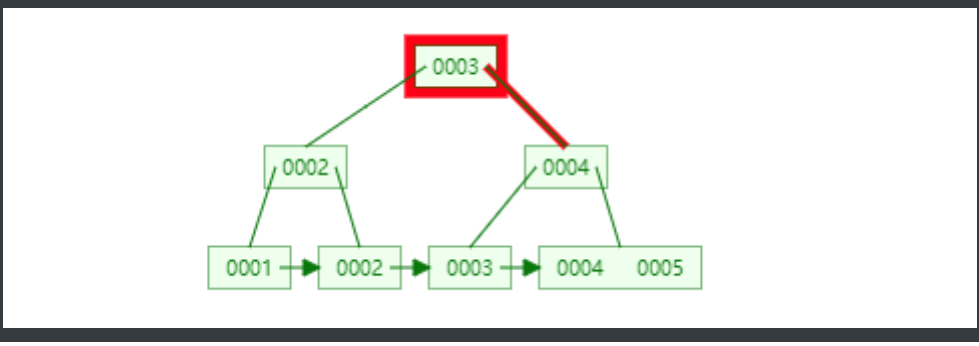
  通常来说, 不同的 type 类型的性能关系如下:
  `ALL < index < range ~ index_merge < ref < eq_ref < const < system`
  `ALL` 类型因为是全表扫描, 因此在相同的查询条件下, 它是速度最慢的.
  而 `index` 类型的查询虽然不是全表扫描, 但是它扫描了所有的索引, 因此比 ALL 类型的稍快.
- possible_key：查询中可能用到的索引(可以把用不到的删掉，降低优化器的优化时间) 。
- key：此字段是 MySQL 在当前查询时所真正使用到的索引。
- filtered：查询器预测满足下一次查询条件的百分比 。
- rows 也是一个重要的字段. MySQL 查询优化器根据统计信息, 估算 SQL 要查找到结果集需要扫描读取的数据行数.
  这个值非常直观显示 SQL 的效率好坏, 原则上 rows 越少越好。
- extra：表示额外信息，如Using where,Start temporary,End temporary,Using temporary等。

13. 为什么官方建议使用自增长主键作为索引?

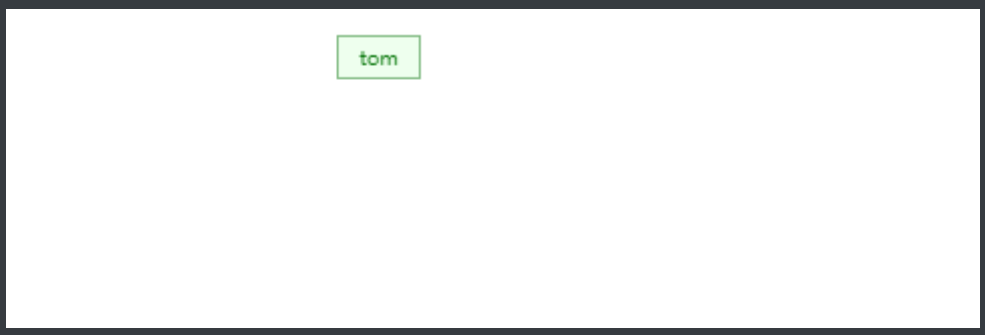
结合B+Tree的特点，自增主键是连续的，在插入过程中尽量减少页分裂，即使要进行页分裂，也只会分裂很少一部分。并且能减少数据的移动，每次插入都是插入到最后。总之就是减少分裂和移动的频率。

插入连续的数据：

> 图片来自：https://www.javazhiyin.com/40232.html



插入非连续的数据：



14. 如何创建索引?

创建索引有三种方式。

1、 在执行CREATE TABLE时创建索引

```
CREATE TABLE user_index2 (
    id INT auto_increment PRIMARY KEY,
    first_name VARCHAR (16),
    last_name VARCHAR (16),
    id_card VARCHAR (18),
    information text,
    KEY name (first_name, last_name),
    FULLTEXT KEY (information),
    UNIQUE KEY (id_card)
);
```

2、 使用ALTER TABLE命令去增加索引。

```
ALTER TABLE table_name ADD INDEX index_name (column_list);
```

ALTER TABLE用来创建普通索引、UNIQUE索引或PRIMARY KEY索引。

其中table_name是要增加索引的表名，column_list指出对哪些列进行索引，多列时各列之间用逗号分隔。

索引名index_name可自己命名，缺省时，MySQL将根据第一个索引列赋一个名称。另外，ALTER TABLE允许在单个语句中更改多个表，因此可以在同时创建多个索引。
3、 使用CREATE INDEX命令创建。

```
CREATE INDEX index_name ON table_name (column_list);
```

15. 创建索引时需要注意什么?

- 非空字段：应该指定列为NOT NULL，除非你想存储NULL。在mysql中，含有空值的列很难进行查询优化，因为它们使得索引、索引的统计信息以及比较运算更加复杂。你应该用0、一个特殊的值或者一个空串代替空值；
- 取值离散大的字段：（变量各个取值之间的差异程度）的列放到联合索引的前面，可以通过count()函数查看字段的差异值，返回值越大说明字段的唯一值越多字段的离散程度高；
- 索引字段越小越好：数据库的数据存储以页为单位一页存储的数据越多一次IO操作获取的数据越大效率越高。

16. 建索引的原则有哪些?

1、最左前缀匹配原则，非常重要的原则，mysql会一直向右匹配直到遇到范围查询(>、<、between、like)就停止匹配，比如a = 1 and b = 2 and c > 3 and d = 4 如果建立(a,b,c,d)顺序的索引，d是用不到索引的，如果建立(a,b,d,c)的索引则都可以用到，a,b,d的顺序可以任意调整。

2、=和in可以乱序，比如a = 1 and b = 2 and c = 3 建立(a,b,c)索引可以任意顺序，mysql的查询优化器会帮你优化成索引可以识别的形式。

3、尽量选择区分度高的列作为索引，区分度的公式是count(distinct col)/count(*)，表示字段不重复的比例，比例越大我们扫描的记录数越少，唯一键的区分度是1，而一些状态、性别字段可能在大数据面前区分度就是0，那可能有人会问，这个比例有什么经验值吗？使用场景不同，这个值也很难确定，一般需要join的字段我们都要求是0.1以上，即平均1条扫描10条记录。

4、索引列不能参与计算，保持列“干净”，比如from_unixtime(create_time) = ’2014-05-29’就不能使用到索引，原因很简单，b+树中存的都是数据表中的字段值，但进行检索时，需要把所有元素都应用函数才能比较，显然成本太大。所以语句应该写成create_time = unix_timestamp(’2014-05-29’)。

5、尽量的扩展索引，不要新建索引。比如表中已经有a的索引，现在要加(a,b)的索引，那么只需要修改原来的索引即可。

17. 使用索引查询一定能提高查询的性能吗?

通常通过索引查询数据比全表扫描要快。但是我们也必须注意到它的代价。

索引需要空间来存储，也需要定期维护， 每当有记录在表中增减或索引列被修改时，索引本身也会被修改。 这意味着每条记录的I* NSERT，DELETE，UPDATE将为此多付出4，5 次的磁盘I/O。 因为索引需要额外的存储空间和处理，那些不必要的索引反而会使查询反应时间变慢。使用索引查询不一定能提高查询性能，索引范围查询(INDEX RANGE SCAN)适用于两种情况:

- 基于一个范围的检索，一般查询返回结果集小于表中记录数的30%。
- 基于非唯一性索引的检索。

18. 什么情况下不走索引（索引失效）？

1、使用!= 或者 <> 导致索引失效

2、类型不一致导致的索引失效

3、函数导致的索引失效

如:

```
SELECT * FROM `user` WHERE DATE(create_time) = '2020-09-03';
```

如果使用函数在索引列，这是不走索引的。

4、运算符导致的索引失效

```
SELECT * FROM `user` WHERE age - 1 = 20;
```

如果你对列进行了（+，-，*，/，!），那么都将不会走索引。

5、OR引起的索引失效

```
SELECT * FROM `user` WHERE `name` = '张三' OR height = '175';
```

OR导致索引是在特定情况下的，并不是所有的OR都是使索引失效，如果OR连接的是同一个字段，那么索引不会失效，反之索引失效。

6、模糊搜索导致的索引失效

```
SELECT * FROM `user` WHERE `name` LIKE '%冰';
```

当 % 放在匹配字段前是不走索引的，放在后面才会走索引。

7、NOT IN、NOT EXISTS导致索引失效

基础

基础

1. 数据库的三范式是什么？
2. MySQL 支持哪些存储引擎?
3. 超键、候选键、主键、外键分别是什么？
4. SQL 约束有哪几种？
5. MySQL 中的 varchar 和 char 有什么区别？
6. MySQL中 in 和 exists 区别
7. drop、delete与truncate的区别
8. 什么是存储过程？有哪些优缺点？
9. MySQL 执行查询的过程

事务

1. 什么是数据库事务？
2. 介绍一下事务具有的四个特征
3. 说一下MySQL 的四种隔离级别
4. 什么是脏读？幻读？不可重复读？
5. 事务的实现原理
6. MySQL事务日志介绍下？
   redo log
   undo log
7. 什么是MySQL的 binlog?
8. 在事务中可以混合使用存储引擎吗？
9. MySQL中是如何实现事务隔离的?
10. 什么是 MVCC？
11. MVCC 的实现原理

锁

1. 为什么要加锁?
2. 按照锁的粒度分数据库锁有哪些？
3. 从锁的类别上分MySQL都有哪些锁呢？
4. 数据库的乐观锁和悲观锁是什么？怎么实现的？
5. InnoDB引擎的行锁是怎么实现的？
6. 什么是死锁？怎么解决？
7. 隔离级别与锁的关系
8. 优化锁方面的意见？

分库分表

1. 为什么要分库分表？

2. 用过哪些分库分表中间件？不同的分库分表中间件都有什么优点和缺点？
   cobar
   TDDL
   atlas
   sharding-jdbc
   mycat

3. 如何对数据库如何进行垂直拆分或水平拆分的？

读写分离、主从同步（复制）

1. 什么是MySQL主从同步？
2. MySQL主从同步的目的？为什么要做主从同步？
3. 如何实现MySQL的读写分离？
4. MySQL主从复制流程和原理？
5. MySQL主从同步延时问题如何解决？

MySQL优化

1. 如何定位及优化SQL语句的性能问题？
2. 大表数据查询，怎么优化
3. 超大分页怎么处理？
4. 统计过慢查询吗？对慢查询都怎么优化过？
5. 如何优化查询过程中的数据访问

1. 数据库的三范式是什么?

- 第一范式：强调的是列的原子性，即数据库表的每一列都是不可分割的原子数据项。
- 第二范式：要求实体的属性完全依赖于主关键字。所谓完全 依赖是指不能存在仅依赖主关键字一部分的属性。
- 第三范式：任何非主属性不依赖于其它非主属性。

2. MySQL 支持哪些存储引擎?

MySQL 支持多种存储引擎,比如 InnoDB,MyISAM,Memory,Archive 等等.在大多数的情况下,直接选择使用 InnoDB 引擎都是最合适的,InnoDB 也是 MySQL 的默认存储引擎。

MyISAM 和 InnoDB 的区别有哪些:

- InnoDB 支持事务，MyISAM 不支持
- InnoDB 支持外键，而 MyISAM 不支持
- InnoDB 是聚集索引，数据文件是和索引绑在一起的，必须要有主键，通过主键索引效率很高；MyISAM 是非聚集索引，数据文件是分离的，索引保存的是数据文件的指针，主键索引和辅助索引是独立的。
- Innodb 不支持全文索引，而 MyISAM 支持全文索引，查询效率上 MyISAM 要高；
- InnoDB 不保存表的具体行数，MyISAM 用一个变量保存了整个表的行数。
- MyISAM 采用表级锁(table-level locking)；InnoDB 支持行级锁(row-level locking)和表级锁,默认为行级锁。

3. 超键、候选键、主键、外键分别是什么?

- 超键：在关系中能唯一标识元组的属性集称为关系模式的超键。一个属性可以为作为一个超键，多个属性组合在一起也可以作为一个超键。超键包含候选键和主键。
- 候选键：是最小超键，即没有冗余元素的超键。
- 主键：数据库表中对储存数据对象予以唯一和完整标识的数据列或属性的组合。一个数据列只能有一个主键，且主键的取值不能缺失，即不能为空值（Null）。
- 外键：在一个表中存在的另一个表的主键称此表的外键。

4. SQL 约束有哪几种?

- NOT NULL: 用于控制字段的内容一定不能为空（NULL）。
- UNIQUE: 控件字段内容不能重复，一个表允许有多个 Unique 约束。
- PRIMARY KEY: 也是用于控件字段内容不能重复，但它在一个表只允许出现一个。
- FOREIGN KEY: 用于预防破坏表之间连接的动作，也能防止非法数据插入外键列，因为它必须是它指向的那个表中的值之一。
- CHECK: 用于控制字段的值范围。

5. MySQL 中的 varchar 和 char 有什么区别?

char 是一个定长字段,假如申请了 `char(10)` 的空间,那么无论实际存储多少内容.该字段都占用 10 个字符,而 varchar 是变长的,也就是说申请的只是最大长度,占用的空间为实际字符长度+1,最后一个字符存储使用了多长的空间.

在检索效率上来讲,char > varchar,因此在使用中,如果确定某个字段的值的长度,可以使用 char,否则应该尽量使用 varchar.例如存储用户 MD5 加密后的密码,则应该使用 char。

6. MySQL中 in 和 exists 区别

MySQL中的in语句是把外表和内表作hash 连接，而exists语句是对外表作loop循环，每次loop循环再对内表进行查询。一直大家都认为exists比in语句的效率要高，这种说法其实是不准确的。这个是要区分环境的。

如果查询的两个表大小相当，那么用in和exists差别不大。
如果两个表中一个较小，一个是大表，则子查询表大的用exists，子查询表小的用in。
not in 和not exists：如果查询语句使用了not in，那么内外表都进行全表扫描，没有用到索引；而not extsts的子查询依然能用到表上的索引。所以无论那个表大，用not exists都比not in要快。

7. drop、delete与truncate的区别

三者都表示删除，但是三者有一些差别：

	Delete	Truncate	Drop
类型	属于DML	属于DDL	属于DDL
回滚	可回滚	不可回滚	不可回滚
删除内容	表结构还在，删除表的全部或者一部分数据行	表结构还在，删除表中的所有数据	从数据库中删除表，所有的数据行，索引和权限也会被删除
删除速度	删除速度慢，需要逐行删除	删除速度快	删除速度最快

8. 什么是存储过程？有哪些优缺点？

存储过程是一些预编译的 SQL 语句。

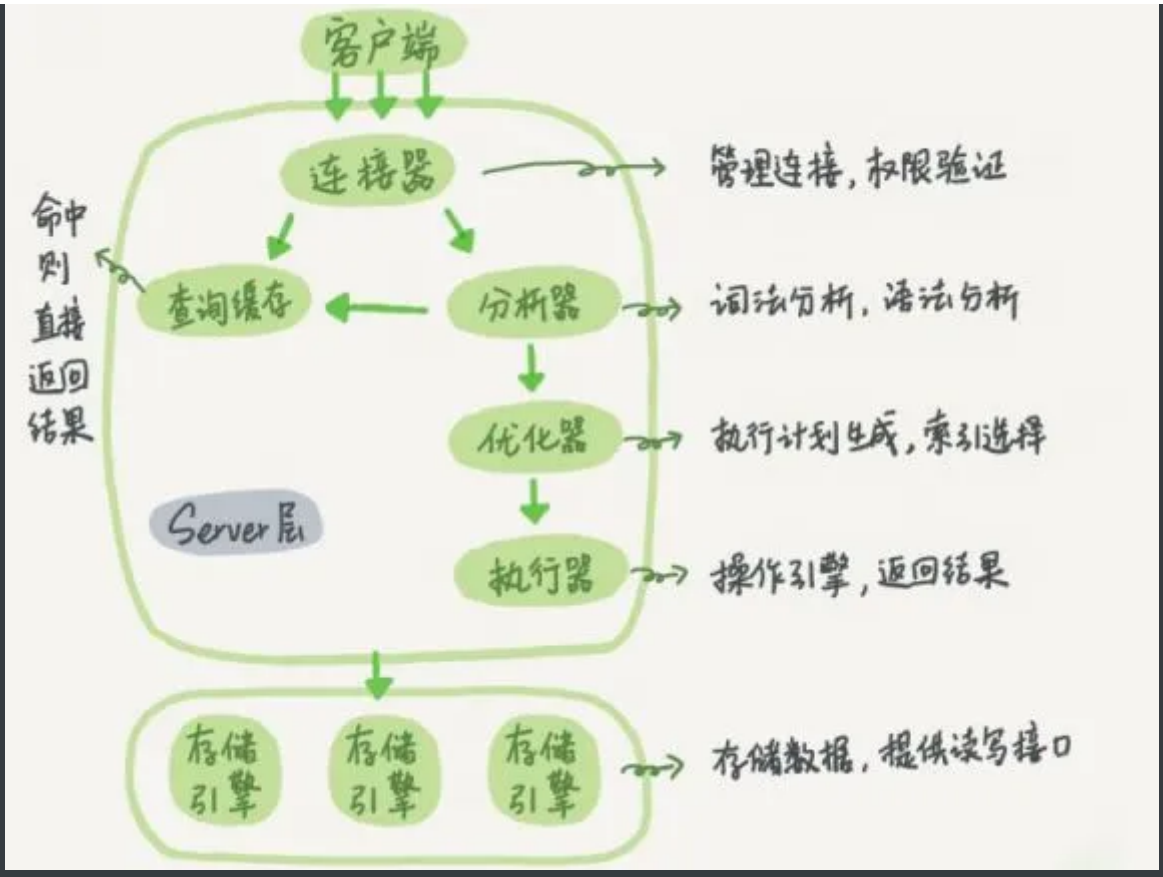
1、更加直白的理解：存储过程可以说是一个记录集，它是由一些 T-SQL 语句组成的代码块，这些 T-SQL 语句代码像一个方法一样实现一些功能（对单表或多表的增删改查），然后再给这个代码块取一个名字，在用到这个功能的时候调用他就行了。

2、存储过程是一个预编译的代码块，执行效率比较高,一个存储过程替代大量 T_SQL 语句，可以降低网络通信量，提高通信速率,可以一定程度上确保数据安全

但是,在互联网项目中,其实是不太推荐存储过程的,比较出名的就是阿里的《Java 开发手册》中禁止使用存储过程,我个人的理解是,在互联网项目中,迭代太快,项目的生命周期也比较短,人员流动相比于传统的项目也更加频繁,在这样的情况下,存储过程的管理确实是没有那么方便,同时,复用性也没有写在服务层那么好。

9. MySQL 执行查询的过程

1. 客户端通过 TCP 连接发送连接请求到 MySQL 连接器，连接器会对该请求进行权限验证及连接资源分配
2. 查缓存。（当判断缓存是否命中时，MySQL 不会进行解析查询语句，而是直接使用 SQL 语句和客户端发送过来的其他原始信息。所以，任何字符上的不同，例如空格、注解等都会导致缓存的不命中。）
3. 语法分析（SQL 语法是否写错了）。 如何把语句给到预处理器，检查数据表和数据列是否存在，解析别名看是否存在歧义。
4. 优化。是否使用索引，生成执行计划。
5. 交给执行器，将数据保存到结果集中，同时会逐步将数据缓存到查询缓存中，最终将结果集返回给客户端。



更新语句执行会复杂一点。需要检查表是否有排它锁，写 binlog，刷盘，是否执行 commit。

事务

1. 什么是数据库事务?

事务是一个不可分割的数据库操作序列，也是数据库并发控制的基本单位，其执行的结果必须使数据库从一种一致性状态变到另一种一致性状态。事务是逻辑上的一组操作，要么都执行，要么都不执行。

事务最经典也经常被拿出来说例子就是转账了。

假如小明要给小红转账1000元，这个转账会涉及到两个关键操作就是：将小明的余额减少1000元，将小红的余额增加1000元。万一在这两个操作之间突然出现错误比如银行系统崩溃，导致小明余额减少而小红的余额没有增加，这样就不对了。事务就是保证这两个关键操作要么都成功，要么都要失败。

2. 介绍一下事务具有的四个特征

事务就是一组原子性的操作，这些操作要么全部发生，要么全部不发生。事务把数据库从一种一致性状态转换成另一种一致性状态。

- 原子性。事务是数据库的逻辑工作单位，事务中包含的各操作要么都做，要么都不做
- 一致性。事 务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。因此当数据库只包含成功事务提交的结果时，就说数据库处于一致性状态。如果数据库系统 运行中发生故障，有些事务尚未完成就被迫中断，这些未完成事务对数据库所做的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态，或者说是 不一致的状态。
- 隔离性。一个事务的执行不能其它事务干扰。即一个事务内部的//操作及使用的数据对其它并发事务是隔离的，并发执行的各个事务之间不能互相干扰。
- 持续性。也称永久性，指一个事务一旦提交，它对数据库中的数据的改变就应该是永久性的。接下来的其它操作或故障不应该对其执行结果有任何影响。

3. 说一下MySQL的四种隔离级别

- Read Uncommitted（读取未提交内容）

在该隔离级别，所有事务都可以看到其他未提交事务的执行结果。本隔离级别很少用于实际应用，因为它的性能也不比其他级别好多少。读取未提交的数据，也被称之为脏读（Dirty Read）。

- Read Committed（读取提交内容）

这是大多数数据库系统的默认隔离级别（但不是 MySQL 默认的）。它满足了隔离的简单定义：一个事务只能看见已经提交事务所做的改变。这种隔离级别 也支持所谓 的 不可重复读（Nonrepeatable Read），因为同一事务的其他实例在该实例处理其间可能会有新的 commit，所以同一 select 可能返回不同结果。

- Repeatable Read（可重读）

这是 MySQL 的默认事务隔离级别，它确保同一事务的多个实例在并发读取数据时，会看到同样的数据行。不过理论上，这会导致另一个棘手的问题：幻读 （Phantom Read）。

- Serializable（可串行化）

通过强制事务排序，使之不可能相互冲突，从而解决幻读问题。简言之，它是在每个读的数据行上加上共享锁。在这个级别，可能导致大量的超时现象和锁竞争。

隔离级别	脏读	不可重复读	幻影读
READ-UNCOMMITTED	√	√	√
READ-COMMITTED	×	√	√
REPEATABLE-READ	×	×	√
SERIALIZABLE	×	×	×

MySQL 默认采用的 REPEATABLE_READ隔离级别 Oracle 默认采用的 READ_COMMITTED 隔离级别

事务隔离机制的实现基于锁机制和并发调度。其中并发调度使用的是MVVC（多版本并发控制），通过保存修改的旧版本信息来支持并发一致性读和回滚等特性。

因为隔离级别越低，事务请求的锁越少，所以大部分数据库系统的隔离级别都是READ-COMMITTED(读取提交内容):，但是你要知道的是InnoDB 存储引擎默认使用 **REPEATABLE-READ（可重读）**并不会有任何性能损失。

InnoDB 存储引擎在 分布式事务 的情况下一般会用到**SERIALIZABLE(可串行化)**隔离级别。

4.什么是脏读？幻读？不可重复读?

1、脏读：事务 A 读取了事务 B 更新的数据，然后 B 回滚操作，那么 A 读取到的数据是脏数据

2、不可重复读：事务 A 多次读取同一数据，事务 B 在事务 A 多次读取的过程中，对数据作了更新并提交，导致事务 A 多次读取同一数据时，结果 不一致。

3、幻读：系统管理员 A 将数据库中所有学生的成绩从具体分数改为 ABCDE 等级，但是系统管理员 B 就在这个时候插入了一条具体分数的记录，当系统管理员 A 改结束后发现还有一条记录没有改过来，就好像发生了幻觉一样，这就叫幻读。

不可重复读侧重于修改，幻读侧重于新增或删除（多了或少量行），脏读是一个事务回滚影响另外一个事务。

5.事务的实现原理

事务是基于重做日志文件(redo log)和回滚日志(undo log)实现的。

每提交一个事务必须先将该事务的所有日志写入到重做日志文件进行持久化，数据库就可以通过重做日志来保证事务的原子性和持久性。

每当有修改事务时，还会产生 undo log，如果需要回滚，则根据 undo log 的反向语句进行逻辑操作，比如 insert 一条记录就 delete 一条记录。undo log 主要实现数据库的一致性。

6. MySQL事务日志介绍下?

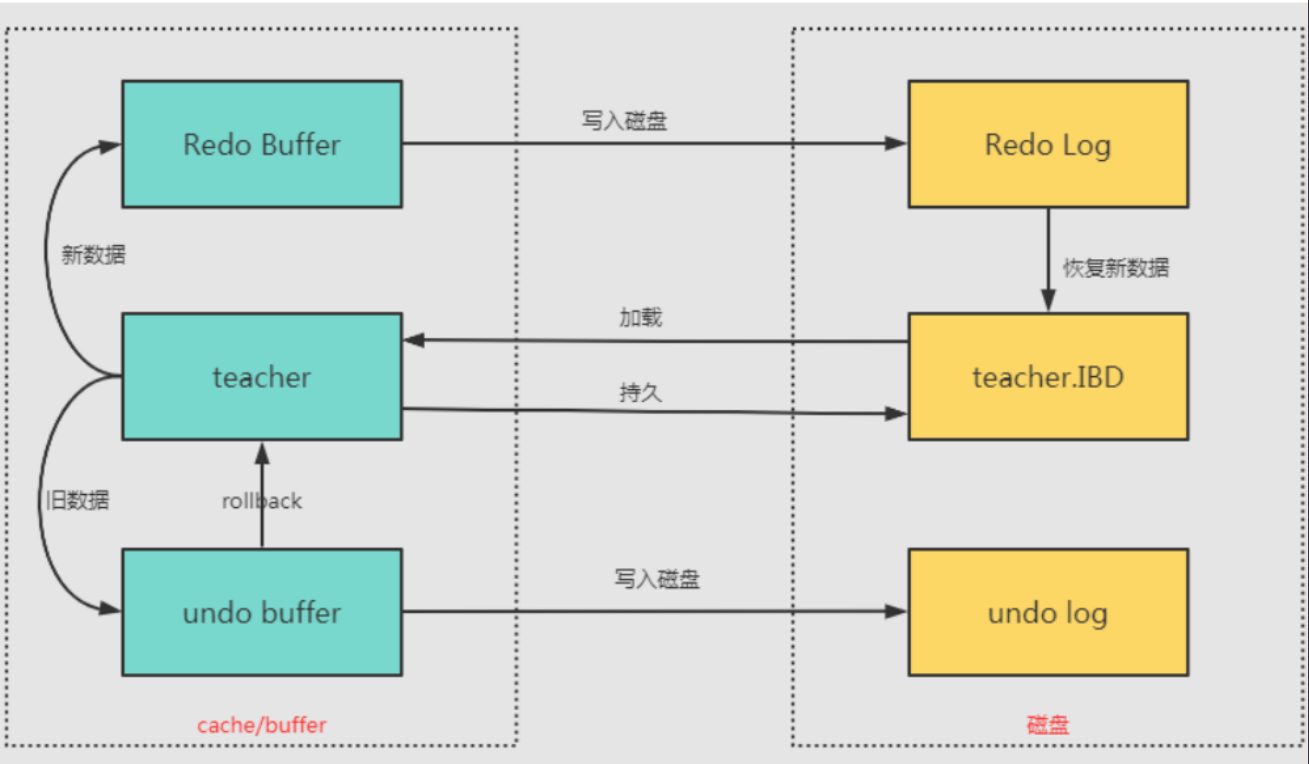
innodb 事务日志包括 redo log 和 undo log。

undo log 指事务开始之前，在操作任何数据之前，首先将需操作的数据备份到一个地方。
redo log 指事务中操作的任何数据，将最新的数据备份到一个地方。

事务日志的目的：实例或者介质失败，事务日志文件就能派上用场。

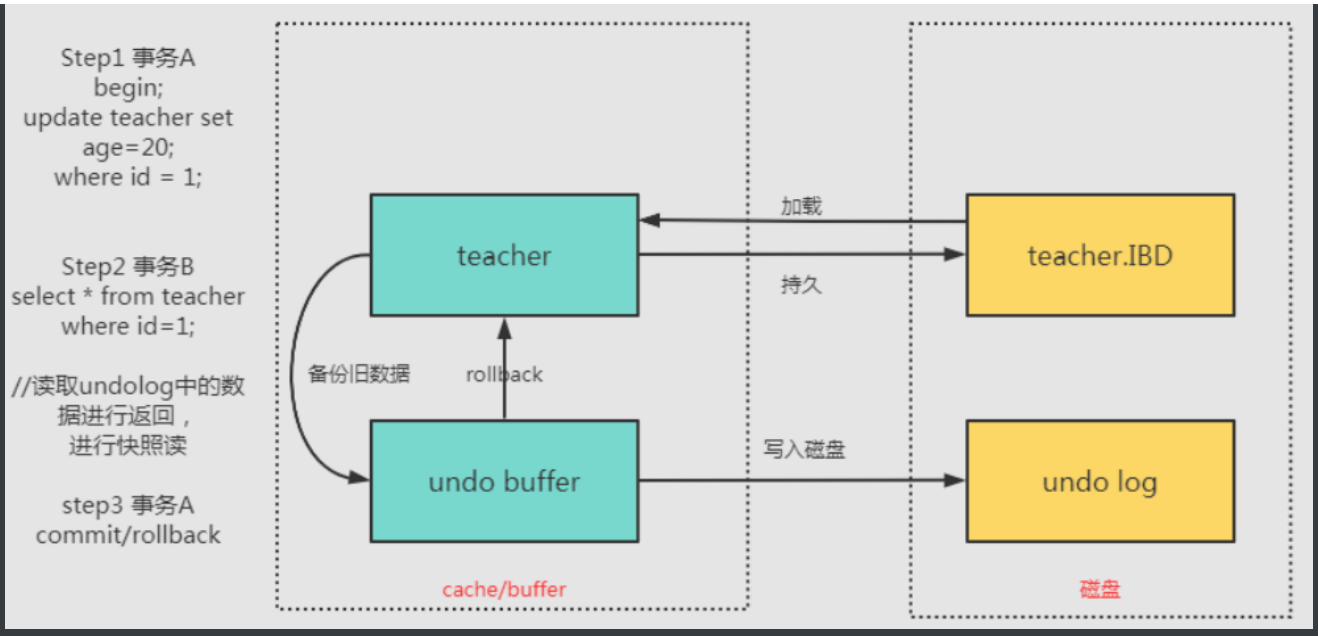
redo log

redo log 不是随着事务的提交才写入的，而是在事务的执行过程中，便开始写入 redo 中。具体的落盘策略可以进行配置 。防止在发生故障的时间点，尚有脏页未写入磁盘，在重启 MySQL 服务的时候，根据 redo log 进行重做，从而达到事务的未入磁盘数据进行持久化这一特性。RedoLog 是为了实现事务的持久性而出现的产物。



undo log

undo log 用来回滚行记录到某个版本。事务未提交之前，Undo 保存了未提交之前的版本数据，Undo 中的数据可作为数据旧版本快照供其他并发事务进行快照读。是为了实现事务的原子性而出现的产物,在 MySQL innodb 存储引擎中用来实现多版本并发控制。



7. 什么是MySQL的binlog?

MySQL的 binlog 是记录所有数据库表结构变更（例如 CREATE、ALTER TABLE）以及表数据修改（INSERT、UPDATE、DELETE）的二进制日志。binlog 不会记录 SELECT 和 SHOW 这类操作，因为这类操作对数据本身并没有修改，但你可以通过查询通用日志来查看 MySQL 执行过的所有语句。

MySQL binlog 以事件形式记录，还包含语句所执行的消耗的时间，MySQL 的二进制日志是事务安全型的。binlog 的主要目的是复制和恢复。

binlog 有三种格式，各有优缺点：

- **statement：** 基于 SQL 语句的模式，某些语句和函数如 UUID, LOAD DATA INFILE 等在复制过程可能导致数据不一致甚至出错。
- **row：** 基于行的模式，记录的是行的变化，很安全。但是 binlog 会比其他两种模式大很多，在一些大表中清除大量数据时在 binlog 中会生成很多条语句，可能导致从库延迟变大。
- **mixed：** 混合模式，根据语句来选用是 statement 还是 row 模式。

8. 在事务中可以混合使用存储引擎吗?

尽量不要在同一个事务中使用多种存储引擎，MySQL服务器层不管理事务，事务是由下层的存储引擎实现的。

如果在事务中混合使用了事务型和非事务型的表（例如InnoDB和MyISAM表）,在正常提交的情况下不会有什么问题。

但如果该事务需要回滚，非事务型的表上的变更就无法撤销，这会导致数据库处于不一致的状态，这种情况很难修复，事务的最终结果将无法确定。所以，为每张表选择合适的存储引擎非常重要。

9. MySQL中是如何实现事务隔离的?

读未提交和串行化基本上是不需要考虑的隔离级别，前者不加锁限制，后者相当于单线程执行，效率太差。

MySQL 在可重复读级别解决了幻读问题，是通过行锁和间隙锁的组合 Next-Key 锁实现的。

详细原理看这篇文章：https://haicoder.net/note/MySQL-interview/MySQL-interview-MySQL-trans-level.html

10. 什么是 MVCC?

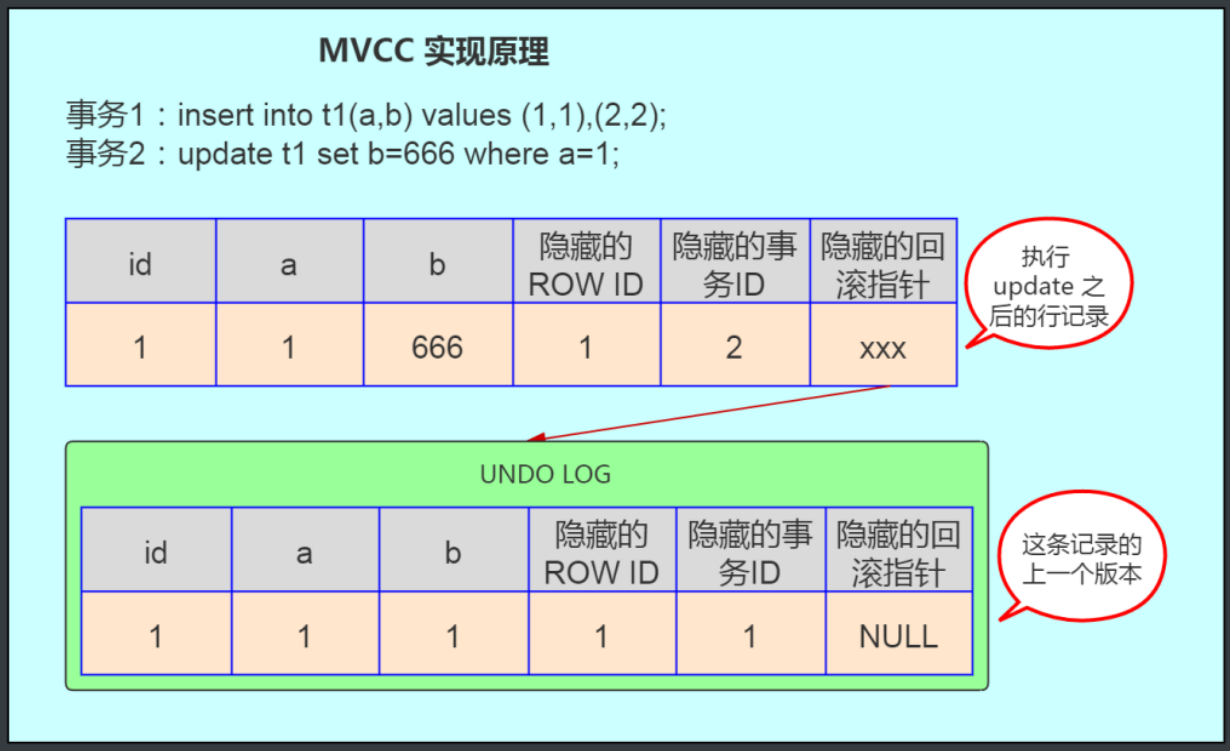
MVCC， 即多版本并发控制。MVCC 的实现，是通过保存数据在某个时间点的快照来实现的。根据事务开始的时间不同，每个事务对同一张表，同一时刻看到的数据可能是不一样的。

11. MVCC 的实现原理

对于 InnoDB ， 聚簇索引记录中包含 3 个隐藏的列：

- ROW ID：隐藏的自增 ID，如果表没有主键，InnoDB 会自动按 ROW ID 产生一个聚集索引树。
- 事务 ID：记录最后一次修改该记录的事务 ID。
- 回滚指针：指向这条记录的上一个版本。

我们拿上面的例子，对应解释下 MVCC 的实现原理，如下图：



如图，首先 insert 语句向表 t1 中插入了一条数据，a 字段为 1，b 字段为 1， ROW ID 也为 1 ，事务 ID 假设为 1，回滚指针假设为 null。当执行 update t1 set b=666 where a=1 时，大致步骤如下：

- 数据库会先对满足 a=1 的行加排他锁；
- 然后将原记录复制到 undo 表空间中；
- 修改 b 字段的值为 666，修改事务 ID 为 2；
- 并通过隐藏的回滚指针指向 undo log 中的历史记录；
- 事务提交，释放前面对满足 a=1 的行所加的排他锁。

在前面实验的第 6 步中，session2 查询的结果是 session1 修改之前的记录，这个记录就是**来自 undolog** 中。

因此可以总结出 MVCC 实现的原理大致是：

InnoDB 每一行数据都有一个隐藏的回滚指针，用于指向该行修改前的最后一个历史版本，这个历史版本存放在 undo log 中。如果要执行更新操作，会将原记录放入 undo log 中，并通过隐藏的回滚指针指向 undo log 中的原记录。其它事务此时需要查询时，就是查询 undo log 中这行数据的最后一个历史版本。

MVCC 最大的好处是读不加锁，读写不冲突，极大地增加了 MySQL 的并发性。通过 MVCC，保证了事务 ACID 中的 I（隔离性）特性。

锁

1. 为什么要加锁?

当多个用户并发地存取数据时，在数据库中就会产生多个事务同时存取同一数据的情况。若对并发操作不加控制就可能会读取和存储不正确的数据，破坏数据库的一致性。

保证多用户环境下保证数据库完整性和一致性。

2. 按照锁的粒度分数据库锁有哪些?

在关系型数据库中，可以**按照锁的粒度把数据库锁分**为行级锁(INNODB引擎)、表级锁(MYISAM引擎)和页级锁(BDB引擎)。

行级锁

- 行级锁是MySQL中锁定粒度最细的一种锁，表示只针对当前操作的行进行加锁。行级锁能大大减少数据库操作的冲突。其加锁粒度最小，但加锁的开销也最大。行级锁分为共享锁 和 排他锁。
- 开销大，加锁慢；会出现死锁；锁定粒度最小，发生锁冲突的概率最低，并发度也最高。

表级锁

- 表级锁是MySQL中锁定粒度最大的一种锁，表示对当前操作的整张表加锁，它实现简单，资源消耗较少，被大部分MySQL引擎支持。最常使用的MYISAM与INNODB都支持表级锁定。表级锁定分为表共享读锁（共享锁）与表独占写锁（排他锁）。
- 开销小，加锁快；不会出现死锁；锁定粒度大，发出锁冲突的概率最高，并发度最低。

页级锁

- 页级锁是MySQL中锁定粒度介于行级锁和表级锁中间的一种锁。表级锁速度快，但冲突多，行级冲突少，但速度慢。所以取了折衷的页级，一次锁定相邻的一组记录。BDB支持页级锁
- 开销和加锁时间界于表锁和行锁之间；会出现死锁；锁定粒度界于表锁和行锁之间，并发度一般

MyISAM和InnoDB存储引擎使用的锁：

- MyISAM采用表级锁(table-level locking)。
- InnoDB支持行级锁(row-level locking)和表级锁，默认为行级锁

3. 从锁的类别上分MySQL都有哪些锁呢?

从锁的类别上来讲，有共享锁和排他锁。

- 共享锁: 又叫做读锁。 当用户要进行数据的读取时，对数据加上共享锁。共享锁可以同时加上多个。
- 排他锁: 又叫做写锁。 当用户要进行数据的写入时，对数据加上排他锁。排他锁只可以加一个，他和其他的排他锁，共享锁都相斥。

用上面的例子来说就是用户的行为有两种，一种是来看房，多个用户一起看房是可以接受的。 一种是真正的入住一晚，在这期间，无论是想入住的还是想看房的都不可以。

锁的粒度取决于具体的存储引擎，InnoDB实现了行级锁，页级锁，表级锁。

他们的加锁开销从大到小，并发能力也是从大到小。

4. 数据库的乐观锁和悲观锁是什么？怎么实现的?

数据库管理系统（DBMS）中的并发控制的任务是确保在多个事务同时存取数据库中同一数据时不破坏事务的隔离性和统一性以及数据库的统一性。乐观并发控制（乐观锁）和悲观并发控制（悲观锁）是并发控制主要采用的技术手段。

- 悲观锁：假定会发生并发冲突，屏蔽一切可能违反数据完整性的操作。在查询完数据的时候就把事务锁起来，直到提交事务。实现方式：使用数据库中的锁机制
- 乐观锁：假设不会发生并发冲突，只在提交操作时检查是否违反数据完整性。在修改数据的时候把事务锁起来，通过version的方式来进行锁定。实现方式：乐一般会使用版本号机制或CAS算法实现。

两种锁的使用场景

从上面对两种锁的介绍，我们知道两种锁各有优缺点，不可认为一种好于另一种，像乐观锁适用于写比较少的情况下（多读场景），即冲突真的很少发生的时候，这样可以省去了锁的开销，加大了系统的整个吞吐量。

但如果是多写的情况，一般会经常产生冲突，这就会导致上层应用会不断的进行retry，这样反倒是降低了性能，所以一般多写的场景下用悲观锁就比较合适。

5. InnoDB引擎的行锁是怎么实现的?

InnoDB是基于索引来完成行锁

例: select * from tab_with_index where id = 1 for update;

for update 可以根据条件来完成行锁锁定，并且 id 是有索引键的列，如果 id 不是索引键那么 InnoDB将完成表锁，并发将无从谈起

6. 什么是死锁？怎么解决?

死锁是指两个或多个事务在同一资源上相互占用，并请求锁定对方的资源，从而导致恶性循环的现象。

常见的解决死锁的方法

1、如果不同程序会并发存取多个表，尽量约定以相同的顺序访问表，可以大大降低死锁机会。

2、在同一个事务中，尽可能做到一次锁定所需要的所有资源，减少死锁产生概率；

3、对于非常容易产生死锁的业务部分，可以尝试使用升级锁定颗粒度，通过表级锁定来减少死锁产生的概率；

如果业务处理不好可以用分布式事务锁或者使用乐观锁

7. 隔离级别与锁的关系

在Read Uncommitted级别下，读取数据不需要加共享锁，这样就不会跟被修改的数据上的排他锁冲突

在Read Committed级别下，读操作需要加共享锁，但是在语句执行完以后释放共享锁；

在Repeatable Read级别下，读操作需要加共享锁，但是在事务提交之前并不释放共享锁，也就是必须等待事务执行完毕以后才释放共享锁。

SERIALIZABLE 是限制性最强的隔离级别，因为该级别锁定整个范围的键，并一直持有锁，直到事务完成。

8. 优化锁方面的意见?

- 使用较低的隔离级别
- 设计索引，尽量使用索引去访问数据，加锁更加精确，从而减少锁冲突

- 选择合理的事务大小，给记录显示加锁时，最好一次性请求足够级别的锁。列如，修改数据的话，最好申请排他锁，而不是先申请共享锁，修改时在申请排他锁，这样会导致死锁
- 不同的程序访问一组表的时候，应尽量约定一个相同的顺序访问各表，对于一个表而言，尽可能的固定顺序的获取表中的行。这样大大的减少死锁的机会。
- 尽量使用相等条件访问数据，这样可以避免间隙锁对并发插入的影响
- 不要申请超过实际需要的锁级别
- 数据查询的时候不是必要，不要使用加锁。MySQL的MVCC可以实现事务中的查询不用加锁，优化事务性能：MVCC只在committed read（读提交）和 repeatable read （可重复读）两种隔离级别
- 对于特定的事务，可以使用表锁来提高处理速度活着减少死锁的可能。

分库分表

1. 为什么要分库分表?

分表

比如你单表都几千万数据了，你确定你能扛住么？绝对不行，单表数据量太大，会极大影响你的 sql执行的性能，到了后面你的 sql 可能就跑的很慢了。一般来说，就以我的经验来看，单表到几百万的时候，性能就会相对差一些了，你就得分表了。

分表就是把一个表的数据放到多个表中，然后查询的时候你就查一个表。比如按照用户 id 来分表，将一个用户的数据就放在一个表中。然后操作的时候你对一个用户就操作那个表就好了。这样可以控制每个表的数据量在可控的范围内，比如每个表就固定在 200 万以内。

分库

分库就是你一个库一般我们经验而言，最多支撑到并发 2000，一定要扩容了，而且一个健康的单库并发值你最好保持在每秒 1000 左右，不要太大。那么你可以将一个库的数据拆分到多个库中，访问的时候就访问一个库好了。

这就是所谓的分库分表。

#	分库分表前	分库分表后
并发支撑情况	MySQL 单机部署，扛不住高并发	MySQL从单机到多机，能承受的并发增加了多倍
磁盘使用情况	MySQL 单机磁盘容量几乎撑满	拆分为多个库，数据库服务器磁盘使用率大大降低
SQL 执行性能	单表数据量太大，SQL 越跑越慢	单表数据量减少，SQL 执行效率明显提升

2. 用过哪些分库分表中间件？不同的分库分表中间件都有什么优点和缺点？

这个其实就是看看你了解哪些分库分表的中间件，各个中间件的优缺点是啥？然后你用过哪些分库分表的中间件。

比较常见的包括：

- cobar
- TDDL
- atlas
- sharding-jdbc
- mycat

cobar

阿里 b2b 团队开发和开源的，属于 proxy 层方案。早些年还可以用，但是最近几年都没更新了，基本没啥人用，差不多算是被抛弃的状态吧。而且不支持读写分离、存储过程、跨库 join 和分页等操作。

TDDL

淘宝团队开发的，属于 client 层方案。支持基本的 crud 语法和读写分离，但不支持 join、多表查询等语法。目前使用的也不多，因为还依赖淘宝的 diamond 配置管理系统。

atlas

360 开源的，属于 proxy 层方案，以前是有一些公司在用的，但是确实有一个很大的问题就是社区最新的维护都在 5 年前了。所以，现在用的公司基本也很少了。

sharding-jdbc

当当开源的，属于 client 层方案。确实之前用的还比较多一些，因为 SQL 语法支持也比较多，没有太多限制，而且目前推出到了 2.0 版本，支持分库分表、读写分离、分布式 id 生成、柔性事务（最大努力送达型事务、TCC 事务）。而且确实之前使用的公司会比较多一些（这个在官网有登记使用的公司，可以看到从 2017 年一直到现在，是有不少公司在用的），目前社区也还一直在开发和维护，还算是比较活跃，个人认为算是一个现在也**可以选择的方案**。

mycat

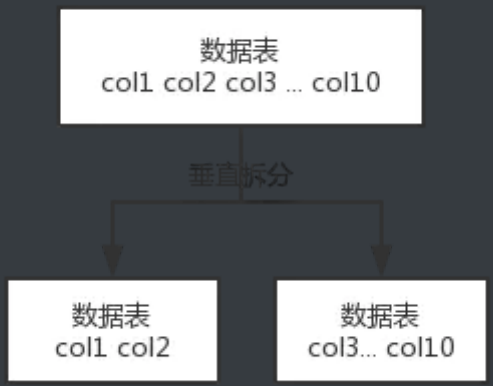
基于 cobar 改造的，属于 proxy 层方案，支持的功能非常完善，而且目前应该是非常火的而且不断流行的数据库中间件，社区很活跃，也有一些公司开始在用了。但是确实相比于 sharding jdbc 来说，年轻一些，经历的锤炼少一些。

3.如何对数据库如何进行垂直拆分或水平拆分的?

水平拆分的意思，就是把一个表的数据给弄到多个库的多个表里去，但是每个库的表结构都一样，只不过每个库表放的数据是不同的，所有库表的数据加起来就是全部数据。水平拆分的意义，就是将数据均匀放更多的库里，然后用多个库来抗更高的并发，还有就是用多个库的存储容量来进行扩容。



垂直拆分的意思，就是**把一个有很多字段的表给拆分成多个表，或者是多个库上去。**每个库表的结构都不一样，每个库表都包含部分字段。一般来说，会**将较少的访问频率很高的字段放到一个表里去**，然后**将较多的访问频率很低的字段放到另外一个表里去。**因为数据库是有缓存的，你访问频率高的行字段越少，就可以在缓存里缓存更多的行，性能就越好。这个一般在表层面做的较多一些。



两种**分库分表的方式**：

- 一种是按照 range 来分，就是每个库一段连续的数据，这个一般是按比如**时间范围**来的，但是这种一般较少用，因为很容易产生热点问题，大量的流量都打在最新的数据上了。
- 或者是按照某个字段hash一下均匀分散，这个较为常用。

range 来分，好处在于说，扩容的时候很简单，因为你只要预备好，给每个月都准备一个库就可以了，到了一个新的月份的时候，自然而然，就会写新的库了；缺点，但是大部分的请求，都是访问最新的数据。实际生产用 range，要看场景。

hash 分发，好处在于说，可以平均分配每个库的数据量和请求压力；坏处在于说扩容起来比较麻烦，会有一个数据迁移的过程，之前的数据需要重新计算 hash 值重新分配到不同的库或表

读写分离、主从同步（复制）

1. 什么是MySQL主从同步?

主从同步使得数据可以从一个数据库服务器复制到其他服务器上，在复制数据时，一个服务器充当主服务器（master），其余的服务器充当从服务器（slave）。

因为复制是异步进行的，所以从服务器不需要一直连接着主服务器，从服务器甚至可以通过拨号断断续续地连接主服务器。通过配置文件，可以指定复制所有的数据库，某个数据库，甚至是某个数据库上的某个表。

2. MySQL主从同步的目的？为什么要做主从同步?

1. 通过增加从服务器来提高数据库的性能，在主服务器上执行写入和更新，在从服务器上向外提供读功能，可以动态地调整从服务器的数量，从而调整整个数据库的性能。
2. 提高数据安全-因为数据已复制到从服务器，从服务器可以终止复制进程，所以，可以在从服务器上备份而不破坏主服务器相应数据
3. 在主服务器上生成实时数据，而在从服务器上分析这些数据，从而提高主服务器的性能
4. 数据备份。一般我们都会做数据备份，可能是写定时任务，一些特殊行业可能还需要手动备份，有些行业要求备份和原数据不能在同一个地方，所以主从就能很好的解决这个问题，不仅备份及时，而且还可以多地备份，保证数据的安全

3. 如何实现MySQL的读写分离?

其实很简单，就是基于主从复制架构，简单来说，就搞一个主库，挂多个从库，然后我们就单单只是写主库，然后主库会自动把数据给同步到从库上去。

4. MySQL主从复制流程和原理?

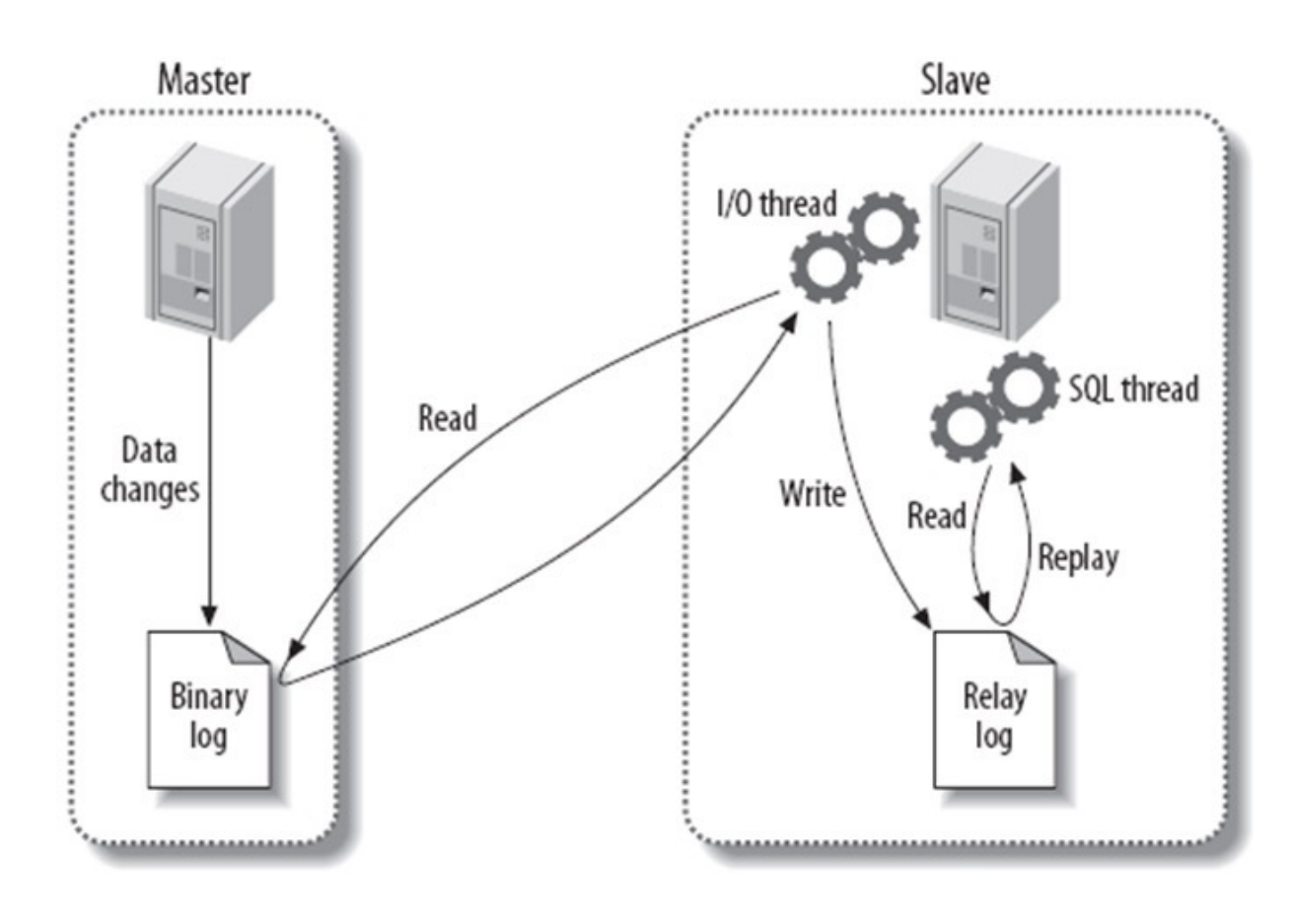
基本原理流程，是3个线程以及之间的关联

主：binlog线程——记录下所有改变了数据库数据的语句，放进master上的binlog中；

从：io线程——在使用start slave 之后，负责从master上拉取 binlog 内容，放进自己的relay log中；

从：sql执行线程——执行relay log中的语句；

复制过程如下：



Binary log：主数据库的二进制日志

Relay log：从服务器的中继日志

第一步：master在每个事务更新数据完成之前，将该操作记录串行地写入到binlog文件中。

第二步：salve开启一个I/O Thread，该线程在master打开一个普通连接，主要工作是binlog dump process。如果读取的进度已经跟上了master，就进入睡眠状态并等待master产生新的事件。I/O线程最终的目的是将这些事件写入到中继日志中。

第三步：SQL Thread会读取中继日志，并顺序执行该日志中的SQL事件，从而与主数据库中的数据保持一致。

5. MySQL主从同步延时问题如何解决?

MySQL 实际上在有两个同步机制，一个是半同步复制，用来 解决主库数据丢失问题；一个是并行复制，用来 解决主从同步延时问题。

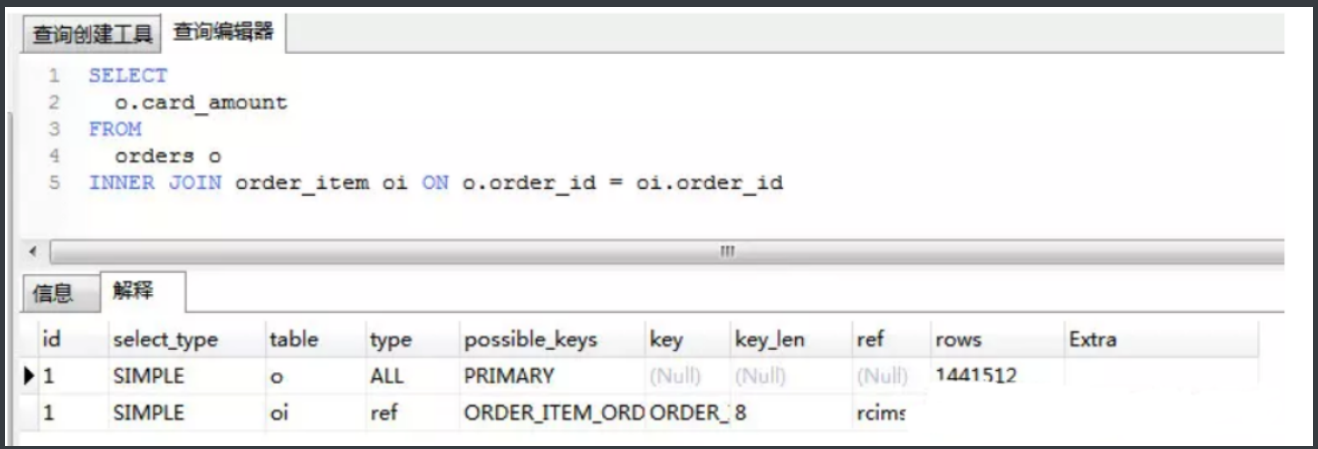
- 半同步复制，也叫 semi-sync 复制，指的就是主库写入 binlog 日志之后，就会将强制此时立即将数据同步到从库，从库将日志写入自己本地的 relay log 之后，接着会返回一个 ack 给主库，主库接收到至少一个从库的 ack 之后才会认为写操作完成了。
- 并行复制，指的是从库开启多个线程，并行读取 relay log 中不同库的日志，然后并行重放不同库的日志，这是库级别的并行。

MySQL优化

1. 如何定位及优化SQL语句的性能问题?

对于低性能的SQL语句的定位，最重要也是最有效的方法就是使用执行计划，MySQL提供了explain命令来查看语句的执行计划。 我们知道，不管是哪种数据库，或者是哪种数据库引擎，在对一条SQL语句进行执行的过程中都会做很多相关的优化，对于查询语句，最重要的优化方式就是使用索引。

而执行计划，就是显示数据库引擎对于SQL语句的执行的详细情况，其中包含了是否使用索引，使用什么索引，使用的索引的相关信息等。



2. 大表数据查询，怎么优化

- 优化shema、sql语句+索引；
- 第二加缓存，memcached, redis；
- 主从复制，读写分离；
- 垂直拆分，根据你模块的耦合度，将一个大的系统分为多个小的系统，也就是分布式系统；
- 水平切分，针对数据量大的表，这一步最麻烦，最能考验技术水平，要选择一个合理的sharding key, 为了有好的查询效率，表结构也要改动，做一定的冗余，应用也要改，sql中尽量带sharding key，将数据定位到限定的表上去查，而不是扫描全部的表；

3. 超大分页怎么处理?

数据库层面,这也是我们主要集中关注的(虽然收效没那么大),类似于 `select * from table where age > 20 limit 1000000` ,10 这种查询其实也是有可以优化的余地的. 这条语句需要 load1000000 数据然后基本上全部丢弃,只取 10 条当然比较慢. 当时我们可以修改为 `select * from table where id in (select id from table where age > 20 limit 1000000,10)` .这样虽然也 load 了一百万的数据,但是由于索引覆盖,要查询的所有字段都在索引中,所以速度会很快。

解决超大分页,其实主要是靠缓存,可预测性的提前查到内容,缓存至redis等k-V数据库中,直接返回即可.

在阿里巴巴《Java开发手册》中,对超大分页的解决办法是类似于上面提到的第一种.

> 【推荐】利用延迟关联或者子查询优化超多分页场景。
>
> 说明：MySQL并不是跳过offset行，而是取offset+N行，然后返回放弃前offset行，返回N行，那当offset特别大的时候，效率就非常的低下，要么控制返回的总页数，要么对超过特定阈值的页数进行SQL改写。
>
> 正例：先快速定位需要获取的id段，然后再关联：
>
> SELECT a.* FROM 表1 a, (select id from 表1 where 条件 LIMIT 100000,20) b where a.id=b.id

4. 统计过慢查询吗？对慢查询都怎么优化过?

在业务系统中，除了使用主键进行的查询，其他的我都会在测试库上测试其耗时，慢查询的统计主要由运维在做，会定期将业务中的慢查询反馈给我们。

慢查询的优化首先要搞明白慢的原因是什么？ 是查询条件没有命中索引？是load了不需要的数据列？还是数据量太大？

所以优化也是针对这三个方向来的，

- 首先分析语句，看看是否load了额外的数据，可能是查询了多余的行并且抛弃掉了，可能是加载了许多结果中并不需要的列，对语句进行分析以及重写。
- 分析语句的执行计划，然后获得其使用索引的情况，之后修改语句或者修改索引，使得语句可以尽可能的命中索引。
- 如果对语句的优化已经无法进行，可以考虑表中的数据量是否太大，如果是的话可以进行横向或者纵向的分表。

5. 如何优化查询过程中的数据访问

- 访问数据太多导致查询性能下降
- 确定应用程序是否在检索大量超过需要的数据，可能是太多行或列
- 确认MySQL服务器是否在分析大量不必要的数据行
- 查询不需要的数据。解决办法：使用limit解决
- 多表关联返回全部列。解决办法：指定列名
- 总是返回全部列。解决办法：避免使用SELECT *
- 重复查询相同的数据。解决办法：可以缓存数据，下次直接读取缓存
- 是否在扫描额外的记录。解决办法：
  使用explain进行分析，如果发现查询需要扫描大量的数据，但只返回少数的行，可以通过如下技巧去优化：
  使用索引覆盖扫描，把所有的列都放到索引中，这样存储引擎不需要回表获取对应行就可以返回结果。
- 改变数据库和表的结构，修改数据表范式
- 重写SQL语句，让优化器可以以更优的方式执行查询。

6. 如何优化关联查询

- 确定ON或者USING子句中是否有索引。
- 确保GROUP BY和ORDER BY只有一个表中的列，这样MySQL才有可能使用索引。

7. 数据库结构优化

一个好的数据库设计方案对于数据库的性能往往会起到事半功倍的效果。

需要考虑数据冗余、查询和更新的速度、字段的数据类型是否合理等多方面的内容。

1. **将字段很多的表分解成多个表**

对于字段较多的表，如果有些字段的使用频率很低，可以将这些字段分离出来形成新表。

因为当一个表的数据量很大时，会由于使用频率低的字段的存在而变慢。

2. **增加中间表**

对于需要经常联合查询的表，可以建立中间表以提高查询效率。

通过建立中间表，将需要通过联合查询的数据插入到中间表中，然后将原来的联合查询改为对中间表的查询。

3. **增加冗余字段**

设计数据表时应尽量遵循范式理论的规约，尽可能的减少冗余字段，让数据库设计看起来精致、优雅。但是，合理的加入冗余字段可以提高查询速度。

表的规范化程度越高，表和表之间的关系越多，需要连接查询的情况也就越多，性能也就越差。

注意：

冗余字段的值在一个表中修改了，就要想办法在其他表中更新，否则就会导致数据不一致的问题。

8. MySQL数据库cpu飙升到500%的话他怎么处理?

当 cpu 飙升到 500%时，先用操作系统命令 top 命令观察是不是 MySQLd 占用导致的，如果不是，找出占用高的进程，并进行相关处理。

如果是 MySQLd 造成的， show processlist，看看里面跑的 session 情况，是不是有消耗资源的 sql 在运行。找出消耗高的 sql，看看执行计划是否准确， index 是否缺失，或者实在是数据量太大造成。

一般来说，肯定要 kill 掉这些线程(同时观察 cpu 使用率是否下降)，等进行相应的调整(比如说加索引、改 sql、改内存参数)之后，再重新跑这些 SQL。

也有可能是每个 sql 消耗资源并不多，但是突然之间，有大量的 session 连进来导致 cpu 飙升，这种情况就需要跟应用一起来分析为何连接数会激增，再做出相应的调整，比如说限制连接数等。

9. 大表怎么优化?

类似的问题：某个表有近千万数据，CRUD比较慢，如何优化？分库分表了是怎么做的？分表分库了有什么问题？有用到中间件么？他们的原理知道么？

当MySQL单表记录数过大时，数据库的CRUD性能会明显下降，一些常见的优化措施如下：

- 限定数据的范围： 务必禁止不带任何限制数据范围条件的查询语句。比如：我们当用户在查询订单历史的时候，我们可以控制在一个月的范围内；
- 读/写分离： 经典的数据库拆分方案，主库负责写，从库负责读；
- 缓存： 使用MySQL的缓存，另外对重量级、更新少的数据可以考虑；
- 通过分库分表的方式进行优化，主要有垂直分表和水平分表。

End

以上是MySQL索引常见面试题，MySQL其他部分正在整理，敬请期待。

若有更新，将第一时间同步公众号【程序员库森】，扫描下方左侧二维码关注！

此外，我还创建了求职交流群，群内分享内推、面试等资源，一起来抱团取暖啊。扫描下方右侧二维码，备注【加群】，拉你进群~



六、Redis

概述

1. Redis是什么？简述它的优缺点？

Redis本质上是一个Key-Value类型的内存数据库，很像Memcached，整个数据库加载在内存当中操作，定期通过异步操作把数据库中的数据flush到硬盘上进行保存。

因为是纯内存操作，Redis的性能非常出色，每秒可以处理超过 10万次读写操作，是已知性能最快的Key-Value 数据库。

优点：

- 读写性能极高， Redis能读的速度是110000次/s，写的速度是81000次/s。
- 支持数据持久化，支持AOF和RDB两种持久化方式。
- 支持事务， Redis的所有操作都是原子性的，意思就是要么成功执行要么失败完全不执行。单个操作是原子性的。多个操作也支持事务，即原子性，通过MULTI和EXEC指令包起来。
- 数据结构丰富，除了支持string类型的value外，还支持hash、set、zset、list等数据结构。
- 支持主从复制，主机会自动将数据同步到从机，可以进行读写分离。
- 丰富的特性 – Redis还支持 publish/subscribe， 通知， key 过期等特性。

缺点：

- 数据库容量受到物理内存的限制，不能用作海量数据的高性能读写，因此Redis适合的场景主要局限在较小数据量的高性能操作和运算上。
- 主机宕机，宕机前有部分数据未能及时同步到从机，切换IP后还会引入数据不一致的问题，降低了系统的可用性。

2. Redis为什么这么快?

- 内存存储：Redis是使用内存(in-memeroy)存储，没有磁盘IO上的开销。数据存在内存中，类似于 HashMap，HashMap 的优势就是查找和操作的时间复杂度都是O(1)。
- 单线程实现（ Redis 6.0以前）：Redis使用单个线程处理请求，避免了多个线程之间线程切换和锁资源争用的开销。注意：单线程是指的是在核心网络模型中，网络请求模块使用一个线程来处理，即一个线程处理所有网络请求。
- 非阻塞IO：Redis使用多路复用IO技术，将epoll作为I/O多路复用技术的实现，再加上Redis自身的事件处理模型将epoll中的连接、读写、关闭都转换为事件，不在网络I/O上浪费过多的时间。
- 优化的数据结构：Redis有诸多可以直接应用的优化数据结构的实现，应用层可以直接使用原生的数据结构提升性能。
- 使用底层模型不同：Redis直接自己构建了 VM (虚拟内存)机制 ，因为一般的系统调用系统函数的话，会浪费一定的时间去移动和请求。

> Redis的VM(虚拟内存)机制就是暂时把不经常访问的数据(冷数据)从内存交换到磁盘中，从而腾出宝贵的内存空间用于其它需要访问的数据(热数据)。通过VM功能可以实现冷热数据分离，使热数据仍在内存中、冷数据保存到磁盘。这样就可以避免因为内存不足而造成访问速度下降的问题。
>
> Redis提高数据库容量的办法有两种：一种是可以将数据分割到多个RedisServer上；另一种是使用虚拟内存把那些不经常访问的数据交换到磁盘上。**需要特别注意的是Redis并没有使用OS提供的Swap，而是自己实现。**

3. Redis相比Memcached有哪些优势?

- 数据类型：Memcached所有的值均是简单的字符串，Redis支持更为丰富的数据类型，支持string(字符串)，list(列表)，Set(集合)、Sorted Set(有序集合)、Hash(哈希)等。
- 持久化：Redis支持数据落地持久化存储，可以将内存中的数据保持在磁盘中，重启的时候可以再次加载进行使用。 memcache不支持数据持久存储 。
- 集群模式：Redis提供主从同步机制，以及 Cluster集群部署能力，能够提供高可用服务。Memcached没有原生的集群模式，需要依靠客户端来实现往集群中分片写入数据
- 性能对比：Redis的速度比Memcached快很多。
- 网络IO模型：Redis使用单线程的多路 IO 复用模型，Memcached使用多线程的非阻塞IO模式。
- Redis支持服务器端的数据操作：Redis相比Memcached来说，拥有更多的数据结构和并支持更丰富的数据操作，通常在Memcached里，你需要将数据拿到客户端来进行类似的修改再set回去。

  这大大增加了网络IO的次数和数据体积。在Redis中，这些复杂的操作通常和一般的GET/SET一样高效。所以，如果需要缓存能够支持更复杂的结构和操作，那么Redis会是不错的选择。

4. 为什么要用 Redis 做缓存?

从高并发上来说：

- 直接操作缓存能够承受的请求是远远大于直接访问数据库的，所以我们可以考虑把数据库中的部分数据转移到缓存中去，这样用户的一部分请求会直接到缓存这里而不用经过数据库。

从高性能上来说：

- 用户第一次访问数据库中的某些数据。 因为是从硬盘上读取的所以这个过程会比较慢。将该用户访问的数据存在缓存中，下一次再访问这些数据的时候就可以直接从缓存中获取了。操作缓存就是直接操作内存，所以速度相当快。如果数据库中的对应数据改变的之后，同步改变缓存中相应的数据。

5. 为什么要用 Redis 而不用 map/guava 做缓存?

缓存分为本地缓存和分布式缓存。以java为例，使用自带的map或者guava实现的是本地缓存，最主要的特点是轻量以及快速，生命周期随着jvm的销毁而结束，并且在多实例的情况下，每个实例都需要各自保存一份缓存，缓存不具有一致性。

使用Redis或memcached之类的称为分布式缓存，在多实例的情况下，各实例共用一份缓存数据，缓存具有一致性。缺点是需要保持Redis或memcached服务的高可用，整个程序架构上较为复杂。

对比:

- Redis 可以用几十 G 内存来做缓存，Map 不行，一般 JVM 也就分几个 G 数据就够大了;
- Redis 的缓存可以持久化，Map 是内存对象，程序一重启数据就没了;
- Redis 可以实现分布式的缓存，Map 只能存在创建它的程序里;
- Redis 可以处理每秒百万级的并发，是专业的缓存服务，Map 只是一个普通的对象;
- Redis 缓存有过期机制，Map 本身无此功能；Redis 有丰富的 API，Map 就简单太多了;
- Redis可单独部署，多个项目之间可以空想，本地内存无法共享;
- Redis有专门的管理工具可以查看缓存数据。

6. Redis的常用场景有哪些?

1、缓存

缓存现在几乎是所有中大型网站都在用的必杀技，合理的利用缓存不仅能够提升网站访问速度，还能大大降低数据库的压力。Redis提供了键过期功能，也提供了灵活的键淘汰策略，所以，现在Redis用在缓存的场合非常多。

2、排行榜

很多网站都有排行榜应用的，如京东的月度销量榜单、商品按时间的上新排行榜等。Redis提供的有序集合数据类构能实现各种复杂的排行榜应用。

3、计数器

什么是计数器，如电商网站商品的浏览量、视频网站视频的播放数等。为了保证数据实时效，每次浏览都得给+1，并发量高时如果每次都请求数据库操作无疑是种挑战和压力。Redis提供的incr命令来实现计数器功能，内存操作，性能非常好，非常适用于这些计数场景。

4、分布式会话

集群模式下，在应用不多的情况下一般使用容器自带的session复制功能就能满足，当应用增多相对复杂的系统中，一般都会搭建以Redis等内存数据库为中心的session服务，session不再由容器管理，而是由session服务及内存数据库管理。

5、分布式锁

在很多互联网公司中都使用了分布式技术，分布式技术带来的技术挑战是对同一个资源的并发访问，如全局ID、减库存、秒杀等场景，并发量不大的场景可以使用数据库的悲观锁、乐观锁来实现，但在并发量高的场合中，利用数据库锁来控制资源的并发访问是不太理想的，大大影响了数据库的性能。可以利用Redis的setnx功能来编写分布式的锁，如果设置返回1说明获取锁成功，否则获取锁失败，实际应用中要考虑的细节要更多。

6、 社交网络

点赞、踩、关注/被关注、共同好友等是社交网站的基本功能，社交网站的访问量通常来说比较大，而且传统的关系数据库类型不适合存储这种类型的数据，Redis提供的哈希、集合等数据结构能很方便的的实现这些功能。如在微博中的共同好友，通过Redis的set能够很方便得出。

7、最新列表

Redis列表结构，LPUSH可以在列表头部插入一个内容ID作为关键字，LTRIM可用来限制列表的数量，这样列表永远为N个ID，无需查询最新的列表，直接根据ID去到对应的内容页即可。

8、消息系统

消息队列是大型网站必用中间件，如ActiveMQ、RabbitMQ、Kafka等流行的消息队列中间件，主要用于业务解耦、流量削峰及异步处理实时性低的业务。Redis提供了发布/订阅及阻塞队列功能，能实现一个简单的消息队列系统。另外，这个不能和专业的消息中间件相比。

7. Redis的数据类型有哪些?

有五种常用数据类型：String、Hash、Set、List、SortedSet。以及三种特殊的数据类型：Bitmap、HyperLogLog、Geospatial ，其中HyperLogLog、Bitmap的底层都是 String 数据类型，Geospatial 的底层是 Sorted Set 数据类型。

五种常用的数据类型：

1、String：String是最常用的一种数据类型，普通的key- value 存储都可以归为此类。其中Value既可以是数字也可以是字符串。使用场景：常规key-value缓存应用。常规计数: 微博数， 粉丝数。

2、Hash：Hash 是一个键值(key => value)对集合。Redishash 是一个 string 类型的 field 和 value 的映射表，hash 特别适合用于存储对象，并且可以像数据库中update一个属性一样只修改某一项属性值。

3、Set：Set是一个无序的天然去重的集合，即Key-Set。此外还提供了交集、并集等一系列直接操作集合的方法，对于求共同好友、共同关注什么的功能实现特别方便。

4、List：List是一个有序可重复的集合，其遵循FIFO的原则，底层是依赖双向链表实现的，因此支持正向、反向双重查找。通过List，我们可以很方面的获得类似于最新回复这类的功能实现。

5、SortedSet：类似于java中的TreeSet，是Set的可排序版。此外还支持优先级排序，维护了一个score的参数来实现。适用于排行榜和带权重的消息队列等场景。

三种特殊的数据类型：

1、Bitmap：位图，Bitmap想象成一个以位为单位数组，数组中的每个单元只能存0或者1，数组的下标在Bitmap中叫做偏移量。使用Bitmap实现统计功能，更省空间。如果只需要统计数据的二值状态，例如商品有没有、用户在不在等，就可以使用 Bitmap，因为它只用一个 bit 位就能表示 0 或 1。

2、Hyperloglog。HyperLogLog 是一种用于统计基数的数据集合类型，HyperLogLog 的优点是，在输入元素的数量或者体积非常非常大

时，计算基数所需的空间总是固定 的、并且是很小的。每个 HyperLogLog 键只需要花费 12 KB 内存，就可以计算接近 2^64 个不同元素的基 数。场景：统计网页的UV（即Unique Visitor，不重复访客，一个人访问某个网站多次，但是还是只计算为一次）。

要注意，HyperLogLog 的统计规则是基于概率完成的，所以它给出的统计结果是有一定误差的，标准误算率是 0.81%。

3、Geospatial： 主要用于存储地理位置信息，并对存储的信息进行操作，适用场景如朋友的定位、附近的人、打车距离计算等。

持久化

1. Redis持久化机制?

为了能够重用Redis数据，或者防止系统故障，我们需要将Redis中的数据写入到磁盘空间中，即持久化。

Redis提供了两种不同的持久化方法可以将数据存储在磁盘中，一种叫快照 `RDB` ，另一种叫只追加文件 `AOF` 。

RDB

在指定的时间间隔内将内存中的数据集快照写入磁盘(`Snapshot`)，它恢复时是将快照文件直接读到内存里。

优势：适合大规模的数据恢复；对数据完整性和一致性要求不高

劣势：在一定间隔时间做一次备份，所以如果Redis意外 `down` 掉的话，就会丢失最后一次快照后的所有修改。

AOF

以日志的形式来记录每个写操作，将Redis执行过的所有写指令记录下来(读操作不记录)，只许追加文件但不可以改写文件，Redis启动之初会读取该文件重新构建数据，换言之，Redis重启的话就根据日志文件的内容将写指令从前到后执行一次以完成数据的恢复工作。

AOF采用文件追加方式，文件会越来越大，为避免出现此种情况，新增了重写机制，当AOF文件的大小超过所设定的阈值时， Redis就会启动AOF文件的内容压缩，只保留可以恢复数据的最小指令集.。

优势

- 每修改同步： `appendfsync always` 同步持久化，每次发生数据变更会被立即记录到磁盘，性能较差但数据完整性比较好
- 每秒同步： `appendfsync everysec` 异步操作，每秒记录，如果一秒内宕机，有数据丢失
- 不同步： `appendfsync no` 从不同步

劣势

- 相同数据集的数据而言 `aof` 文件要远大于 `rdb` 文件，恢复速度慢于 `rdb`
- `aof` 运行效率要慢于 `rdb` ，每秒同步策略效率较好，不同步效率和 `rdb` 相同

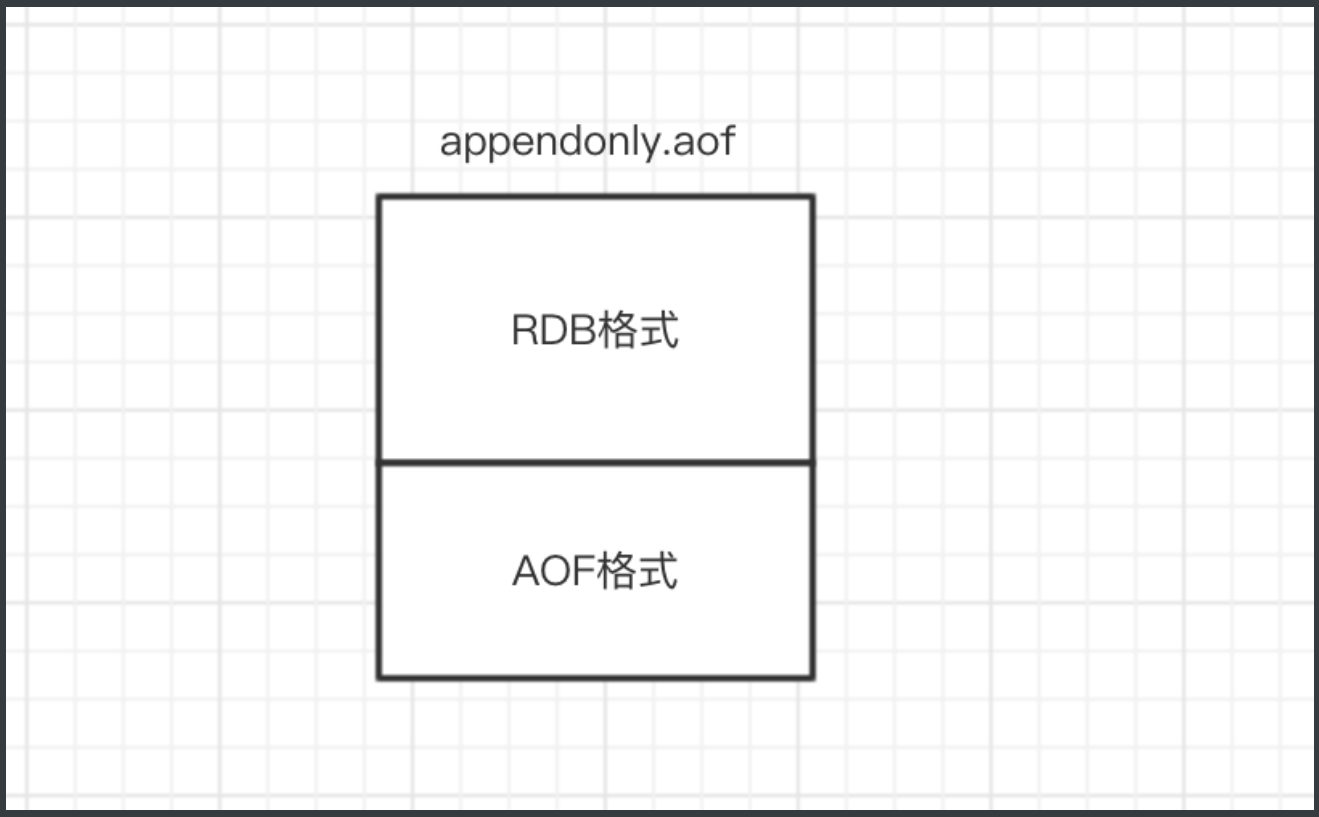
2. 如何选择合适的持久化方式

- 如果是数据不那么敏感，且可以从其他地方重新生成补回的，那么可以关闭持久化。
- 如果是数据比较重要，不想再从其他地方获取，且可以承受数分钟的数据丢失，比如缓存等，那么可以只使用RDB。
- 如果是用做内存数据库，要使用Redis的持久化，建议是RDB和AOF都开启，或者定期执行bgsave做快照备份，RDB方式更适合做数据的备份，AOF可以保证数据的不丢失。

补充：Redis4.0 对于持久化机制的优化

Redis4.0相对与3.X版本其中一个比较大的变化是4.0添加了新的混合持久化方式。

简单的说：新的AOF文件前半段是RDB格式的全量数据后半段是AOF格式的增量数据，如下图：



优势：混合持久化结合了RDB持久化 和 AOF 持久化的优点， 由于绝大部分都是RDB格式，加载速度快，同时结合AOF，增量的数据以AOF方式保存了，数据更少的丢失。

劣势：兼容性差，一旦开启了混合持久化，在4.0之前版本都不识别该aof文件，同时由于前部分是RDB格式，阅读性较差。

3. Redis持久化数据和缓存怎么做扩容?

- 如果Redis被当做缓存使用，使用一致性哈希实现动态扩容缩容。
- 如果Redis被当做一个持久化存储使用，必须使用固定的keys-to-nodes映射关系，节点的数量一旦确定不能变化。否则的话(即Redis节点需要动态变化的情况），必须使用可以在运行时进行数据再平衡的一套系统，而当前只有Redis集群可以做到这样。

过期键的删除策略、淘汰策略

1. Redis过期键的删除策略

Redis的过期删除策略就是：惰性删除和定期删除两种策略配合使用。

惰性删除：惰性删除不会去主动删除数据，而是在访问数据的时候，再检查当前键值是否过期，如果过期则执行删除并返回 null 给客户端，如果没有过期则返回正常信息给客户端。它的优点是简单，不需要对过期的数据做额外的处理，只有在每次访问的时候才会检查键值是否过期，缺点是删除过期键不及时，造成了一定的空间浪费。

定期删除：Redis会周期性的随机测试一批设置了过期时间的key并进行处理。测试到的已过期的key将被删除。

附：删除key常见的三种处理方式。

1、定时删除

在设置某个key 的过期时间同时，我们创建一个定时器，让定时器在该过期时间到来时，立即执行对其进行删除的操作。

优点：定时删除对内存是最友好的，能够保存内存的key一旦过期就能立即从内存中删除。

缺点：对CPU最不友好，在过期键比较多的时候，删除过期键会占用一部分 CPU 时间，对服务器的响应时间和吞吐量造成影响。

2、惰性删除

设置该key 过期时间后，我们不去管它，当需要该key时，我们在检查其是否过期，如果过期，我们就删掉它，反之返回该key。

优点：对 CPU友好，我们只会在使用该键时才会进行过期检查，对于很多用不到的key不用浪费时间进行过期检查。

缺点：对内存不友好，如果一个键已经过期，但是一直没有使用，那么该键就会一直存在内存中，如果数据库中有很多这种使用不到的过期键，这些键便永远不会被删除，内存永远不会释放。从而造成内存泄漏。

3、定期删除

每隔一段时间，我们就对一些key进行检查，删除里面过期的key。

优点：可以通过限制删除操作执行的时长和频率来减少删除操作对 CPU 的影响。另外定期删除，也能有效释放过期键占用的内存。

缺点：难以确定删除操作执行的时长和频率。如果执行的太频繁，定期删除策略变得和定时删除策略一样，对CPU不友好。如果执行的太少，那又和惰性删除一样了，过期键占用的内存不会及时得到释放。另外最重要的是，在获取某个键时，如果某个键的过期时间已经到了，但是还没执行定期删除，那么就会返回这个键的值，这是业务不能忍受的错误。

2. Redis key的过期时间和永久有效分别怎么设置?

通过expire或pexpire命令，客户端可以以秒或毫秒的精度为数据库中的某个键设置生存时间。

与expire和pexpire命令类似，客户端可以通过expireat和pexpireat命令，以秒或毫秒精度给数据库中的某个键设置过期时间，可以理解为：让某个键在某个时间点过期。

3. Redis内存淘汰策略

Redis是不断的删除一些过期数据，但是很多没有设置过期时间的数据也会越来越多，那么Redis内存不够用的时候是怎么处理的呢？答案就是淘汰策略。此类的

当Redis的内存超过最大允许的内存之后，Redis会触发内存淘汰策略，删除一些不常用的数据，以保证Redis服务器的正常运行。

Redisv4.0前提供 6种数据淘汰策略:

- volatile-lru：利用LRU算法移除设置过过期时间的key (LRU:最近使用 Least Recently Used)
- allkeys-lru：当内存不足以容纳新写入数据时，在键空间中，移除最近最少使用的key（这个是最常用的）

- volatile-ttl：从已设置过期时间的数据集（server.db[i].expires）中挑选将要过期的数据淘汰
- volatile-random：从已设置过期时间的数据集（server.db[i].expires）中任意选择数据淘汰
- allkeys-random：从数据集（server.db[i].dict）中任意选择数据淘汰
- no-eviction：禁止驱逐数据，也就是说当内存不足以容纳新写入数据时，新写入操作会报错。这个应该没人使用吧！

Redisv4.0后增加以下两种：

- volatile-lfu：从已设置过期时间的数据集(server.db[i].expires)中挑选最不经常使用的数据淘汰(LFU(Least Frequently Used)算法，也就是最频繁被访问的数据将来最有可能被访问到)
- allkeys-lfu：当内存不足以容纳新写入数据时，在键空间中，移除最不经常使用的key。

内存淘汰策略可以通过配置文件来修改，Redis.conf对应的配置项是maxmemory-policy 修改对应的值就行，默认是noeviction。

缓存异常

> 缓存异常有四种类型，分别是缓存和数据库的数据不一致、缓存雪崩、缓存击穿和缓存穿透。

1. 如何保证缓存与数据库双写时的数据一致性?

> 背景：使用到缓存，无论是本地内存做缓存还是使用 Redis 做缓存，那么就会存在数据同步的问题，因为配置信息缓存在内存中，而内存时无法感知到数据在数据库的修改。这样就会造成数据库中的数据与缓存中数据不一致的问题。

共有四种方案：

1. 先更新数据库，后更新缓存
2. 先更新缓存，后更新数据库
3. 先删除缓存，后更新数据库
4. 先更新数据库，后删除缓存

第一种和第二种方案，没有人使用的，因为第一种方案存在问题是：并发更新数据库场景下，会将脏数据刷到缓存。

第二种方案存在的问题是：如果先更新缓存成功，但是数据库更新失败，则肯定会造成数据不一致。

目前主要用第三和第四种方案。

2. 先删除缓存，后更新数据库

该方案也会出问题，此时来了两个请求，请求 A（更新操作） 和请求 B（查询操作）

1. 请求A进行写操作，删除缓存
2. 请求B查询发现缓存不存在
3. 请求B去数据库查询得到旧值
4. 请求B将旧值写入缓存
5. 请求A将新值写入数据库

上述情况就会导致不一致的情形出现。而且，如果不采用给缓存设置过期时间策略，该数据永远都是脏数据。

答案一：延时双删

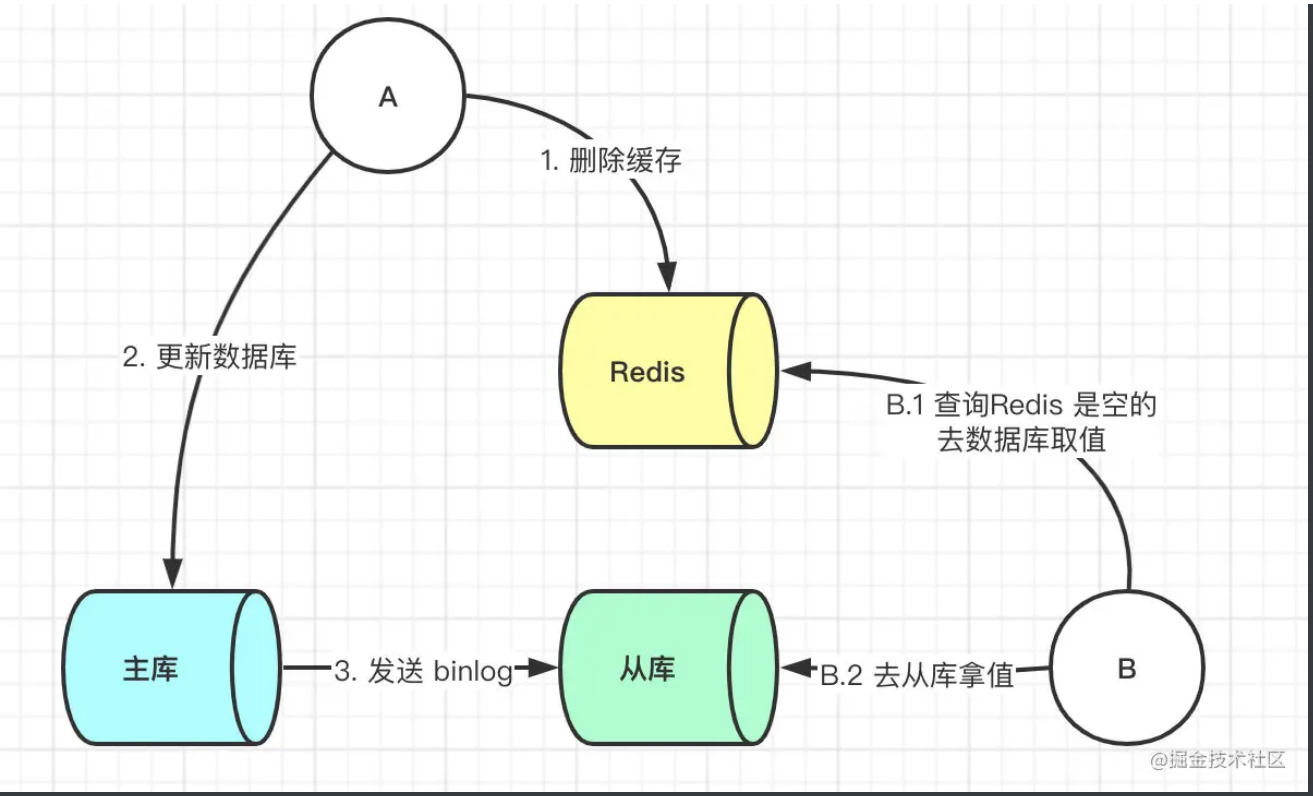
最简单的解决办法延时双删

使用伪代码如下：

```
    public void write(String key,Object data){
            Redis.delKey(key);
            db.updateData(data);
            Thread.sleep(1000);
            Redis.delKey(key);
        }
```

转化为中文描述就是
（1）先淘汰缓存
（2）再写数据库（这两步和原来一样）
（3）休眠1秒，再次淘汰缓存，这么做，可以将1秒内所造成的缓存脏数据，再次删除。确保读请求结束，写请求可以删除读请求造成的缓存脏数据。自行评估自己的项目的读数据业务逻辑的耗时，写数据的休眠时间则在读数据业务逻辑的耗时基础上，加几百ms即可。

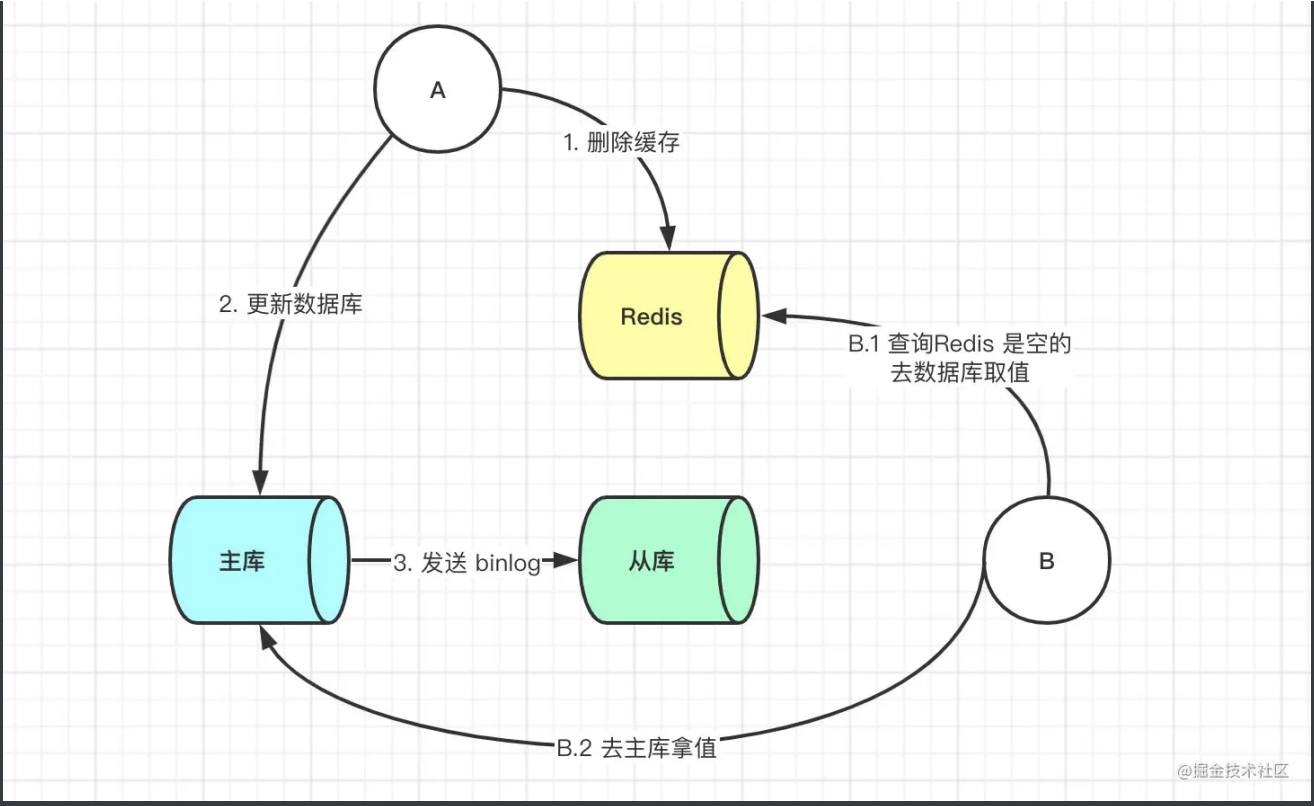
如果使用的是 Mysql 的读写分离的架构的话，那么其实主从同步之间也会有时间差。



此时来了两个请求，请求 A（更新操作） 和请求 B（查询操作）

1. 请求 A 更新操作，删除了 Redis
2. 请求主库进行更新操作，主库与从库进行同步数据的操作
3. 请 B 查询操作，发现 Redis 中没有数据
4. 去从库中拿去数据
5. 此时同步数据还未完成，拿到的数据是旧数据

此时的解决办法就是如果是对 Redis 进行填充数据的查询数据库操作，那么就强制将其指向主库进行查询。



答案二： 更新与读取操作进行异步串行化

采用**更新与读取操作进行异步串行化**

异步串行化

我在系统内部维护n个内存队列，更新数据的时候，根据数据的唯一标识，将该操作路由之后，发送到其中一个jvm内部的内存队列中（对同一数据的请求发送到同一个队列）。读取数据的时候，如果发现数据不在缓存中，并且此时队列里有更新库存的操作，那么将重新读取数据+更新缓存的操作，根据唯一标识路由之后，也将发送到同一个jvm内部的内存队列中。然后每个队列对应一个工作线程，每个工作线程串行地拿到对应的操作，然后一条一条的执行。

这样的话，一个数据变更的操作，先执行删除缓存，然后再去更新数据库，但是还没完成更新的时候，如果此时一个读请求过来，读到了空的缓存，那么可以先将缓存更新的请求发送到队列中，此时会在队列中积压，排在刚才更新库的操作之后，然后同步等待缓存更新完成，再读库。

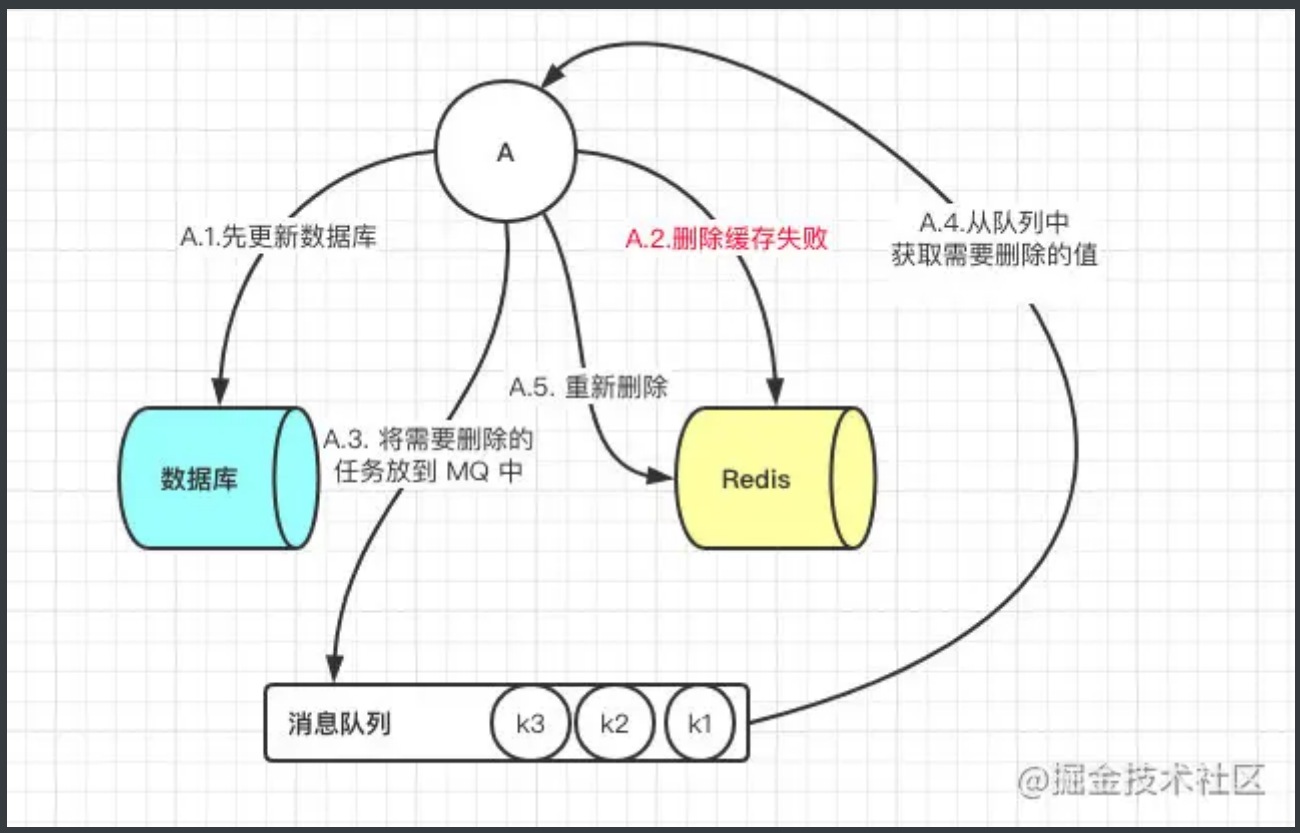
读操作去重

多个读库更新缓存的请求串在同一个队列中是没意义的，因此可以做过滤，如果发现队列中已经有了该数据的更新缓存的请求了，那么就不用再放进去了，直接等待前面的更新操作请求完成即可，待那个队列对应的工作线程完成了上一个操作（数据库的修改）之后，才会去执行下一个操作（读库更新缓存），此时会从数据库中读取最新的值，然后写入缓存中。

如果请求还在等待时间范围内，不断轮询发现可以取到值了，那么就直接返回；如果请求等待的时间超过一定时长，那么这一次直接从数据库中读取当前的旧值。（返回旧值不是又导致缓存和数据库不一致了么？那至少可以减少这个情况发生，因为等待超时也不是每次都是，几率很小吧。这里我想的是，如果超时了就直接读旧值，这时候仅仅是读库后返回而不放缓存）

3.先更新数据库，后删除缓存

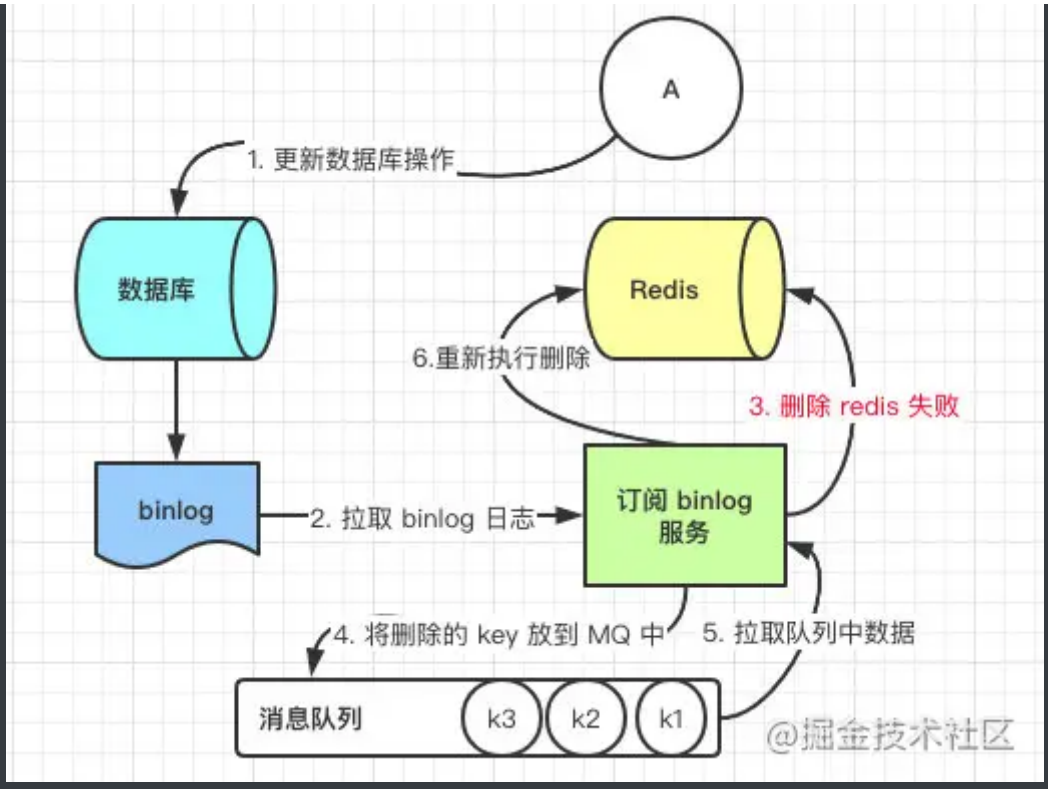
这一种情况也会出现问题，比如更新数据库成功了，但是在删除缓存的阶段出错了没有删除成功，那么此时再读取缓存的时候每次都是错误的数据了。



此时解决方案就是利用消息队列进行删除的补偿。具体的业务逻辑用语言描述如下：

1. 请求 A 先对数据库进行更新操作
2. 在对 Redis 进行删除操作的时候发现报错，删除失败
3. 此时将Redis 的 key 作为消息体发送到消息队列中
4. 系统接收到消息队列发送的消息后再次对 Redis 进行删除操作

但是这个方案会有一个缺点就是会对业务代码造成大量的侵入，深深的耦合在一起，所以这时会有一个优化的方案，我们知道对 Mysql 数据库更新操作后再 binlog 日志中我们都能够找到相应的操作，那么我们可以订阅 Mysql 数据库的 binlog 日志对缓存进行操作。



4. 什么是缓存击穿?

缓存击穿跟缓存雪崩有点类似，缓存雪崩是大规模的key失效，而缓存击穿是某个热点的key失效，大并发集中对其进行请求，就会造成大量请求读缓存没读到数据，从而导致高并发访问数据库，引起数据库压力剧增。这种现象就叫做缓存击穿。

从两个方面解决，第一是否可以考虑热点key不设置过期时间，第二是否可以考虑降低打在数据库上的请求数量。

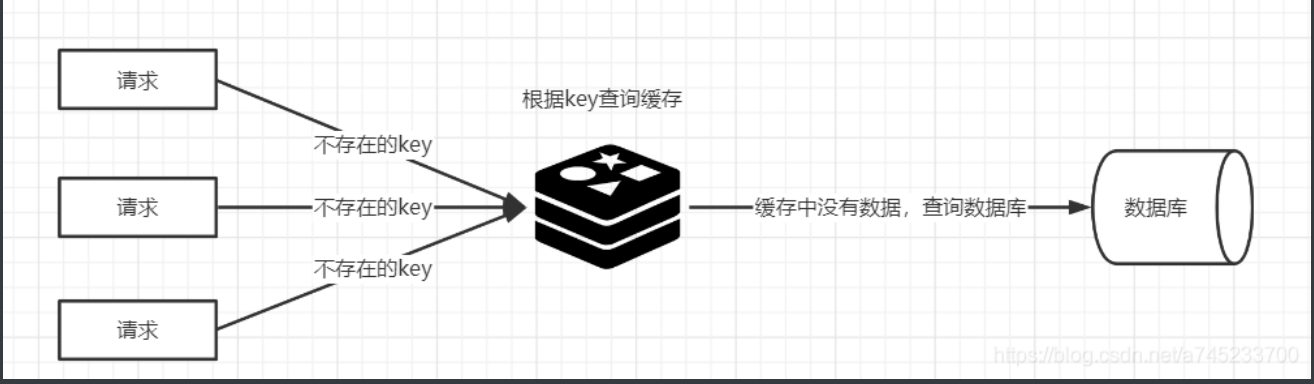
解决方案：

- 在缓存失效后，通过互斥锁或者队列来控制读数据写缓存的线程数量，比如某个key只允许一个线程查询数据和写缓存，其他线程等待。这种方式会阻塞其他的线程，此时系统的吞吐量会下降
- 热点数据缓存永远不过期。永不过期实际包含两层意思：
  - 物理不过期，针对热点key不设置过期时间
  - 逻辑过期，把过期时间存在key对应的value里，如果发现要过期了，通过一个后台的异步线程进行缓存的构建

5. 什么是缓存穿透?

缓存穿透是指用户请求的数据在缓存中不存在即没有命中，同时在数据库中也不存在，导致用户每次请求该数据都要去数据库中查询一遍。如果有恶意攻击者不断请求系统中不存在的数据，会导致短时间大量请求落在数据库上，造成数据库压力过大，甚至导致数据库承受不住而宕机崩溃。

> 缓存穿透的关键在于在Redis中查不到key值，它和缓存击穿的根本区别在于传进来的key在Redis中是不存在的。假如有黑客传进大量的不存在的key，那么大量的请求打在数据库上是很致命的问题，所以在日常开发中要对参数做好校验，一些非法的参数，不可能存在的key就直接返回错误提示。



解决方法：

- 将无效的key存放进Redis中：

当出现Redis查不到数据，数据库也查不到数据的情况，我们就把这个key保存到Redis中，设置value="null"，并设置其过期时间极短，后面再出现查询这个key的请求的时候，直接返回null，就不需要再查询数据库了。但这种处理方式是有问题的，假如传进来的这个不存在的Key值每次都是随机的，那存进Redis也没有意义。

- 使用布隆过滤器：

如果布隆过滤器判定某个 key 不存在布隆过滤器中，那么就一定不存在，如果判定某个 key 存在，那么很大可能是存在(存在一定的误判率)。于是我们可以在缓存之前再加一个布隆过滤器，将数据库中的所有key都存储在布隆过滤器中，在查询Redis前先去布隆过滤器查询 key 是否存在，如果不存在就直接返回，不让其访问数据库，从而避免了对底层存储系统的查询压力。

> 如何选择：针对一些恶意攻击，攻击带过来的大量key是随机，那么我们采用第一种方案就会缓存大量不存在key的数据。那么这种方案就不合适了，我们可以先对使用布隆过滤器方案进行过滤掉这些key。所以，针对这种key异常多、请求重复率比较低的数据，优先使用第二种方案直接过滤掉。而对于空数据的key有限的，重复率比

> 较高的，则可优先采用第一种方式进行缓存。

6.什么是缓存雪崩?

如果缓在某一个时刻出现大规模的key失效，那么就会导致大量的请求打在了数据库上面，导致数据库压力巨大，如果在高并发的情况下，可能瞬间就会导致数据库宕机。这时候如果运维马上又重启数据库，马上又会有新的流量把数据库打死。这就是缓存雪崩。

造成缓存雪崩的关键在于同一时间的大规模的key失效，主要有两种可能：第一种是Redis宕机，第二种可能就是采用了相同的过期时间。

解决方案：

1、事前：

- 均匀过期：设置不同的过期时间，让缓存失效的时间尽量均匀，避免相同的过期时间导致缓存雪崩，造成大量数据库的访问。如把每个Key的失效时间都加个随机值，`setRedis(Key, value, time + Math.random() * 10000);`，保证数据不会在同一时间大面积失效。
- 分级缓存：第一级缓存失效的基础上，访问二级缓存，每一级缓存的失效时间都不同。
- 热点数据缓存永远不过期。永不过期实际包含两层意思：
  - 物理不过期，针对热点key不设置过期时间
  - 逻辑过期，把过期时间存在key对应的value里，如果发现要过期了，通过一个后台的异步线程进行缓存的构建
- 保证Redis缓存的高可用，防止Redis宕机导致缓存雪崩的问题。可以使用 主从+ 哨兵，Redis集群来避免 Redis 全盘崩溃的情况。

2、事中：

- 互斥锁：在缓存失效后，通过互斥锁或者队列来控制读数据写缓存的线程数量，比如某个key只允许一个线程查询数据和写缓存，其他线程等待。这种方式会阻塞其他的线程，此时系统的吞吐量会下降
- 使用熔断机制，限流降级。当流量达到一定的阈值，直接返回“系统拥挤”之类的提示，防止过多的请求打在数据库上将数据库击垮，至少能保证一部分用户是可以正常使用，其他用户多刷新几次也能得到结果。

3、事后：

开启Redis持久化机制，尽快恢复缓存数据，一旦重启，就能从磁盘上自动加载数据恢复内存中的数据。

7. 什么是缓存预热?

缓存预热是指系统上线后，提前将相关的缓存数据加载到缓存系统。避免在用户请求的时候，先查询数据库，然后再将数据缓存的问题，用户直接查询事先被预热的缓存数据。

如果不进行预热，那么Redis初始状态数据为空，系统上线初期，对于高并发的流量，都会访问到数据库中， 对数据库造成流量的压力。

缓存预热解决方案：

- 数据量不大的时候，工程启动的时候进行加载缓存动作；
- 数据量大的时候，设置一个定时任务脚本，进行缓存的刷新；
- 数据量太大的时候，优先保证热点数据进行提前加载到缓存。

8. 什么是缓存降级?

缓存降级是指缓存失效或缓存服务器挂掉的情况下，不去访问数据库，直接返回默认数据或访问服务的内存数据。降级一般是有损的操作，所以尽量减少降级对于业务的影响程度。

在进行降级之前要对系统进行梳理，看看系统是不是可以丢卒保帅；从而梳理出哪些必须誓死保护，哪些可降级；比如可以参考日志级别设置预案：

- 一般：比如有些服务偶尔因为网络抖动或者服务正在上线而超时，可以自动降级；
- 警告：有些服务在一段时间内成功率有波动（如在95~100%之间），可以自动降级或人工降级，并发送告警；
- 错误：比如可用率低于90%，或者数据库连接池被打爆了，或者访问量突然猛增到系统能承受的最大阀值，此时可以根据情况自动降级或者人工降级；
- 严重错误：比如因为特殊原因数据错误了，此时需要紧急人工降级。

线程模型

1. Redis为何选择单线程?

在Redis 6.0以前，Redis的核心网络模型选择用单线程来实现。先来看下官方的回答：

> It's not very frequent that CPU becomes your bottleneck with Redis， as usually Redisis either memory or network bound. For instance， using pipelining Redisrunning on an average Linux system can deliver even 1 million requests per second， so if your application mainly uses O(N) or O(log(N)) commands， it is hardly going to use too much CPU.

核心意思就是，对于一个 DB 来说，CPU 通常不会是瓶颈，因为大多数请求不会是 CPU 密集型的，而是 I/O 密集型。具体到 Redis的话，如果不考虑 RDB/AOF 等持久化方案，Redis是完全的纯内存操作，执行速度是非常快的，因此这部分操作通常不会是性能瓶颈，Redis真正的性能瓶颈在于网络 I/O，也就是客户端和服务端之间的网络传输延迟，因此 Redis选择了单线程的 I/O 多路复用来实现它的核心网络模型。

实际上更加具体的选择单线程的原因如下：

- 避免过多的上下文切换开销：如果是单线程则可以规避进程内频繁的线程切换开销，因为程序始终运行在进程中单个线程内，没有多线程切换的场景。
- 避免同步机制的开销：如果 Redis选择多线程模型，又因为 Redis是一个数据库，那么势必涉及到底层数据同步的问题，则必然会引入某些同步机制，比如锁，而我们知道 Redis不仅仅提供了简单的 key-value 数据结构，还有 list、set 和 hash 等等其他丰富的数据结构，而不同的数据结构对同步访问的加锁粒度又不尽相同，可能会导致在操作数据过程中带来很多加锁解锁的开销，增加程序复杂度的同时还会降低性能。
- 简单可维护：如果 Redis使用多线程模式，那么所有的底层数据结构都必须实现成线程安全的，这无疑又使得 Redis的实现变得更加复杂。

总而言之，Redis选择单线程可以说是多方博弈之后的一种权衡：在保证足够的性能表现之下，使用单线程保持代码的简单和可维护性。

2. Redis真的是单线程?

讨论 这个问题前，先看下 Redis的版本中两个重要的节点：

1. Redisv4.0（引入多线程处理异步任务）
2. Redis 6.0（在网络模型中实现多线程 I/O ）

所以，网络上说的Redis是单线程，通常是指在Redis 6.0之前，其核心网络模型使用的是单线程。

且Redis6.0引入**多线程I/O**，只是用来**处理网络数据的读写和协议的解析**，而**执行命令依旧是单线程**。

> Redis在 v4.0 版本的时候就已经引入了的多线程来做一些异步操作，此举主要针对的是那些非常耗时的命令，通过将这些命令的执行进行异步化，避免阻塞单线程的事件循环。
>
> 在 Redisv4.0 之后增加了一些的非阻塞命令如 `UNLINK`、`FLUSHALL ASYNC`、`FLUSHDB ASYNC`。

3. Redis 6.0为何引入多线程?

很简单，就是 Redis的网络 I/O 瓶颈已经越来越明显了。

随着互联网的飞速发展，互联网业务系统所要处理的线上流量越来越大，Redis的单线程模式会导致系统消耗很多 CPU 时间在网络 I/O 上从而降低吞吐量，要提升 Redis的性能有两个方向：

- 优化网络 I/O 模块
- 提高机器内存读写的速度

后者依赖于硬件的发展，暂时无解。所以只能从前者下手，网络 I/O 的优化又可以分为两个方向：

- 零拷贝技术或者 DPDK 技术
- 利用多核优势

零拷贝技术有其局限性，无法完全适配 Redis这一类复杂的网络 I/O 场景，更多网络 I/O 对 CPU 时间的消耗和 Linux 零拷贝技术。而 DPDK 技术通过旁路网卡 I/O 绕过内核协议栈的方式又太过于复杂以及需要内核甚至是硬件的支持。

总结起来，Redis支持多线程主要就是两个原因：

- 可以充分利用服务器 CPU 资源，目前主线程只能利用一个核
- 多线程任务可以分摊 Redis 同步 IO 读写负荷

4. Redis 6.0 采用多线程后，性能的提升效果如何?

Redis 作者 antirez 在 RedisConf 2019 分享时曾提到：Redis 6 引入的多线程 IO 特性对性能提升至少是一倍以上。

国内也有大牛曾使用 unstable 版本在阿里云 esc 进行过测试，GET/SET 命令在 4 线程 IO 时性能相比单线程是几乎是翻倍了。

5. 介绍下Redis的线程模型

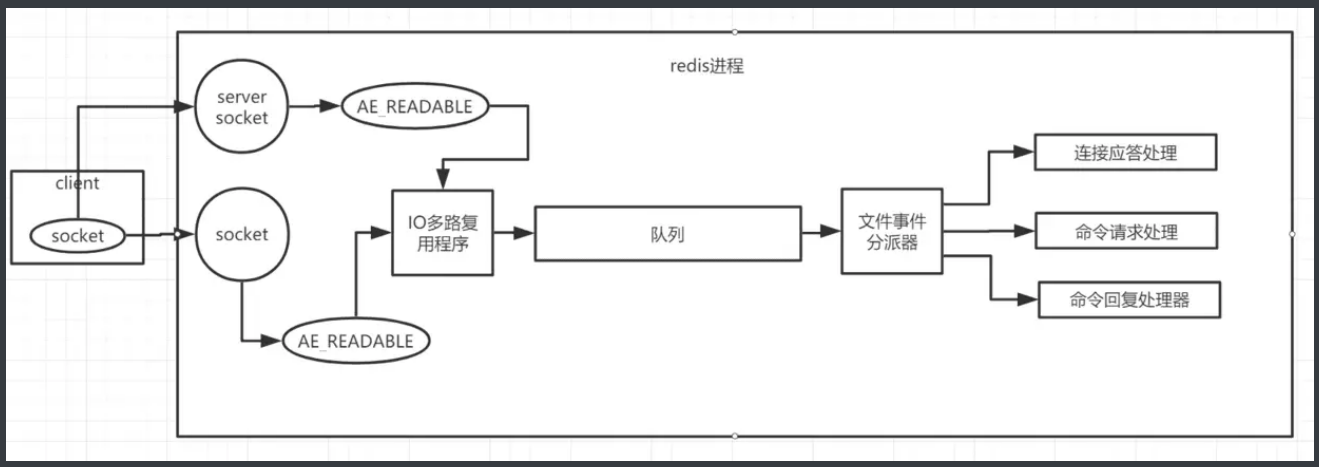
Redis的线程模型包括Redis 6.0之前和Redis 6.0。

下面介绍的是Redis 6.0之前。

Redis 是基于 reactor 模式开发了网络事件处理器，这个处理器叫做文件事件处理器（file event handler）。由于这个文件事件处理器是单线程的，所以 Redis 才叫做单线程的模型。采用 IO 多路复用机制同时监听多个 Socket，根据 socket 上的事件来选择对应的事件处理器来处理这个事件。

> IO多路复用是 IO 模型的一种，有时也称为异步阻塞 IO，是基于经典的 Reactor 设计模式设计的。多路指的是多个 Socket 连接，复用指的是复用一个线程。多路复用主要有三种技术：Select，Poll，Epoll。
>
> Epoll 是最新的也是目前最好的多路复用技术。

模型如下图：



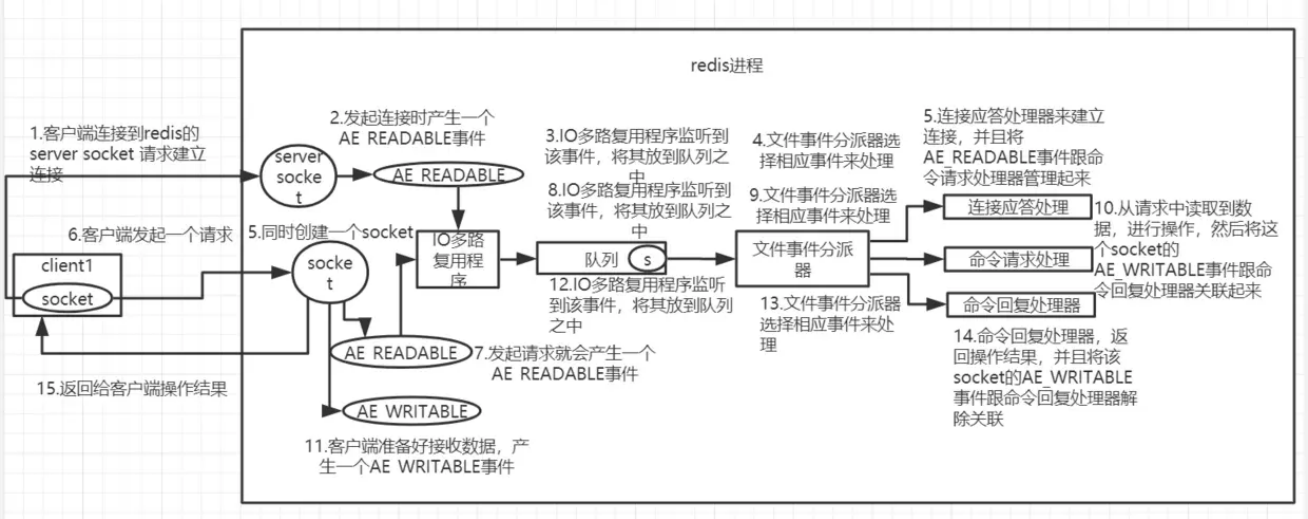
文件事件处理器的结构包含了四个部分：

- 多个 Socket。Socket 会产生 AE_READABLE 和 AE_WRITABLE 事件：
  - 当 socket 变得可读时或者有新的可以应答的 socket 出现时，socket 就会产生一个 AE_READABLE 事件
  - 当 socket 变得可写时，socket 就会产生一个 AE_WRITABLE 事件。
- IO 多路复用程序
- 文件事件分派器
- 事件处理器。事件处理器包括：连接应答处理器、命令请求处理器、命令回复处理器，每个处理器对应不同的 socket 事件：

- 如果是客户端要连接 Redis，那么会为 socket 关联连接应答处理器
- 如果是客户端要写数据到 Redis（读、写请求命令），那么会为 socket 关联命令请求处理器
- 如果是客户端要从 Redis 读数据，那么会为 socket 关联命令回复处理器

多个 socket 会产生不同的事件，不同的事件对应着不同的操作，IO 多路复用程序监听着这些 Socket，当这些 Socket 产生了事件，IO 多路复用程序会将这些事件放到一个队列中，通过这个队列，以有序、同步、每次一个事件的方式向文件时间分派器中传送。当事件处理器处理完一个事件后，IO 多路复用程序才会继续向文件分派器传送下一个事件。

下图是客户端与 Redis 通信的一次完整的流程：

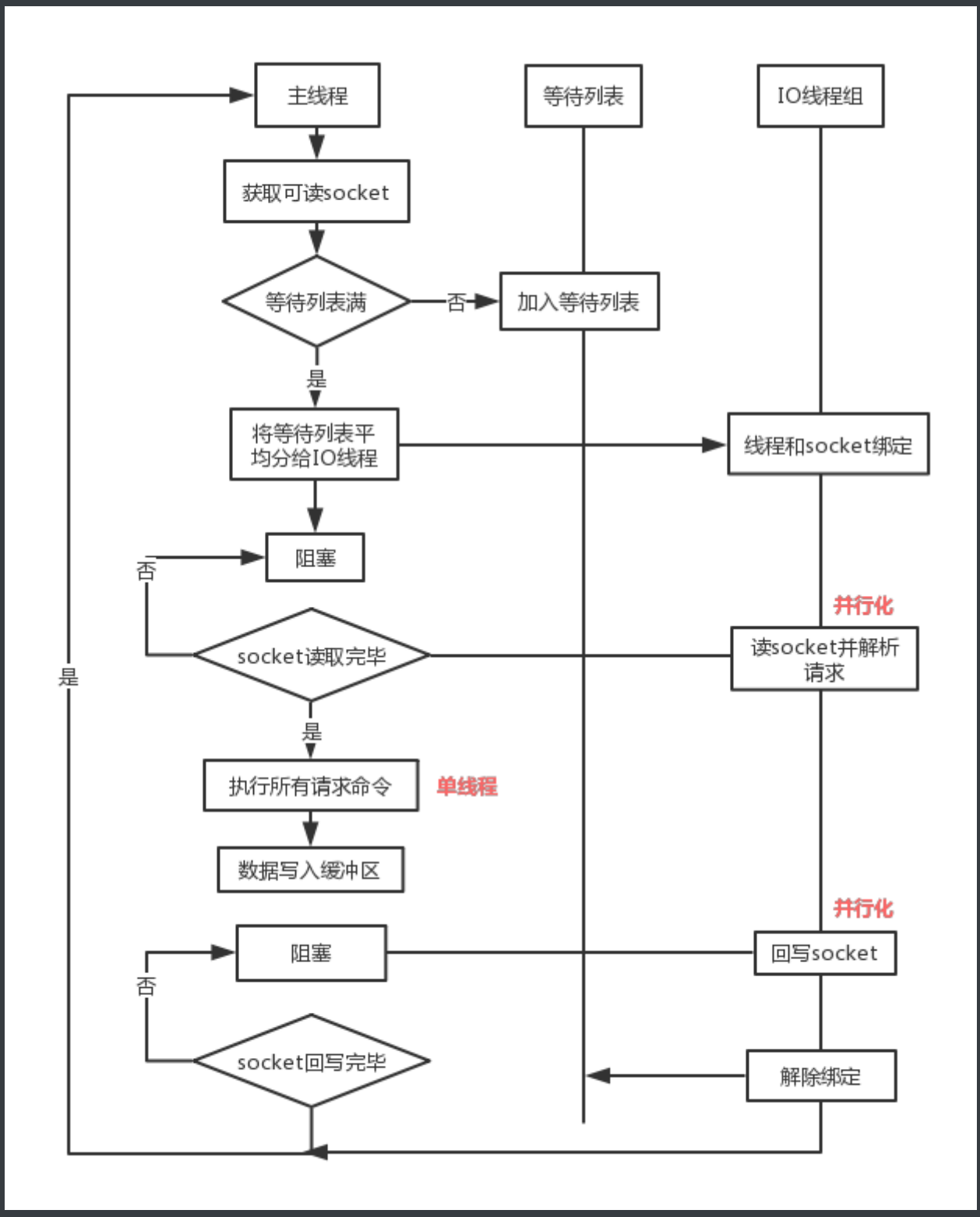


1. Redis 启动初始化的时候，Redis 会将连接应答处理器与 AE_READABLE 事件关联起来。
2. 如果一个客户端跟 Redis 发起连接，此时 Redis 会产生一个 AE_READABLE 事件，由于开始之初 AE_READABLE 是与连接应答处理器关联，所以由连接应答处理器来处理该事件，这时连接应答处理器会与客户端建立连接，创建客户端响应的 socket，同时将这个 socket 的 AE_READABLE 事件与命令请求处理器关联起来。
3. 如果这个时间客户端向 Redis 发送一个命令（set k1 v1），这时 socket 会产生一个 AE_READABLE 事件，IO 多路复用程序会将该事件压入队列中，此时事件分派器从队列中取得该事件，由于该 socket 的 AE_READABLE 事件已经和命令请求处理器关联了，因此事件分派器会将该事件交给命令请求处理器处理，命令请求处理器读取事件中的命令并完成。操作完成后，Redis 会将该 socket 的 AE_WRITABLE 事件与命令回复处理器关联。
4. 如果客户端已经准备好接受数据后，Redis 中的该 socket 会产生一个 AE_WRITABLE 事件，同样会压入队列然后被事件派发器取出交给相对应的命令回复处理器，由该命令回复处理器将准备好的响应数据写入 socket 中，供客户端读取。
5. 命令回复处理器写完后，就会删除该 socket 的 AE_WRITABLE 事件与命令回复处理器的关联关系。

6. Redis 6.0 多线程的实现机制?

流程简述如下:

- 主线程负责接收建立连接请求，获取 Socket 放入全局等待读处理队列。
- 主线程处理完读事件之后，通过 RR（Round Robin）将这些连接分配给这些 IO 线程。
- 主线程阻塞等待 IO 线程读取 Socket 完毕。
- 主线程通过单线程的方式执行请求命令，请求数据读取并解析完成，但并不执行。
- 主线程阻塞等待 IO 线程将数据回写 Socket 完毕。



该设计有如下特点：

- IO 线程要么同时在读 Socket，要么同时在写，不会同时读或写。
- IO 线程只负责读写 Socket 解析命令，不负责命令处理。

7. Redis 6.0开启多线程后，是否会存在线程并发安全问题?

从实现机制可以看出，Redis 的多线程部分只是用来处理网络数据的读写和协议解析，执行命令仍然是单线程顺序执行。

所以我们不需要去考虑控制 Key、Lua、事务，LPUSH/LPOP 等等的并发及线程安全问题。

8. Redis 6.0 与 Memcached 多线程模型的对比

- **相同点**：都采用了 Master 线程 -Worker 线程的模型。
- **不同点**：Memcached 执行主逻辑也是在 Worker 线程里，模型更加简单，实现了真正的线程隔离，符合我们对线程隔离的常规理解。

  而 Redis 把处理逻辑交还给 Master 线程，虽然一定程度上增加了模型复杂度，但也解决了线程并发安全等问题。

事务

1. Redis事务的概念

Redis的事务并不是我们传统意义上理解的事务，我们都知道 单个 Redis 命令的执行是原子性的，但 Redis 没有在事务上增加任何维持原子性的机制，所以 Redis **事务的执行并不是原子性的**。

事务可以理解为一个**打包的批量执行脚本**，但**批量指令并非原子化**的操作，中间某条指令的失败不会导致前面已做指令的回滚，也不会造成后续的指令不做。

总结：

1. Redis事务中如果有某一条命令执行失败，之前的命令不会回滚，其后的命令仍然会被继续执行。**鉴于这个原因，所以说Redis的事务严格意义上来说是不具备原子性的。**
2. Redis事务中所有命令都会序列化、按顺序地执行。事务在执行的过程中，不会被其他客户端发送来的命令请求所打断。
3. 在事务开启之前，如果客户端与服务器之间出现通讯故障并导致网络断开，其后所有待执行的语句都将不会被服务器执行。然而如果网络中断事件是发生在客户端执行EXEC命令之后，那么该事务中的所有命令都会被服务器执行。

> 当使用Append-Only模式时，Redis会通过调用系统函数write将该事务内的所有写操作在本次调用中全部写入磁盘。然而如果在写入的过程中出现系统崩溃，如电源故障导致的宕机，那么此时也许只有部分数据被写入到磁盘，而另外一部分数据却已经丢失。Redis服务器会在重新启动时执行一系列必要的一致性检测，一旦发现类似问题，就会立即退出并给出相应的错误提示。此时，我们就要充分利用Redis工具包中提供的Redis-check-aof工具，该工具可以帮助我们定位到数据不一致的错误，并将已经写入的部分数据进行回滚。修复之后我们就可以再次重新启动Redis服务器了。

2. Redis事务的三个阶段

1. multi 开启事务
2. 大量指令入队
3. exec执行事务块内命令，**截止此处一个事务已经结束。**
4. discard 取消事务
5. watch 监视一个或多个key，如果事务执行前key被改动，事务将打断。unwatch 取消监视。

事务执行过程中，如果服务端收到有EXEC、DISCARD、WATCH、MULTI之外的请求，将会把请求放入队列中排队.

3. Redis事务相关命令

Redis事务功能是通过MULTI、EXEC、DISCARD和WATCH 四个原语实现的

- WATCH 命令是一个乐观锁，可以为 Redis 事务提供 check-and-set （CAS）行为。 可以监控一个或多个键，一旦其中有一个键被修改（或删除），之后的事务就不会执行，监控一直持续到EXEC命令。
- MULTI命令用于开启一个事务，它总是返回OK。 MULTI执行之后，客户端可以继续向服务器发送任意多条命令，这些命令不会立即被执行，而是被放到一个队列中，当EXEC命令被调用时，所有队列中的命令才会被执行。
- EXEC：执行所有事务块内的命令。返回事务块内所有命令的返回值，按命令执行的先后顺序排列。 当操作被打断时，返回空值 nil 。
  通过调用DISCARD，客户端可以清空事务队列，并放弃执行事务， 并且客户端会从事务状态中退出。
- UNWATCH命令可以取消watch对所有key的监控。

4. Redis事务支持隔离性吗?

Redis 是单进程程序，并且它保证在执行事务时，不会对事务进行中断，事务可以运行直到执行完所有事务队列中的命令为止。因此，**Redis 的事务是总是带有隔离性的**。

5. Redis为什么不支持事务回滚?

- Redis 命令只会因为错误的语法而失败，或是命令用在了错误类型的键上面，这些问题不能在入队时发现，这也就是说，从实用性的角度来说，失败的命令是由编程错误造成的，而这些错误应该在开发的过程中被发现，而不应该出现在生产环境中.
- 因为不需要对回滚进行支持，所以 Redis 的内部可以保持简单且快速。

6. Redis事务其他实现

- 基于Lua脚本，Redis可以保证脚本内的命令一次性、按顺序地执行，
  其同时也不提供事务运行错误的回滚，执行过程中如果部分命令运行错误，剩下的命令还是会继续运行完。
- 基于中间标记变量，通过另外的标记变量来标识事务是否执行完成，读取数据时先读取该标记变量判断是否事务执行完成。但这样会需要额外写代码实现，比较繁琐。

主从、哨兵、集群

1. Redis常见使用方式有哪些?

Redis的几种常见使用方式包括：

- Redis单副本；
- Redis多副本（主从）；
- Redis Sentinel（哨兵）；
- Redis Cluster；
- Redis自研。

使用场景：

如果数据量很少，主要是承载高并发高性能的场景，比如缓存一般就几个G的话，单机足够了。

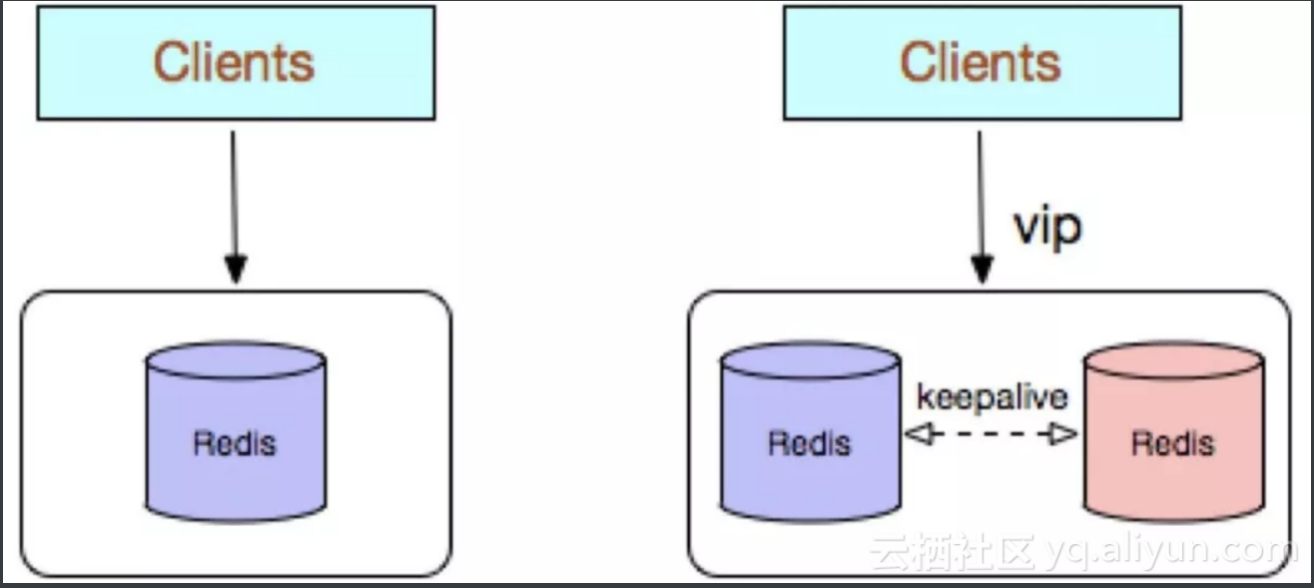
主从模式：master 节点挂掉后，需要手动指定新的 master，可用性不高，基本不用。

哨兵模式：master 节点挂掉后，哨兵进程会主动选举新的 master，可用性高，但是每个节点存储的数据是一样的，浪费内存空间。数据量不是很多，集群规模不是很大，需要自动容错容灾的时候使用。

Redis cluster 主要是针对海量数据+高并发+高可用的场景，如果是海量数据，如果你的数据量很大，那么建议就用Redis cluster，所有master的容量总和就是Redis cluster可缓存的数据容量。

2. 介绍下Redis单副本

Redis单副本，采用单个Redis节点部署架构，没有备用节点实时同步数据，不提供数据持久化和备份策略，适用于数据可靠性要求不高的纯缓存业务场景。



优点：

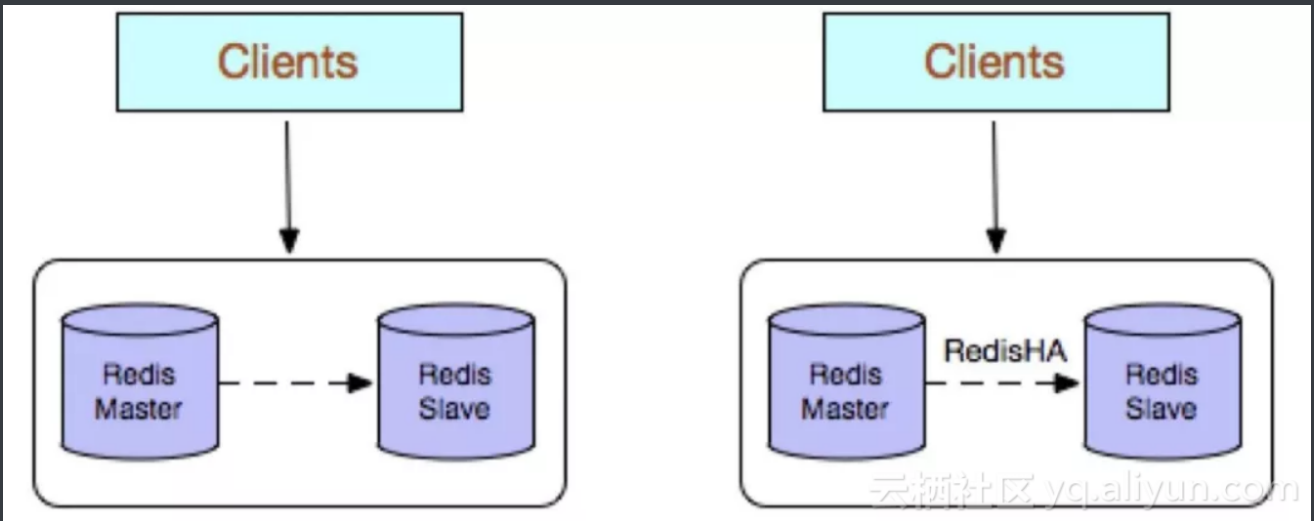
- 架构简单，部署方便；
- 高性价比：缓存使用时无需备用节点（单实例可用性可以用supervisor或crontab保证），当然为了满足业务的高可用性，也可以牺牲一个备用节点，但同时刻只有一个实例对外提供服务；
- 高性能。

缺点：

- 不保证数据的可靠性；
- 在缓存使用，进程重启后，数据丢失，即使有备用的节点解决高可用性，但是仍然不能解决缓存预热问题，因此不适用于数据可靠性要求高的业务；
- 高性能受限于单核CPU的处理能力（Redis是单线程机制），CPU为主要瓶颈，所以适合操作命令简单，排序、计算较少的场景。也可以考虑用Memcached替代。

3. 介绍下Redis多副本（主从）

Redis多副本，采用主从（replication）部署结构，相较于单副本而言最大的特点就是主从实例间数据实时同步，并且提供数据持久化和备份策略。主从实例部署在不同的物理服务器上，根据公司的基础环境配置，可以实现同时对外提供服务和读写分离策略。



优点：

- 高可靠性：一方面，采用双机主备架构，能够在主库出现故障时自动进行主备切换，从库提升为主库提供服务，保证服务平稳运行；另一方面，开启数据持久化功能和配置合理的备份策略，能有效的解决数据误操作和数据异常丢失的问题；
- 读写分离策略：从节点可以扩展主库节点的读能力，有效应对大并发量的读操作。

缺点：

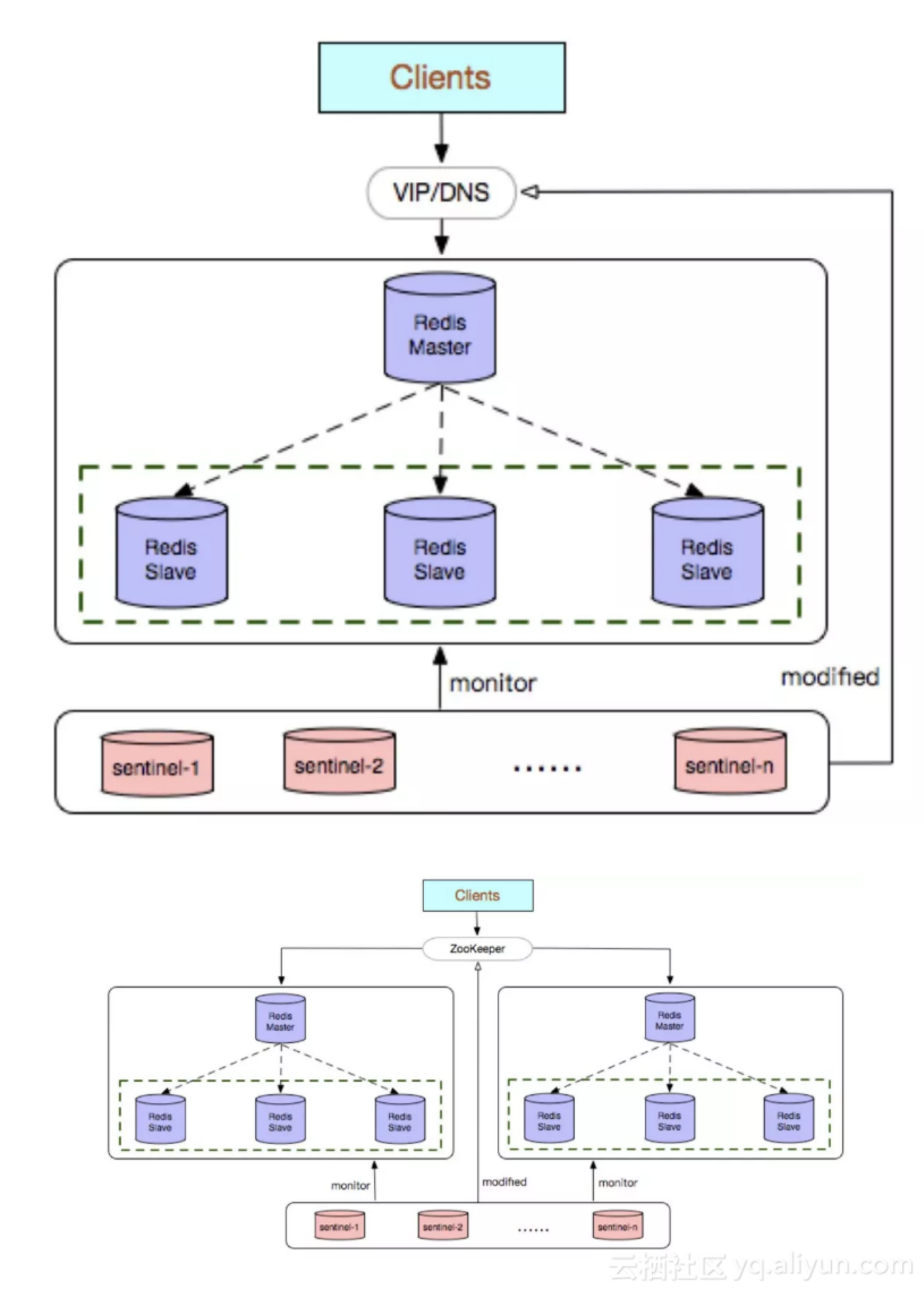
- 故障恢复复杂，如果没有RedisHA系统（需要开发），当主库节点出现故障时，需要手动将一个从节点晋升为主节点，同时需要通知业务方变更配置，并且需要让其它从库节点去复制新主库节点，整个过程需要人为干预，比较繁琐；
- 主库的写能力受到单机的限制，可以考虑分片；
- 主库的存储能力受到单机的限制，可以考虑Pika；
- 原生复制的弊端在早期的版本中也会比较突出，如：Redis复制中断后，Slave会发起psync，此时如果同步不成功，则会进行全量同步，主库执行全量备份的同时可能会造成毫秒或秒级的卡顿；又由于COW机制，导致极端情况下的主库内存溢出，程序异常退出或宕机；主库节点生成备份文件导致服务器磁盘IO和CPU（压缩）资源消耗；发送数GB大小的备份文件导致服务器出口带宽暴增，阻塞请求，建议升级到最新版本。

4. 介绍下Redis Sentinel（哨兵）

> 主从模式下，当主服务器宕机后，需要手动把一台从服务器切换为主服务器，这就需要人工干预，费事费力，还会造成一段时间内服务不可用。这种方式并不推荐，实际生产中，我们优先考虑哨兵模式。这种模式下，master 宕机，哨兵会自动选举 master 并将其他的 slave 指向新的 master。

Redis Sentinel是社区版本推出的原生高可用解决方案，其部署架构主要包括两部分：Redis Sentinel集群和Redis数据集群。

其中Redis Sentinel集群是由若干Sentinel节点组成的分布式集群，可以实现故障发现、故障自动转移、配置中心和客户端通知。Redis Sentinel的节点数量要满足2n+1（n>=1）的奇数个。



优点：

- Redis Sentinel集群部署简单；
- 能够解决Redis主从模式下的高可用切换问题；

- 很方便实现Redis数据节点的线形扩展，轻松突破Redis自身单线程瓶颈，可极大满足Redis大容量或高性能的业务需求；
- 可以实现一套Sentinel监控一组Redis数据节点或多组数据节点。

缺点：

- 部署相对Redis主从模式要复杂一些，原理理解更繁琐；
- 资源浪费，Redis数据节点中slave节点作为备份节点不提供服务；
- Redis Sentinel主要是针对Redis数据节点中的主节点的高可用切换，对Redis的数据节点做失败判定分为主观下线和客观下线两种，对于Redis的从节点有对节点做主观下线操作，并不执行故障转移。
- 不能解决读写分离问题，实现起来相对复杂。

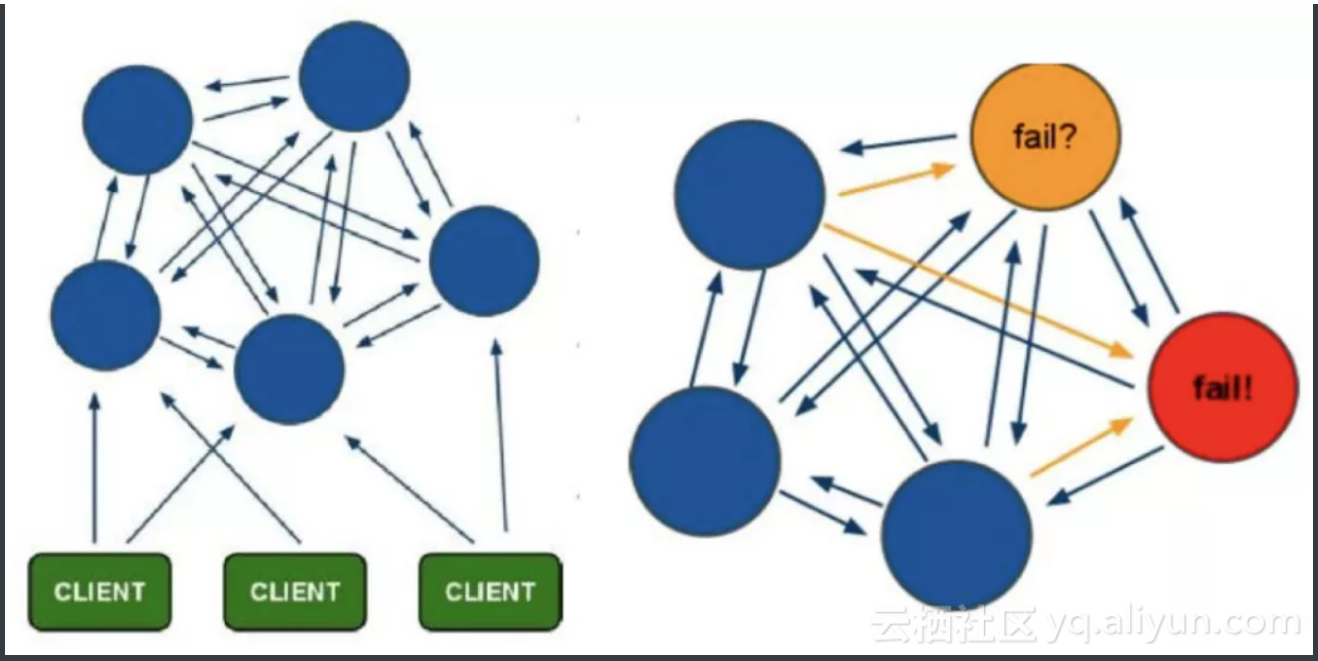
5. 介绍下Redis Cluster

> Redis 的哨兵模式基本已经可以实现高可用，读写分离 ，但是在这种模式下每台 Redis 服务器都存储相同的数据，很浪费内存，所以在 Redis3.0 上加入了 Cluster 集群模式，实现了 Redis 的分布式存储，对数据进行分片，也就是说每台 Redis 节点上存储不同的内容。

Redis Cluster是社区版推出的Redis分布式集群解决方案，主要解决Redis分布式方面的需求，比如，当遇到单机内存，并发和流量等瓶颈的时候，Redis Cluster能起到很好的负载均衡的目的。

Redis Cluster集群节点最小配置6个节点以上（3主3从），其中主节点提供读写操作，从节点作为备用节点，不提供请求，只作为故障转移使用。

Redis Cluster采用虚拟槽分区，所有的键根据哈希函数映射到0 ~ 16383个整数槽内，每个节点负责维护一部分槽以及槽所印映射的键值数据。



优点：

- 无中心架构；
- 数据按照slot存储分布在多个节点，节点间数据共享，可动态调整数据分布；
- 可扩展性：可线性扩展到1000多个节点，节点可动态添加或删除；
- 高可用性：部分节点不可用时，集群仍可用。通过增加Slave做standby数据副本，能够实现故障自动failover，节点之间通过gossip协议交换状态信息，用投票机制完成Slave到Master的角色提升；
- 降低运维成本，提高系统的扩展性和可用性。

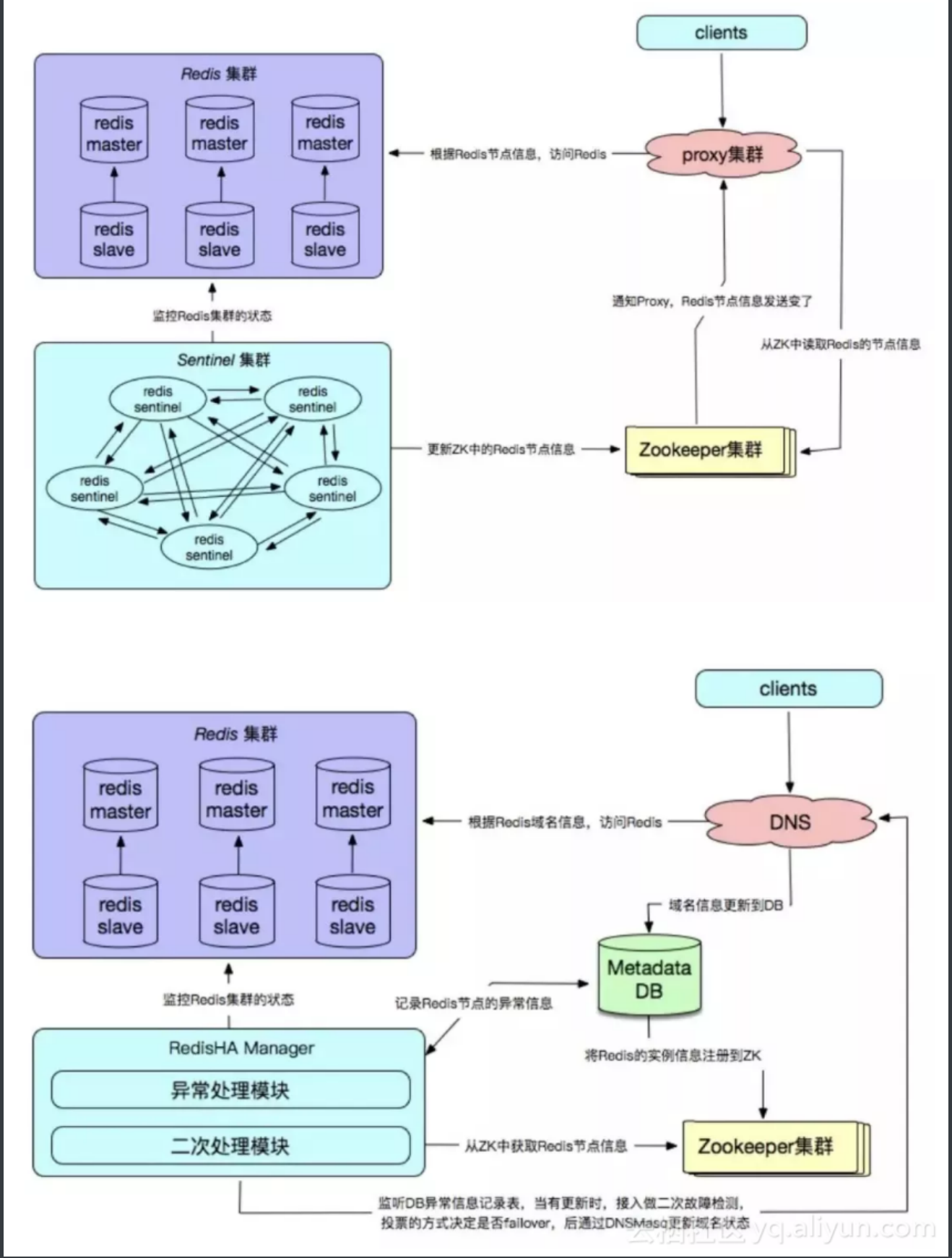
缺点：

- Client实现复杂，驱动要求实现Smart Client，缓存slots mapping信息并及时更新，提高了开发难度，客户端的不成熟影响业务的稳定性。目前仅JedisCluster相对成熟，异常处理部分还不完善，比如常见的“max redirect exception”。
- 节点会因为某些原因发生阻塞（阻塞时间大于clutser-node-timeout），被判断下线，这种failover是没有必要的。
- 数据通过异步复制，不保证数据的强一致性。
- 多个业务使用同一套集群时，无法根据统计区分冷热数据，资源隔离性较差，容易出现相互影响的情况。
- Slave在集群中充当“冷备”，不能缓解读压力，当然可以通过SDK的合理设计来提高Slave资源的利用率。
- Key批量操作限制，如使用mset、mget目前只支持具有相同slot值的Key执行批量操作。对于映射为不同slot值的Key由于Keys不支持跨slot查询，所以执行mset、mget、sunion等操作支持不友好。
- Key事务操作支持有限，只支持多key在同一节点上的事务操作，当多个Key分布于不同的节点上时无法使用事务功能。

- Key作为数据分区的最小粒度，不能将一个很大的键值对象如hash、list等映射到不同的节点。
- 不支持多数据库空间，单机下的Redis可以支持到16个数据库，集群模式下只能使用1个数据库空间，即db 0。
- 复制结构只支持一层，从节点只能复制主节点，不支持嵌套树状复制结构。
- 避免产生hot-key，导致主库节点成为系统的短板。
- 避免产生big-key，导致网卡撑爆、慢查询等。
- 重试时间应该大于cluster-node-time时间。
- Redis Cluster不建议使用pipeline和multi-keys操作，减少max redirect产生的场景。

6. 介绍下Redis自研

Redis自研的高可用解决方案，主要体现在配置中心、故障探测和failover的处理机制上，通常需要根据企业业务的实际线上环境来定制化。



优点：

- 高可靠性、高可用性；
- 自主可控性高；
- 贴切业务实际需求，可缩性好，兼容性好。

缺点：

- 实现复杂，开发成本高；
- 需要建立配套的周边设施，如监控，域名服务，存储元数据信息的数据库等；
- 维护成本高。

7. Redis高可用方案具体怎么实施?

使用官方推荐的哨兵(sentinel)机制就能实现，当主节点出现故障时，由Sentinel自动完成故障发现和转移，并通知应用方，实现高可用性。它有四个主要功能：

- 集群监控，负责监控Redis master和slave进程是否正常工作。
- 消息通知，如果某个Redis实例有故障，那么哨兵负责发送消息作为报警通知给管理员。
- 故障转移，如果master node挂掉了，会自动转移到slave node上。
- 配置中心，如果故障转移发生了，通知client客户端新的master地址。

43. 了解主从复制的原理吗?

1、主从架构的核心原理

当启动一个slave node的时候，它会发送一个PSYNC命令给master node

如果这是slave node重新连接master node，那么master node仅仅会复制给slave部分缺少的数据; 否则如果是slave node第一次连接master node，那么会触发一次full resynchronization

开始full resynchronization的时候，master会启动一个后台线程，开始生成一份RDB快照文件，同时还会将从客户端收到的所有写命令缓存在内存中。RDB文件生成完毕之后，master会将这个RDB发送给slave，slave会先写入本地磁盘，然后再从本地磁盘加载到内存中。然后master会将内存中缓存的写命令发送给slave，slave也会同步这些数据。

slave node如果跟master node有网络故障，断开了连接，会自动重连。master如果发现有多个slave node都来重新连接，仅仅会启动一个rdb save操作，用一份数据服务所有slave node。

2、主从复制的断点续传

从Redis 2.8开始，就支持主从复制的断点续传，如果主从复制过程中，网络连接断掉了，那么可以接着上次复制的地方，继续复制下去，而不是从头开始复制一份

master node会在内存中常见一个backlog，master和slave都会保存一个replica offset还有一个master id，offset就是保存在backlog中的。如果master和slave网络连接断掉了，slave会让master从上次的replica offset开始继续复制

但是如果没有找到对应的offset，那么就会执行一次resynchronization

3、无磁盘化复制

master在内存中直接创建rdb，然后发送给slave，不会在自己本地落地磁盘了

repl-diskless-sync repl-diskless-sync-delay，等待一定时长再开始复制，因为要等更多slave重新连接过来

4、过期key处理

slave不会过期key，只会等待master过期key。如果master过期了一个key，或者通过LRU淘汰了一个key，那么会模拟一条del命令发送给slave。

8. 由于主从延迟导致读取到过期数据怎么处理?

1. 通过scan命令扫库：当Redis中的key被scan的时候，相当于访问了该key，同样也会做过期检测，充分发挥Redis惰性删除的策略。这个方法能大大降低了脏数据读取的概率，但缺点也比较明显，会造成一定的数据库压力，否则影响线上业务的效率。
2. Redis加入了一个新特性来解决主从不一致导致读取到过期数据问题，增加了key是否过期以及对主从库的判断，如果key已过期，当前访问的master则返回null；当前访问的是从库，且执行的是只读命令也返回null。

9. 主从复制的过程中如果因为网络原因停止复制了会怎么样?

如果出现网络故障断开连接了，会自动重连的，从Redis 2.8开始，就支持主从复制的断点续传，可以接着上次复制的地方，继续复制下去，而不是从头开始复制一份。

master如果发现有多个slave node都来重新连接，仅仅会启动一个rdb save操作，用一份数据服务所有slave node。

master node会在内存中创建一个 `backlog` ，master和slave都会保存一个 `replica offset` ，还有一个 `master id` ，offset就是保存在backlog中的。如果master和slave网络连接断掉了，slave会让master从上次的replica offset开始继续复制。

但是如果没有找到对应的offset，那么就会执行一次 `resynchronization` 全量复制。

10. Redis主从架构数据会丢失吗，为什么?

有两种数据丢失的情况：

1. 异步复制导致的数据丢失：因为master -> slave的复制是异步的，所以可能有部分数据还没复制到slave，master就宕机了，此时这些部分数据就丢失了。
2. 脑裂导致的数据丢失：某个master所在机器突然脱离了正常的网络，跟其他slave机器不能连接，但是实际上master还运行着，此时哨兵可能就会认为master宕机了，然后开启选举，将其他slave切换成了master。这个时候，集群里就会有两个master，也就是所谓的脑裂。此时虽然某个slave被切换成了master，但是可能client还没来得及切换到新的master，还继续写向旧master的数据可能也丢失了。因此旧master再次恢复的时候，会被作为一个slave挂到新的master上去，自己的数据会清空，重新从新的master复制数据。

11. 如何解决主从架构数据丢失的问题?

数据丢失的问题是不可避免的，但是我们可以尽量减少。

在Redis的配置文件里设置参数

```
min-slaves-to-write 1
min-slaves-max-lag 10
```

`min-slaves-to-write` 默认情况下是0， `min-slaves-max-lag` 默认情况下是10。

上面的配置的意思是要求至少有1个slave，数据复制和同步的延迟不能超过10秒。如果说一旦所有的slave，数据复制和同步的延迟都超过了10秒钟，那么这个时候，master就不会再接收任何请求了。

减小 `min-slaves-max-lag` 参数的值，这样就可以避免在发生故障时大量的数据丢失，一旦发现延迟超过了该值就不会往master中写入数据。

那么对于client，我们可以采取降级措施，将数据暂时写入本地缓存和磁盘中，在一段时间后重新写入master来保证数据不丢失；也可以将数据写入kafka消息队列，隔一段时间去消费kafka中的数据。

12. Redis哨兵是怎么工作的?

1. 每个Sentinel以每秒钟一次的频率向它所知的Master，Slave以及其他 Sentinel 实例发送一个 PING 命令。

2. 如果一个实例（instance）距离最后一次有效回复 PING 命令的时间超过 down-after-milliseconds 选项所指定的值， 则这个实例会被当前 Sentinel 标记为主观下线。
3. 如果一个Master被标记为主观下线，则正在监视这个Master的所有 Sentinel 要以每秒一次的频率确认Master的确进入了主观下线状态。
4. 当有足够数量的 Sentinel（大于等于配置文件指定的值）在指定的时间范围内确认Master的确进入了主观下线状态， 则Master会被标记为客观下线 。
5. 当Master被 Sentinel 标记为客观下线时，Sentinel 向下线的 Master 的所有 Slave 发送 INFO 命令的频率会从 10 秒一次改为每秒一次 （在一般情况下， 每个 Sentinel 会以每 10 秒一次的频率向它已知的所有Master，Slave发送 INFO 命令 ）。
6. 若没有足够数量的 Sentinel 同意 Master 已经下线， Master 的客观下线状态就会变成主观下线。若 Master 重新向 Sentinel 的 PING 命令返回有效回复， Master 的主观下线状态就会被移除。
7. sentinel节点会与其他sentinel节点进行“沟通”，投票选举一个sentinel节点进行故障处理，在从节点中选取一个主节点，其他从节点挂载到新的主节点上自动复制新主节点的数据。

13. 故障转移时会从剩下的slave选举一个新的master，被选举为master的标准是什么?

如果一个master被认为odown了，而且majority哨兵都允许了主备切换，那么某个哨兵就会执行主备切换操作，此时首先要选举一个slave来，会考虑slave的一些信息。

- 跟master断开连接的时长。
  如果一个slave跟master断开连接已经超过了down-after-milliseconds的10倍，外加master宕机的时长，那么slave就被认为不适合选举为master.

```
( down-after-milliseconds * 10) +
milliseconds_since_master_is_in_SDOWN_state
```

- slave优先级。
  按照slave优先级进行排序，slave priority越低，优先级就越高
- 复制offset。
  如果slave priority相同，那么看replica offset，哪个slave复制了越多的数据，offset越靠后，优先级就越高
- run id
  如果上面两个条件都相同，那么选择一个run id比较小的那个slave。

14. 同步配置的时候其他哨兵根据什么更新自己的配置呢?

执行切换的那个哨兵，会从要切换到的新master（salve->master）那里得到一个configuration epoch，这就是一个version号，每次切换的version号都必须是唯一的。

如果第一个选举出的哨兵切换失败了，那么其他哨兵，会等待failover-timeout时间，然后接替继续执行切换，此时会重新获取一个新的configuration epoch 作为新的version号。

这个version号就很重要了，因为各种消息都是通过一个channel去发布和监听的，所以一个哨兵完成一次新的切换之后，新的master配置是跟着新的version号的，其他的哨兵都是根据版本号的大小来更新自己的master配置的。

15. 为什么Redis哨兵集群只有2个节点无法正常工作?

哨兵集群必须部署2个以上节点。

如果两个哨兵实例，即两个Redis实例，一主一从的模式。

则Redis的配置quorum=1，表示一个哨兵认为master宕机即可认为master已宕机。

但是如果是机器1宕机了，那哨兵1和master都宕机了，虽然哨兵2知道master宕机了，但是这个时候，需要majority，也就是大多数哨兵都是运行的，2个哨兵的majority就是2（2的majority=2，3的majority=2，5的majority=3，4的majority=2），2个哨兵都运行着，就可以允许执行故障转移。

但此时哨兵1没了就只有1个哨兵了了，此时就没有majority来允许执行故障转移，所以故障转移不会执行。

16. Redis cluster中是如何实现数据分布的？这种方式有什么优点?

Redis cluster有固定的16384个hash slot（哈希槽），对每个key计算CRC16值，然后对16384取模，可以获取key对应的hash slot。

Redis cluster中每个master都会持有部分slot（槽），比如有3个master，那么可能每个master持有5000多个hash slot。

hash slot让node的增加和移除很简单，增加一个master，就将其他master的hash slot移动部分过去，减少一个master，就将它的hash slot移动到其他master上去。每次增加或减少master节点都是对16384取模，而不是根据master数量，这样原本在老的master上的数据不会因master的新增或减少而找不到。并且增加或减少master时Redis cluster移动hash slot的成本是非常低的。

17. Redis cluster节点间通信是什么机制?

Redis cluster节点间采取gossip协议进行通信，所有节点都持有一份元数据，不同的节点如果出现了元数据的变更之后U不断地i将元数据发送给其他节点让其他节点进行数据变更。

> 节点互相之间不断通信，保持整个集群所有节点的数据是完整的。
> 主要交换故障信息、节点的增加和移除、hash slot信息等。

这种机制的好处在于，元数据的更新比较分散，不是集中在一个地方，更新请求会陆陆续续，打到所有节点上去更新，有一定的延时，降低了压力;

缺点，元数据更新有延时，可能导致集群的一些操作会有一些滞后。

分布式问题

1. 什么是分布式锁？为什么用分布式锁?

锁在程序中的作用就是同步工具，保证共享资源在同一时刻只能被一个线程访问，Java中的锁我们都很熟悉了，像synchronized 、Lock都是我们经常使用的，但是Java的锁只能保证单机的时候有效，分布式集群环境就无能为力了，这个时候我们就需要用到分布式锁。

分布式锁，顾名思义，就是分布式项目开发中用到的锁，可以用来控制分布式系统之间同步访问共享资源。

思路是：在整个系统提供一个**全局、唯一**的获取锁的“东西”，然后每个系统在需要加锁时，都去问这个“东西”拿到一把锁，这样不同的系统拿到的就可以认为是同一把锁。至于这个“东西”，可以是Redis、Zookeeper，也可以是数据库。

一般来说，分布式锁需要满足的特性有这么几点：

1、互斥性：在任何时刻，对于同一条数据，只有一台应用可以获取到分布式锁;

2、高可用性：在分布式场景下，一小部分服务器宕机不影响正常使用，这种情况就需要将提供分布式锁的服务以集群的方式部署;

3、防止锁超时：如果客户端没有主动释放锁，服务器会在一段时间之后自动释放锁，防止客户端宕机或者网络不可达时产生死锁;

4、独占性：加锁解锁必须由同一台服务器进行，也就是锁的持有者才可以释放锁，不能出现你加的锁，别人给你解锁了。

2. 常见的分布式锁有哪些解决方案?

实现分布式锁目前有三种流行方案，即基于关系型数据库、Redis、ZooKeeper 的方案

1、基于关系型数据库，如MySQL
基于关系型数据库实现分布式锁，是依赖数据库的唯一性来实现资源锁定，比如主键和唯一索引等。

缺点：

- 这把锁强依赖数据库的可用性，数据库是一个单点，一旦数据库挂掉，会导致业务系统不可用。
- 这把锁没有失效时间，一旦解锁操作失败，就会导致锁记录一直在数据库中，其他线程无法再获得到锁。
- 这把锁只能是非阻塞的，因为数据的insert操作，一旦插入失败就会直接报错。没有获得锁的线程并不会进入排队队列，要想再次获得锁就要再次触发获得锁操作。
- 这把锁是非重入的，同一个线程在没有释放锁之前无法再次获得该锁。因为数据中数据已经存在了。

2、基于Redis实现

优点：

Redis 锁实现简单，理解逻辑简单，性能好，可以支撑高并发的获取、释放锁操作。

缺点：

- Redis 容易单点故障，集群部署，并不是强一致性的，锁的不够健壮；
- key 的过期时间设置多少不明确，只能根据实际情况调整；
- 需要自己不断去尝试获取锁，比较消耗性能。

3、基于zookeeper

优点：

zookeeper 天生设计定位就是分布式协调，强一致性，锁很健壮。如果获取不到锁，只需要添加一个监听器就可以了，不用一直轮询，性能消耗较小。

缺点：

在高请求高并发下，系统疯狂的加锁释放锁，最后 zk 承受不住这么大的压力可能会存在宕机的风险。

3. Redis实现分布式锁

分布式锁的三个核心要素

1、加锁

使用setnx来加锁。key是锁的唯一标识，按业务来决定命名，value这里设置为test。

```
setx key test
```

当一个线程执行setnx返回1，说明key原本不存在，该线程成功得到了锁；当一个线程执行setnx返回0，说明key已经存在，该线程抢锁失败；

2、解锁

有加锁就得有解锁。当得到的锁的线程执行完任务，需要释放锁，以便其他线程可以进入。释放锁的最简单方式就是执行del指令。

```
del key
```

释放锁之后，其他线程就可以继续执行setnx命令来获得锁。

3、锁超时

锁超时知道的是：如果一个得到锁的线程在执行任务的过程中挂掉，来不及显式地释放锁，这块资源将会永远被锁住，别的线程北向进来。

所以，setnx的key必须设置一个超时时间，以保证即使没有被显式释放，这把锁也要在一段时间后自动释放。setnx不支持超时参数，所以需要额外指令，

```
expire key 30
```

上述分布式锁存在的问题

通过上述 `setnx` 、 `del` 和 `expire` 实现的分布式锁还是存在着一些问题。

1、SETNX 和 EXPIRE 非原子性

假设一个场景中，某一个线程刚执行setnx，成功得到了锁。此时setnx刚执行成功，还未来得及执行expire命令，节点就挂掉了。此时这把锁就没有设置过期时间，别的线程就再也无法获得该锁。

解决措施:

由于 `setnx` 指令本身是不支持传入超时时间的，而在Redis2.6.12版本上为 `set` 指令增加了可选参数, 用法如下：

```
SET key value [EX seconds][PX milliseconds] [NX|XX]
```

- EX second: 设置键的过期时间为second秒；
- PX millisecond：设置键的过期时间为millisecond毫秒；
- NX：只在键不存在时，才对键进行设置操作；
- XX：只在键已经存在时，才对键进行设置操作；
- SET操作完成时，返回OK，否则返回nil。

2、锁误解除

如果线程 A 成功获取到了锁，并且设置了过期时间 30 秒，但线程 A 执行时间超过了 30 秒，锁过期自动释放，此时线程 B 获取到了锁；随后 A 执行完成，线程 A 使用 DEL 命令来释放锁，但此时线程 B 加的锁还没有执行完成，线程 A 实际释放的线程 B 加的锁。

解决办法:

在del释放锁之前加一个判断，验证当前的锁是不是自己加的锁。

具体在加锁的时候把当前线程的id当做value，可生成一个 UUID 标识当前线程，在删除之前验证key对应的value是不是自己线程的id。

还可以使用 lua 脚本做验证标识和解锁操作。

3、超时解锁导致并发

如果线程 A 成功获取锁并设置过期时间 30 秒，但线程 A 执行时间超过了 30 秒，锁过期自动释放，此时线程 B 获取到了锁，线程 A 和线程 B 并发执行。

A、B 两个线程发生并发显然是不被允许的，一般有两种方式解决该问题：

- 将过期时间设置足够长，确保代码逻辑在锁释放之前能够执行完成。

- 为获取锁的线程增加守护线程，为将要过期但未释放的锁增加有效时间。

4、不可重入

当线程在持有锁的情况下再次请求加锁，如果一个锁支持一个线程多次加锁，那么这个锁就是可重入的。如果一个不可重入锁被再次加锁，由于该锁已经被持有，再次加锁会失败。Redis 可通过对锁进行重入计数，加锁时加 1，解锁时减 1，当计数归 0 时释放锁。

5、无法等待锁释放

上述命令执行都是立即返回的，如果客户端可以等待锁释放就无法使用。

- 可以通过客户端轮询的方式解决该问题，当未获取到锁时，等待一段时间重新获取锁，直到成功获取锁或等待超时。这种方式比较消耗服务器资源，当并发量比较大时，会影响服务器的效率。
- 另一种方式是使用 Redis 的发布订阅功能，当获取锁失败时，订阅锁释放消息，获取锁成功后释放时，发送锁释放消息。

具体实现参考：https://xiaomi-info.github.io/2019/12/17/Redis-distributed-lock/

4. 了解RedLock吗?

Redlock是一种算法，Redlock也就是 Redis Distributed Lock，可用实现多节点Redis的分布式锁。

RedLock官方推荐，Redisson完成了对Redlock算法封装。

此种方式具有以下特性：

- 互斥访问：即永远只有一个 client 能拿到锁
- 避免死锁：最终 client 都可能拿到锁，不会出现死锁的情况，即使锁定资源的服务崩溃或者分区，仍然能释放锁。
- 容错性：只要大部分 Redis 节点存活（一半以上），就可以正常提供服务

5. RedLock的原理

假设有5个完全独立的Redis主服务器

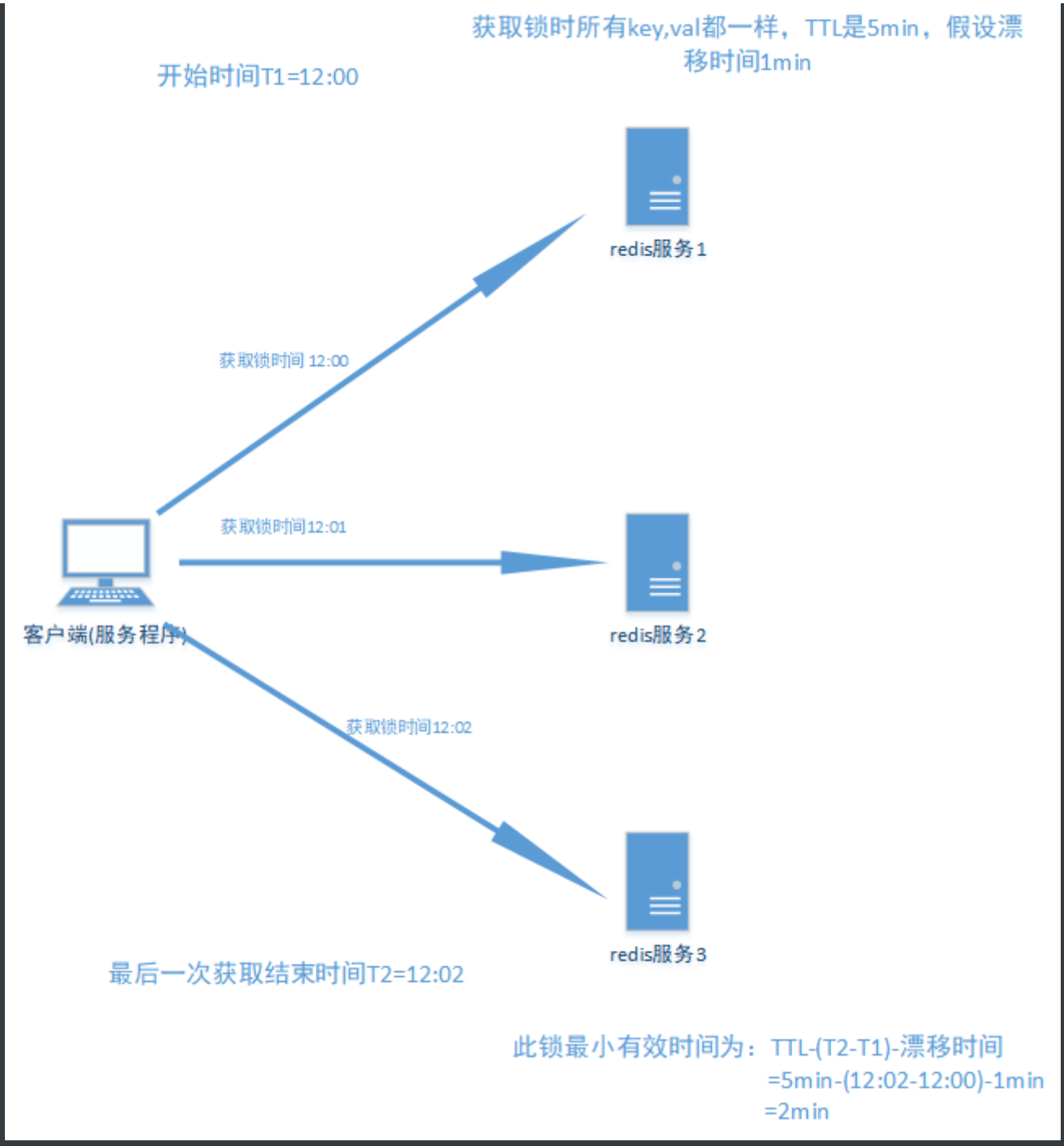
1. 获取当前时间戳

2. client尝试按照顺序使用相同的key,value获取所有Redis服务的锁，在获取锁的过程中的获取时间比锁过期时间短很多，这是为了不要过长时间等待已经关闭的Redis服务。并且试着获取下一个Redis实例。

比如：TTL为5s,设置获取锁最多用1s，所以如果一秒内无法获取锁，就放弃获取这个锁，从而尝试获取下个锁

3. client通过获取所有能获取的锁后的时间减去第一步的时间，这个时间差要小于TTL时间并且至少有3个Redis实例成功获取锁，才算真正的获取锁成功
4. 如果成功获取锁，则锁的真正有效时间是 TTL减去第三步的时间差 的时间；比如：TTL是5s,获取所有锁用了2s,则真正锁有效时间为3s(其实应该再减去时钟漂移);
5. 如果客户端由于某些原因获取锁失败，便会开始解锁所有Redis实例；因为可能已经获取了小于3个锁，必须释放，否则影响其他client获取锁

算法示意图如下：



其他

1. Redis如何做内存优化?

- **控制key的数量**。当使用Redis存储大量数据时，通常会存在大量键，过多的键同样会消耗大量内存。Redis本质是一个数据结构服务器，它为我们提供多种数据结构，如hash，list，set，zset 等结构。使用Redis时不要进入一个误区，大量使用get/set这样的API，把Redis当成Memcached使用。对于存储相同的数据内容利用Redis的数据结构降低外层键的数量，也可以节省大量内存。

- **缩减键值对象**，降低Redis内存使用最直接的方式就是缩减键（key）和值（value）的长度。
  - key长度：如在设计键时，在完整描述业务情况下，键值越短越好。
  - value长度：值对象缩减比较复杂，常见需求是把业务对象序列化成二进制数组放入Redis。首先应该在业务上精简业务对象，去掉不必要的属性避免存储无效数据。其次在序列化工具选择上，应该选择更高效的序列化工具来降低字节数组大小。
- **编码优化**。Redis对外提供了string,list,hash,set,zet等类型，但是Redis内部针对不同类型存在编码的概念，所谓编码就是具体使用哪种底层数据结构来实现。编码不同将直接影响数据的内存占用和读写效率。可参考文章：https://cloud.tencent.com/developer/article/1162213

2. 如果现在有个读超高并发的系统，用Redis来抗住大部分读请求，你会怎么设计?

如果是读高并发的话，先看读并发的数量级是多少，因为Redis单机的读QPS在万级，每秒几万没问题，使用一主多从+哨兵集群的缓存架构来承载每秒10W+的读并发，主从复制，读写分离。

使用哨兵集群主要是提高缓存架构的可用性，解决单点故障问题。主库负责写，多个从库负责读，支持水平扩容，根据读请求的QPS来决定加多少个Redis从实例。如果读并发继续增加的话，只需要增加Redis从实例就行了。

如果需要缓存1T+的数据，选择Redis cluster模式，每个主节点存一部分数据，假设一个master存32G，那只需要n*32G>=1T，n个这样的master节点就可以支持1T+的海量数据的存储了。

> Redis单主的瓶颈不在于读写的并发，而在于内存容量，即使是一主多从也是不能解决该问题，因为一主多从架构下，多个slave的数据和master的完全一样。假如master是10G那slave也只能存10G数据。所以数据量受单主的影响。
> 而这个时候又需要缓存海量数据，那就必须得有多主了，并且多个主保存的数据还不能一样。Redis官方给出的 Redis cluster 模式完美的解决了这个问题。

End

此外，我还创建了求职交流群，群内分享内推、面试等资源，一起来抱团取暖啊。扫描下方右侧二维码，备注【加群】，拉你进群~



长按关注库森



来找库森唠嗑

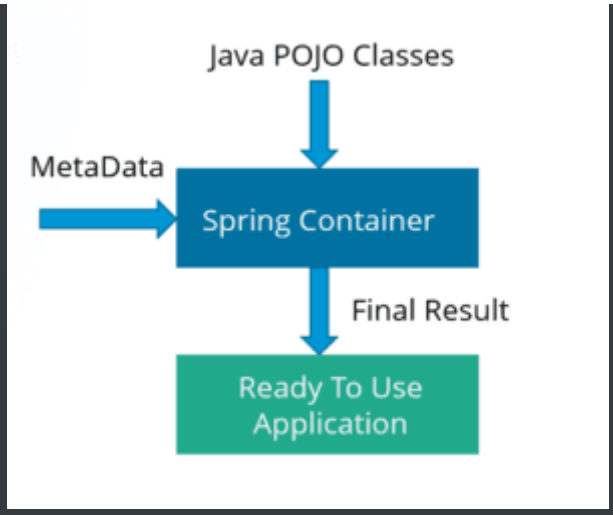
七、Spring

1. 使用Spring框架的好处是什么?

- **轻量：**Spring 是轻量的，基本的版本大约2MB
- **控制反转：**Spring通过控制反转实现了松散耦合，对象们给出它们的依赖，而不是创建或查找依赖的对象们
- **面向切面的编程(AOP)：**Spring支持面向切面的编程，并且把应用业务逻辑和系统服务分开
- **容器：**Spring 包含并管理应用中对象的生命周期和配置
- **MVC框架：**Spring的WEB框架是个精心设计的框架，是Web框架的一个很好的替代品
- **事务管理：**Spring 提供一个持续的事务管理接口，可以扩展到上至本地事务下至全局事务（JTA）
- **异常处理：**Spring 提供方便的API把具体技术相关的异常（比如由JDBC，Hibernate or JDO抛出的）转化为一致的unchecked 异常。

2. 什么是 Spring IOC 容器?

Spring 框架的核心是 Spring 容器。容器创建对象，将它们装配在一起，配置它们并管理它们的完整生命周期。Spring 容器使用依赖注入来管理组成应用程序的组件。容器通过读取提供的配置元数据来接收对象进行实例化，配置和组装的指令。该元数据可以通过 XML，Java 注解或 Java 代码提供。



3. 什么是依赖注入？可以通过多少种方式完成依赖注入？

在依赖注入中，您不必创建对象，但必须描述如何创建它们。您不是直接在代码中将组件和服务连接在一起，而是描述配置文件中哪些组件需要哪些服务。由 IoC 容器将它们装配在一起。

通常，依赖注入可以通过三种方式完成，即：

- 构造函数注入
- setter 注入
- 接口注入

在 Spring Framework 中，仅使用构造函数和 setter 注入。

4. 区分 BeanFactory 和 ApplicationContext?

BeanFactory	ApplicationContext
它使用懒加载	它使用即时加载
它使用语法显式提供资源对象	它自己创建和管理资源对象
不支持国际化	支持国际化
不支持基于依赖的注解	支持基于依赖的注解

BeanFactory和ApplicationContext的优缺点分析：

BeanFactory的优缺点：

- 优点：应用启动的时候占用资源很少，对资源要求较高的应用，比较有优势；

- 缺点：运行速度会相对来说慢一些。而且有可能会出现空指针异常的错误，而且通过Bean工厂创建的Bean生命周期会简单一些。

ApplicationContext的优缺点：

- 优点：所有的Bean在启动的时候都进行了加载，系统运行的速度快；在系统启动的时候，可以发现系统中的配置问题。
- 缺点：把费时的操作放到系统启动中完成，所有的对象都可以预加载，缺点就是内存占用较大。

5. 区分构造函数注入和 setter 注入

构造函数注入	setter 注入
没有部分注入	有部分注入
不会覆盖 setter 属性	会覆盖 setter 属性
任意修改都会创建一个新实例	任意修改不会创建一个新实例
适用于设置很多属性	适用于设置少量属性

6. spring 提供了哪些配置方式?

- 基于 xml 配置

bean 所需的依赖项和服务在 XML 格式的配置文件中指定。这些配置文件通常包含许多 bean 定义和特定于应用程序的配置选项。它们通常以 bean 标签开头。例如：

```
<bean id="studentbean" class="org.edureka.firstSpring.StudentBean">
 <property name="name" value="Edureka"></property>
</bean>
```

- 基于注解配置

您可以通过在相关的类，方法或字段声明上使用注解，将 bean 配置为组件类本身，而不是使用 XML 来描述 bean 装配。默认情况下，Spring 容器中未打开注解装配。因此，您需要在使用它之前在 Spring 配置文件中启用它。例如：

```
<beans>
<context:annotation-config/>
<!-- bean definitions go here -->
</beans>
```

- 基于 Java API 配置

Spring 的 Java 配置是通过使用 @Bean 和 @Configuration 来实现。

1. @Bean 注解扮演与 `<bean />` 元素相同的角色。
2. @Configuration 类允许通过简单地调用同一个类中的其他 @Bean 方法来定义 bean 间依赖关系。

例如：

```
@Configuration
public class StudentConfig {
    @Bean
    public StudentBean myStudent() {
        return new StudentBean();
    }
}
```

7. Spring 中的 bean 的作用域有哪些?

- singleton : 唯一 bean 实例，Spring 中的 bean 默认都是单例的。
- prototype : 每次请求都会创建一个新的 bean 实例。
- request : 每一次HTTP请求都会产生一个新的bean，该bean仅在当前HTTP request内有效。
- session : ： 在一个HTTP Session中，一个Bean定义对应一个实例。该作用域仅在基于web的Spring ApplicationContext情形下有效。
- global-session： 全局session作用域，仅仅在基于portlet的web应用中才有意义，Spring5已经没有了。Portlet是能够生成语义代码(例如：HTML)片段的小型Java Web插件。它们基于portlet容器，可以像servlet一样处理HTTP请求。但是，与 servlet 不同，每个 portlet 都有不同的会话

8. 如何理解IoC和DI?

IOC就是控制反转，通俗的说就是我们不用自己创建实例对象，这些都交给Spring的bean工厂帮我们创建管理。这也是Spring的核心思想，通过面向接口编程的方式来是实现对业务组件的动态依赖。这就意味着IOC是Spring针对解决程序耦合而存在的。在实际应用中，Spring通过配置文件（xml或者properties）指定需要实例化的java类（类名的完整字符串），包括这些java类的一组初始化值，通过加载读取配置文件，用Spring提供的方法（getBean()）就可以获取到我们想要的根据指定配置进行初始化的实例对象。

- 优点：IOC或依赖注入减少了应用程序的代码量。它使得应用程序的测试很简单，因为在单元测试中不再需要单例或JNDI查找机制。简单的实现以及较少的干扰机制使得松耦合得以实现。IOC容器支持勤性单例及延迟加载服务。

DI：DI—Dependency Injection，即“依赖注入”：组件之间依赖关系由容器在运行期决定，形象的说，即由容器动态的将某个依赖关系注入到组件之中。依赖注入的目的并非为软件系统带来更多功能，而是为了提升组件重用的频率，并为系统搭建一个灵活、可扩展的平台。通过依赖注入机制，我们只需要通过简单的配置，而无需任何代码就可指定目标需要的资源，完成自身的业务逻辑，而不需要关心具体的资源来自何处，由谁实现。

9. 将一个类声明为Spring的 bean 的注解有哪些?

我们一般使用 @Autowired 注解自动装配 bean，要想把类标识成可用于 @Autowired 注解自动装配的 bean 的类,采用以下注解可实现：

- @Component ： 通用的注解，可标注任意类为 Spring 组件。如果一个Bean不知道属于哪个层，可以使用@Component 注解标注。
  8 @Repository : 对应持久层即 Dao 层，主要用于数据库相关操作。
- @Service : 对应服务层，主要涉及一些复杂的逻辑，需要用到 Dao层。
- @Controller : 对应 Spring MVC 控制层，主要用户接受用户请求并调用 Service 层返回数据给前端页面。

10. spring 支持几种 bean scope?

Spring bean 支持 5 种 scope：

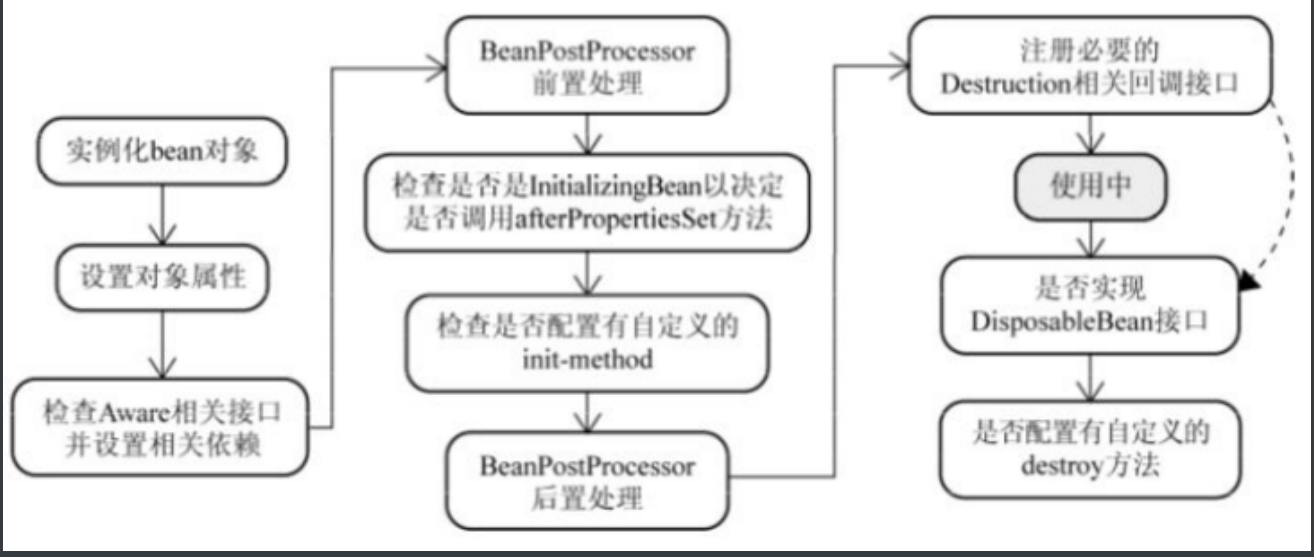
- **Singleton** - 每个 Spring IoC 容器仅有一个单实例。
- **Prototype** - 每次请求都会产生一个新的实例。
- **Request** - 每一次 HTTP 请求都会产生一个新的实例，并且该 bean 仅在当前 HTTP 请求内有效。
- **Session** - 每一次 HTTP 请求都会产生一个新的 bean，同时该 bean 仅在当前 HTTP session 内有效。

- **Global-session** - 类似于标准的 HTTP Session 作用域，不过它仅仅在基于 portlet 的 web 应用中才有意义。Portlet 规范定义了全局 Session 的概念，它被所有构成某个 portlet web 应用的各种不同的 portlet 所共享。在 global session 作用域中定义的 bean 被限定于全局 portlet Session 的生命周期范围内。如果你在 web 中使用 global session 作用域来标识 bean，那么 web 会自动当成 session 类型来使用。

仅当用户使用支持 Web 的 ApplicationContext 时，最后三个才可用。

11. Spring 中的 bean 生命周期?

Bean的生命周期是由容器来管理的。主要在创建和销毁两个时期。



创建过程：

1，实例化bean对象，以及设置bean属性；
2，如果通过Aware接口声明了依赖关系，则会注入Bean对容器基础设施层面的依赖，Aware接口是为了感知到自身的一些属性。容器管理的Bean一般不需要知道容器的状态和直接使用容器。但是在某些情况下是需要在Bean中对IOC容器进行操作的。这时候需要在bean中设置对容器的感知。SpringIOC容器也提供了该功能，它是通过特定的Aware接口来完成的。
比如BeanNameAware接口，可以知道自己在容器中的名字。
如果这个Bean已经实现了BeanFactoryAware接口，可以用这个方式来获取其它Bean。
（如果Bean实现了BeanNameAware接口，调用setBeanName()方法，传入Bean的名字。
如果Bean实现了BeanClassLoaderAware接口，调用setBeanClassLoader()方法，传入ClassLoader对象的实例。
如果Bean实现了BeanFactoryAware接口，调用setBeanFactory()方法，传入BeanFactory对象的实例。）
3，紧接着会调用BeanPostProcess的前置初始化方法postProcessBeforeInitialization，主要作用是在Spring完成实例化之后，初始化之前，对Spring容器实例化的Bean添加自定义的处理逻辑。有点类似于AOP。

4，如果实现了BeanFactoryPostProcessor接口的afterPropertiesSet方法，做一些属性被设定后的自定义的事情。
5，调用Bean自身定义的init方法，去做一些初始化相关的工作。
6，调用BeanPostProcess的后置初始化方法，postProcessAfterInitialization去做一些bean初始化之后的自定义工作。
7，完成以上创建之后就可以在应用里使用这个Bean了。

销毁过程：

当Bean不再用到，便要销毁
1，若实现了DisposableBean接口，则会调用destroy方法；
2，若配置了destry-method属性，则会调用其配置的销毁方法；

总结

主要把握创建过程和销毁过程这两个大的方面；
创建过程：首先实例化Bean，并设置Bean的属性，根据其实现的Aware接口（主要是BeanFactoryAware接口，BeanFactoryAware，ApplicationContextAware）设置依赖信息，接下来调用BeanPostProcess的postProcessBeforeInitialization方法，完成initial前的自定义逻辑；afterPropertiesSet方法做一些属性被设定后的自定义的事情;调用Bean自身定义的init方法，去做一些初始化相关的工作;然后再调用postProcessAfterInitialization去做一些bean初始化之后的自定义工作。这四个方法的调用有点类似AOP。
此时，Bean初始化完成，可以使用这个Bean了。
销毁过程：如果实现了DisposableBean的destroy方法，则调用它，如果实现了自定义的销毁方法，则调用之。

12. 什么是 spring 的内部 bean?

只有将 bean 用作另一个 bean 的属性时，才能将 bean 声明为内部 bean。为了定义 bean，Spring 的基于 XML 的配置元数据在 `<property>` 或 `<constructor-arg>` 中提供了 `<bean>` 元素的使用。内部 bean 总是匿名的，它们总是作为原型。

例如，假设我们有一个 Student 类，其中引用了 Person 类。这里我们将只创建一个 Person 类实例并在 Student 中使用它。

Student.java

```
public class Student {
    private Person person;
    //Setters and Getters
}
public class Person {
    private String name;
    private String address;
    //Setters and Getters
}
```

bean.xml

```
<bean id="StudentBean" class="com.edureka.Student">
    <property name="person">
        <!--This is inner bean -->
        <bean class="com.edureka.Person">
            <property name="name" value="Scott"></property>
            <property name="address" value="Bangalore"></property>
        </bean>
    </property>
</bean>
```

13. 什么是 spring 装配?

当 bean 在 Spring 容器中组合在一起时，它被称为装配或 bean 装配。 Spring 容器需要知道需要什么 bean 以及容器应该如何使用依赖注入来将 bean 绑定在一起，同时装配 bean。

Spring 容器能够自动装配 bean。也就是说，可以通过检查 BeanFactory 的内容让 Spring 自动解析 bean 的协作者。

自动装配的不同模式：

- **no** - 这是默认设置，表示没有自动装配。应使用显式 bean 引用进行装配。
- **byName** - 它根据 bean 的名称注入对象依赖项。它匹配并装配其属性与 XML 文件中由相同名称定义的 bean。
- **byType** - 它根据类型注入对象依赖项。如果属性的类型与 XML 文件中的一个 bean 名称匹配，则匹配并装配属性。
- **构造函数** - 它通过调用类的构造函数来注入依赖项。它有大量的参数。

- **autodetect** - 首先容器尝试通过构造函数使用 autowire 装配，如果不能，则尝试通过 byType 自动装配。

14. 自动装配有什么局限?

- 覆盖的可能性 - 您始终可以使用 `<constructor-arg>` 和 `<property>` 设置指定依赖项，这将覆盖自动装配。
- 基本元数据类型 - 简单属性（如原数据类型，字符串和类）无法自动装配。
- 令人困惑的性质 - 总是喜欢使用明确的装配，因为自动装配不太精确。

15. Spring中出现同名bean怎么办?

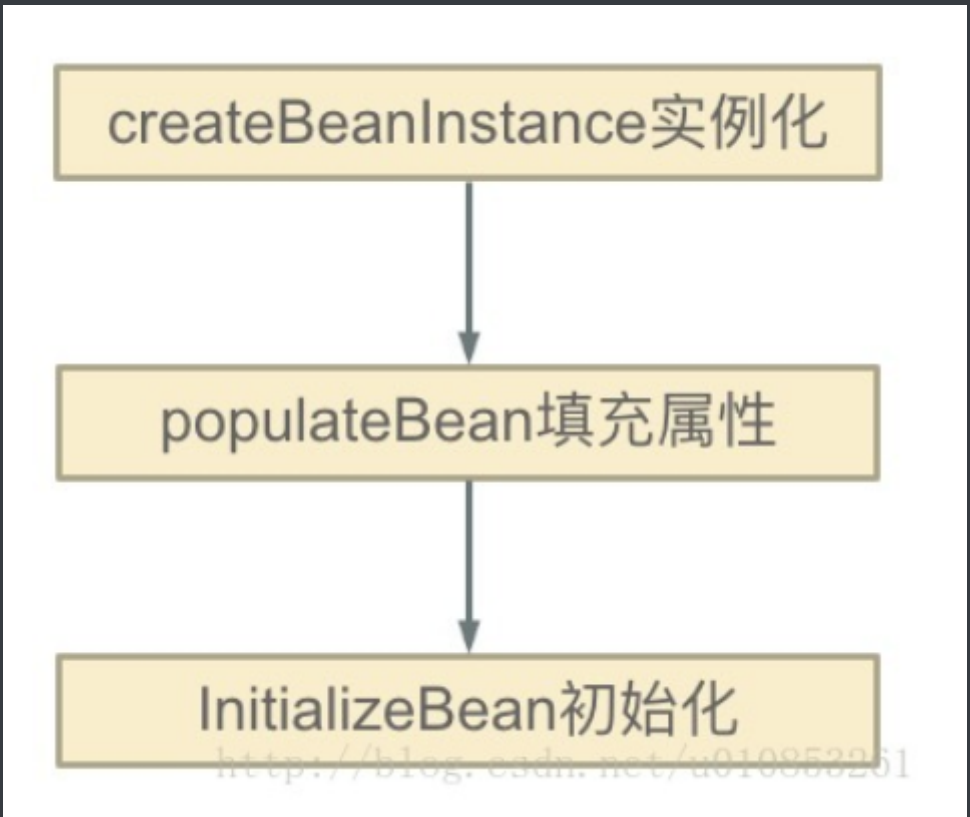
- 同一个配置文件内同名的Bean，以最上面定义的为准
- 不同配置文件中存在同名Bean，后解析的配置文件会覆盖先解析的配置文件
- 同文件中ComponentScan和@Bean出现同名Bean。同文件下@Bean的会生效，@ComponentScan扫描进来不会生效。通过@ComponentScan扫描进来的优先级是最低的，原因就是它扫描进来的Bean定义是最先被注册的~

16. Spring 怎么解决循环依赖问题?

spring对循环依赖的处理有三种情况：
①构造器的循环依赖：这种依赖spring是处理不了的，直 接抛出BeanCurrentlyInCreationException异常。
②单例模式下的setter循环依赖：通过“三级缓存”处理循环依赖。
③非单例循环依赖：无法处理。

下面分析单例模式下的setter循环依赖如何解决

Spring的单例对象的初始化主要分为三步：



（1）createBeanInstance：实例化，其实也就是调用对象的构造方法实例化对象

（2）populateBean：填充属性，这一步主要是多bean的依赖属性进行填充

（3）initializeBean：调用spring xml中的init 方法。

从上面讲述的单例bean初始化步骤我们可以知道，循环依赖主要发生在第一、第二部。也就是构造器循环依赖和field循环依赖。

缓存	用途
singletonObjects	用于存放完全初始化好的 bean，从该缓存中取出的 bean 可以直接使用
earlySingletonObjects	存放原始的 bean 对象（尚未填充属性），用于解决循环依赖
singletonFactories	存放 bean 工厂对象，用于解决循环依赖

举例：A的某个field或者setter依赖了B的实例对象，同时B的某个field或者setter依赖了A的实例对象”这种循环依赖的情况。A首先完成了

初始化的第一步（createBeanINstance实例化），并且将自己提前曝光到singletonFactories中。

此时进行初始化的第二步，发现自己依赖对象B，此时就尝试去get(B)，发现B还没有被create，所以走create流程，B在初始化第一步的时候发现自己依赖了对象A，于是尝试get(A)，尝试一级缓存singletonObjects(肯定没有，因为A还没初始化完全)，尝试二级缓存earlySingletonObjects（也没有），尝试三级缓存singletonFactories，由于A通过ObjectFactory将自己提前曝光了，所以B能够通过

ObjectFactory.getObject拿到A对象(虽然A还没有初始化完全，但是总比没有好呀)，B拿到A对象后顺利完成了初始化阶段1、2、3，完全初始化之后将自己放入到一级缓存singletonObjects中。

此时返回A中，A此时能拿到B的对象顺利完成自己的初始化阶段2、3，最终A也完成了初始化，进去了一级缓存singletonObjects中，而且更加幸运的是，由于B拿到了A的对象引用，所以B现在hold住的A对象完成了初始化。

17. Spring 中的单例 bean 的线程安全问题?

当多个用户同时请求一个服务时，容器会给每一个请求分配一个线程，这时多个线程会并发执行该请求对应的业务逻辑（成员方法），此时就要注意了，如果该处理逻辑中有对单例状态的修改（体现为该单例的成员属性），则必须考虑线程同步问题。
线程安全问题都是由全局变量及静态变量引起的。
若每个线程中对全局变量、静态变量只有读操作，而无写操作，一般来说，这个全局变量是线程安全的；若有多个线程同时执行写操作，一般都需要考虑线程同步，否则就可能影响线程安全.

无状态bean和有状态bean

- 有状态就是有数据存储功能。有状态对象(Stateful Bean)，就是有实例变量的对象，可以保存数据，是非线程安全的。在不同方法调用间不保留任何状态。
- 无状态就是一次操作，不能保存数据。无状态对象(Stateless Bean)，就是没有实例变量的对象 .不能保存数据，是不变类，是线程安全的。

在spring中无状态的Bean适合用不变模式，就是单例模式，这样可以共享实例提高性能。有状态的Bean在多线程环境下不安全，适合用Prototype原型模式。
Spring使用ThreadLocal解决线程安全问题。如果你的Bean有多种状态的话（比如 View Model 对象），就需要自行保证线程安全 。

18. 什么是 AOP?

AOP(Aspect-Oriented Programming), 即 **面向切面编程**, 它与 OOP(Object-Oriented Programming, 面向对象编程) 相辅相成, 提供了与 OOP 不同的抽象软件结构的视角.
在 OOP 中, 我们以类(class)作为我们的基本单元, 而 AOP 中的基本单元是 **Aspect(切面)**

19. AOP 有哪些实现方式?

实现 AOP 的技术，主要分为两大类：

- 静态代理 - 指使用 AOP 框架提供的命令进行编译，从而在编译阶段就可生成 AOP 代理类，因此也称为编译时增强；
  - 编译时编织（特殊编译器实现）
  - 类加载时编织（特殊的类加载器实现）。
- 动态代理 - 在运行时在内存中“临时”生成 AOP 动态代理类，因此也被称为运行时增强。
  - `JDK` 动态代理：通过反射来接收被代理的类，并且要求被代理的类必须实现一个接口 。JDK 动态代理的核心是 InvocationHandler 接口和 Proxy 类 。
  - `CGLIB` 动态代理： 如果目标类没有实现接口，那么 `Spring AOP` 会选择使用 `CGLIB` 来动态代理目标类 。 `CGLIB` （ Code Generation Library ），是一个代码生成的类库，可以在运行时动态的生成某个类的子类，注意， `CGLIB` 是通过继承的方式做的动态代理，因此如果某个类被标记为 `final` ，那么它是无法使用 `CGLIB` 做动态代理的。

20. Spring AOP and AspectJ AOP 有什么区别?

Spring AOP 基于动态代理方式实现；AspectJ 基于静态代理方式实现。
Spring AOP 仅支持方法级别的 PointCut；提供了完全的 AOP 支持，它还支持属性级别的 PointCut。

21. Spring 框架中用到了哪些设计模式?

工厂设计模式 : Spring使用工厂模式通过 `BeanFactory`、 `ApplicationContext` 创建 bean 对象。

代理设计模式 : Spring AOP 功能的实现。

单例设计模式 : Spring 中的 Bean 默认都是单例的。

模板方法模式 : Spring 中 `jdbcTemplate`、 `hibernateTemplate` 等以 Template 结尾的对数据库操作的类，它们就使用到了模板模式。

包装器设计模式：我们的项目需要连接多个数据库，而且不同的客户在每次访问中根据需要会去访问不同的数据库。这种模式让我们可以根据客户的需求能够动态切换不同的数据源。

观察者模式: Spring 事件驱动模型就是观察者模式很经典的一个应用。

适配器模式：Spring AOP 的增强或通知(Advice)使用到了适配器模式、spring MVC 中也是用到了适配器模式适配 `Controller`。

22. Spring 事务实现方式有哪些?

- 编程式事务管理：这意味着你可以通过编程的方式管理事务，这种方式带来了很大的灵活性，但很难维护。
- 声明式事务管理：这种方式意味着你可以将事务管理和业务代码分离。你只需要通过注解或者XML配置管理事务。

23. Spring框架的事务管理有哪些优点?

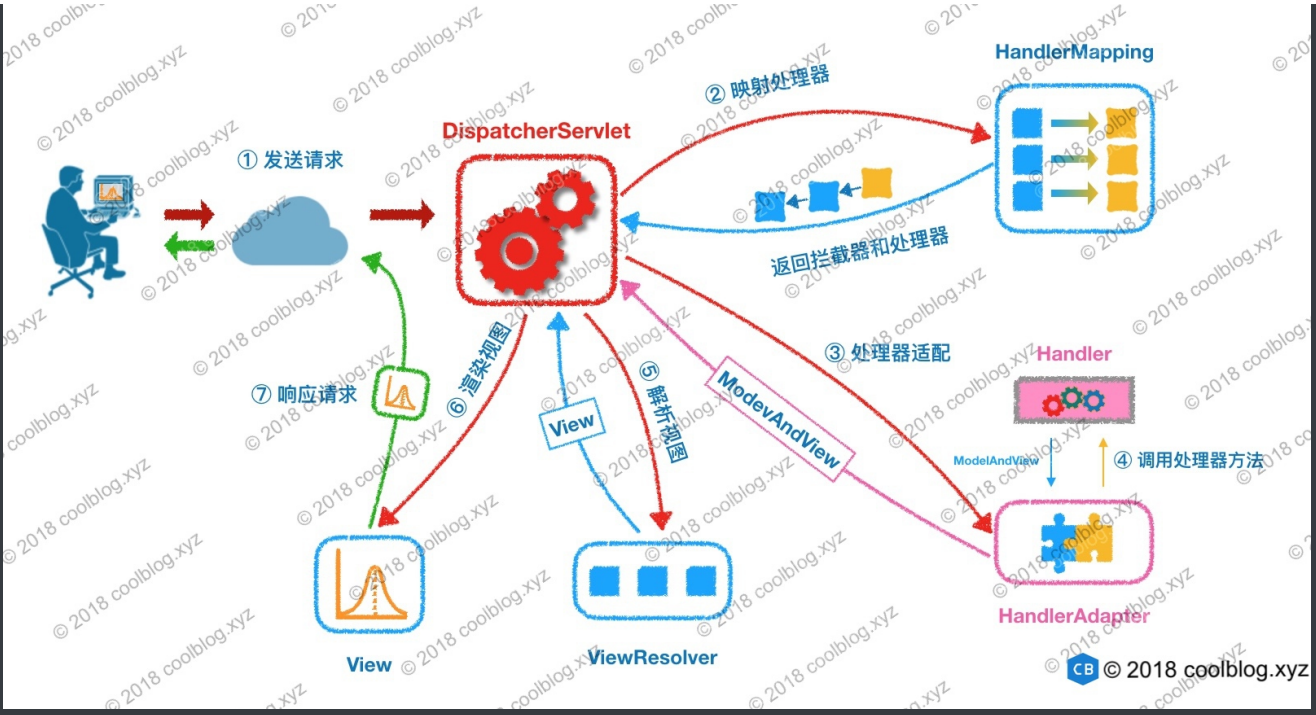
- 它提供了跨不同事务api（如JTA、JDBC、Hibernate、JPA和JDO）的一致编程模型。
- 它为编程事务管理提供了比JTA等许多复杂事务API更简单的API。
- 它支持声明式事务管理。
- 它很好地集成了Spring的各种数据访问抽象。

24. spring事务定义的传播规则

- PROPAGATION_REQUIRED: 支持当前事务，如果当前没有事务，就新建一个事务。这是最常见的选择。
- PROPAGATION_SUPPORTS: 支持当前事务，如果当前没有事务，就以非事务方式执行。
- PROPAGATION_MANDATORY: 支持当前事务，如果当前没有事务，就抛出异常。
- PROPAGATION_REQUIRES_NEW: 新建事务，如果当前存在事务，把当前事务挂起。
- PROPAGATION_NOT_SUPPORTED: 以非事务方式执行操作，如果当前存在事务，就把当前事务挂起。
- PROPAGATION_NEVER: 以非事务方式执行，如果当前存在事务，则抛出异常。
- PROPAGATION_NESTED:如果当前存在事务，则在嵌套事务内执行。如果当前没有事务，则进行与PROPAGATION_REQUIRED类似的操作。

25. SpringMVC 工作原理了解吗?

原理如下图所示：



上图的一个笔误的小问题：Spring MVC 的入口函数也就是前端控制器 `DispatcherServlet` 的作用是接收请求，响应结果。

流程说明（重要）：

1. 客户端（浏览器）发送请求，直接请求到 `DispatcherServlet`。
2. `DispatcherServlet` 根据请求信息调用 `HandlerMapping`，解析请求对应的 `Handler`。
3. 解析到对应的 `Handler`（也就是我们平常说的 `Controller` 控制器）后，开始由 `HandlerAdapter` 适配器处理。
4. `HandlerAdapter` 会根据 `Handler` 来调用真正的处理器开处理请求，并处理相应的业务逻辑。
5. 处理器处理完业务后，会返回一个 `ModelAndView` 对象，`Model` 是返回的数据对象，`View` 是个逻辑上的 `View`。
6. `ViewResolver` 会根据逻辑 `View` 查找实际的 `View`。
7. `DispaterServlet` 把返回的 `Model` 传给 `View`（视图渲染）。
8. 把 `View` 返回给请求者（浏览器）

26. 简单介绍 Spring MVC 的核心组件

那么接下来就简单介绍一下 `DispatcherServlet` 和九大组件（按使用顺序排序的）：

组件	说明
DispatcherServlet	Spring MVC 的核心组件，是请求的入口，负责协调各个组件工作

组件	说明
MultipartResolver	内容类型(`Content-Type`)为 `multipart/*` 的请求的解析器，例如解析处理文件上传的请求，便于获取参数信息以及上传的文件
HandlerMapping	请求的处理器匹配器，负责为请求找到合适的 `HandlerExecutionChain` 处理器执行链，包含处理器（ `handler` ）和拦截器们（ `interceptors` ）
HandlerAdapter	处理器的适配器。因为处理器 `handler` 的类型是 Object 类型，需要有一个调用者来实现 `handler` 是怎么被执行。Spring 中的处理器的实现多变，比如用户处理器可以实现 Controller 接口、HttpRequestHandler 接口，也可以用 `@RequestMapping` 注解将方法作为一个处理器等，这就导致 Spring MVC 无法直接执行这个处理器。所以这里需要一个处理器适配器，由它去执行处理器
HandlerExceptionResolver	处理器异常解析器，将处理器（ `handler` ）执行时发生的异常，解析(转换)成对应的 ModelAndView 结果
RequestToViewNameTranslator	视图名称转换器，用于解析出请求的默认视图名
LocaleResolver	本地化（国际化）解析器，提供国际化支持
ThemeResolver	主题解析器，提供可设置应用整体样式风格的支持
ViewResolver	视图解析器，根据视图名和国际化，获得最终的视图 View 对象
FlashMapManager	FlashMap 管理器，负责重定向时，保存参数至临时存储（默认 Session）

Spring MVC 对各个组件的职责划分的比较清晰。`DispatcherServlet` 负责协调，其他组件则各自做分内之事，互不干扰。

27. @Controller 注解有什么用?

`@Controller` 注解标记一个类为 Spring Web MVC **控制器** Controller。Spring MVC 会将扫描到该注解的类，然后扫描这个类下面带有 `@RequestMapping` 注解的方法，根据注解信息，为这个方法生成一个对应的**处理器**对象，在上面的 HandlerMapping 和 HandlerAdapter组件中讲到过。

当然，除了添加 `@Controller` 注解这种方式以外，你还可以实现 Spring MVC 提供的 `Controller` 或者 `HttpRequestHandler` 接口，对应的实现类也会被作为一个**处理器**对象

28. @RequestMapping 注解有什么用?

`@RequestMapping` 注解，在上面已经讲过了，配置**处理器**的 HTTP 请求方法，URI等信息，这样才能将请求和方法进行映射。这个注解可以作用于类上面，也可以作用于方法上面，在类上面一般是配置这个**控制器**的 URI 前缀

29. @RestController 和 @Controller 有什么区别?

`@RestController` 注解，在 `@Controller` 基础上，增加了 `@ResponseBody` 注解，更加适合目前前后端分离的架构下，提供 Restful API ，返回例如 JSON 数据格式。当然，返回什么样的数据格式，根据客户端的 `ACCEPT` 请求头来决定。

30. @RequestMapping 和 @GetMapping 注解的不同之处在哪里?

1. `@RequestMapping`： 可注解在类和方法上； `@GetMapping` 仅可注册在方法上
2. `@RequestMapping`： 可进行 GET、POST、PUT、DELETE 等请求方法； `@GetMapping` 是 `@RequestMapping` 的 GET 请求方法的特例，目的是为了提高清晰度。

31. @RequestParam 和 @PathVariable 两个注解的区别

两个注解都用于方法参数，获取参数值的方式不同， `@RequestParam` 注解的参数从请求携带的参数中获取，而 `@PathVariable` 注解从请求的 URI 中获取

32. 返回 JSON 格式使用什么注解?

可以使用 **`@ResponseBody`** 注解，或者使用包含 `@ResponseBody` 注解的 **`@RestController`** 注解。

当然，还是需要配合相应的支持 JSON 格式化的 HttpMessageConverter 实现类。例如，Spring MVC 默认使用 MappingJackson2HttpMessageConverter。

33. 什么是springmvc拦截器以及如何使用它?

Spring的处理程序映射机制包括处理程序拦截器，当你希望将特定功能应用于某些请求时，例如，检查用户主题时，这些拦截器非常有用。拦截器必须实现org.springframework.web.servlet包的HandlerInterceptor。此接口定义了三种方法：

- preHandle：在执行实际处理程序之前调用。
- postHandle：在执行完实际程序之后调用。
- afterCompletion：在完成请求后调用。

34. Spring MVC 和 Struts2 的异同?

入口不同

- Spring MVC 的入门是一个 Servlet **控制器**。
- Struts2 入门是一个 Filter **过滤器**。

配置映射不同，

- Spring MVC 是基于**方法**开发，传递参数是通过**方法形参**，一般设置为**单例**。
- Struts2 是基于**类**开发，传递参数是通过**类的属性**，只能设计为**多例**。

视图不同

- Spring MVC 通过参数解析器是将 Request 对象内容进行解析成方法形参，将响应数据和页面封装成 **ModelAndView** 对象，最后又将模型数据通过 **Request** 对象传输到页面。其中，如果视图使用 JSP 时，默认使用 **JSTL** 。
- Struts2 采用**值栈**存储请求和响应的数据，通过 **OGNL** 存取数据。

35. REST 代表着什么?

REST 代表着抽象状态转移，它是根据 HTTP 协议从客户端发送数据到服务端，例如：服务端的一本书可以以 XML 或 JSON 格式传递到客户端

可以看看 REST API design and development，知乎上的 《怎样用通俗的语言解释 REST，以及 RESTful？》了解。

36. 什么是安全的 REST 操作?

REST 接口是通过 HTTP 方法完成操作

- 一些 HTTP 操作是安全的，如 GET 和 HEAD ，它不能在服务端修改资源

- 换句话说，PUT、POST 和 DELETE 是不安全的，因为他们能修改服务端的资源

所以，是否安全的界限，在于**是否修改**服务端的资源

37. REST API 是无状态的吗?

是的，REST API 应该是无状态的，因为它是基于 HTTP 的，它也是无状态的

REST API 中的请求应该包含处理它所需的所有细节。它**不应该**依赖于以前或下一个请求或服务器端维护的一些数据，例如会话

REST 规范为使其无状态设置了一个约束，在设计 REST API 时，你应该记住这一点

38. REST安全吗? 你能做什么来保护它?

安全是一个宽泛的术语。它可能意味着消息的安全性，这是通过认证和授权提供的加密或访问限制提供的

REST 通常不是安全的，需要开发人员自己实现安全机制

39. 为什么要用SpringBoot?

在使用Spring框架进行开发的过程中，需要配置很多Spring框架包的依赖，如spring-core、spring-bean、spring-context等，而这些配置通常都是重复添加的，而且需要做很多框架使用及环境参数的重复配置，如开启注解、配置日志等。Spring Boot致力于弱化这些不必要的操作，提供默认配置，当然这些默认配置是可以按需修改的，快速搭建、开发和运行Spring应用。

以下是使用SpringBoot的一些好处：

- 自动配置，使用基于类路径和应用程序上下文的智能默认值，当然也可以根据需要重写它们以满足开发人员的需求。
- 创建Spring Boot Starter 项目时，可以选择选择需要的功能，Spring Boot将为你管理依赖关系。
- SpringBoot项目可以打包成jar文件。可以使用Java-jar命令从命令行将应用程序作为独立的Java应用程序运行。
- 在开发web应用程序时，springboot会配置一个嵌入式Tomcat服务器，以便它可以作为独立的应用程序运行。（Tomcat是默认的，当然你也可以配置Jetty或Undertow）
- SpringBoot包括许多有用的非功能特性（例如安全和健康检查）。

40. Spring Boot中如何实现对不同环境的属性配置文件的支持?

Spring Boot支持不同环境的属性配置文件切换，通过创建application-{profile}.properties文件，其中{profile}是具体的环境标识名称，例如：application-dev.properties用于开发环境，application-test.properties用于测试环境，application-uat.properties用于uat环境。如果要想使用application-dev.properties文件，则在application.properties文件中添加spring.profiles.active=dev。

如果要想使用application-test.properties文件，则在application.properties文件中添加spring.profiles.active=test。

41. Spring Boot 的核心注解是哪个？它主要由哪几个注解组成的?

启动类上面的注解是@SpringBootApplication，它也是 Spring Boot 的核心注解，主要组合包含了以下 3 个注解：

@SpringBootConfiguration：组合了 @Configuration 注解，实现配置文件的功能。

@EnableAutoConfiguration：打开自动配置的功能，也可以关闭某个自动配置的选项，如关闭数据源自动配置功能： @SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })。

@ComponentScan：Spring组件扫描。

42. 你如何理解 Spring Boot 中的 Starters?

Starters可以理解为启动器，它包含了一系列可以集成到应用里面的依赖包，你可以一站式集成 Spring 及其他技术，而不需要到处找示例代码和依赖包。如你想使用 Spring JPA 访问数据库，只要加入 spring-boot-starter-data-jpa 启动器依赖就能使用了。

Starters包含了许多项目中需要用到的依赖，它们能快速持续的运行，都是一系列得到支持的管理传递性依赖。

43. Spring Boot Starter 的工作原理是什么?

Spring Boot 在启动的时候会干这几件事情：

- Spring Boot 在启动时会去依赖的 Starter 包中寻找 resources/META-INF/spring.factories 文件，然后根据文件中配置的 Jar 包去扫描项目所依赖的 Jar 包。
- 根据 spring.factories 配置加载 AutoConfigure 类
- 根据 @Conditional 注解的条件，进行自动配置并将 Bean 注入 Spring Context

总结一下，其实就是 Spring Boot 在启动的时候，按照约定去读取 Spring Boot Starter 的配置信息，再根据配置信息对资源进行初始化，并注入到 Spring 容器中。这样 Spring Boot 启动完毕后，就已经准备好了一切资源，使用过程中直接注入对应 Bean 资源即可

44. 保护 Spring Boot 应用有哪些方法?

- 在生产中使用HTTPS
- 使用Snyk检查你的依赖关系
- 升级到最新版本
- 启用CSRF保护
- 使用内容安全策略防止XSS攻击

45. Spring 、 Spring Boot 和 Spring Cloud 的关系?

Spring 最初最核心的两大核心功能 Spring Ioc 和 Spring Aop 成就了 Spring，Spring 在这两大核心的功能上不断的发展，才有了 Spring 事务、Spring Mvc 等一系列伟大的产品，最终成就了 Spring 帝国，到了后期 Spring 几乎可以解决企业开发中的所有问题。

Spring Boot 是在强大的 Spring 帝国生态基础上面发展而来，发明 Spring Boot 不是为了取代 Spring ,是为了让人们更容易的使用 Spring 。

Spring Cloud 是一系列框架的有序集合。它利用 Spring Boot 的开发便利性巧妙地简化了分布式系统基础设施的开发，如服务发现注册、配置中心、消息总线、负载均衡、断路器、数据监控等，都可以用 Spring Boot 的开发风格做到一键启动和部署。

Spring Cloud 是为了解决微服务架构中服务治理而提供的一系列功能的开发框架，并且 Spring Cloud 是完全基于 Spring Boot 而开发，Spring Cloud 利用 Spring Boot 特性整合了开源行业中优秀的组件，整体对外提供了一套在微服务架构中服务治理的解决方案。

用一组不太合理的包含关系来表达它们之间的关系。

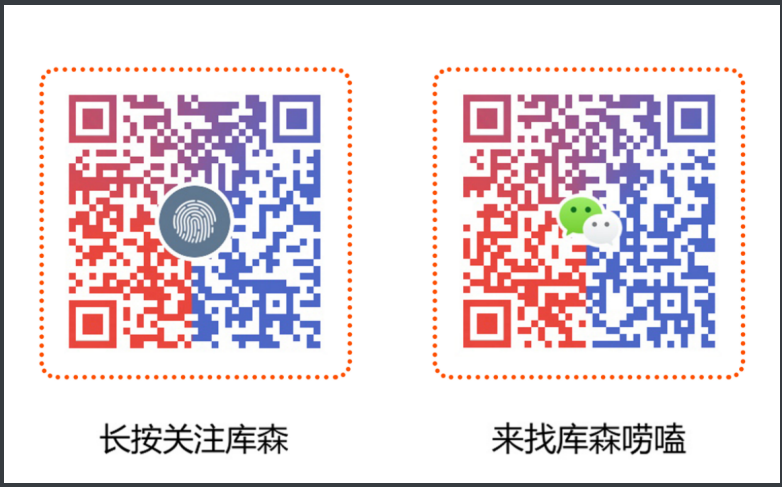
Spring ioc/aop > Spring > Spring Boot > Spring Cloud

End

以上是Spring基础的面试题，Spring其他部分正在整理中，敬请期待！

若有更新，将第一时间同步公众号【程序员库森】，扫描下方左侧二维码关注！

此外，我还创建了求职交流群，群内分享内推、面试等资源，一起来抱团取暖啊。扫描下方右侧二维码，备注【加群】，拉你进群~



八、MyBatis

1. MyBatis是什么?

- Mybatis是一个半ORM（对象关系映射）框架，它内部封装了JDBC，加载驱动、创建连接、创建statement等繁杂的过程，开发者开发时只需要关注如何编写SQL语句，可以严格控制sql执行性能，灵活度高。
- 作为一个半ORM框架，MyBatis 可以使用 XML 或注解来配置和映射原生信息，将 POJO 映射成数据库中的记录，避免了几乎所有的 JDBC 代码和手动设置参数以及获取结果集。
- 通过xml 文件或注解的方式将要执行的各种 statement 配置起来，并通过java对象和statement中sql的动态参数进行映射生成最终执行的sql语句，最后由mybatis框架执行sql并将结果映射为java对象并返回。（从执行sql到返回result的过程）。
- 由于MyBatis专注于SQL本身，灵活度高，所以比较适合对性能的要求很高，或者需求变化较多的项目，如互联网项目。

2. Mybaits的优缺点

优点：

- 基于SQL语句编程，相当灵活，不会对应用程序或者数据库的现有设计造成任何影响，SQL写在XML里，解除sql与程序代码的耦合，便于统一管理；提供XML标签，支持编写动态SQL语句，并可重用。
- 与JDBC相比，减少了50%以上的代码量，消除了JDBC大量冗余的代码，不需要手动开关连接；
- 很好的与各种数据库兼容（因为MyBatis使用JDBC来连接数据库，所以只要JDBC支持的数据库MyBatis都支持）。
- 能够与Spring很好的集成；
- 提供映射标签，支持对象与数据库的ORM字段关系映射；提供对象关系映射标签，支持对象关系组件维护。

缺点：

- SQL语句的编写工作量较大，尤其当字段多、关联表多时，对开发人员编写SQL语句的功底有一定要求。
- SQL语句依赖于数据库，导致数据库移植性差，不能随意更换数据库。

3. 为什么说Mybatis是半自动ORM映射工具？它与全自动的区别在哪里？

Hibernate属于全自动ORM映射工具，使用Hibernate查询关联对象或者关联集合对象时，可以根据对象关系模型直接获取，所以它是全自动的。

而Mybatis在查询关联对象或关联集合对象时，需要手动编写sql来完成，所以，称之为半自动ORM映射工具。

4. Hibernate 和 MyBatis 的区别

相同点：都是对jdbc的封装，都是持久层的框架，都用于dao层的开发。

不同点

1、映射关系

MyBatis 是一个半自动映射的框架，配置Java对象与sql语句执行结果的对应关系，多表关联关系配置简单。

Hibernate 是一个全表映射的框架，配置Java对象与数据库表的对应关系，多表关联关系配置复杂。

2、 SQL优化和移植性

Hibernate 对SQL语句封装，提供了日志、缓存、级联（级联比 MyBatis 强大）等特性，此外还提供 HQL（Hibernate Query Language）操作数据库，数据库无关性支持好，但会多消耗性能。如果项目需要支持多种数据库，代码开发量少，但SQL语句优化困难。
MyBatis 需要手动编写 SQL，支持动态 SQL、处理列表、动态生成表名、支持存储过程。开发工作量相对大些。直接使用SQL语句操作数据库，不支持数据库无关性，但sql语句优化容易。

3、开发难易程度和学习成本

Hibernate 是重量级框架，学习使用门槛高，适合于需求相对稳定，中小型的项目，比如：办公自动化系统

MyBatis 是轻量级框架，学习使用门槛低，适合于需求变化频繁，大型的项目，比如：互联网电子商务系统

总结

MyBatis 是一个小巧、方便、高效、简单、直接、半自动化的持久层框架，

Hibernate 是一个强大、方便、高效、复杂、间接、全自动化的持久层框架。

5. JDBC编程有哪些不足之处，MyBatis是如何解决这些问题的?

1、数据库链接创建、释放频繁造成系统资源浪费从而影响系统性能，如果使用数据库链接池可解决此问题。

解决：在SqlMapConfig.xml中配置数据链接池，使用连接池管理数据库链接。

2、Sql语句写在代码中造成代码不易维护，实际应用sql变化的可能较大，sql变动需要改变java代码。

解决：将Sql语句配置在XXXXmapper.xml文件中与java代码分离。

3、 向sql语句传参数麻烦，因为sql语句的where条件不一定，可能多也可能少，占位符需要和参数一一对应。

解决： Mybatis自动将java对象映射至sql语句。

4、 对结果集解析麻烦，sql变化导致解析代码变化，且解析前需要遍历，如果能将数据库记录封装成pojo对象解析比较方便。

解决：Mybatis自动将sql执行结果映射至java对象。

6. MyBatis编程步骤是什么样的?

1、创建SqlSessionFactory
2、通过SqlSessionFactory创建SqlSession
3、 通过sqlsession执行数据库操作
4、 调用session.commit()提交事务
5、 调用session.close()关闭会话

7. MyBatis与Hibernate有哪些不同?

1、Mybatis 和 hibernate 不同，它不完全是一个 ORM 框架，因为 MyBatis 需要 程序员自己编写 Sql 语句。

2、Mybatis 直接编写原生态 sql，可以严格控制 sql 执行性能，灵活度高，非常 适合对关系数据模型要求不高的软件开发，因为这类软件需求变化频繁，一但需 求变化要求迅速输出成果。但是灵活的前提是 mybatis 无法做到数据库无关性， 如果需要实现支持多种数据库的软件，则需要自定义多套 sql 映射文件，工作量大。

3、Hibernate 对象/关系映射能力强，数据库无关性好，对于关系模型要求高的 软件，如果用 hibernate 开发可以节省很多代码，提高效率

8. Mybaits 的优点：

1、基于 SQL 语句编程，相当灵活，不会对应用程序或者数据库的现有设计造成任 何影响，SQL 写在 XML 里，解除 sql 与程序代码的耦合，便于统一管理；提供 XML 标签，支持编写动态 SQL 语句，并可重用。

2、与 JDBC 相比，减少了 50%以上的代码量，消除了 JDBC 大量冗余的代码，不 需要手动开关连接；

3、很好的与各种数据库兼容（因为 MyBatis 使用 JDBC 来连接数据库，所以只要 JDBC 支持的数据库 MyBatis 都支持）。

4、能够与 Spring 很好的集成； 5、提供映射标签，支持对象与数据库的 ORM 字段关系映射；提供对象关系映射 标签，支持对象关系组件维护

9. MyBatis 框架的缺点：

1、SQL 语句的编写工作量较大，尤其当字段多、关联表多时，对开发人员编写 SQL 语句的功底有一定要求。

2、SQL 语句依赖于数据库，导致数据库移植性差，不能随意更换数据库。

10. #{}和${}的区别?

- #{}是占位符，预编译处理；${}是拼接符，字符串替换，没有预编译处理。
- Mybatis在处理#{}时，#{}传入参数是以字符串传入，会将SQL中的#{}替换为?号，调用PreparedStatement的set方法来赋值。
- Mybatis在处理时， 是 原 值 传 入， 就 是 把 {}时，是原值传入，就是把时，是原值传入，就是把{}替换成变量的值，相当于JDBC中的Statement编译
- 变量替换后，#{} 对应的变量自动加上单引号 ‘’；变量替换后，${} 对应的变量不会加上单引号 ‘’
- #{} 可以有效的防止SQL注入，提高系统安全性；${} 不能防止SQL 注入
- #{} 的变量替换是在DBMS 中；${} 的变量替换是在 DBMS 外

11. 通常一个Xml映射文件，都会写一个Dao接口与之对应，那么这个Dao接口的工作原理是什么？Dao接口里的方法、参数不同时，方法能重载吗?

Dao接口即Mapper接口。接口的全限名就是映射文件中的namespace的值；接口的方法名，就是映射文件中Mapper的Statement的id值；接口方法内的参数，就是传递给sql的参数。Mapper接口是没有实现类的，当调用接口方法时，接口全限名+方法名的拼接字符串作为key值，可唯一定位一个MapperStatement。

Dao接口里的方法，是不能重载的，因为是全限名+方法名的保存和寻找策略。

Dao接口的工作原理是JDK动态代理，Mybatis运行时会使用JDK动态代理为Dao接口生成代理proxy对象，代理对象proxy会拦截接口方法，转而执行MappedStatement所代表的sql，然后将sql执行结果返回。

12. 在Mapper中如何传递多个参数?

1、若Dao层函数有多个参数，那么其对应的xml中，#{0}代表接收的是Dao层中的第一个参数，#{1}代表Dao中的第二个参数，以此类推。

2、使用@Param注解：在Dao层的参数中前加@Param注解,注解内的参数名为传递到Mapper中的参数名。

3、多个参数封装成Map，以HashMap的形式传递到Mapper中。

13. Mybatis动态sql有什么用？执行原理是什么？有哪些动态sql？

Mybatis动态sql可以在xml映射文件内，以标签的形式编写动态sql，执行原理是根据表达式的值完成逻辑判断，并动态拼接sql的功能。

Mybatis提供了9种动态sql标签：trim、where、set、foreach、if、choose、when、otherwise、bind

14. xml映射文件中，不同的xml映射文件id是否可以重复？

不同的xml映射文件，如果配置了namespace，那么id可以重复；如果没有配置namespace，那么id不能重复；

原因是namespace+id是作为Map<String,MapperStatement>的key使用的，如果没有namespace，就剩下id，那么id重复会导致数据互相覆盖。有了namespace，自然id就可以重复，namespace不同，namespace+id自然也不同。

15. Mybatis实现一对一有几种方式？具体是怎么操作的？

有联合查询和嵌套查询两种方式。

联合查询是几个表联合查询，通过在resultMap里面配置association节点配置一对一的类就可以完成；

嵌套查询是先查一个表，根据这个表里面的结果的外键id，再去另外一个表里面查询数据，也是通过association配置，但另外一个表的查询是通过select配置的。

16. Mybatis实现一对多有几种方式？具体是怎么操作的？

有联合查询和嵌套查询两种方式。

联合查询是几个表联合查询，只查询一次，通过在resultMap里面的collection节点配置一对多的类就可以完成；

嵌套查询是先查一个表，根据这个表里面的结果的外键id，再去另外一个表里面查询数据，也是通过collection，但另外一个表的查询是通过select配置的。

17. Mybatis的一级、二级缓存

1、 一级缓存：基于PerpetualCache的HashMap本地缓存，其存储作用域为Session，当Session flush或close之后，该Session中的所有Cache就将清空，默认打开一级缓存。
2、 二级缓存与一级缓存机制相同，默认也是采用PerpetualCache，HashMap存储，不同在于其存储作用域为Mapper（namespace），并且可自定义存储源，如Ehcache。默认打不开二级缓存，要开启二级缓存，使用二级缓存属性类需要实现Serializable序列化接口（可用来保存对象的状态），可在它的映射文件中配置。

对于缓存数据更新机制，当某一个作用域（一级缓存Session/二级缓存Namespace）进行了增/删/改操作后，默认该作用域下所有select中的缓存将被clear。

18. 使用MyBatis的Mapper接口调用时有哪些要求?

1、Mapper接口方法名和mapper.xml中定义的每个sql的id相同；
2、Mapper接口方法的输入参数类型和mapper.xml中定义的每个sql的parameterType类型相同；
3、Mapper接口方法的输出参数类型和mapper.xml中定义的每个sql的resultType的类型相同；
4、Mapper.xml文件中的namespace即是mapper接口的类路径。

19. Mybatis动态sql是做什么的？都有哪些动态sql?

Mybatis动态sql可以让我们在Xml映射文件内，以标签的形式编写动态sql，完成逻辑判断和动态拼接sql的功能，Mybatis提供了9种动态sql标签
trim|where|set|foreach|if|choose|when|otherwise|bind。

其执行原理为，使用OGNL从sql参数对象中计算表达式的值，根据表达式的值动态拼接sql，以此来完成动态sql的功能。

20. Mybatis的Xml映射文件中，不同的Xml映射文件，id是否可以重复?

不同的Xml映射文件，如果配置了namespace，那么id可以重复；如果没有配置namespace，那么id不能重复；毕竟namespace不是必须的，只是最佳实践而已。

原因就是namespace+id是作为Map<String, MappedStatement>的key使用的，如果没有namespace，就剩下id，那么，id重复会导致数据互相覆盖。有了namespace，自然id就可以重复，namespace不同，namespace+id自然也就不同。

九、MQ

概述

为什么使用MQ?

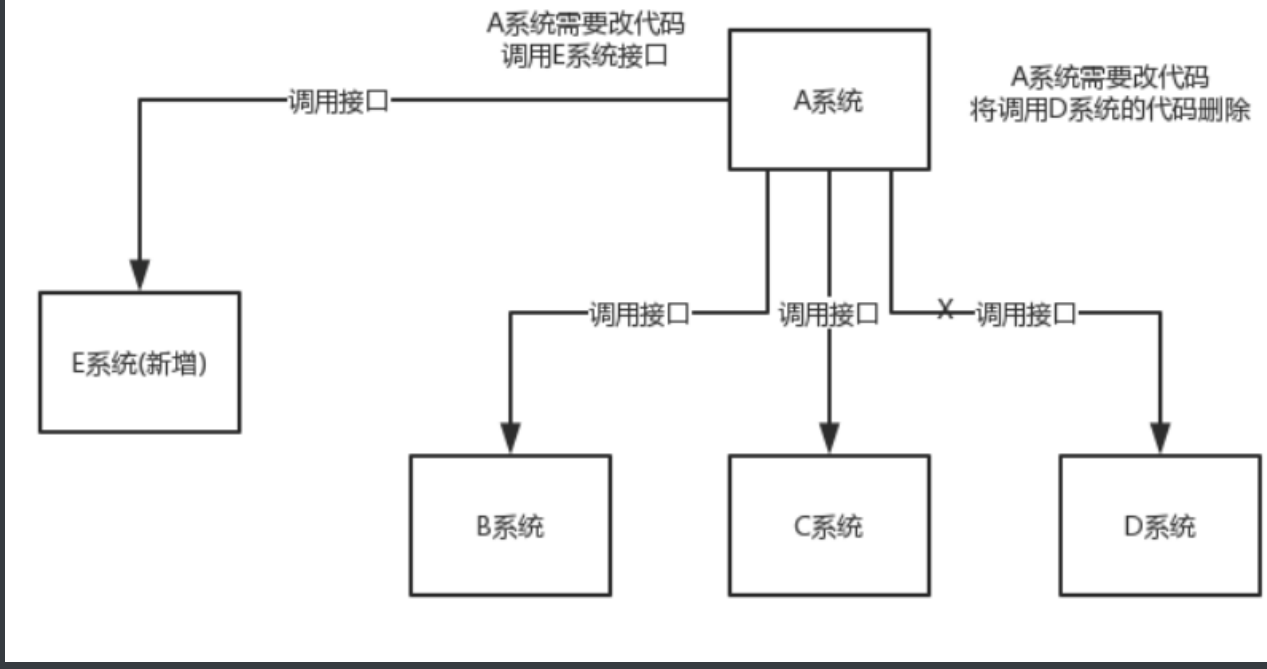
使用MQ的场景很多，主要有三个：解耦、异步、削峰。

- 解耦：假设现在，日志不光要插入到数据库里，还要在硬盘中增加文件类型的日志，同时，一些关键日志还要通过邮件的方式发送给指定的人。那么，如果按照原来的逻辑，A可能就需要在原来的代码上做扩展，除了B服务，还要加上日志文件的存储和日志邮件的发送。但是，如果你使用了MQ，那么，A服务是不需要做更改的，它还是将消息放到MQ中即可，其它的服务，无论是原来的B服务还是新增的日志文件存储服务或日志邮件发送服务，都直接从MQ中获取消息并处理即可。这就是解耦，它的好处是提高系统灵活性，扩展性。
- 异步：可以将一些非核心流程，如日志，短信，邮件等，通过MQ的方式异步去处理。这样做的好处是缩短主流程的响应时间，提升用户体验。
- 削峰：MQ的本质就是业务的排队。所以，面对突然到来的高并发，MQ也可以不用慌忙，先排好队，不要着急，一个一个来。削峰的好处就是避免高并发压垮系统的关键组件，如某个核心服务或数据库等。

下面附场景解释：

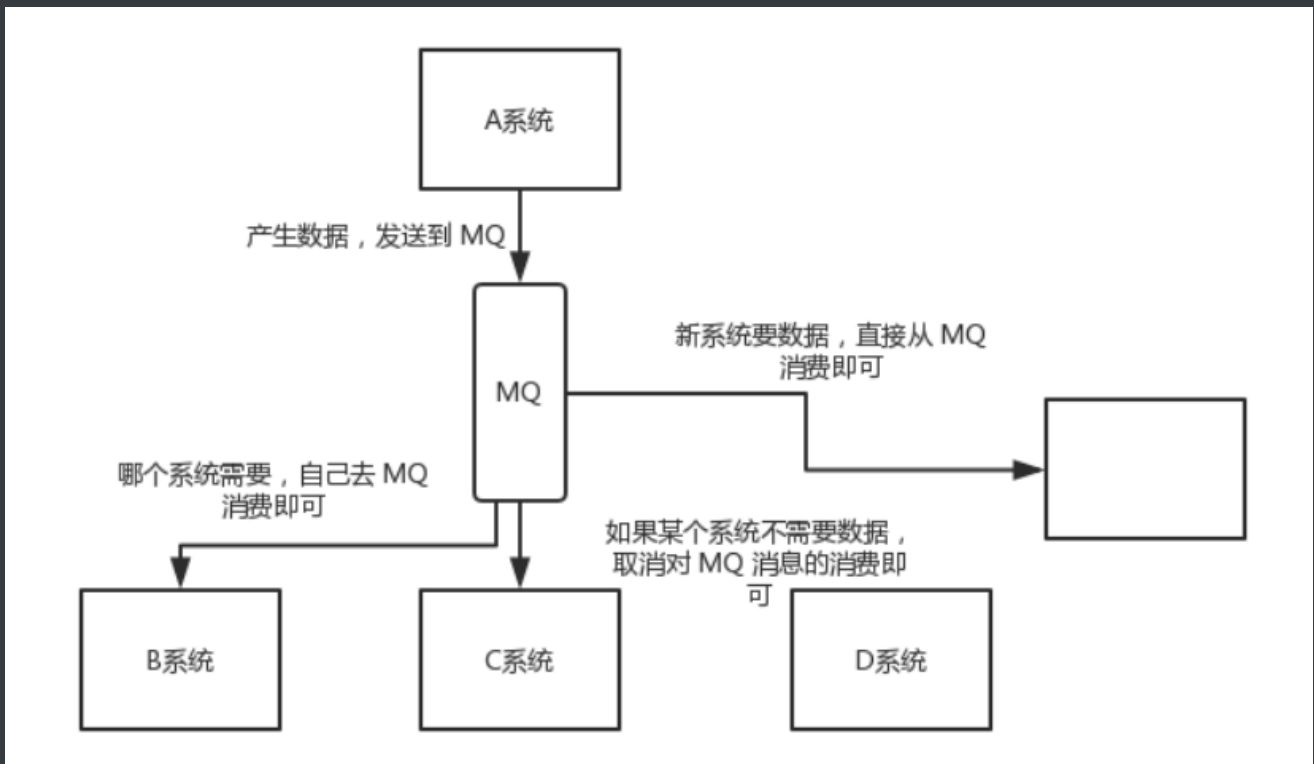
解耦

场景：A 系统发送数据到 BCD 三个系统，通过接口调用发送。如果 E 系统也要这个数据呢？那如果 C 系统现在不需要了呢？A 系统负责人几乎崩溃.....



在这个场景中，A 系统跟其它各种乱七八糟的系统严重耦合，A 系统产生一条比较关键的数据，很多系统都需要 A 系统将这个数据发送过来。A 系统要时时刻刻考虑 BCDE 四个系统如果挂了该咋办？要不要重发，要不要把消息存起来？头发都白了啊！

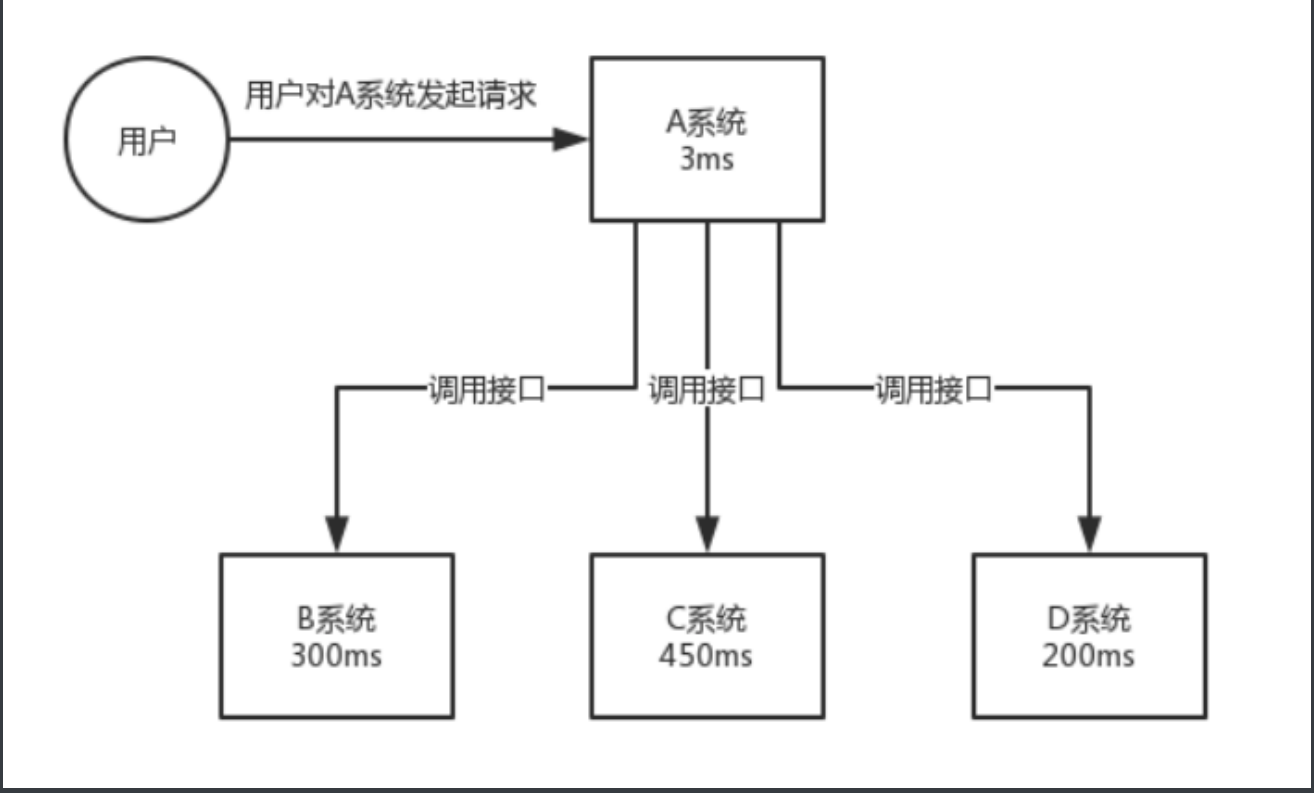
如果使用 MQ，A 系统产生一条数据，发送到 MQ 里面去，哪个系统需要数据自己去 MQ 里面消费。如果新系统需要数据，直接从 MQ 里消费即可；如果某个系统不需要这条数据了，就取消对 MQ 消息的消费即可。这样下来，A 系统压根儿不需要去考虑要给谁发送数据，不需要维护这个代码，也不需要考虑人家是否调用成功、失败超时等情况。



总结：通过一个 MQ，Pub/Sub 发布订阅消息这么一个模型，A 系统就跟其它系统彻底解耦了。

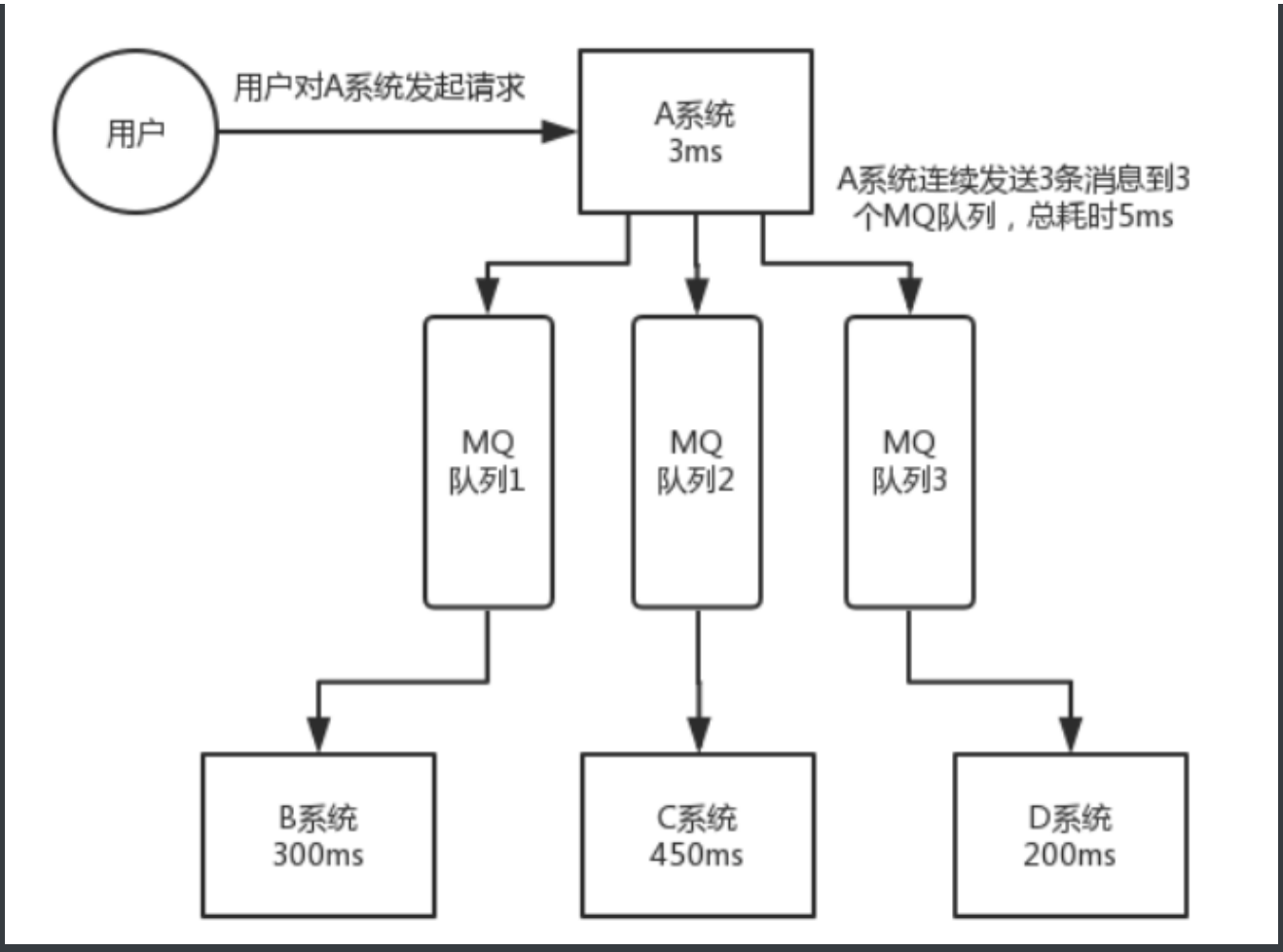
异步

场景：A 系统接收一个请求，需要在自己本地写库，还需要在 BCD 三个系统写库，自己本地写库要 3ms，BCD 三个系统分别写库要 300ms、450ms、200ms。最终请求总延时是 3 + 300 + 450 + 200 = 953ms，接近 1s，用户感觉搞个什么东西，慢死了慢死了。用户通过浏览器发起请求，等待个 1s，这几乎是不可接受的。



一般互联网类的企业，对于用户直接的操作，一般要求是每个请求都必须在 200 ms 以内完成，对用户几乎是无感知的。

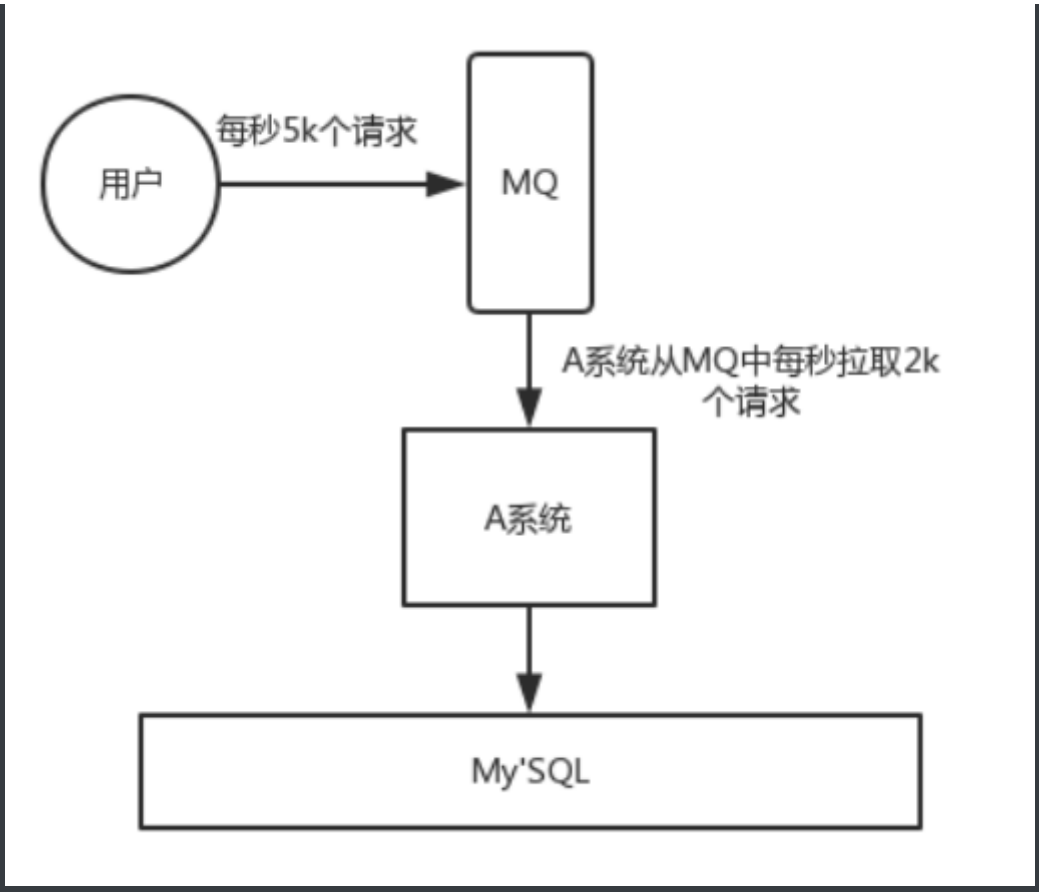
如果使用 MQ，那么 A 系统连续发送 3 条消息到 MQ 队列中，假如耗时 5ms，A 系统从接受一个请求到返回响应给用户，总时长是 3 + 5 = 8ms，对于用户而言，其实感觉上就是点个按钮，8ms 以后就直接返回了。



削峰

场景：每天 0:00 到 12:00，A 系统风平浪静，每秒并发请求数量就 50 个。结果每次一到 12:00 ~ 13:00 ，每秒并发请求数量突然会暴增到 5k+ 条。但是系统是直接基于 MySQL 的，大量的请求涌入 MySQL，每秒钟对 MySQL 执行约 5k 条 SQL。

使用 MQ，每秒 5k 个请求写入 MQ，A 系统每秒钟最多处理 2k 个请求，因为 MySQL 每秒钟最多处理 2k 个。A 系统从 MQ 中慢慢拉取请求，每秒钟就拉取 2k 个请求，不要超过自己每秒能处理的最大请求数量就 ok，这样下来，哪怕是高峰期的时候，A 系统也绝对不会挂掉。而 MQ 每秒钟 5k 个请求进来，就 2k 个请求出去，结果就导致在中午高峰期（1 个小时），可能有几十万甚至几百万的请求积压在 MQ 中。



这个短暂的高峰期积压是 ok 的，因为高峰期过了之后，每秒钟就 50 个请求进 MQ，但是 A 系统依然会按照每秒 2k 个请求的速度在处理。所以说，只要高峰期一过，A 系统就会快速将积压的消息给解决掉。

消息队列的缺点

1、 系统可用性降低

系统引入的外部依赖越多，越容易挂掉。

2、 系统复杂度提高

加入了消息队列，要多考虑很多方面的问题，比如：一致性问题、如何保证消息不被重复消费、如何保证消息可靠性传输等。因此，需要考虑的东西更多，复杂性增大。

3、 一致性问题

A 系统处理完了直接返回成功了，人都以为你这个请求就成功了；但是问题是，要是 BCD 三个系统那里，BD 两个系统写库成功了，结果 C 系统写库失败了，这就数据不一致了。

Kafka、ActiveMQ、RabbitMQ、RocketMQ 有什么优缺点?

特性	ActiveMQ	RabbitMQ	RocketMQ	Kafka
开发语言	java	erlang	java	scala
单机吞吐量	万级，比 RocketMQ、Kafka 低一个数量级	同 ActiveMQ	10 万级，支撑高吞吐	10 万级，高吞吐，一般配合大数据类的系统来进行实时数据计算、日志采集等场景
topic 数量对吞吐量的影响			topic 可以达到几百/几千的级别，吞吐量会有较小幅度的下降，这是 RocketMQ 的一大优势，在同等机器下，可以支撑大量的 topic	topic 从几十到几百个时候，吞吐量会大幅度下降，在同等机器下，Kafka 尽量保证 topic 数量不要过多，如果要支撑大规模的 topic，需要增加更多的机器资源
时效性	ms 级	微秒级，这是 RabbitMQ 的一大特点，延迟最低	ms 级	延迟在 ms 级以内
可用性	高，基于主从架构实现高可用	同 ActiveMQ	非常高，分布式架构	非常高，分布式，一个数据多个副本，少数机器宕机，不会丢失数据，不会导致不可用
消息可靠性	有较低的概率丢失数据	基本不丢	经过参数优化配置，可以做到 0 丢失	同 RocketMQ
功能支持	MQ 领域的功能极其完备	基于 erlang 开发，并发能力很强，性能极好，延时很低	MQ 功能较为完善，还是分布式的，扩展性好	功能较为简单，主要支持简单的 MQ 功能，在大数据领域的实时计算以及日志采集被大规模使用

特性	ActiveMQ	RabbitMQ	RocketMQ	Kafka
社区活跃度	低	很高	一般	很高

- 中小型公司，技术实力较为一般，技术挑战不是特别高，用 RabbitMQ 是不错的选择；
- 大型公司，基础架构研发实力较强，用 RocketMQ 是很好的选择。
- 大数据领域的实时计算、日志采集等场景，用 Kafka 是业内标准的，几乎是全世界这个领域的事实性规范。

RabbitMQ

1. RabbitMQ是什么?

RabbitMQ是实现了高级消息队列协议（AMQP）的开源消息代理软件（亦称面向消息的中间件）。RabbitMQ服务器是用Erlang语言编写的，而群集和故障转移是构建在开放电信平台框架上的。所有主要的编程语言均有与代理接口通讯的客户端库。

2. RabbitMQ特点?

可靠性: RabbitMQ使用一些机制来保证可靠性， 如持久化、传输确认及发布确认等。

灵活的路由 : 在消息进入队列之前，通过交换器来路由消息。对于典型的路由功能，RabbitMQ 己经提供了一些内置的交换器来实现。针对更复杂的路由功能，可以将多个 交换器绑定在一起， 也可以通过插件机制来实现自己的交换器。

扩展性: 多个RabbitMQ节点可以组成一个集群，也可以根据实际业务情况动态地扩展 集群中节点。

高可用性 : 队列可以在集群中的机器上设置镜像，使得在部分节点出现问题的情况下队 列仍然可用。

多种协议: RabbitMQ除了原生支持AMQP协议，还支持STOMP， MQTT等多种消息 中间件协议。

多语言客户端 :RabbitMQ 几乎支持所有常用语言，比如 Java、 Python、 Ruby、 PHP、 C#、 JavaScript 等。

管理界面 : RabbitMQ 提供了一个易用的用户界面，使得用户可以监控和管理消息、集 群中的节点等。

令插件机制: RabbitMQ 提供了许多插件， 以实现从多方面进行扩展，当然也可以编写自 己的插件。

3. AMQP是什么?

RabbitMQ就是 AMQP 协议的 `Erlang` 的实现(当然 RabbitMQ 还支持 `STOMP2` 、 `MQTT3` 等协议) AMQP 的模型架构 和 RabbitMQ 的模型架构是一样的，生产者将消息发送给交换器，交换器和队列绑定 。

RabbitMQ 中的交换器、交换器类型、队列、绑定、路由键等都是遵循的 AMQP 协议中相 应的概念。目前 RabbitMQ 最新版本默认支持的是 AMQP 0-9-1。

4. AMQP的3层协议?

Module Layer:协议最高层，主要定义了一些客户端调用的命令，客户端可以用这些命令实现自己的业务逻辑。

Session Layer:中间层，主要负责客户端命令发送给服务器，再将服务端应答返回客户端，提供可靠性同步机制和错误处理。

TransportLayer:最底层，主要传输二进制数据流，提供帧的处理、信道服用、错误检测和数据表示等。

5. 说说Broker服务节点、Queue队列、Exchange交换器?

- Broker可以看做RabbitMQ的服务节点。一般请下一个Broker可以看做一个RabbitMQ服务器。
- Queue:RabbitMQ的内部对象，用于存储消息。多个消费者可以订阅同一队列，这时队列中的消息会被平摊（轮询）给多个消费者进行处理。
- Exchange:生产者将消息发送到交换器，由交换器将消息路由到一个或者多个队列中。当路由不到时，或返回给生产者或直接丢弃。

6. 如何保证消息的可靠性?

分三点：

- 生产者到RabbitMQ：事务机制和Confirm机制，注意：事务机制和 Confirm 机制是互斥的，两者不能共存，会导致 RabbitMQ 报错。
- RabbitMQ自身：持久化、集群、普通模式、镜像模式。
- RabbitMQ到消费者：basicAck机制、死信队列、消息补偿机制。

7. 生产者消息运转的流程?

1. `Producer` 先连接到Broker,建立连接Connection,开启一个信道(Channel)。
2. `Producer` 声明一个交换器并设置好相关属性。
3. `Producer` 声明一个队列并设置好相关属性。
4. `Producer` 通过路由键将交换器和队列绑定起来。
5. `Producer` 发送消息到 `Broker` ,其中包含路由键、交换器等信息。
6. 相应的交换器根据接收到的路由键查找匹配的队列。
7. 如果找到，将消息存入对应的队列，如果没有找到，会根据生产者的配置丢弃或者退回给生产者。
8. 关闭信道。
9. 管理连接。

8.消费者接收消息过程?

1. `Producer` 先连接到 `Broker` ,建立连接 `Connection` ,开启一个信道(`Channel`)。
2. 向 `Broker` 请求消费响应的队列中消息，可能会设置响应的回调函数。
3. 等待 `Broker` 回应并投递相应队列中的消息，接收消息。
4. 消费者确认收到的消息, `ack` 。
5. `RabbitMq` 从队列中删除已经确定的消息。
6. 关闭信道。
7. 关闭连接。

9. 生产者如何将消息可靠投递到RabbitMQ?

1. Client发送消息给MQ
2. MQ将消息持久化后，发送Ack消息给Client，此处有可能因为网络问题导致Ack消息无法发送到Client，那么Client在等待超时后，会重传消息；
3. Client收到Ack消息后，认为消息已经投递成功。

10. RabbitMQ如何将消息可靠投递到消费者?

1. MQ将消息push给Client（或Client来pull消息）
2. Client得到消息并做完业务逻辑
3. Client发送Ack消息给MQ，通知MQ删除该消息，此处有可能因为网络问题导致Ack失败，那么Client会重复消息，这里就引出消费幂等的问题；
4. MQ将已消费的消息删除。

11. 如何保证RabbitMQ消息队列的高可用?

RabbitMQ 有三种模式：`单机模式`， `普通集群模式`， `镜像集群模式`。

单机模式：就是demo级别的，一般就是你本地启动了玩玩儿的，没人生产用单机模式

普通集群模式：意思就是在多台机器上启动多个RabbitMQ实例，每个机器启动一个。

镜像集群模式：这种模式，才是所谓的RabbitMQ的高可用模式，跟普通集群模式不一样的是，你创建的queue，无论元数据(元数据指RabbitMQ的配置数据)还是queue里的消息都会存在于多个实例上，然后每次你写消息到queue的时候，都会自动把消息到多个实例的queue里进行消息同步。

RocketMQ

1. RocketMQ是什么?

RocketMQ 是阿里巴巴开源的分布式消息中间件。支持事务消息、顺序消息、批量消息、定时消息、消息回溯等。它里面有几个区别于标准消息中件间的概念，如Group、Topic、Queue等。系统组成则由Producer、Consumer、Broker、NameServer等。

RocketMQ 特点

- 是一个队列模型的消息中间件，具有高性能、高可靠、高实时、分布式等特点
- Producer、Consumer、队列都可以分布式
- Producer 向一些队列轮流发送消息，队列集合称为 Topic，Consumer 如果做广播消费，则一个 Consumer 实例消费这个 Topic 对应的所有队列，如果做集群消费，则多个 Consumer 实例平均消费这个 Topic 对应的队列集合
- 能够保证严格的消息顺序
- 支持拉（pull）和推（push）两种消息模式
- 高效的订阅者水平扩展能力
- 实时的消息订阅机制
- 亿级消息堆积能力
- 支持多种消息协议，如 JMS、OpenMessaging 等
- 较少的依赖

2. RocketMQ由哪些角色组成，每个角色作用和特点是什么?

角色	作用
Nameserver	无状态，动态列表；这也是和zookeeper的重要区别之一。zookeeper是有状态的。

角色	作用
Producer	消息生产者，负责发消息到Broker。
Broker	就是MQ本身，负责收发消息、持久化消息等。
Consumer	消息消费者，负责从Broker上拉取消息进行消费，消费完进行ack。

3. RocketMQ消费模式有几种?

消费模型由Consumer决定，消费维度为Topic。

1、集群消费

- 一条消息只会被同Group中的一个Consumer消费
- 多个Group同时消费一个Topic时，每个Group都会有一个Consumer消费到数据

2、广播消费

消息将对一 个Consumer Group 下的各个 Consumer 实例都消费一遍。即即使这些 Consumer 属于同一个Consumer Group ，消息也会被 Consumer Group 中的每个 Consumer 都消费一次。

4. RocketMQ消费消息是push还是pull?

RocketMQ没有真正意义的push，都是pull，虽然有push类，但实际底层实现采用的是**长轮询机制**，即拉取方式

> broker端属性 longPollingEnable 标记是否开启长轮询。默认开启

追问：为什么要主动拉取消息而不使用事件监听方式?

事件驱动方式是建立好长连接，由事件（发送数据）的方式来实时推送。

如果broker主动推送消息的话有可能push速度快，消费速度慢的情况，那么就会造成消息在consumer端堆积过多，同时又不能被其他consumer消费的情况。而pull的方式可以根据当前自身情况来pull，不会造成过多的压力而造成瓶颈。所以采取了pull的方式。

5. broker如何处理拉取请求的?

Consumer首次请求Broker

- Broker中是否有符合条件的消息
- 有
    - 响应Consumer
    - 等待下次Consumer的请求
- 没有
    - DefaultMessageStore#ReputMessageService#run方法
    - PullRequestHoldService 来Hold连接，每个5s执行一次检查pullRequestTable有没有消息，有的话立即推送
    - 每隔1ms检查commitLog中是否有新消息，有的话写入到pullRequestTable
    - 当有新消息的时候返回请求
    - 挂起consumer的请求，即不断开连接，也不返回数据
    - 使用consumer的offset，

6. 如何让RocketMQ保证消息的顺序消费?

首先多个queue只能保证单个queue里的顺序，queue是典型的FIFO，天然顺序。多个queue同时消费是无法绝对保证消息的有序性的。所以总结如下：

同一topic，同一个QUEUE，发消息的时候一个线程去发送消息，消费的时候 一个线程去消费一个queue里的消息。

7. RocketMQ如何保证消息不丢失?

首先在如下三个部分都可能会出现丢失消息的情况：

- Producer端
- Broker端
- Consumer端

1 、Producer端如何保证消息不丢失

- 采取send()同步发消息，发送结果是同步感知的。
- 发送失败后可以重试，设置重试次数。默认3次。
- 集群部署，比如发送失败了的原因可能是当前Broker宕机了，重试的时候会发送到其他Broker上。

2、Broker端如何保证消息不丢失

- 修改刷盘策略为同步刷盘。默认情况下是异步刷盘的。
- 集群部署，主从模式，高可用。

3、Consumer端如何保证消息不丢失

- 完全消费正常后在进行手动ack确认。

7. rocketMQ的消息堆积如何处理?

首先要找到是什么原因导致的消息堆积，是Producer太多了，Consumer太少了导致的还是说其他情况，总之先定位问题。

然后看下消息消费速度是否正常，正常的话，可以通过上线更多consumer临时解决消息堆积问题

追问：如果Consumer和Queue不对等，上线了多台也在短时间内无法消费完堆积的消息怎么办?

- 准备一个临时的topic
- queue的数量是堆积的几倍
- queue分布到多Broker中
- 上线一台Consumer做消息的搬运工，把原来Topic中的消息挪到新的Topic里，不做业务逻辑处理，只是挪过去
- 上线N台Consumer同时消费临时Topic中的数据
- 改bug
- 恢复原来的Consumer，继续消费之前的Topic

追问：堆积时间过长消息超时了?

RocketMQ中的消息只会在commitLog被删除的时候才会消失，不会超时。也就是说未被消费的消息不会存在超时删除这情况。

追问：堆积的消息会不会进死信队列?

不会，消息在消费失败后会进入重试队列（%RETRY%+ConsumerGroup），18次（默认18次，网上所有文章都说是16次，无一例外。但是我没搞懂为啥是16次，这不是18个时间吗？）才会进入死信队列（%DLQ%+ConsumerGroup）。

8. RocketMQ为什么自研nameserver而不用zk?

1. RocketMQ只需要一个轻量级的维护元数据信息的组件，为此引入zk增加维护成本还强依赖另一个中间件了。
2. RocketMQ追求的是AP，而不是CP，也就是需要高可用。

- zk是CP，因为zk节点间通过zap协议有数据共享，每个节点数据会一致，但是zk集群当挂了一半以上的节点就没法使用了。
- nameserver是AP，节点间不通信，这样会导致节点间数据信息会发生短暂的不一致，但每个broker都会定时向所有nameserver上报路由信息和心跳。当某个broker下线了，nameserver也会延时30s才知道，而且不会通知客户端（生产和消费者），只能靠客户端自己来拉，rocketMQ是靠消息重试机制解决这个问题的，所以是最终一致性。但nameserver集群只要有一个节点就可用。https://juejin.cn/post/6844904068771479559

Kafka

1. Apache Kafka是什么?

Apach Kafka是一款分布式流处理平台，用于实时构建流处理应用。它有一个核心的功能广为人知，即作为企业级的消息引擎被广泛使用（通常也会称之为消息总线message bus）。

2. Kafka 的设计是什么样的?

Kafka 将消息以 topic 为单位进行归纳

将向 Kafka topic 发布消息的程序成为 producers.

将预订 topics 并消费消息的程序成为 consumer.

Kafka 以集群的方式运行，可以由一个或多个服务组成，每个服务叫做一个 broker.

producers 通过网络将消息发送到 Kafka 集群，集群向消费者提供消息

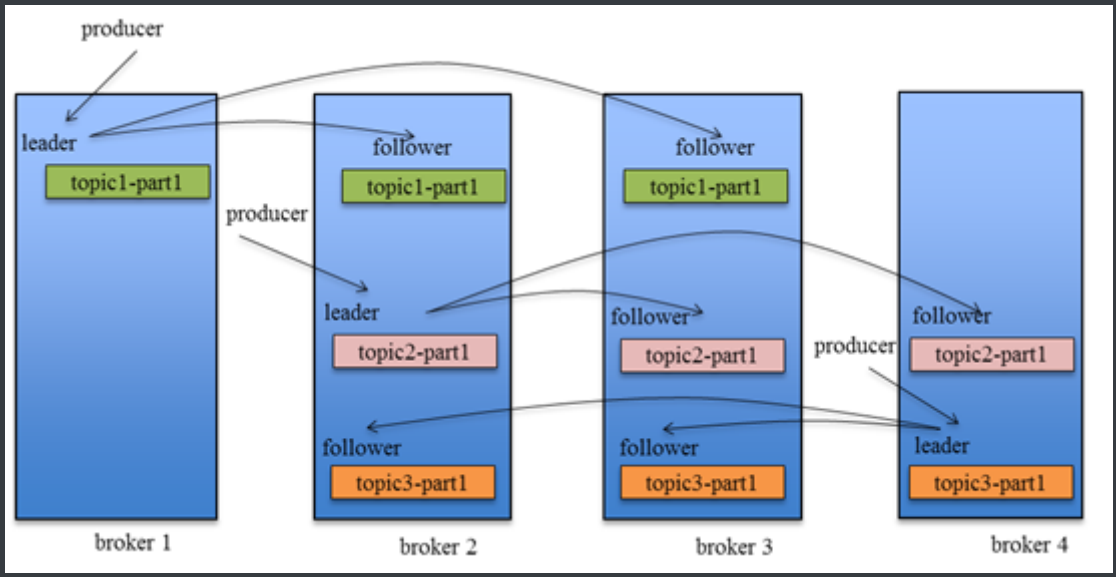
3. Kafka 如何保证高可用?

`Kafka` 的基本架构组成是：由多个 `broker` 组成一个集群，每个 `broker` 是一个节点；当创建一个 `topic` 时，这个 `topic` 会被划分为多个 `partition`，每个 `partition` 可以存在于不同的 `broker` 上，每个 `partition` 只存放一部分数据。

这就是**天然的分布式消息队列**，就是说一个 `topic` 的数据，是**分散放在多个机器上的，每个机器就放一部分数据。**

在 `Kafka 0.8` 版本之前，是没有 `HA` 机制的，当任何一个 `broker` 所在节点宕机了，这个 `broker` 上的 `partition` 就无法提供读写服务，所以这个版本之前， `Kafka` 没有什么高可用性可言。

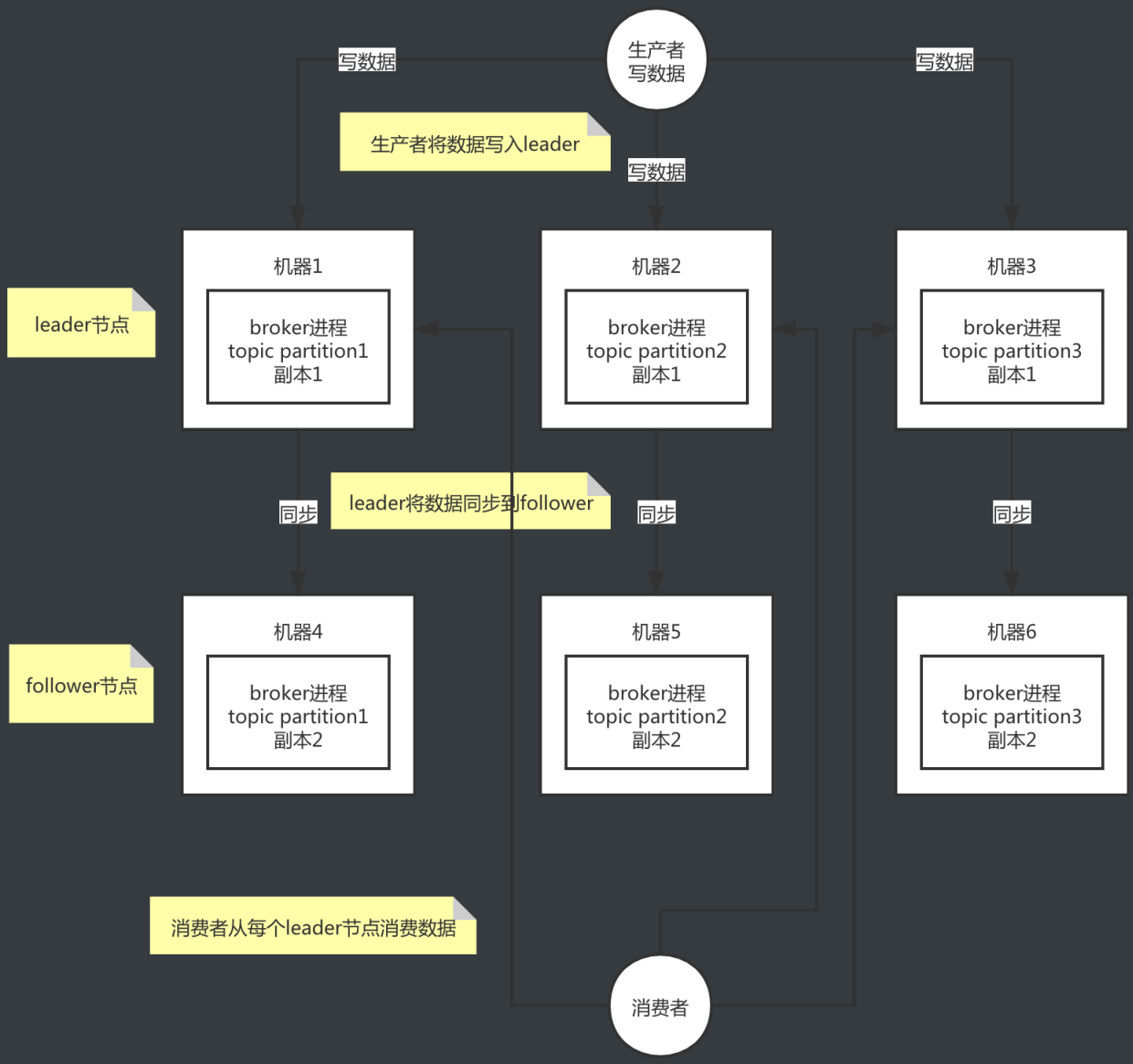
在 Kafka 0.8 以后，提供了 HA 机制，就是 replica 副本机制。每个 partition 上的数据都会同步到其它机器，形成自己的多个 replica 副本。所有 replica 会选举一个 leader 出来，消息的生产者和消费者都跟这个 leader 打交道，其他 replica 作为 follower 。写的时候， leader 会负责把数据同步到所有 follower 上去，读的时候就直接读 leader 上的数据即可。 Kafka 负责均匀的将一个 partition 的所有 replica 分布在不同的机器上，这样才可以提高容错性。



拥有了 replica 副本机制，如果某个 broker 宕机了，这个 broker 上的 partition 在其他机器上还存在副本。如果这个宕机的 broker 上面有某个 partition 的 leader ，那么此时会从其 follower 中重新选举一个新的 leader 出来，这个新的 leader 会继续提供读写服务，这就有达到了所谓的高可用性。

写数据的时候，生产者只将数据写入 leader 节点， leader 会将数据写入本地磁盘，接着其他 follower 会主动从 leader 来拉取数据， follower 同步好数据了，就会发送 ack 给 leader ， leader 收到所有 follower 的 ack 之后，就会返回写成功的消息给生产者。

消费数据的时候，消费者只会从 leader 节点去读取消息，但是只有当一个消息已经被所有 follower 都同步成功返回 ack 的时候，这个消息才会被消费者读到。



4. Kafka 消息是采用 Pull 模式，还是 Push 模式?

生产者使用push模式将消息发布到Broker，消费者使用pull模式从Broker订阅消息。

push模式很难适应消费速率不同的消费者，如果push的速度太快，容易造成消费者拒绝服务或网络拥塞；如果push的速度太慢，容易造成消费者性能浪费。但是采用pull的方式也有一个缺点，就是当Broker没有消息时，消费者会陷入不断地轮询中，为了避免这点，kafka有个参数可以让消费者阻塞知道是否有新消息到达。

5. Kafka 与传统消息系统之间的区别

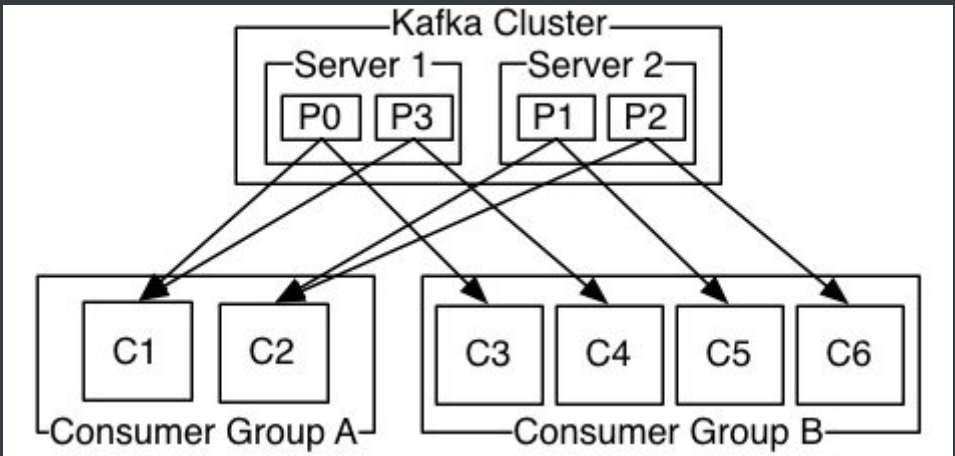
- Kafka 持久化日志，这些日志可以被重复读取和无限期保留
- Kafka 是一个分布式系统：它以集群的方式运行，可以灵活伸缩，在内部通过复制数据提升容错能力和高可用性
- Kafka 支持实时的流式处理

6. 什么是消费者组?

消费者组是Kafka独有的概念，即消费者组是Kafka提供的可扩展且具有容错性的消费者机制。

但实际上，消费者组（Consumer Group）其实包含两个概念，作为队列，消费者组允许你分割数据处理到一组进程集合上（即一个消费者组中可以包含多个消费者进程，他们共同消费该topic的数据），这有助于你的消费能力的动态调整；作为发布-订阅模型（publish-subscribe），Kafka允许你将同一份消息广播到多个消费者组里，以此来丰富多种数据使用场景。

需要注意的是：在消费者组中，多个实例共同订阅若干个主题，实现共同消费。同一个组下的每个实例都配置有相同的组ID，被分配不同的订阅分区。当某个实例挂掉的时候，其他实例会自动地承担起它负责消费的分区。 因此，消费者组在一定程度上也保证了消费者程序的高可用性。



7. 在Kafka中，ZooKeeper的作用是什么?

目前，Kafka使用ZooKeeper存放集群元数据、成员管理、Controller选举，以及其他一些管理类任务。之后，等KIP-500提案完成后，Kafka将完全不再依赖于ZooKeeper。

- “存放元数据”是指主题分区的所有数据都保存在 ZooKeeper 中，且以它保存的数据为权威，其他 “人” 都要与它保持对齐。
- “成员管理” 是指 Broker 节点的注册、注销以及属性变更，等等。
- “Controller 选举” 是指选举集群 Controller，而其他管理类任务包括但不限于主题删除、参数配置等。

KIP-500 思想，是使用社区自研的基于Raft的共识算法，替代ZooKeeper，实现Controller自选举。

8. 解释下Kafka中位移（offset）的作用

在Kafka中，每个主题分区下的每条消息都被赋予了一个唯一的ID数值，用于标识它在分区中的位置。这个ID数值，就被称为位移，或者叫偏移量。一旦消息被写入到分区日志，它的位移值将不能被修改。

9. kafka为什么那么快?

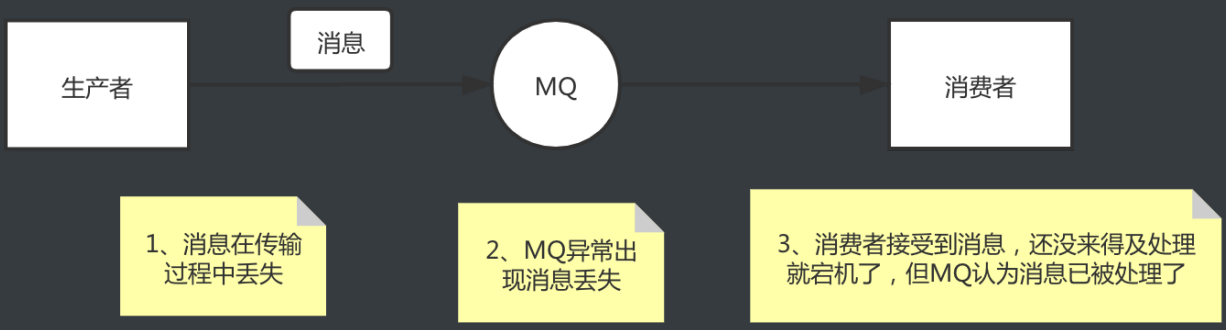
- Cache Filesystem Cache PageCache缓存
- `顺序写`： 由于现代的操作系统提供了预读和写技术，磁盘的顺序写大多数情况下比随机写内存还要快。
- `Zero-copy`：零拷技术减少拷贝次数
- `Batching of Messages`：批量量处理。合并小的请求，然后以流的方式进行交互，直顶网络上限。
- `Pull 拉模式`：使用拉模式进行消息的获取消费，与消费端处理能力相符。

10. kafka producer发送数据，ack为0，1，-1分别是什么意思?

- `1`（默认） 数据发送到Kafka后，经过leader成功接收消息的的确认，就算是发送成功了。在这种情况下，如果leader宕机了，则会丢失数据。
- `0` 生产者将数据发送出去就不管了，不去等待任何返回。这种情况下数据传输效率最高，但是数据可靠性确是最低的。
- `-1` producer需要等待ISR中的所有follower都确认接收到数据后才算一次发送完成，可靠性最高。当ISR中所有Replica都向Leader发送ACK时，leader才commit，这时候producer才能认为一个请求中的消息都commit了。

11. Kafka如何保证消息不丢失?

首先需要弄明白消息为什么会丢失，对于一个消息队列，会有 `生产者`、`MQ`、`消费者` 这三个角色，在这三个角色数据处理和传输过程中，都有可能会出现消息丢失。



消息丢失的原因以及解决办法：

消费者异常导致的消息丢失

消费者可能导致数据丢失的情况是：消费者获取到了这条消息后，还未处理，`Kafka` 就自动提交了 `offset`，这时 `Kafka` 就认为消费者已经处理完这条消息，其实消费者才刚准备处理这条消息，这时如果消费者宕机，那这条消息就丢失了。

消费者引起消息丢失的主要原因就是消息还未处理完 `Kafka` 会自动提交了 `offset`，那么只要关闭自动提交 `offset`，消费者在处理完之后手动提交 `offset`，就可以保证消息不会丢失。但是此时需要注意重复消费问题，比如消费者刚处理完，还没提交 `offset`，这时自己宕机了，此时这条消息肯定会被重复消费一次，这就需要消费者根据实际情况保证幂等性。

生产者数据传输导致的消息丢失

对于生产者数据传输导致的数据丢失主常见情况是生产者发送消息给 `Kafka`，由于网络等原因导致消息丢失，对于这种情况也是通过在 **producer** 端设置 **acks=all** 来处理，这个参数是要求 `leader` 接收到消息后，需要等到所有的 `follower` 都同步到了消息之后，才认为本次写成功了。如果没满足这个条件，生产者会自动不断的重试。

Kafka 导致的消息丢失

`Kafka` 导致的数据丢失一个常见的场景就是 `Kafka` 某个 `broker` 宕机，，而这个节点正好是某个 `partition` 的 `leader` 节点，这时需要重新重新选举该 `partition` 的 `leader`。如果该 `partition` 的 `leader` 在宕机时刚好还有些数据没有同步到 `follower`，此时 `leader` 挂了，在选举某个 `follower` 成 `leader` 之后，就会丢失一部分数据。

对于这个问题，`Kafka` 可以设置如下 4 个参数，来尽量避免消息丢失：

- 给 `topic` 设置 `replication.factor` 参数：这个值必须大于 `1`，要求每个 `partition` 必须有至少 `2` 个副本；
- 在 `Kafka` 服务端设置 `min.insync.replicas` 参数：这个值必须大于 `1`，这个参数的含义是一个 `leader` 至少感知到有至少一个 `follower` 还跟自己保持联系，没掉队，这样才能确保 `leader` 挂了还有一个 `follower` 节点。
- 在 `producer` 端设置 `acks=all`，这个是要求每条数据，必须是写入所有 `replica` 之后，才能认为是写成功了；
- 在 `producer` 端设置 `retries=MAX`（很大很大很大的一个值，无限次重试的意思）：这个参数的含义是一旦写入失败，就无限重试，卡在这里了。

12. Kafka 如何保证消息的顺序性

在某些业务场景下，我们需要保证对于有逻辑关联的多条MQ消息被按顺序处理，比如对于某一条数据，正常处理顺序是 `新增-更新-删除` ，最终结果是数据被删除；如果消息没有按序消费，处理顺序可能是 `删除-新增-更新` ，最终数据没有被删掉，可能会产生一些逻辑错误。对于如何保证消息的顺序性，主要需要考虑如下两点：

- 如何保证消息在 `Kafka` 中顺序性；
- 如何保证消费者处理消费的顺序性。

如何保证消息在 Kafka 中顺序性

对于 `Kafka` ，如果我们创建了一个 `topic` ，默认有三个 `partition` 。生产者在写数据的时候，可以指定一个 `key` ，比如在订单 `topic` 中我们可以指定订单 `id` 作为 `key` ，那么相同订单 `id` 的数据，一定会被分发到同一个 `partition` 中去，而且这个 `partition` 中的数据一定是有顺序的。消费者从 `partition` 中取出来数据的时候，也一定是有顺序的。通过制定 `key` 的方式首先可以保证在 `kafka` 内部消息是有序的。

如何保证消费者处理消费的顺序性

对于某个 `topic` 的一个 `partition` ，只能被同组内部的一个 `consumer` 消费，如果这个 `consumer` 内部还是单线程处理，那么其实只要保证消息在 `MQ` 内部是有顺序的就可以保证消费也是有顺序的。但是单线程吞吐量太低，在处理大量 `MQ` 消息时，我们一般会开启多线程消费机制，那么如何保证消息在多个线程之间是被顺序处理的呢？对于多线程消费我们可以预先设置 `N` 个内存 `Queue` ，具有相同 `key` 的数据都放到同一个内存 `Queue` 中；然后开启 `N` 个线程，每个线程分别消费一个内存 `Queue` 的数据即可，这样就能保证顺序性。当然，消息放到内存 `Queue` 中，有可能还未被处理， `consumer` 发生宕机，内存 `Queue` 中的数据会全部丢失，这就转变为上面提到的**如何保证消息的可靠传输**的问题了。

13. Kafka中的ISR、AR代表什么？ISR的伸缩指什么？

- `ISR`： In-Sync Replicas 副本同步队列
- `AR` :Assigned Replicas 所有副本

ISR是由leader维护，follower从leader同步数据有一些延迟（包括 `延迟时间replica.lag.time.max.ms` 和 `延迟条数replica.lag.max.messages` 两个维度，当前最新的版本0.10.x中只支持 `replica.lag.time.max.ms` 这个维度），任意一个超过阈值都会把follower剔除出ISR，存入OSR（Outof-Sync Replicas）列表，新加入的follower也会先存放在OSR中。

> AR=ISR+OSR。

14. 描述下Kafka中的领导者副本（Leader Replica）和追随者副本（Follower Replica）的区别

Kafka副本当前分为领导者副本和追随者副本。只有Leader副本才能对外提供读写服务，响应Clients端的请求。Follower副本只是采用拉（PULL）的方式，被动地同步Leader副本中的数据，并且在Leader副本所在的Broker宕机后，随时准备应聘Leader副本。

加分点：

- 强调Follower副本也能对外提供读服务。自Kafka 2.4版本开始，社区通过引入新的Broker端参数，允许Follower副本有限度地提供读服务。
- 强调Leader和Follower的消息序列在实际场景中不一致。通常情况下，很多因素可能造成Leader和Follower之间的不同步，比如程序问题，网络问题，broker问题等，短暂的不同步我们可以关注（秒级别），但长时间的不同步可能就需要深入排查了，因为一旦Leader所在节点异常，可能直接影响可用性。

注意：之前确保一致性的主要手段是高水位机制（HW），但高水位值无法保证Leader连续变更场景下的数据一致性，因此，社区引入了Leader Epoch机制，来修复高水位值的弊端。

15. 分区Leader选举策略有几种?

分区的Leader副本选举对用户是完全透明的，它是由Controller独立完成的。你需要回答的是，在哪些场景下，需要执行分区Leader选举。每一种场景对应于一种选举策略。

- OfflinePartition Leader选举：每当有分区上线时，就需要执行Leader选举。所谓的分区上线，可能是创建了新分区，也可能是之前的下线分区重新上线。这是最常见的分区Leader选举场景。
- ReassignPartition Leader选举：当你手动运行kafka-reassign-partitions命令，或者是调用Admin的alterPartitionReassignments方法执行分区副本重分配时，可能触发此类选举。假设原来的AR是[1，2，3]，Leader是1，当执行副本重分配后，副本集合AR被设置成[4，5，6]，显然，Leader必须要变更，此时会发生Reassign Partition Leader选举。
- PreferredReplicaPartition Leader选举：当你手动运行kafka-preferred-replica-election命令，或自动触发了Preferred Leader选举时，该类策略被激活。所谓的Preferred Leader，指的是AR中的第一个副本。比如AR是[3，2，1]，那么，Preferred Leader就是3。
- ControlledShutdownPartition Leader选举：当Broker正常关闭时，该Broker上的所有Leader副本都会下线，因此，需要为受影响的分区执行相应的Leader选举。

这4类选举策略的大致思想是类似的，即从AR中挑选首个在ISR中的副本，作为新Leader。

16. Kafka的哪些场景中使用了零拷贝（Zero Copy）？

在Kafka中，体现Zero Copy使用场景的地方有两处：基于mmap的索引和日志文件读写所用的TransportLayer。

先说第一个。索引都是基于MappedByteBuffer的，也就是让用户态和内核态共享内核态的数据缓冲区，此时，数据不需要复制到用户态空间。不过，mmap虽然避免了不必要的拷贝，但不一定就能保证很高的性能。在不同的操作系统下，mmap的创建和销毁成本可能是不一样的。很高的创建和销毁开销会抵消Zero Copy带来的性能优势。由于这种不确定性，在Kafka中，只有索引应用了mmap，最核心的日志并未使用mmap机制。

再说第二个。TransportLayer是Kafka传输层的接口。它的某个实现类使用了FileChannel的transferTo方法。该方法底层使用sendfile实现了Zero Copy。对Kafka而言，如果I/O通道使用普通的PLAINTEXT，那么，Kafka就可以利用Zero Copy特性，直接将页缓存中的数据发送到网卡的Buffer中，避免中间的多次拷贝。相反，如果I/O通道启用了SSL，那么，Kafka便无法利用Zero Copy特性了。

17. 为什么Kafka不支持读写分离?

在 Kafka 中，生产者写入消息、消费者读取消息的操作都是与 leader 副本进行交互的，从 而实现的是一种主写主读的生产消费模型。

Kafka 并不支持主写从读，因为主写从读有 2 个很明 显的缺点:

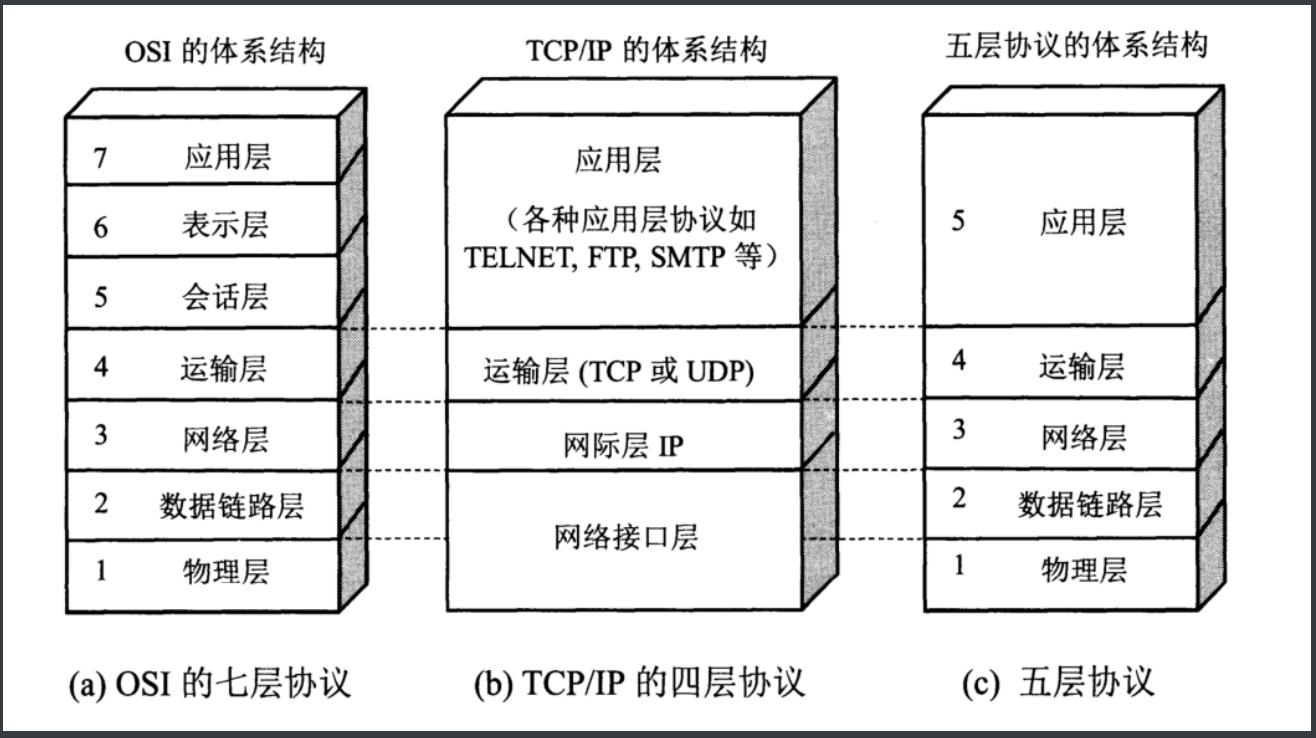
- **数据一致性问题**。数据从主节点转到从节点必然会有一个延时的时间窗口，这个时间 窗口会导致主从节点之间的数据不一致。某一时刻，在主节点和从节点中 A 数据的值都为 X， 之后将主节点中 A 的值修改为 Y，那么在这个变更通知到从节点之前，应用读取从节点中的 A 数据的值并不为最新的 Y，由此便产生了数据不一致的问题。
- **延时问题**。类似 Redis 这种组件，数据从写入主节点到同步至从节点中的过程需要经历 网络→主节点内存→网络→从节点内存 这几个阶段，整个过程会耗费一定的时间。而在 Kafka 中，主从同步会比 Redis 更加耗时，它需要经历 网络→主节点内存→主节点磁盘→网络→从节点内存→从节点磁盘 这几个阶段。对延时敏感的应用而言，主写从读的功能并不太适用。

十、计算机网络

1. 计算机网络的各层协议及作用?

计算机网络体系可以大致分为一下三种，OSI七层模型、TCP/IP四层模型和五层模型。

- OSI七层模型：大而全，但是比较复杂、而且是先有了理论模型，没有实际应用。
- TCP/IP四层模型：是由实际应用发展总结出来的，从实质上讲，TCP/IP只有最上面三层，最下面一层没有什么具体内容，TCP/IP参考模型没有真正描述这一层的实现。
- 五层模型：五层模型只出现在计算机网络教学过程中，这是对七层模型和四层模型的一个折中，既简洁又能将概念阐述清楚。



七层网络体系结构各层的主要功能：

- 应用层：为应用程序提供交互服务。在互联网中的应用层协议很多，如域名系统DNS，支持万维网应用的HTTP协议，支持电子邮件的SMTP协议等。
- 表示层：主要负责数据格式的转换，如加密解密、转换翻译、压缩解压缩等。
- 会话层：负责在网络中的两节点之间建立、维持和终止通信，如服务器验证用户登录便是由会话层完成的。
- 运输层：有时也译为传输层，向主机进程提供通用的数据传输服务。该层主要有以下两种协议：
  - TCP：提供面向连接的、可靠的数据传输服务；
  - UDP：提供无连接的、尽最大努力的数据传输服务，但不保证数据传输的可靠性。
- 网络层：选择合适的路由和交换结点，确保数据及时传送。主要包括IP协议。
- 数据链路层：数据链路层通常简称为链路层。将网络层传下来的IP数据包组装成帧，并再相邻节点的链路上传送帧。
- `物理层`：实现相邻节点间比特流的透明传输，尽可能屏蔽传输介质和通信手段的差异。

2. TCP和UDP的区别?

对比如下：

	UDP	TCP
是否连接	无连接	面向连接
是否可靠	不可靠传输，不使用流量控制和拥塞控制	可靠传输，使用流量控制和拥塞控制
是否有序	无序	有序，消息在传输过程中可能会乱序，TCP 会重新排序
传输速度	快	慢
连接对象个数	支持一对一，一对多，多对一和多对多交互通信	只能是一对一通信
传输方式	面向报文	面向字节流
首部开销	首部开销小，仅8字节	首部最小20字节，最大60字节
适用场景	适用于实时应用（IP电话、视频会议、直播等）	适用于要求可靠传输的应用，例如文件传输

总结：

TCP 用于在传输层有必要实现可靠传输的情况，UDP 用于对高速传输和实时性有较高要求的通信。TCP 和 UDP 应该根据应用目的按需使用。

3. UDP 和 TCP 对应的应用场景是什么?

TCP 是面向连接，能保证数据的可靠性交付，因此经常用于：

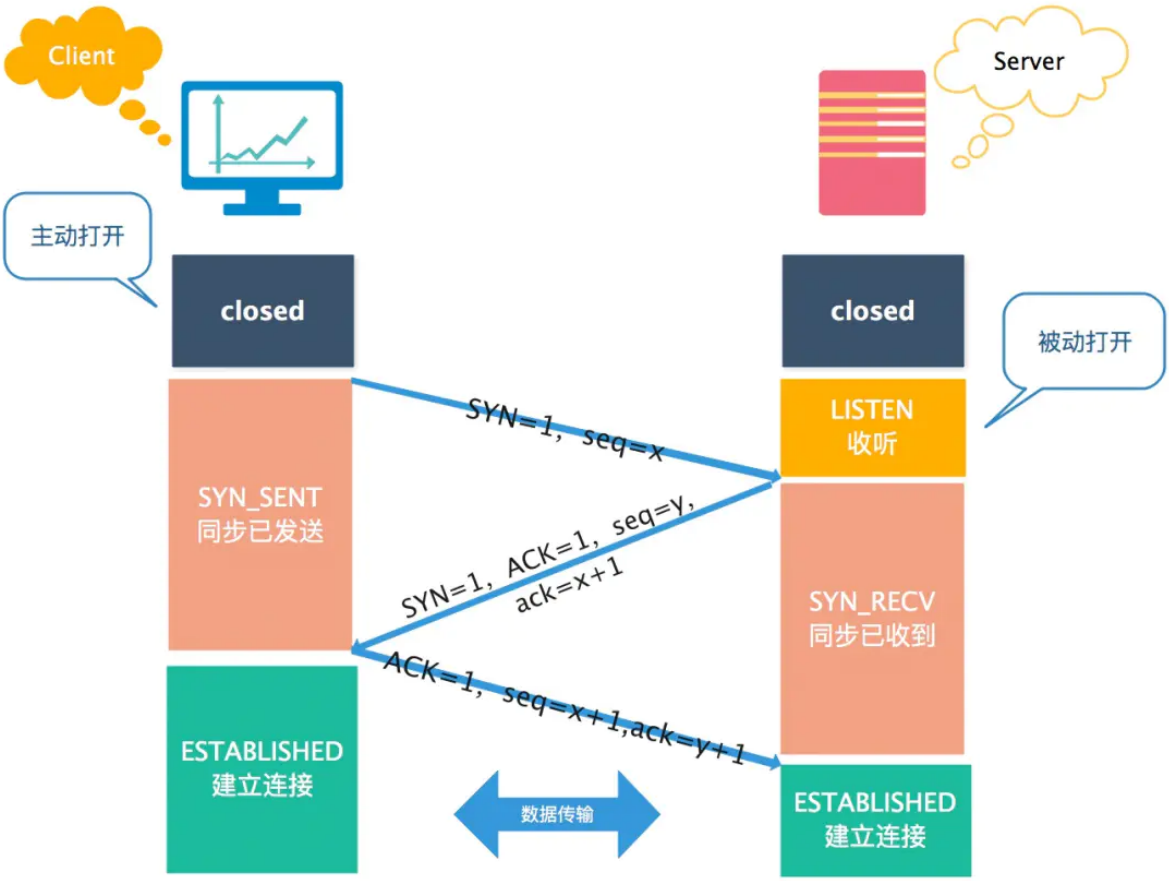
- FTP文件传输
- HTTP / HTTPS

UDP 面向无连接，它可以随时发送数据，再加上UDP本身的处理既简单又高效，因此经常用于：

- 包总量较少的通信，如 DNS 、SNMP等
- 视频、音频等多媒体通信
- 广播通信

应用层协议	应用	传输层协议
SMTP	电子邮件	TCP
TELNET	远程终端接入	
HTTP	万维网	
FTP	文件传输	
DNS	域名转换	UDP
TFTP	文件传输	
SNMP	网络管理	
NFS	远程文件服务器	

4. 详细介绍一下 TCP 的三次握手机制?



> 图片来自：https://juejin.cn/post/6844904005315854343

三次握手机制：

- 第一次握手：客户端请求建立连接，向服务端发送一个**同步报文**（SYN=1），同时选择一个随机数 seq = x 作为**初始序列号**，并进入SYN_SENT状态，等待服务器确认。
- 第二次握手：：服务端收到连接请求报文后，如果同意建立连接，则向客户端发送**同步确认报文**（SYN=1，ACK=1），确认号为 ack = x + 1，同时选择一个随机数 seq = y 作为初始序列号，此时服务器进入SYN_RECV状态。
- 第三次握手：客户端收到服务端的确认后，向服务端发送一个**确认报文**（ACK=1），确认号为 ack = y + 1，序列号为 seq = x + 1，客户端和服务器进入ESTABLISHED状态，完成三次握手。

理想状态下，TCP连接一旦建立，在通信双方中的任何一方主动关闭连接之前，TCP 连接都将被一直保持下去。

5. 为什么需要三次握手，而不是两次?

主要有三个原因：

1. 防止已过期的连接请求报文突然又传送到服务器，因而产生错误和资源浪费。

   在双方两次握手即可建立连接的情况下，假设客户端发送 A 报文段请求建立连接，由于网络原因造成 A 暂时无法到达服务器，服务器接收不到请求报文段就不会返回确认报文段。

   客户端在长时间得不到应答的情况下重新发送请求报文段 B，这次 B 顺利到达服务器，服务器随即返回确认报文并进入 ESTABLISHED 状态，客户端在收到 确认报文后也进入 ESTABLISHED 状态，双方建立连接并传输数据，之后正常断开连接。

   此时姗姗来迟的 A 报文段才到达服务器，服务器随即返回确认报文并进入 ESTABLISHED 状态，但是已经进入 CLOSED 状态的客户端无法再接受确认报文段，更无法进入 ESTABLISHED 状态，这将导致服务器长时间单方面等待，造成资源浪费。

2. 三次握手才能让双方均确认自己和对方的发送和接收能力都正常。

   第一次握手：客户端只是发送处请求报文段，什么都无法确认，而服务器可以确认自己的接收能力和对方的发送能力正常；

   第二次握手：客户端可以确认自己发送能力和接收能力正常，对方发送能力和接收能力正常；

   第三次握手：服务器可以确认自己发送能力和接收能力正常，对方发送能力和接收能力正常；

   可见三次握手才能让双方都确认自己和对方的发送和接收能力全部正常，这样就可以愉快地进行通信了。

3. 告知对方自己的初始序号值，并确认收到对方的初始序号值。

   TCP 实现了可靠的数据传输，原因之一就是 TCP 报文段中维护了序号字段和确认序号字段，通过这两个字段双方都可以知道在自己发出的数据中，哪些是已经被对方确认接收的。这两个字段的值会在初始序号值得基础递增，如果是两次握手，只有发起方的初始序号可以得到确认，而另一方的初始序号则得不到确认。

6. 为什么要三次握手，而不是四次?

因为三次握手已经可以确认双方的发送接收能力正常，双方都知道彼此已经准备好，而且也可以完成对双方初始序号值得确认，也就无需再第四次握手了。

- 第一次握手：服务端确认“自己收、客户端发”报文功能正常。
- 第二次握手：客户端确认“自己发、自己收、服务端收、客户端发”报文功能正常，客户端认为连接已建立。
- 第三次握手：服务端确认“自己发、客户端收”报文功能正常，此时双方均建立连接，可以正常通信。

7. 什么是 SYN洪泛攻击？如何防范?

SYN洪泛攻击属于 DOS 攻击的一种，它利用 TCP 协议缺陷，通过发送大量的半连接请求，耗费 CPU 和内存资源。

原理：

- 在三次握手过程中，服务器发送 `[SYN/ACK]` 包（第二个包）之后、收到客户端的 `[ACK]` 包（第三个包）之前的 TCP 连接称为半连接（half-open connect），此时服务器处于 `SYN_RECV` （等待客户端响应）状态。如果接收到客户端的 `[ACK]`，则 TCP 连接成功，如果未接受到，则会**不断重发请求**直至成功。
- SYN 攻击的攻击者在短时间内**伪造大量不存在的 IP 地址**，向服务器不断地发送 `[SYN]` 包，服务器回复 `[SYN/ACK]` 包，并等待客户的确认。由于源地址是不存在的，服务器需要不断的重发直至超时。
- 这些伪造的 `[SYN]` 包将长时间占用未连接队列，影响了正常的 SYN，导致目标系统运行缓慢、网络堵塞甚至系统瘫痪。

检测：当在服务器上看到大量的半连接状态时，特别是源 IP 地址是随机的，基本上可以断定这是一次 SYN 攻击。

防范：

- 通过防火墙、路由器等过滤网关防护。
- 通过加固 TCP/IP 协议栈防范，如增加最大半连接数，缩短超时时间。
- SYN cookies技术。SYN Cookies 是对 TCP 服务器端的三次握手做一些修改，专门用来防范 SYN 洪泛攻击的一种手段。

8. 三次握手连接阶段，最后一次ACK包丢失，会发生什么?

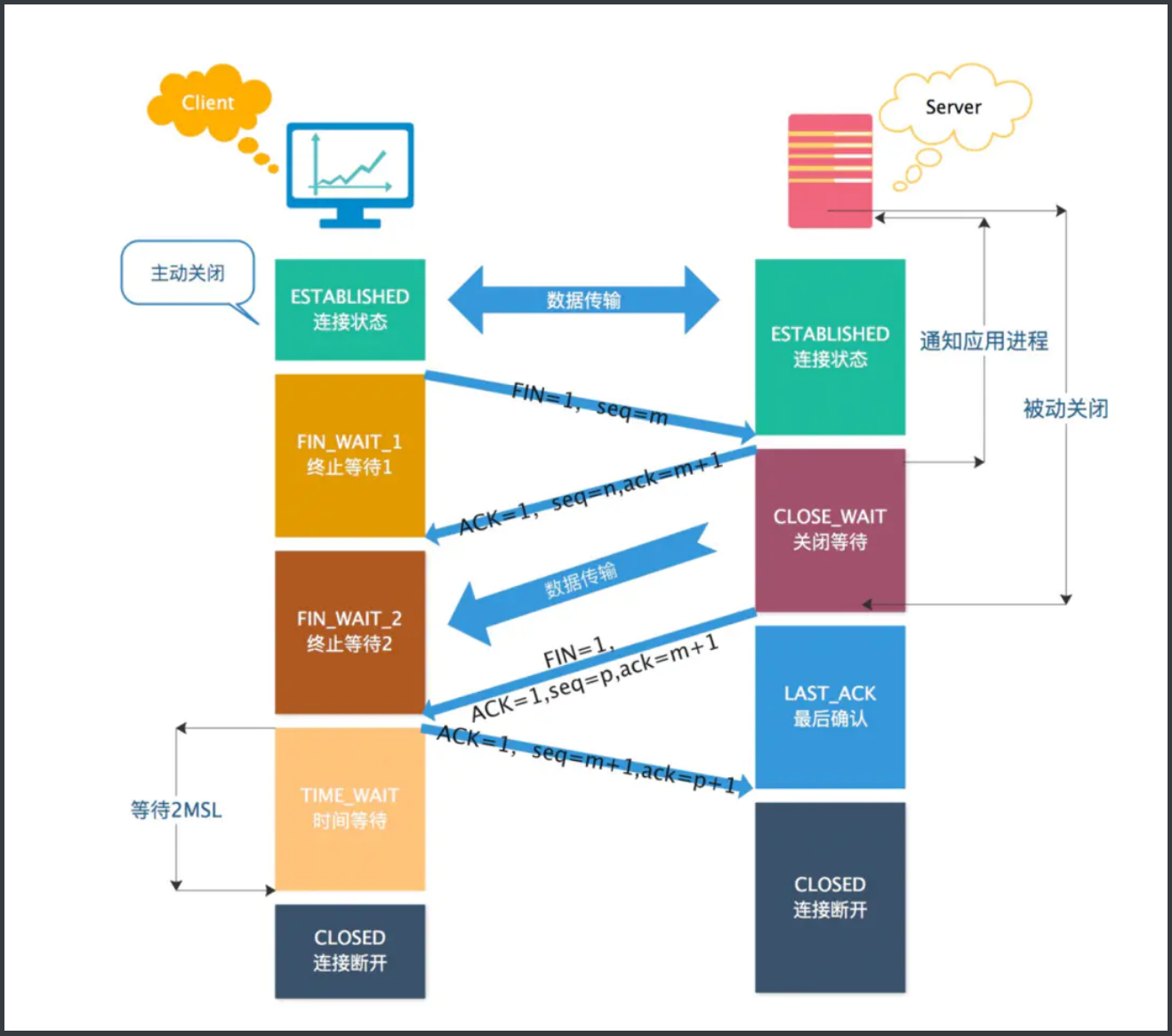
服务端：

- 第三次的ACK在网络中丢失，那么服务端该TCP连接的状态为SYN_RECV,并且会根据TCP的超时重传机制，会等待3秒、6秒、12秒后重新发送SYN+ACK包，以便客户端重新发送ACK包。
- 如果重发指定次数之后，仍然未收到 客户端的ACK应答，那么一段时间后，服务端自动关闭这个连接。

客户端：

客户端认为这个连接已经建立，如果客户端向服务端发送数据，服务端将以RST包（Reset，标示复位，用于异常的关闭连接）响应。此时，客户端知道第三次握手失败。

9. 详细介绍一下 TCP 的四次挥手过程?



> 图片来源：https://juejin.im/post/5ddd1f30e51d4532c42c5abe

- 第一次挥手：客户端向服务端发送连接释放报文（FIN=1，ACK=1），主动关闭连接，同时等待服务端的确认。
  - 序列号 seq = u，即客户端上次发送的报文的最后一个字节的序号 + 1
  - 确认号 ack = k, 即服务端上次发送的报文的最后一个字节的序号 + 1
- 第二次挥手：服务端收到连接释放报文后，立即发出**确认报文**（ACK=1），序列号 seq = k，确认号 ack = u + 1。

  这时 TCP 连接处于半关闭状态，即客户端到服务端的连接已经释放了，但是服务端到客户端的连接还未释放。这表示客户端已经没有数据发送了，但是服务端可能还要给客户端发送数据。
- 第三次挥手：服务端向客户端发送连接释放报文（FIN=1，ACK=1），主动关闭连接，同时等待 A 的确认。

    - 序列号 seq = w，即服务端上次发送的报文的最后一个字节的序号 + 1。
    - 确认号 ack = u + 1，与第二次挥手相同，因为这段时间客户端没有发送数据
- 第四次挥手：客户端收到服务端的连接释放报文后，立即发出**确认报文**（ACK=1），序列号 seq = u + 1，确认号为 ack = w + 1。

  此时，客户端就进入了 `TIME-WAIT` 状态。注意此时客户端到 TCP 连接还没有释放，必须经过 2*MSL（最长报文段寿命）的时间后，才进入 `CLOSED` 状态。而服务端只要收到客户端发出的确认，就立即进入 `CLOSED` 状态。可以看到，服务端结束 TCP 连接的时间要比客户端早一些。

10. 为什么连接的时候是三次握手，关闭的时候却是四次握手?

服务器在收到客户端的 FIN 报文段后，可能还有一些数据要传输，所以不能马上关闭连接，但是会做出应答，返回 ACK 报文段.

接下来可能会继续发送数据，在数据发送完后，服务器会向客户单发送 FIN 报文，表示数据已经发送完毕，请求关闭连接。服务器的**ACK和FIN一般都会分开发送**，从而导致多了一次，因此一共需要四次挥手。

11. 为什么客户端的 TIME-WAIT 状态必须等待 2MSL？

主要有两个原因：

1. 确保 ACK 报文能够到达服务端，从而使服务端正常关闭连接。

   第四次挥手时，客户端第四次挥手的 ACK 报文不一定会到达服务端。服务端会超时重传 FIN/ACK 报文，此时如果客户端已经断开了连接，那么就无法响应服务端的二次请求，这样服务端迟迟收不到 FIN/ACK 报文的确认，就无法正常断开连接。

   MSL 是报文段在网络上存活的最长时间。客户端等待 2MSL 时间，即「客户端 ACK 报文 1MSL 超时 + 服务端 FIN 报文 1MSL 传输」，就能够收到服务端重传的 FIN/ACK 报文，然后客户端重传一次 ACK 报文，并重新启动 2MSL 计时器。如此保证服务端能够正常关闭。

   如果服务端重发的 FIN 没有成功地在 2MSL 时间里传给客户端，服务端则会继续超时重试直到断开连接。

2. 防止已失效的连接请求报文段出现在之后的连接中。

   TCP 要求在 2MSL 内不使用相同的序列号。客户端在发送完最后一个 ACK 报文段后，再经过时间 2MSL，就可以保证本连接持续的时间内产生的所有报文段都从网络中消失。这样就可以使下一个连接中不会出现这种旧的连接请求报文段。或者即使收到这些过时的报文，也可以不处理它。

12. 如果已经建立了连接，但是客户端出现故障了怎么办?

或者说，如果三次握手阶段、四次挥手阶段的包丢失了怎么办？如“服务端重发 FIN丢失”的问题。

简而言之，通过**定时器 + 超时重试机制**，尝试获取确认，直到最后会自动断开连接。

具体而言，TCP 设有一个保活计时器。服务器每收到一次客户端的数据，都会重新复位这个计时器，时间通常是设置为 2 小时。若 2 小时还没有收到客户端的任何数据，服务器就开始重试：每隔 75 分钟发送一个探测报文段，若一连发送 10 个探测报文后客户端依然没有回应，那么服务器就认为连接已经断开了。

13. TIME-WAIT 状态过多会产生什么后果？怎样处理?

从服务器来讲，短时间内关闭了大量的Client连接，就会造成服务器上出现大量的TIME_WAIT连接，严重消耗着服务器的资源，此时部分客户端就会显示连接不上。

从客户端来讲，客户端TIME_WAIT过多，就会导致端口资源被占用，因为端口就65536个，被占满就会导致无法创建新的连接。

解决办法：

- 服务器可以设置 SO_REUSEADDR 套接字选项来避免 TIME_WAIT状态，此套接字选项告诉内核，即使此端口正忙（处于TIME_WAIT状态），也请继续并重用它。
- 调整系统内核参数，修改/etc/sysctl.conf文件，即修改 `net.ipv4.tcp_tw_reuse` 和 `tcp_timestamps`

  ```
  net.ipv4.tcp_tw_reuse = 1 表示开启重用。允许将TIME-WAIT sockets重新用于新的TCP连接，默认为0，表示关闭;
  net.ipv4.tcp_tw_recycle = 1 表示开启TCP连接中TIME-WAIT sockets的快速回收，默认为0，表示关闭。
  ```

- 强制关闭，发送 RST 包越过TIME_WAIT状态，直接进入CLOSED状态。

14. TIME_WAIT 是服务器端的状态?还是客户端的状态?

TIME_WAIT 是主动断开连接的一方会进入的状态，一般情况下，都是客户端所处的状态;服务器端一般设置不主动关闭连接。

TIME_WAIT 需要等待 2MSL，在大量短连接的情况下，TIME_WAIT会太多，这也会消耗很多系统资源。对于服务器来说，在 HTTP 协议里指定 KeepAlive（浏览器重用一个 TCP 连接来处理多个 HTTP 请求），由浏览器来主动断开连接，可以一定程度上减少服务器的这个问题。

15. TCP协议如何保证可靠性?

TCP主要提供了检验和、序列号/确认应答、超时重传、滑动窗口、拥塞控制和 流量控制等方法实现了可靠性传输。

- 检验和：通过检验和的方式，接收端可以检测出来数据是否有差错和异常，假如有差错就会直接丢弃TCP段，重新发送。
- 序列号/确认应答：

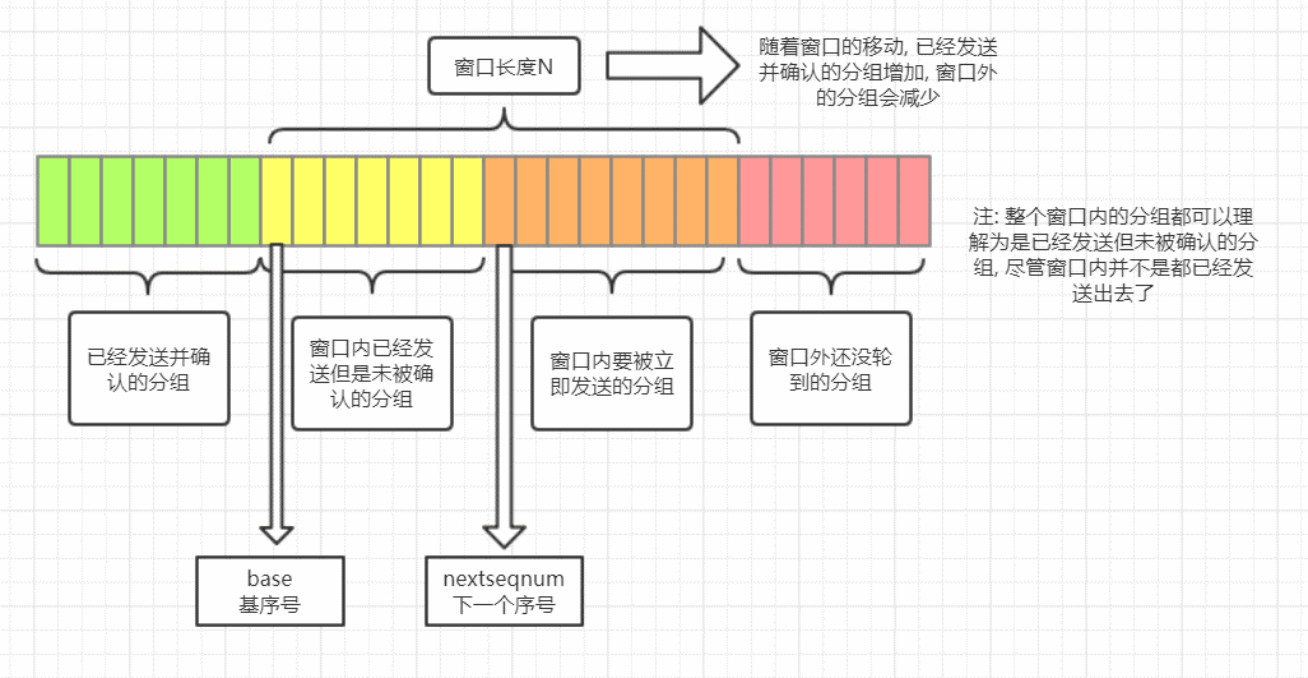
  序列号的作用不仅仅是应答的作用，有了序列号能够将接收到的数据根据序列号排序，并且去掉重复序列号的数据。

  TCP传输的过程中，每次接收方收到数据后，都会对传输方进行确认应答。也就是发送ACK报文，这个ACK报文当中带有对应的确认序列号，告诉发送方，接收到了哪些数据，下一次的数据从哪里发。
- 滑动窗口：滑动窗口既提高了报文传输的效率，也避免了发送方发送过多的数据而导致接收方无法正常处理的异常。
- 超时重传：超时重传是指发送出去的数据包到接收到确认包之间的时间，如果超过了这个时间会被认为是丢包了，需要重传。最大超时时间是动态计算的。
- 拥塞控制：在数据传输过程中，可能由于网络状态的问题，造成网络拥堵，此时引入拥塞控制机制，在保证TCP可靠性的同时，提高性能。
- 流量控制：如果主机A 一直向主机B发送数据，不考虑主机B的接受能力，则可能导致主机B的接受缓冲区满了而无法再接受数据，从而会导致大量的数据丢包，引发重传机制。而在重传的过程中，若主机B的接收缓冲区情况仍未好转，则会将大量的时间浪费在重传数据上，降低传送数据的效率。所以引入流量控制机制，主机B通过告诉主机A自己接收缓冲区的大小，来使主机A控制发送的数据量。流量控制与TCP协议报头中的窗口大小有关。

16. 详细讲一下TCP的滑动窗口?

在进行数据传输时，如果传输的数据比较大，就需要拆分为多个数据包进行发送。TCP 协议需要对数据进行确认后，才可以发送下一个数据包。这样一来，就会在等待确认应答包环节浪费时间。

为了避免这种情况，TCP引入了窗口概念。窗口大小指的是不需要等待确认应答包而可以继续发送数据包的最大值。



从上面的图可以看到滑动窗口左边的是已发送并且被确认的分组，滑动窗口右边是还没有轮到的分组。

滑动窗口里面也分为两块，一块是已经发送但是未被确认的分组，另一块是窗口内等待发送的分组。随着已发送的分组不断被确认，窗口内等待发送的分组也会不断被发送。整个窗口就会往右移动，让还没轮到的分组进入窗口内。

可以看到滑动窗口起到了一个限流的作用，也就是说当前滑动窗口的大小决定了当前 TCP 发送包的速率，而滑动窗口的大小取决于拥塞控制窗口和流量控制窗口的两者间的最小值。

17. 详细讲一下拥塞控制?

TCP 一共使用了四种算法来实现拥塞控制：

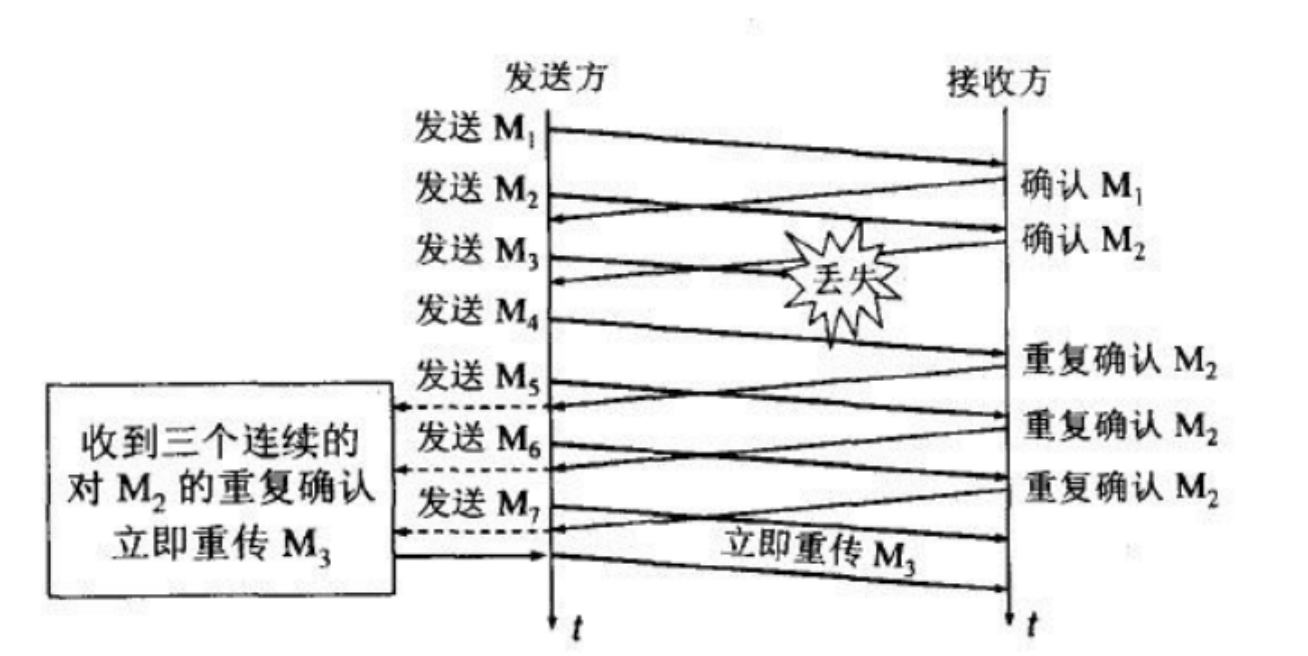
- 慢开始 (slow-start)；
- 拥塞避免 (congestion avoidance)；
- 快速重传 (fast retransmit)；
- 快速恢复 (fast recovery)。

发送方维持一个叫做拥塞窗口cwnd（congestion window）的状态变量。当cwndssthresh时，改用拥塞避免算法。

慢开始：不要一开始就发送大量的数据，由小到大逐渐增加拥塞窗口的大小。

拥塞避免：拥塞避免算法让拥塞窗口缓慢增长，即每经过一个往返时间RTT就把发送方的拥塞窗口cwnd加1而不是加倍。这样拥塞窗口按线性规律缓慢增长。

快重传：我们可以剔除一些不必要的拥塞报文，提高网络吞吐量。比如接收方在**收到一个失序的报文段后就立即发出重复确认，而不要等到自己发送数据时捎带确认。**快重传规定：发送方只要**一连收到三个**重复确认就应当立即重传对方尚未收到的报文段，而不必继续等待设置的重传计时器时间到期。



快恢复：主要是配合快重传。当发送方连续收到三个重复确认时，就执行“乘法减小”算法，把ssthresh门限减半（为了预防网络发生拥塞），但**接下来并不执行慢开始算法**，因为如果网络出现拥塞的话就不会收到好几个重复的确认，收到三个重复确认说明网络状况还可以。

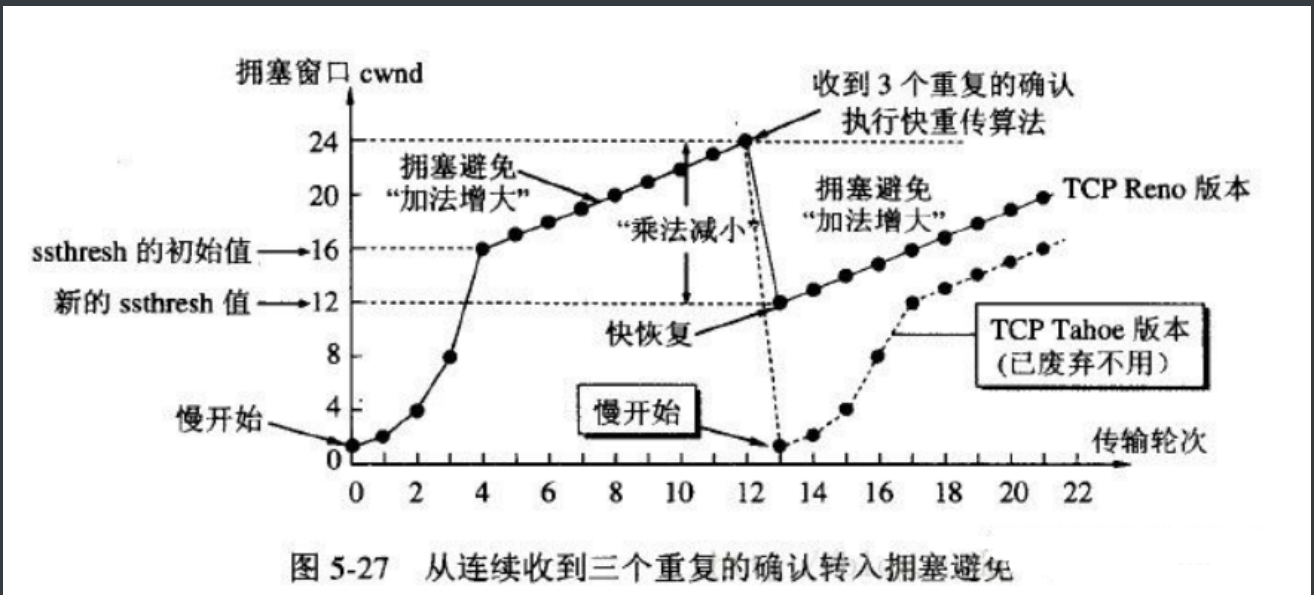


图 5-27　从连续收到三个重复的确认转入拥塞避免

18. HTTP常见的状态码有哪些?

常见状态码：

- 200：服务器已成功处理了请求。 通常，这表示服务器提供了请求的网页。
- 301： (永久移动) 请求的网页已永久移动到新位置。 服务器返回此响应(对 GET 或 HEAD 请求的响应)时，会自动将请求者转到新位置。
- 302：(临时移动) 服务器目前从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。
- 400： 客户端请求有语法错误，不能被服务器所理解。
- 403： 服务器收到请求，但是拒绝提供服务。
- 404： (未找到) 服务器找不到请求的网页。
- 500： (服务器内部错误) 服务器遇到错误，无法完成请求。

状态码开头代表类型：

	类别	原因短语
1XX	Informational（信息性状态码）	接收的请求正在处理
2XX	Success（成功状态码）	请求正常处理完毕
3XX	Redirection（重定向状态码）	需要进行附加操作以完成请求
4XX	Client Error（客户端错误状态码）	服务器无法处理请求
5XX	Server Error（服务器错误状态码）	服务器处理请求出错

19. 状态码301和302的区别是什么?

共同点：301和302状态码都表示重定向，就是说浏览器在拿到服务器返回的这个状态码后会自动跳转到一个新的URL地址，这个地址可以从响应的Location首部中获取（**用户看到的效果就是他输入的地址A瞬间变成了另一个地址B**）。
不同点：301表示旧地址A的资源已经被永久地移除了(这个资源不可访问了)，搜索引擎在抓取新内容的同时也将旧的网址交换为重定向之后的网址；302表示旧地址A的资源还在（仍然可以访问），这个重定向只是临时地从旧地址A跳转到地址B，搜索引擎会抓取新的内容而保存旧的网址。 SEO中302好于301。

补充，重定向原因：

1. 网站调整（如改变网页目录结构）；
2. 网页被移到一个新地址；
3. 网页扩展名改变(如应用需要把.php改成.Html或.shtml)。

20. HTTP 常用的请求方式?

方法	作用
GET	获取资源
POST	传输实体主体
PUT	上传文件
DELETE	删除文件
HEAD	和GET方法类似，但只返回报文首部，不返回报文实体主体部分
PATCH	对资源进行部分修改
OPTIONS	查询指定的URL支持的方法
CONNECT	要求用隧道协议连接代理
TRACE	服务器会将通信路径返回给客户端

为了方便记忆，可以将PUT、DELETE、POST、GET理解为客户端对服务端的增删改查。

- PUT：上传文件，向服务器添加数据，可以看作增
- DELETE：删除文件
- POST：传输数据，向服务器提交数据，对服务器数据进行更新。
- GET：获取资源，查询服务器资源

21. GET请求和POST请求的区别?

使用上的区别:

- GET使用URL或Cookie传参，而POST将数据放在BODY中”，这个是因为HTTP协议用法的约定。
- GET方式提交的数据有长度限制，则POST的数据则可以非常大”，这个是因为它们使用的操作系统和浏览器设置的不同引起的区别。
- POST比GET安全，因为数据在地址栏上不可见”，这个说法没毛病，但依然不是GET和POST本身的区别。

本质区别

GET和POST最大的区别主要是GET请求是幂等性的，POST请求不是。这个是它们本质区别。

幂等性是指一次和多次请求某一个资源应该具有同样的副作用。简单来说意味着对同一URL的多个请求应该返回同样的结果。

22. 解释一下HTTP长连接和短连接?

在HTTP/1.0中，默认使用的是短连接。也就是说，浏览器和服务器每进行一次HTTP操作，就建立一次连接，但任务结束就中断连接。如果客户端浏览器访问的某个HTML或其他类型的Web页中包含有其他的Web资源，如JavaScript文件、图像文件、CSS文件等；当浏览器每遇到这样一个Web资源，就会建立一个HTTP会话。

但从 **HTTP/1.1起，默认使用长连接**，用以保持连接特性。使用长连接的HTTP协议，会在响应头有加入这行代码：`Connection:keep-alive`

在使用长连接的情况下，当一个网页打开完成后，客户端和服务器之间用于传输HTTP数据的TCP连接不会关闭，如果客户端再次访问这个服务器上的网页，会继续使用这一条已经建立的连接。Keep-Alive不会永久保持连接，它有一个保持时间，可以在不同的服务器软件（如Apache）中设定这个时间。实现长连接要客户端和服务端都支持长连接。

HTTP协议的长连接和短连接，实质上是TCP协议的长连接和短连接。

23. HTTP请求报文和响应报文的格式?

请求报文格式:

1. 请求行（请求方法+URI协议+版本）
2. 请求头部
3. 空行
4. 请求主体

```
GET/sample.jspHTTP/1.1 请求行
Accept:image/gif.image/jpeg, 请求头部
Accept-Language:zh-cn
Connection:Keep-Alive
Host:localhost
User-Agent:Mozila/4.0(compatible;MSIE5.01;Window NT5.0)
Accept-Encoding:gzip,deflate

username=jinqiao&password=1234 请求主体
```

响应报文:

1. 状态行（版本+状态码+原因短语）
2. 响应首部

3. 空行
4. 响应主体

```
HTTP/1.1 200 OK
Server:Apache Tomcat/5.0.12
Date:Mon,6Oct2003 13:23:42 GMT
Content-Length:112

<html>
    <head>
        <title>HTTP响应示例<title>
    </head>
    <body>
        Hello HTTP!
    </body>
</html>
```

24. HTTP1.0和HTTP1.1的区别?

- **长连接**：HTTP 1.1支持长连接（Persistent Connection）和请求的流水线（Pipelining）处理，在一个TCP连接上可以传送多个HTTP请求和响应，减少了建立和关闭连接的消耗和延迟，在HTTP1.1中默认开启 `Connection： keep-alive` ，一定程度上弥补了HTTP1.0每次请求都要创建连接的缺点。
- **缓存处理**：在HTTP1.0中主要使用header里的If-Modified-Since,Expires来做为缓存判断的标准，HTTP1.1则引入了更多的缓存控制策略，可供选择的缓存头来控制缓存策略。
- **带宽优化及网络连接的使用**：HTTP1.0中，存在一些浪费带宽的现象，例如客户端只是需要某个对象的一部分，而服务器却将整个对象送过来了，并且不支持断点续传功能，HTTP1.1则在请求头引入了range头域，它允许只请求资源的某个部分，即返回码是206（Partial Content），这样就方便了开发者自由的选择以便于充分利用带宽和连接。
- **错误通知的管理**：在HTTP1.1中新增了24个错误状态响应码，如409（Conflict）表示请求的资源与资源的当前状态发生冲突；410（Gone）表示服务器上的某个资源被永久性的删除。
- **Host头处理**：在HTTP1.0中认为每台服务器都绑定一个唯一的IP地址，因此，请求消息中的URL并没有传递主机名（hostname）。但随着虚拟主机技术的发展，在一台物理服务器上可以存在多个虚拟主机（Multi-homed Web Servers），并且它们共享一个IP地址。HTTP1.1的请求消息和响应消息都应支持Host头域，且请求消息中如果没有Host头域会报告一个错误（400 Bad Request）。

25. HTTP1.1和 HTTP2.0的区别?

HTTP2.0相比HTTP1.1支持的特性：

- **新的二进制格式**：HTTP1.1的解析是基于文本。基于文本协议的格式解析存在天然缺陷，文本的表现形式有多样性，要做到健壮性考虑的场景必然很多，二进制则不同，只认0和1的组合。基于这种考虑HTTP2.0的协议解析决定采用二进制格式，实现方便且健壮。
- **多路复用**，即连接共享，即每一个request都是用作连接共享机制的。一个request对应一个id，这样一个连接上可以有多个request，每个连接的request可以随机的混杂在一起，接收方可以根据request的 id将request再归属到各自不同的服务端请求里面。
- **头部压缩**，HTTP1.1的头部（header）带有大量信息，而且每次都要重复发送；HTTP2.0使用encoder来减少需要传输的header大小，通讯双方各自cache一份header fields表，既避免了重复header的传输，又减小了需要传输的大小。
- **服务端推送**：服务器除了对最初请求的响应外，服务器还可以额外的向客户端推送资源，而无需客户端明确的请求。

26. HTTP 与 HTTPS 的区别?

	HTTP	HTTPS
端口	80	443
安全性	无加密，安全性较差	有加密机制，安全性较高
资源消耗	较少	由于加密处理，资源消耗更多
是否需要证书	不需要	需要
协议	运行在TCP协议之上	运行在SSL协议之上，SSL运行在TCP协议之上

27. HTTPS 的优缺点?

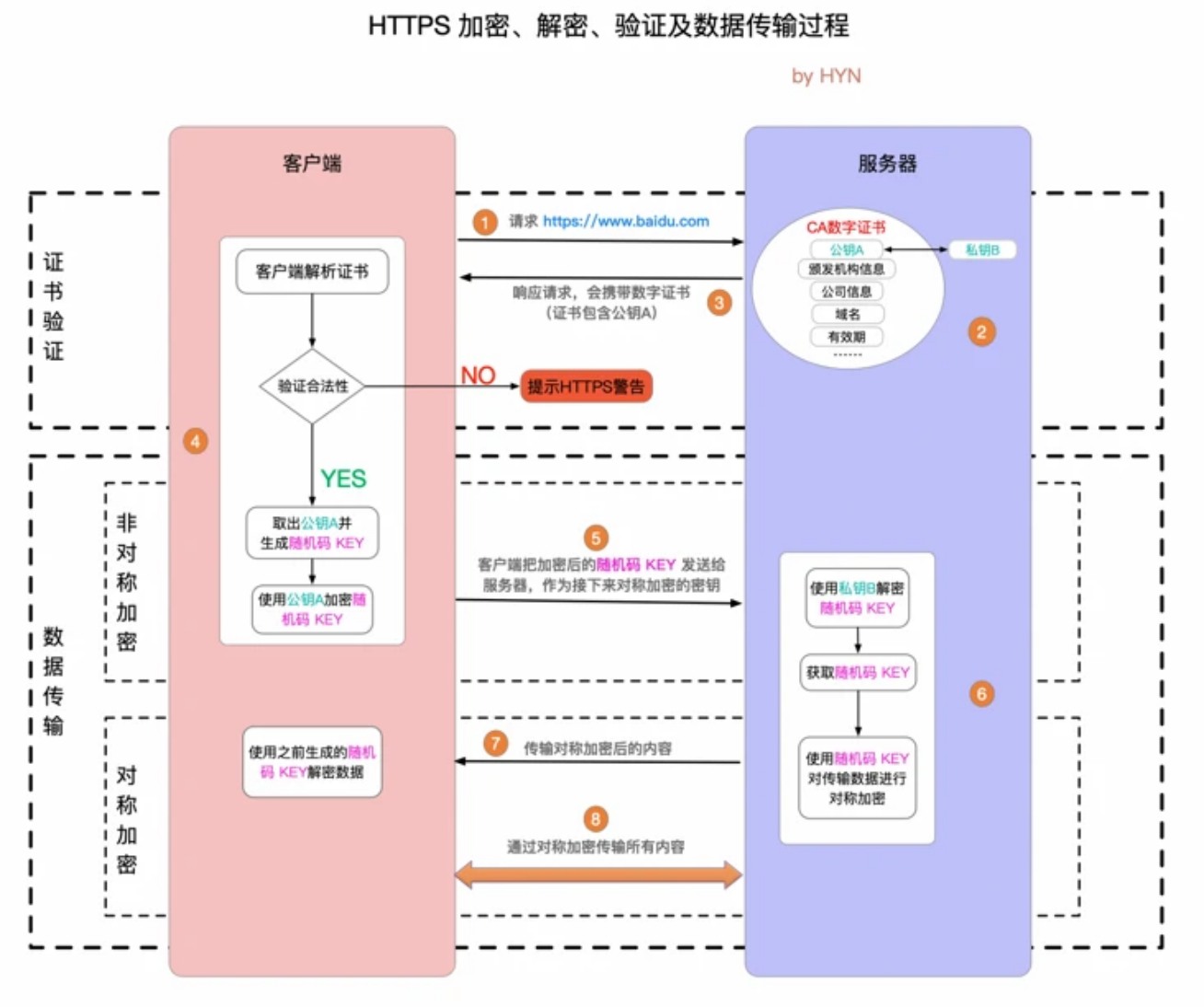
优点：

- 安全性：
  - 使用HTTPS协议可认证用户和服务器，确保数据发送到正确的客户机和服务器；
  - HTTPS协议是由SSL+HTTP协议构建的可进行加密传输、身份认证的网络协议，要比http协议安全，可防止数据在传输过程中不被窃取、改变，确保数据的完整性。
  - HTTPS是现行架构下最安全的解决方案，虽然不是绝对安全，但它大幅增加了中间人攻击的成本。
- SEO方面：谷歌曾在2014年8月份调整搜索引擎算法，并称“比起同等HTTP网站，采用HTTPS加密的网站在搜索结果中的排名将会更高”。

缺点：

- 在相同网络环境中，HTTPS 相比 HTTP 无论是响应时间还是耗电量都有大幅度上升。
- HTTPS 的安全是有范围的，在黑客攻击、服务器劫持等情况下几乎起不到作用。
- 在现有的证书机制下，中间人攻击依然有可能发生。
- HTTPS 需要更多的服务器资源，也会导致成本的升高。

28. 讲一讲HTTPS 的原理?



> 图片来源：https://segmentfault.com/a/1190000021494676

加密流程按图中的序号分为：

1. 客户端请求 HTTPS 网址，然后连接到 server 的 443 端口 (HTTPS 默认端口，类似于 HTTP 的80端口)。

2. 采用 HTTPS 协议的服务器必须要有一套数字 CA (Certification Authority)证书。颁发证书的同时会产生一个私钥和公钥。私钥由服务端自己保存，不可泄漏。公钥则是附带在证书的信息中，可以公开的。证书本身也附带一个证书电子签名，这个签名用来验证证书的完整性和真实性，可以防止证书被篡改。
3. 服务器响应客户端请求，将证书传递给客户端，证书包含公钥和大量其他信息，比如证书颁发机构信息，公司信息和证书有效期等。
4. 客户端解析证书并对其进行验证。如果证书不是可信机构颁布，或者证书中的域名与实际域名不一致，或者证书已经过期，就会向访问者显示一个警告，由其选择是否还要继续通信。

   如果证书没有问题，客户端就会从服务器证书中取出服务器的公钥A。然后客户端还会生成一个随机码 KEY，并使用公钥A将其加密。
5. 客户端把加密后的随机码 KEY 发送给服务器，作为后面对称加密的密钥。
6. 服务器在收到随机码 KEY 之后会使用私钥B将其解密。经过以上这些步骤，客户端和服务器终于建立了安全连接，完美解决了对称加密的密钥泄露问题，接下来就可以用对称加密愉快地进行通信了。
7. 服务器使用密钥 (随机码 KEY)对数据进行对称加密并发送给客户端，客户端使用相同的密钥 (随机码 KEY)解密数据。
8. 双方使用对称加密愉快地传输所有数据。

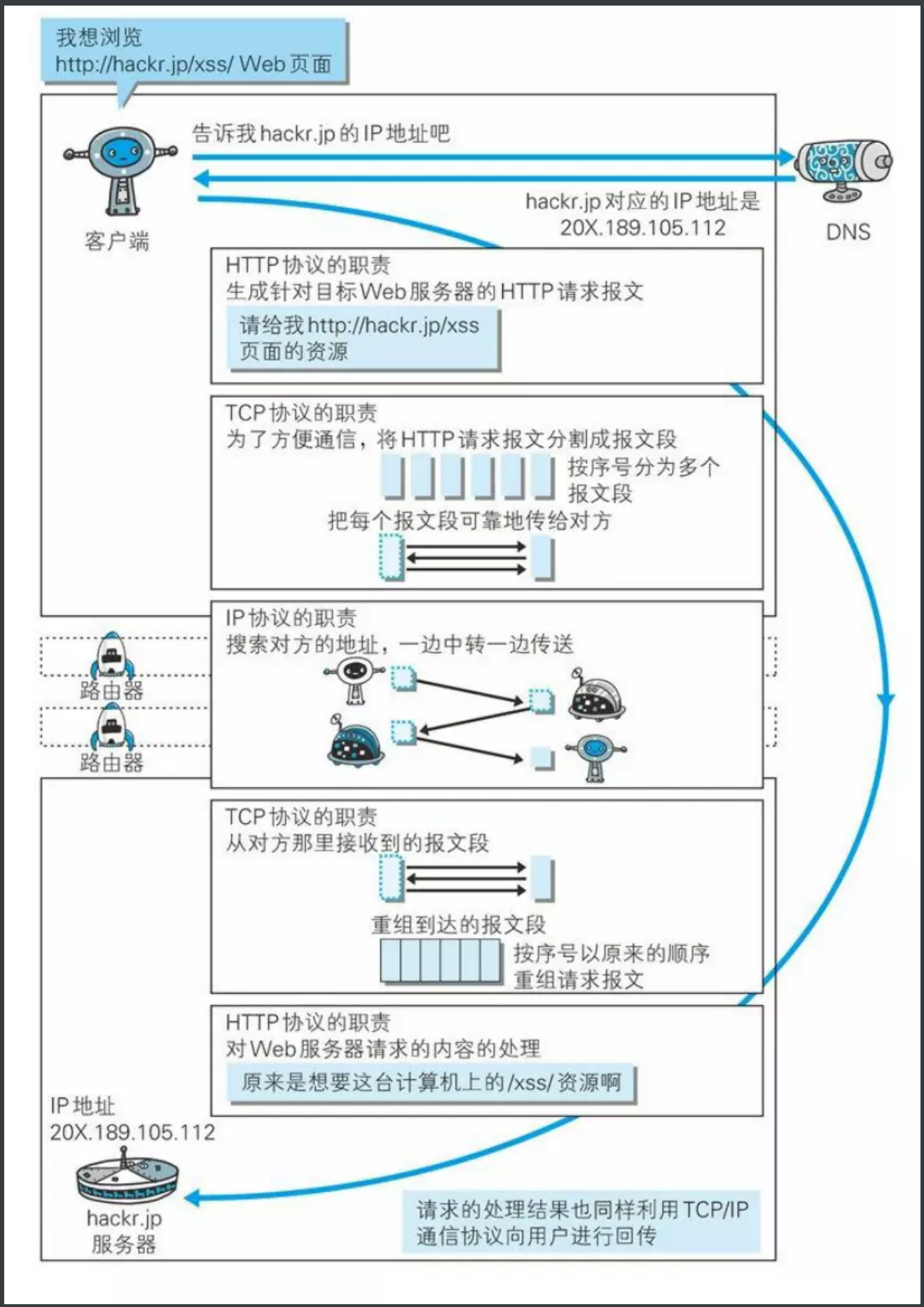
29. 在浏览器中输入www.baidu.com后执行的全部过程?

1. 域名解析（域名 www.baidu.com 变为 ip 地址）。

   浏览器搜索自己的DNS缓存（维护一张域名与IP的对应表）；若没有，则搜索**操作系统的DNS缓存**（维护一张域名与IP的对应表）；若没有，则搜索操作系统的**hosts文件**（维护一张域名与IP的对应表）。

   若都没有，则找 tcp/ip 参数中设置的首选 dns 服务器，即**本地 dns 服务器**（递归查询），**本地域名服务器查询自己的dns缓存**，如果没有，则进行迭代查询。将本地dns服务器将IP返回给操作系统，同时缓存IP。
2. 发起 tcp 的三次握手，建立 tcp 连接。浏览器会以一个随机端口（1024-65535）向服务端的 web 程序 **80** 端口发起 tcp 的连接。
3. 建立 tcp 连接后发起 http 请求。
4. 服务器响应 http 请求，客户端得到 html 代码。服务器 web 应用程序收到 http 请求后，就开始处理请求，处理之后就返回给浏览器 html 文件。
5. 浏览器解析 html 代码，并请求 html 中的资源。
6. 浏览器对页面进行渲染，并呈现给用户。

附一张形象的图片：



30. 什么是 Cookie 和 Session？

什么是 Cookie

HTTP Cookie（也叫 Web Cookie或浏览器 Cookie）是服务器发送到用户浏览器并保存在本地的一小块数据，它会在浏览器下次向同一服务器再发起请求时被携带并发送到服务器上。通常，它用于告知服务端两个请求是否来自同一浏览器，如保持用户的登录状态。Cookie 使基于无状态的 HTTP 协议记录稳定的状态信息成为了可能。

Cookie 主要用于以下三个方面：

- 会话状态管理（如用户登录状态、购物车、游戏分数或其它需要记录的信息）
- 个性化设置（如用户自定义设置、主题等）
- 浏览器行为跟踪（如跟踪分析用户行为等）

什么是 Session

Session 代表着服务器和客户端一次会话的过程。Session 对象存储特定用户会话所需的属性及配置信息。这样，当用户在应用程序的 Web 页之间跳转时，存储在 Session 对象中的变量将不会丢失，而是在整个用户会话中一直存在下去。当客户端关闭会话，或者 Session 超时失效时会话结束。

31. Cookie 和 Session 是如何配合的呢?

用户第一次请求服务器的时候，服务器根据用户提交的相关信息，创建对应的 Session ，请求返回时将此 Session 的唯一标识信息 SessionID 返回给浏览器，浏览器接收到服务器返回的 SessionID 信息后，会将此信息存入到 Cookie 中，同时 Cookie 记录此 SessionID 属于哪个域名。

当用户第二次访问服务器的时候，请求会自动判断此域名下是否存在 Cookie 信息，如果存在自动将 Cookie 信息也发送给服务端，服务端会从 Cookie 中获取 SessionID，再根据 SessionID 查找对应的 Session 信息，如果没有找到说明用户没有登录或者登录失效，如果找到 Session 证明用户已经登录可执行后面操作。

根据以上流程可知，SessionID 是连接 Cookie 和 Session 的一道桥梁，大部分系统也是根据此原理来验证用户登录状态。

32. Cookie和Session的区别?

- 作用范围不同，Cookie 保存在客户端（浏览器），Session 保存在服务器端。
- 存取方式的不同，Cookie 只能保存 ASCII，Session 可以存任意数据类型，一般情况下我们可以在 Session 中保持一些常用变量信息，比如说 UserId 等。

- 有效期不同，Cookie 可设置为长时间保持，比如我们经常使用的默认登录功能，Session 一般失效时间较短，客户端关闭或者 Session 超时都会失效。
- 隐私策略不同，Cookie 存储在客户端，比较容易遭到不法获取，早期有人将用户的登录名和密码存储在 Cookie 中导致信息被窃取；Session 存储在服务端，安全性相对 Cookie 要好一些。
- 存储大小不同， 单个 Cookie 保存的数据不能超过 4K，Session 可存储数据远高于 Cookie。

33. 如何考虑分布式 Session 问题?

在互联网公司为了可以支撑更大的流量，后端往往需要多台服务器共同来支撑前端用户请求，那如果用户在 A 服务器登录了，第二次请求跑到服务 B 就会出现登录失效问题。

分布式 Session 一般会有以下几种解决方案：

- **客户端存储**：直接将信息存储在cookie中，cookie是存储在客户端上的一小段数据，客户端通过http协议和服务器进行cookie交互，通常用来存储一些不敏感信息

- **Nginx ip_hash 策略**：服务端使用 Nginx 代理，每个请求按访问 IP 的 hash 分配，这样来自同一 IP 固定访问一个后台服务器，避免了在服务器 A 创建 Session，第二次分发到服务器 B 的现象。
- **Session 复制**：任何一个服务器上的 Session 发生改变（增删改），该节点会把这个 Session 的所有内容序列化，然后广播给所有其它节点。
- **共享 Session**：服务端无状态话，将用户的 Session 等信息使用缓存中间件（如Redis）来统一管理，保障分发到每一个服务器的响应结果都一致。

建议采用共享 Session的方案。

34. 什么是DDos攻击?

DDos全称Distributed Denial of Service，分布式拒绝服务攻击。最基本的DOS攻击过程如下：

1. 客户端向服务端发送请求链接数据包。
2. 服务端向客户端发送确认数据包。
3. 客户端不向服务端发送确认数据包，服务器一直等待来自客户端的确认

DDoS则是采用分布式的方法，通过在网络上占领多台“肉鸡”，用多台计算机发起攻击。

DOS攻击现在基本没啥作用了，因为服务器的性能都很好，而且是多台服务器共同作用，1V1的模式黑客无法占上风。对于DDOS攻击，预防方法有：

- **减少SYN timeout时间**。在握手的第三步，服务器会等待30秒-120秒的时间，减少这个等待时间就能释放更多的资源。
- **限制同时打开的SYN半连接数目。**

35. 什么是XSS攻击?

XSS也称 cross-site scripting，**跨站脚本**。这种攻击是**由于服务器将攻击者存储的数据原原本本地显示给其他用户所致的**。比如一个存在XSS漏洞的论坛，用户发帖时就可以引入**带有 < script > 标签的代码**，导致恶意代码的执行。

预防措施有：

- 前端：过滤。
- 后端：转义，比如go自带的处理器就具有转义功能。

36. SQL注入是什么，如何避免SQL注入?

SQL 注入就是在用户输入的字符串中加入 SQL 语句，如果在设计不良的程序中忽略了检查，那么这些注入进去的 SQL 语句就会被数据库服务器误认为是正常的 SQL 语句而运行，攻击者就可以执行计划外的命令或访问未被授权的数据。

SQL注入的原理主要有以下 4 点

- 恶意拼接查询
- 利用注释执行非法命令
- 传入非法参数
- 添加额外条件

避免SQL注入的一些方法：

- 限制数据库权限，给用户提供仅仅能够满足其工作的最低权限。
- 对进入数据库的特殊字符（’”\尖括号&*;等）转义处理。
- 提供参数化查询接口，不要直接使用原生SQL。

37. 负载均衡算法有哪些?

多台服务器以对称的方式组成一个服务器集合，每台服务器都具有等价的地位，能互相分担负载。

- 轮询法：将请求按照顺序轮流的分配到服务器上。大锅饭，不能发挥某些高性能服务器的优势。
- 随机法：随机获取一台，和轮询类似。
- 哈希法：通过ip地址哈希化来确定要选择的服务器编号。好处是,每次客户端访问的服务器都是同一个服务器，能很好地利用session或者cookie。
- 加权轮询：根据服务器性能不同加权。

十一、操作系统

1. 进程和线程的区别?

- 调度：进程是资源管理的基本单位，线程是程序执行的基本单位。
- 切换：线程上下文切换比进程上下文切换要快得多。
- 拥有资源： 进程是拥有资源的一个独立单位，线程不拥有系统资源，但是可以访问隶属于进程的资源。
- 系统开销： 创建或撤销进程时，系统都要为之分配或回收系统资源，如内存空间，I/O设备等，OS所付出的开销显著大于在创建或撤销线程时的开销，进程切换的开销也远大于线程切换的开销。

2. 协程与线程的区别?

- 线程和进程都是同步机制，而协程是异步机制。
- 线程是抢占式，而协程是非抢占式的。需要用户释放使用权切换到其他协程，因此同一时间其实只有一个协程拥有运行权，相当于单线程的能力。
- 一个线程可以有多个协程，一个进程也可以有多个协程。
- 协程不被操作系统内核管理，而完全是由程序控制。线程是被分割的CPU资源，协程是组织好的代码流程，线程是协程的资源。但协程不会直接使用线程，协程直接利用的是执行器关联任意线程或线程池。
- 协程能保留上一次调用时的状态。

3. 并发和并行有什么区别?

并发就是在一段时间内，多个任务都会被处理；但在某一时刻，只有一个任务在执行。单核处理器可以做到并发。比如有两个进程 `A` 和 `B`， `A` 运行一个时间片之后，切换到 `B`， `B` 运行一个时间片之后又切换到 `A`。因为切换速度足够快，所以宏观上表现为在一段时间内能同时运行多个程序。

并行就是在同一时刻，有多个任务在执行。这个需要多核处理器才能完成，在微观上就能同时执行多条指令，不同的程序被放到不同的处理器上运行，这个是物理上的多个进程同时进行。

4. 进程与线程的切换流程?

进程切换分两步：

1、切换**页表**以使用新的地址空间，一旦去切换上下文，处理器中所有已经缓存的内存地址一瞬间都作废了。

2、切换内核栈和硬件上下文。

对于linux来说，线程和进程的最大区别就在于地址空间，对于线程切换，第1步是不需要做的，第2步是进程和线程切换都要做的。

因为每个进程都有自己的虚拟地址空间，而线程是共享所在进程的虚拟地址空间的，因此同一个进程中的线程进行线程切换时不涉及虚拟地址空间的转换。

5. 为什么虚拟地址空间切换会比较耗时?

进程都有自己的虚拟地址空间，把虚拟地址转换为物理地址需要查找页表，页表查找是一个很慢的过程，因此通常使用Cache来缓存常用的地址映射，这样可以加速页表查找，这个Cache就是TLB（translation Lookaside Buffer，TLB本质上就是一个Cache，是用来加速页表查找的）。

由于每个进程都有自己的虚拟地址空间，那么显然每个进程都有自己的页表，那么**当进程切换后页表也要进行切换，页表切换后TLB就失效了**，Cache失效导致命中率降低，那么虚拟地址转换为物理地址就会变慢，表现出来的就是程序运行会变慢，而线程切换则不会导致TLB失效，因为线程无需切换地址空间，因此我们通常说线程切换要比较进程切换块，原因就在这里。

6. 进程间通信方式有哪些?

- 管道：管道这种通讯方式有两种限制，一是半双工的通信，数据只能单向流动，二是只能在具有亲缘关系的进程间使用。进程的亲缘关系通常是指父子进程关系。

  管道可以分为两类：匿名管道和命名管道。匿名管道是单向的，只能在有亲缘关系的进程间通信；命名管道以磁盘文件的方式存在，可以实现本机任意两个进程通信。

- 信号 ： 信号是一种比较复杂的通信方式，信号可以在任何时候发给某一进程，而无需知道该进程的状态。

  > **Linux系统中常用信号**：
  > （1）**SIGHUP**：用户从终端注销，所有已启动进程都将收到该进程。系统缺省状态下对该信号的处理是终止进程。
  >
  > （2）**SIGINT**：程序终止信号。程序运行过程中，按 `Ctrl+C` 键将产生该信号。
  >
  > （3）**SIGQUIT**：程序退出信号。程序运行过程中，按 `Ctrl+\\` 键将产生该信号。
  >
  > （4）**SIGBUS和SIGSEGV**：进程访问非法地址。
  >
  > （5）**SIGFPE**：运算中出现致命错误，如除零操作、数据溢出等。
  >
  > （6）**SIGKILL**：用户终止进程执行信号。shell下执行 `kill -9` 发送该信号。
  >
  > （7）**SIGTERM**：结束进程信号。shell下执行 `kill 进程pid` 发送该信号。
  >
  > （8）**SIGALRM**：定时器信号。
  >
  > （9）**SIGCLD**：子进程退出信号。如果其父进程没有忽略该信号也没有处理该信号，则子进程退出后将形成僵尸进程。

- 信号量：信号量是一个**计数器**，可以用来控制多个进程对共享资源的访问。它常作为一种**锁机制**，防止某进程正在访问共享资源时，其他进程也访问该资源。因此，主要作为进程间以及同一进程内不同线程之间的同步手段。
- 消息队列：消息队列是消息的链接表，包括Posix消息队列和System V消息队列。有足够权限的进程可以向队列中添加消息，被赋予读权限的进程则可以读走队列中的消息。消息队列克服了信号承载信息量少，管道只能承载无格式字节流以及缓冲区大小受限等缺点。
- 共享内存：共享内存就是映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但多个进程都可以访问。共享内存是最快的 IPC 方式，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，如信号量，配合使用，来实现进程间的同步和通信。
- Socket：与其他通信机制不同的是，它可用于不同机器间的进程通信。

优缺点：

- 管道：速度慢，容量有限；
- Socket：任何进程间都能通讯，但速度慢；
- 消息队列：容量受到系统限制，且要注意第一次读的时候，要考虑上一次没有读完数据的问题；
- 信号量：不能传递复杂消息，只能用来同步；
- 共享内存区：能够很容易控制容量，速度快，但要保持同步，比如一个进程在写的时候，另一个进程要注意读写的问题，相当于线程中的线程安全，当然，共享内存区同样可以用作线程间通讯，不过没这个必要，线程间本来就已经共享了同一进程内的一块内存。

7. 进程间同步的方式有哪些?

1、临界区：通过对多线程的串行化来访问公共资源或一段代码，速度快，适合控制数据访问。

优点：保证在某一时刻只有一个线程能访问数据的简便办法。

缺点：虽然临界区同步速度很快，但却只能用来同步本进程内的线程，而不可用来同步多个进程中的线程。

2、互斥量：为协调共同对一个共享资源的单独访问而设计的。互斥量跟临界区很相似，比临界区复杂，互斥对象只有一个，只有拥有互斥对象的线程才具有访问资源的权限。

优点：使用互斥不仅仅能够在同一应用程序不同线程中实现资源的安全共享，而且可以在不同应用程序的线程之间实现对资源的安全共享。

缺点：

- 互斥量是可以命名的，也就是说它可以跨越进程使用，所以创建互斥量需要的资源更多，所以如果只为了在进程内部是用的话使用临界区会带来速度上的优势并能够减少资源占用量。
- 通过互斥量可以指定资源被独占的方式使用，但如果有下面一种情况通过互斥量就无法处理，比如现在一位用户购买了一份三个并发访问许可的数据库系统，可以根据用户购买的访问许可数量来决定有多少个线程/进程能同时进行数据库操作，这时候如果利用互斥量就没有办法完成这个要求，信号量对象可以说是一种资源计数器。

3、信号量：为控制一个具有有限数量用户资源而设计。它允许多个线程在同一时刻访问同一资源，但是需要限制在同一时刻访问此资源的最大线程数目。互斥量是信号量的一种特殊情况，当信号量的最大资源数=1就是互斥量了。

优点：适用于对Socket（套接字）程序中线程的同步。

缺点:

- 信号量机制必须有公共内存，不能用于分布式操作系统，这是它最大的弱点；
- 信号量机制功能强大，但使用时对信号量的操作分散， 而且难以控制，读写和维护都很困难，加重了程序员的编码负担；
- 核心操作P-V分散在各用户程序的代码中，不易控制和管理，一旦错误，后果严重，且不易发现和纠正。

4、事件： 用来通知线程有一些事件已发生，从而启动后继任务的开始。

优点：事件对象通过通知操作的方式来保持线程的同步，并且可以实现不同进程中的线程同步操作。

8. 线程同步的方式有哪些?

1、临界区：当多个线程访问一个独占性共享资源时，可以使用临界区对象。拥有临界区的线程可以访问被保护起来的资源或代码段，其他线程若想访问，则被挂起，直到拥有临界区的线程放弃临界区为止，以此达到用原子方式操 作共享资源的目的。

2、事件：事件机制，则允许一个线程在处理完一个任务后，主动唤醒另外一个线程执行任务。

3、互斥量：互斥对象和临界区对象非常相似，只是其允许在进程间使用，而临界区只限制与同一进程的各个线程之间使用，但是更节省资源，更有效率。

4、信号量：当需要一个计数器来限制可以使用某共享资源的线程数目时，可以使用“信号量”对象。

区别:

- 互斥量与临界区的作用非常相似，但互斥量是可以命名的，也就是说互斥量可以跨越进程使用，但创建互斥量需要的资源更多，所以如果只为了在进程内部是用的话使用临界区会带来速度上的优势并能够减少资源占用量 。因为互斥量是跨进程的互斥量一旦被创建，就可以通过名字打开它。
- 互斥量，信号量，事件都可以被跨越进程使用来进行同步数据操作。

9. 线程的分类?

从线程的运行空间来说，分为用户级线程（user-level thread, ULT）和内核级线程（kernel-level, KLT）

内核级线程：这类线程依赖于内核，又称为内核支持的线程或轻量级进程。无论是在用户程序中的线程还是系统进程中的线程，它们的创建、撤销和切换都由内核实现。比如英特尔i5-8250U是4核8线程，这里的线程就是内核级线程

用户级线程：它仅存在于用户级中，这种线程是**不依赖于操作系统核心**的。应用进程利用**线程库来完成其创建和管理**，速度比较快，**操作系统内核无法感知用户级线程的存在**。

10. 什么是临界区，如何解决冲突?

每个进程中访问临界资源的那段程序称为临界区，**一次仅允许一个进程使用的资源称为临界资源。**

解决冲突的办法：

- 如果有若干进程要求进入空闲的临界区，**一次仅允许一个进程进入**，如已有进程进入自己的临界区，则其它所有试图进入临界区的进程必须等待；
- 进入临界区的进程要在**有限时间内退出**。
- 如果进程不能进入自己的临界区，则应**让出CPU**，避免进程出现“忙等”现象。

11. 什么是死锁？死锁产生的条件?

什么是死锁：

在两个或者多个并发进程中，如果每个进程持有某种资源而又等待其它进程释放它或它们现在保持着的资源，在未改变这种状态之前都不能向前推进，称这一组进程产生了死锁。通俗的讲就是两个或多个进程无限期的阻塞、相互等待的一种状态。

死锁产生的四个必要条件：（有一个条件不成立，则不会产生死锁）

- 互斥条件：一个资源一次只能被一个进程使用
- 请求与保持条件：一个进程因请求资源而阻塞时，对已获得资源保持不放
- 不剥夺条件：进程获得的资源，在未完全使用完之前，不能强行剥夺
- 循环等待条件：若干进程之间形成一种头尾相接的环形等待资源关系

如何处理死锁问题

常用的处理死锁的方法有：死锁预防、死锁避免、死锁检测、死锁解除、鸵鸟策略。

（1）死锁的预防：基本思想就是确保死锁发生的四个必要条件中至少有一个不成立：

> - ① 破除资源互斥条件
> - ② 破除“请求与保持”条件：实行资源预分配策略，进程在运行之前，必须一次性获取所有的资源。缺点：在很多情况下，无法预知进程执行前所需的全部资源，因为进程是动态执行的，同时也会降低资源利用率，导致降低了进程的并发性。
> - ③ 破除“不可剥夺”条件：允许进程强行从占有者那里夺取某些资源。当一个已经保持了某些不可被抢占资源的进程，提出新的资源请求而不能得到满足时，它必须释放已经保持的所有资源，待以后需要时再重新申请。这意味着进程已经占有的资源会被暂时被释放，或者说被抢占了。
> - ④ 破除“循环等待”条件：实行资源有序分配策略，对所有资源排序编号，按照顺序获取资源，将紧缺的，稀少的采用较大的编号，在申请资源时必须按照编号的顺序进行，一个进程只有获得较小编号的进程才能申请较大编号的进程。

（2）死锁避免：

死锁预防通过约束资源请求，防止4个必要条件中至少一个的发生，可以通过直接或间接预防方法，但是都会导致低效的资源使用和低效的进程执行。而死锁避免则允许前三个必要条件，但是通过动态地检测资源分配状态，以确保循环等待条件不成立，从而确保系统处于安全状态。所谓安全状态是指：如果系统能按某个顺序为每个进程分配资源（不超过其最大值），那么系统状态是安全的，换句话说就是，如果存在一个安全序列，那么系统处于安全状态。银行家算法是经典的死锁避免的算法。

（3）死锁检测：

死锁预防策略是非常保守的，他们通过限制访问资源和在进程上强加约束来解决死锁的问题。死锁检测则是完全相反，它不限制资源访问或约束进程行为，只要有可能，被请求的资源就被授权给进程。但是操作系统会周期性地执行一个算法检测前面的循环等待的条件。死锁检测算法是通过资源分配图来检测是否存在环来实现，从一个节点出发进行深度优先搜索，对访问过的节点进行标记，如果访问了已经标记的节点，就表示有存在环，也就是检测到死锁的发生。

> - （1）如果进程-资源分配图中无环路，此时系统没有死锁。
> - （2）如果进程-资源分配图中有环路，且每个资源类中只有一个资源，则系统发生死锁。
> - （3）如果进程-资源分配图中有环路，且所涉及的资源类有多个资源，则不一定会发生死锁。

（4）死锁解除：

死锁解除的常用方法就是终止进程和资源抢占，回滚。所谓进程终止就是简单地终止一个或多个进程以打破循环等待，包括两种方式：终止所有死锁进程和一次只终止一个进程直到取消死锁循环为止；所谓资源抢占就是从一个或者多个死锁进程那里抢占一个或多个资源。

（5）鸵鸟策略：

把头埋在沙子里，假装根本没发生问题。因为解决死锁问题的代价很高，因此鸵鸟策略这种不采取任何措施的方案会获得更高的性能。当发生死锁时不会对用户造成多大影响，或发生死锁的概率很低，可以采用鸵鸟策略。大多数操作系统，包括 Unix，Linux 和 Windows，处理死锁问题的办法仅仅是忽略它。

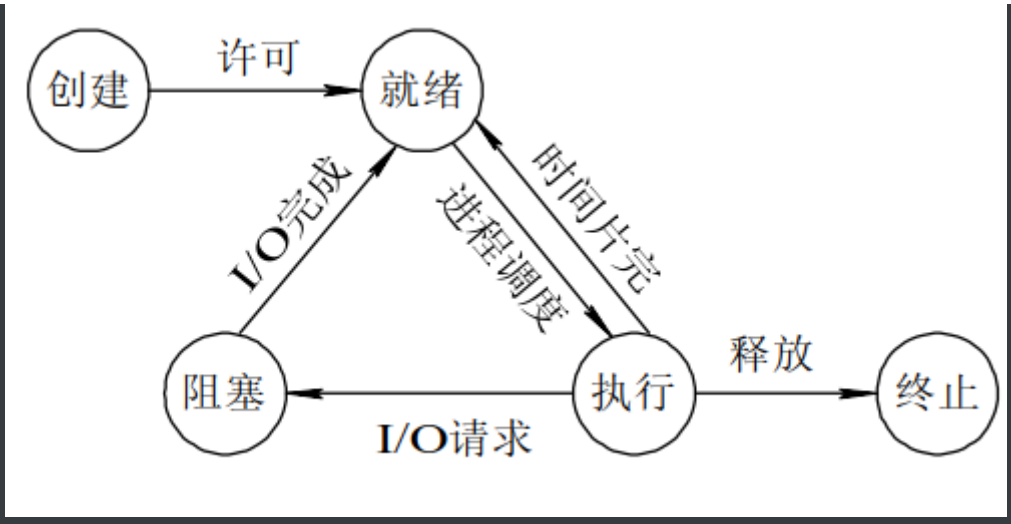
12. 进程调度策略有哪几种?

- **先来先服务**：非抢占式的调度算法，按照请求的顺序进行调度。有利于长作业，但不利于短作业，因为短作业必须一直等待前面的长作业执行完毕才能执行，而长作业又需要执行很长时间，造成了短作业等待时间过长。另外，对 `I/O` 密集型进程也不利，因为这种进程每次进行 `I/O` 操作之后又得重新排队。
- **短作业优先**：非抢占式的调度算法，按估计运行时间最短的顺序进行调度。长作业有可能会饿死，处于一直等待短作业执行完毕的状态。因为如果一直有短作业到来，那么长作业永远得不到调度。
- **最短剩余时间优先**：最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。 当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较。如果新的进程需要的时间更少，则挂起当前进程，运行新的进程。否则新的进程等待。
- **时间片轮转**：将所有就绪进程按 `FCFS` 的原则排成一个队列，每次调度时，把 `CPU` 时间分配给队首进程，该进程可以执行一个时间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 `CPU` 时间分配给队首的进程。

  时间片轮转算法的效率和时间片的大小有很大关系：因为进程切换都要保存进程的信息并且载入新进程的信息，如果时间片太小，会导致进程切换得太频繁，在进程切换上就会花过多时间。 而如果时间片过长，那么实时性就不能得到保证。
- **优先级调度**：为每个进程分配一个优先级，按优先级进行调度。为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级。

13. 进程有哪些状态?

进程一共有 5 种状态，分别是创建、就绪、运行（执行）、终止、阻塞。



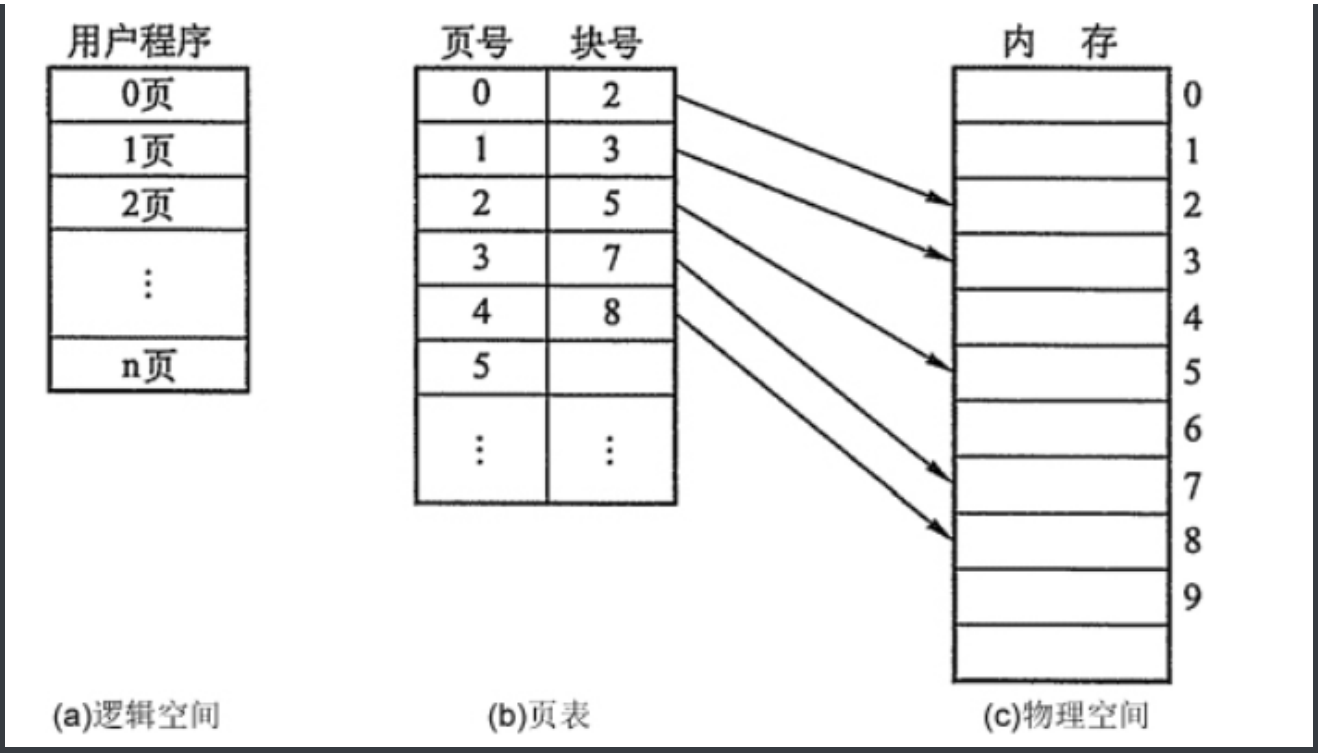
- 运行状态就是进程正在 CPU 上运行。在单处理机环境下，每一时刻最多只有一个进程处于运行状态。
- 就绪状态就是说进程已处于准备运行的状态，即进程获得了除 CPU 之外的一切所需资源，一旦得到 CPU 即可运行。
- 阻塞状态就是进程正在等待某一事件而暂停运行，比如等待某资源为可用或等待 I/O 完成。即使 CPU 空闲，该进程也不能运行。

运行态→阻塞态：往往是由于等待外设，等待主存等资源分配或等待人工干预而引起的。
阻塞态→就绪态：则是等待的条件已满足，只需分配到处理器后就能运行。
运行态→就绪态：不是由于自身原因，而是由外界原因使运行状态的进程让出处理器，这时候就变成就绪态。例如时间片用完，或有更高优先级的进程来抢占处理器等。
就绪态→运行态：系统按某种策略选中就绪队列中的一个进程占用处理器，此时就变成了运行态。

14. 什么是分页?

把内存空间划分为**大小相等且固定的块**，作为主存的基本单位。因为程序数据存储在不同的页面中，而页面又离散的分布在内存中，**因此需要一个页表来记录映射关系，以实现从页号到物理块号的映射。**

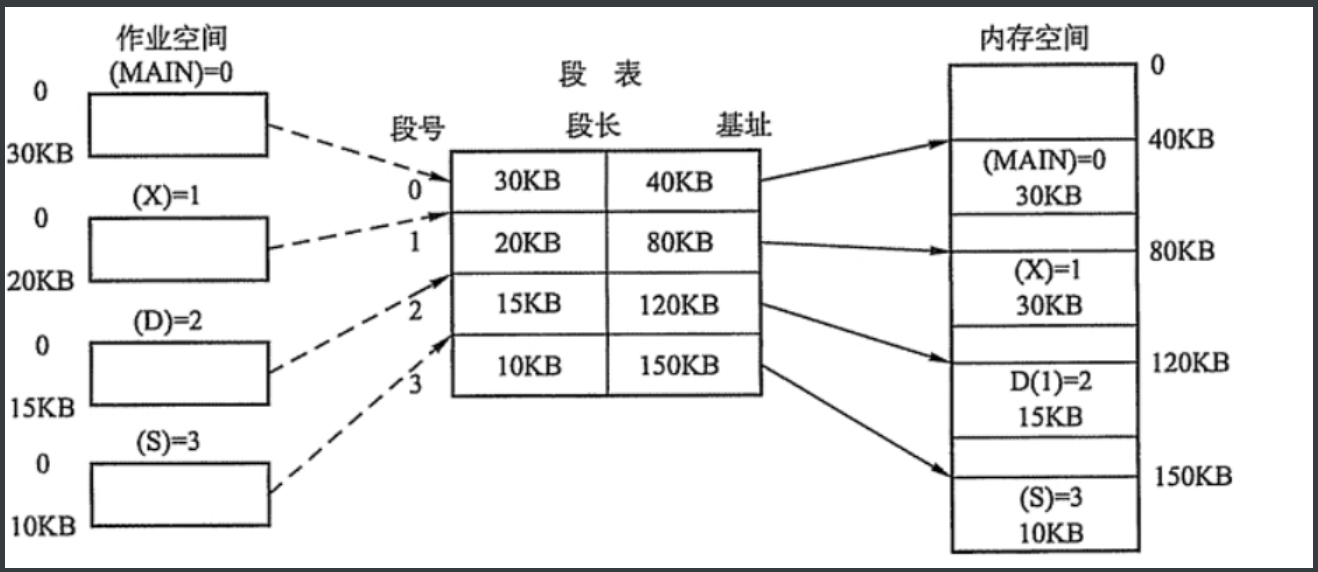
访问分页系统中内存数据需要**两次的内存访问** (一次是从内存中访问页表，从中找到指定的物理块号，加上页内偏移得到实际物理地址；第二次就是根据第一次得到的物理地址访问内存取出数据)。



15. 什么是分段?

分页是为了提高内存利用率，而分段是为了满足程序员在编写代码的时候的一些逻辑需求(比如数据共享，数据保护，动态链接等)。

分段内存管理当中，**地址是二维的，一维是段号，二维是段内地址；其中每个段的长度是不一样的，而且每个段内部都是从0开始编址的。**由于分段管理中，每个段内部是连续内存分配，但是段和段之间是离散分配的，因此也存在一个逻辑地址到物理地址的映射关系，相应的就是段表机制。



16. 分页和分段有什区别?

- 分页对程序员是透明的，但是分段需要程序员显式划分每个段。

- 分页的地址空间是一维地址空间，分段是二维的。
- 页的大小不可变，段的大小可以动态改变。
- 分页主要用于实现虚拟内存，从而获得更大的地址空间；分段主要是为了使程序和数据可以被划分为逻辑上独立的地址空间并且有助于共享和保护。

17. 什么是交换空间?

操作系统把物理内存(physical RAM)分成一块一块的小内存，每一块内存被称为**页(page)**。当内存资源不足时，**Linux把某些页的内容转移至硬盘上的一块空间上，以释放内存空间**。硬盘上的那块空间叫做**交换空间**(swap space),而这一过程被称为交换(swapping)。**物理内存和交换空间的总容量就是虚拟内存的可用容量。**

用途：

- 物理内存不足时一些不常用的页可以被交换出去，腾给系统。
- 程序启动时很多内存页被用来初始化，之后便不再需要，可以交换出去。

18. 物理地址、逻辑地址、有效地址、线性地址、虚拟地址的区别?

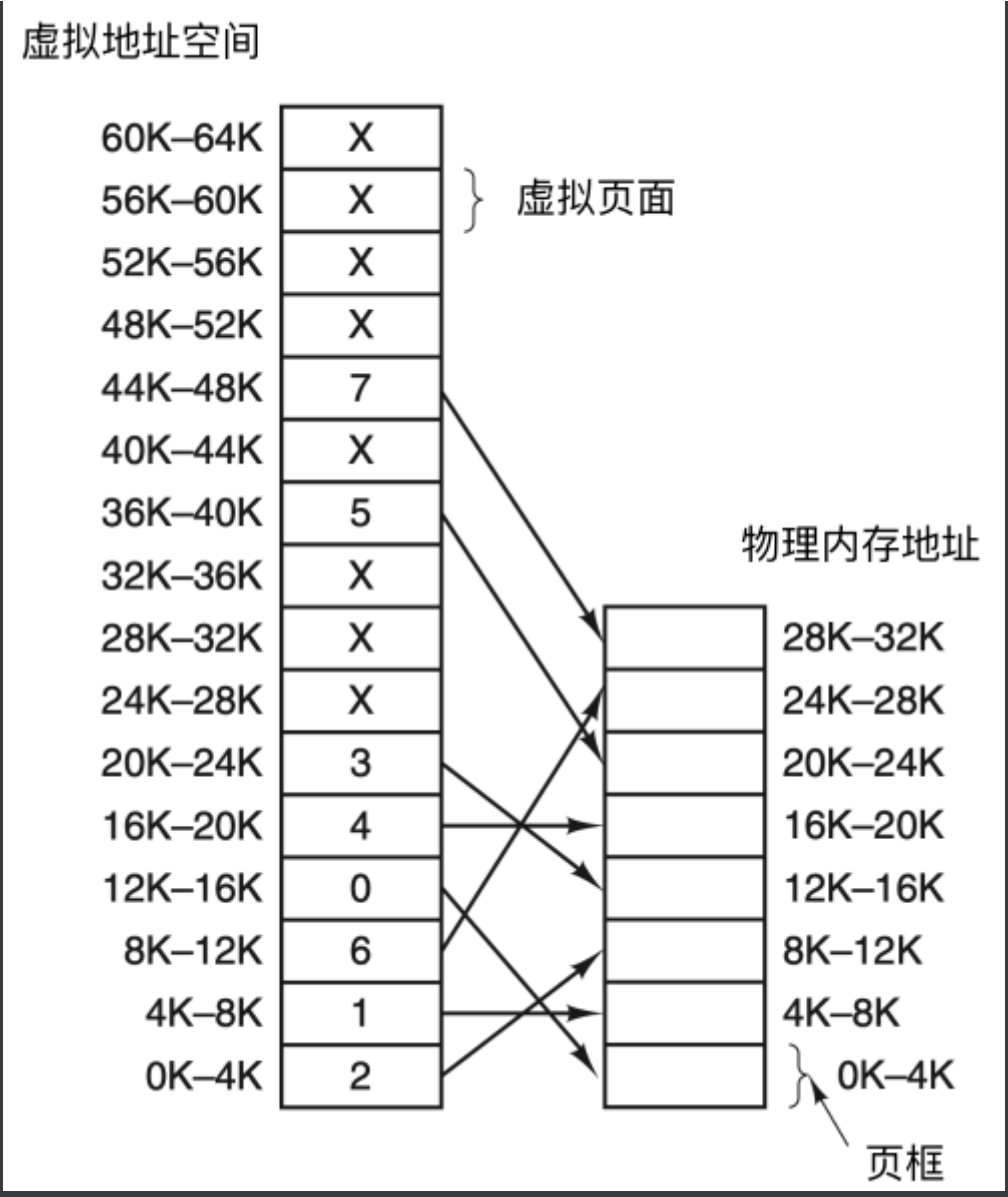
物理地址就是内存中真正的地址，它就相当于是你家的门牌号，你家就肯定有这个门牌号，具有唯一性。**不管哪种地址，最终都会映射为物理地址**。

在 `实模式` 下，段基址 + 段内偏移经过地址加法器的处理，经过地址总线传输，最终也会转换为 `物理地址` 。

但是在 `保护模式` 下，段基址 + 段内偏移被称为 `线性地址` ，不过此时的段基址不能称为真正的地址，而是会被称作为一个 `选择子` 的东西，选择子就是个索引，相当于数组的下标，通过这个索引能够在 GDT 中找到相应的段描述符，段描述符记录了**段的起始、段的大小**等信息，这样便得到了基地址。如果此时没有开启内存分页功能，那么这个线性地址可以直接当做物理地址来使用，直接访问内存。如果开启了分页功能，那么这个线性地址又多了一个名字，这个名字就是 `虚拟地址` 。

不论在实模式还是保护模式下，段内偏移地址都叫做 `有效地址` 。有效抵制也是逻辑地址。

线性地址可以看作是 `虚拟地址` ，虚拟地址不是真正的物理地址，但是虚拟地址会最终被映射为物理地址。下面是虚拟地址 -> 物理地址的映射。



19. 页面替换算法有哪些?

在程序运行过程中，如果要访问的页面不在内存中，就发生缺页中断从而将该页调入内存中。此时如果内存已无空闲空间，系统必须从内存中调出一个页面到磁盘对换区中来腾出空间。

算法	注释
最优算法	不可实现，但可以用作基准
NRU(最近未使用) 算法	和 LRU 算法很相似
FIFO(先进先出) 算法	有可能会抛弃重要的页面
第二次机会算法	比 FIFO 有较大的改善
时钟算法	实际使用
LRU(最近最少)算法	比较优秀，但是很难实现
NFU(最不经常食用)算法	和 LRU 很类似
老化算法	近似 LRU 的高效算法
工作集算法	实施起来开销很大
工作集时钟算法	比较有效的算法

- `最优算法` 在当前页面中置换最后要访问的页面。不幸的是，没有办法来判定哪个页面是最后一个要访问的， `因此实际上该算法不能使用` 。然而，它可以作为衡量其他算法的标准。
- `NRU` 算法根据 R 位和 M 位的状态将页面分为四类。从编号最小的类别中随机选择一个页面。NRU 算法易于实现，但是性能不是很好。存在更好的算法。
- `FIFO` 会跟踪页面加载进入内存中的顺序，并把页面放入一个链表中。有可能删除存在时间最长但是还在使用的页面，因此这个算法也不是一个很好的选择。
- `第二次机会` 算法是对 FIFO 的一个修改，它会在删除页面之前检查这个页面是否仍在使用。如果页面正在使用，就会进行保留。这个改进大大提高了性能。
- `时钟` 算法是第二次机会算法的另外一种实现形式，时钟算法和第二次算法的性能差不多，但是会花费更少的时间来执行算法。
- `LRU` 算法是一个非常优秀的算法，但是没有 `特殊的硬件(TLB)` 很难实现。如果没有硬件，就不能使用 LRU 算法。
- `NFU` 算法是一种近似于 LRU 的算法，它的性能不是非常好。
- `老化` 算法是一种更接近 LRU 算法的实现，并且可以更好的实现，因此是一个很好的选择
- 最后两种算法都使用了工作集算法。工作集算法提供了合理的性能开销，但是它的实现比较复杂。 `WSClock` 是另外一种变体，它不仅能够提供良好的性能，而且可以高效地实现。

最好的算法是老化算法和WSClock算法。他们分别是基于 LRU 和工作集算法。他们都具有良好的性能并且能够被有效的实现。还存在其他一些好的算法，但实际上这两个可能是最重要的。

20. 什么是缓冲区溢出？有什么危害？

缓冲区溢出是指当计算机向缓冲区填充数据时超出了缓冲区本身的容量，溢出的数据覆盖在合法数据上。

危害有以下两点：

- 程序崩溃，导致拒绝额服务
- 跳转并且执行一段恶意代码

造成缓冲区溢出的主要原因是程序中没有仔细检查用户输入。

21. 什么是虚拟内存?

虚拟内存就是说，让物理内存扩充成更大的逻辑内存，从而让程序获得更多的可用内存。虚拟内存使用部分加载的技术，让一个进程或者资源的某些页面加载进内存，从而能够加载更多的进程，甚至能加载比内存大的进程，这样看起来好像内存变大了，这部分内存其实包含了磁盘或者硬盘，并且就叫做虚拟内存。

22. 虚拟内存的实现方式有哪些?

虚拟内存中，允许将一个作业分多次调入内存。采用连续分配方式时，会使相当一部分内存空间都处于暂时或 永久 的空闲状态，造成内存资源的严重浪费，而且也无法从逻辑上扩大内存容量。因此，虚拟内存的实需要建立在离散分配的内存管理方式的基础上。虚拟内存的实现有以下三种方式：

- 请求分页存储管理。
- 请求分段存储管理。
- 请求段页式存储管理。

23. 讲一讲IO多路复用?

IO多路复用是指内核一旦发现进程指定的一个或者多个IO条件准备读取，它就通知该进程。
IO多路复用适用如下场合：

- 当客户处理多个描述字时（一般是交互式输入和网络套接口），必须使用I/O复用。
- 当一个客户同时处理多个套接口时，而这种情况是可能的，但很少出现。
- 如果一个TCP服务器既要处理监听套接口，又要处理已连接套接口，一般也要用到I/O复用。
- 如果一个服务器即要处理TCP，又要处理UDP，一般要使用I/O复用。
- 如果一个服务器要处理多个服务或多个协议，一般要使用I/O复用。

- 与多进程和多线程技术相比，I/O多路复用技术的最大优势是系统开销小，系统不必创建进程/线程，也不必维护这些进程/线程，从而大大减小了系统的开销。

24. 硬链接和软链接有什么区别?

- 硬链接就是在目录下创建一个条目，记录着文件名与 `inode` 编号，这个 `inode` 就是源文件的 `inode` 。删除任意一个条目，文件还是存在，只要引用数量不为 `0` 。但是硬链接有限制，它不能跨越文件系统，也不能对目录进行链接。
- 符号链接文件保存着源文件所在的绝对路径，在读取时会定位到源文件上，可以理解为 `Windows` 的快捷方式。当源文件被删除了，链接文件就打不开了。因为记录的是路径，所以可以为目录建立符号链接。

25. 中断的处理过程?

1. 保护现场：将当前执行程序的相关数据保存在寄存器中，然后入栈。
2. 开中断：以便执行中断时能响应较高级别的中断请求。
3. 中断处理
4. 关中断：保证恢复现场时不被新中断打扰
5. 恢复现场：从堆栈中按序取出程序数据，恢复中断前的执行状态。

26. 中断和轮询有什么区别?

- 轮询：CPU对**特定设备**轮流询问。中断：通过**特定事件**提醒CPU。
- 轮询：效率低等待时间长，CPU利用率不高。中断：容易遗漏问题，CPU利用率不高。

27. 什么是用户态和内核态?

用户态和系统态是操作系统的两种运行状态：

> - 内核态：内核态运行的程序可以访问计算机的任何数据和资源，不受限制，包括外围设备，比如网卡、硬盘等。处于内核态的 CPU 可以从一个程序切换到另外一个程序，并且占用 CPU 不会发生抢占情况。
> - 用户态：用户态运行的程序只能受限地访问内存，只能直接读取用户程序的数据，并且不允许访问外围设备，用户态下的 CPU 不允许独占，也就是说 CPU 能够被其他程序获取。

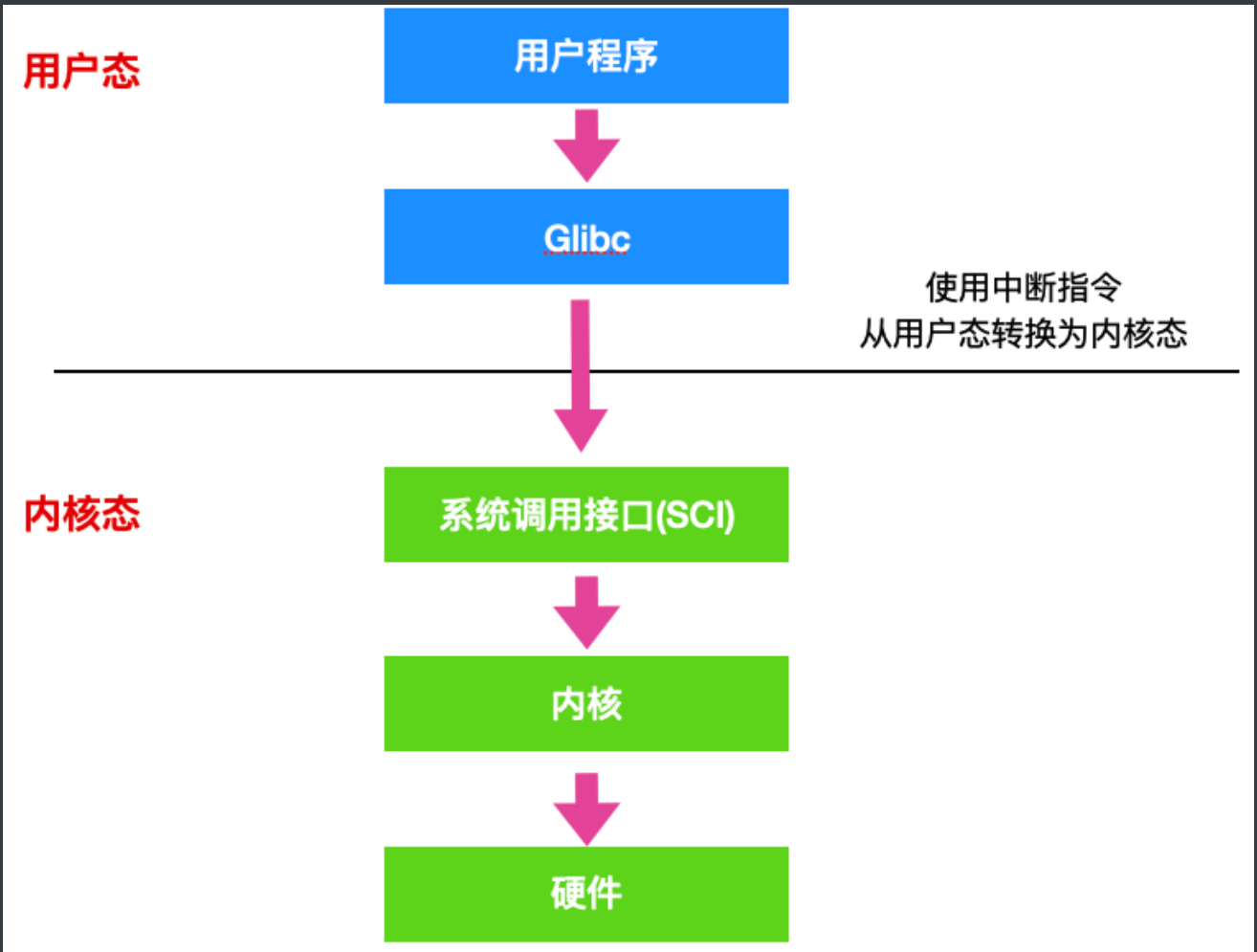
将操作系统的运行状态分为用户态和内核态，主要是为了对访问能力进行限制，防止随意进行一些比较危险的操作导致系统的崩溃，比如设置时钟、内存清理，这些都需要在内核态下完成 。

28. 用户态和内核态是如何切换的?

所有的用户进程都是运行在用户态的，但是我们上面也说了，用户程序的访问能力有限，一些比较重要的比如从硬盘读取数据，从键盘获取数据的操作则是内核态才能做的事情，而这些数据却又对用户程序来说非常重要。所以就涉及到两种模式下的转换，即**用户态 -> 内核态 -> 用户态**，而唯一能够做这些操作的只有 `系统调用` ，而能够执行系统调用的就只有 `操作系统` 。

一般用户态 -> 内核态的转换我们都称之为 trap 进内核，也被称之为 `陷阱指令(trap instruction)` 。

他们的工作流程如下：



- 首先用户程序会调用 `glibc` 库，glibc 是一个标准库，同时也是一套核心库，库中定义了很多关键 API。
- glibc 库知道针对不同体系结构调用 `系统调用` 的正确方法，它会根据体系结构应用程序的二进制接口设置用户进程传递的参数，来准备系统调用。
- 然后，glibc 库调用 `软件中断指令(SWI)` ，这个指令通过更新 `CPSR` 寄存器将模式改为超级用户模式，然后跳转到地址 `0x08` 处。

- 到目前为止，整个过程仍处于用户态下，在执行 SWI 指令后，允许进程执行内核代码，MMU 现在允许内核虚拟内存访问
- 从地址 0x08 开始，进程执行加载并跳转到中断处理程序，这个程序就是 ARM 中的 `vector_swi()`。
- 在 vector_swi() 处，从 SWI 指令中提取系统调用号 SCNO，然后使用 SCNO 作为系统调用表 `sys_call_table` 的索引，调转到系统调用函数。
- 执行系统调用完成后，将还原用户模式寄存器，然后再以用户模式执行。

29. Unix 常见的IO模型：

对于一次IO访问（以read举例），数据会先被拷贝到操作系统内核的缓冲区中，然后才会从操作系统内核的缓冲区拷贝到应用程序的地址空间。所以说，当一个read操作发生时，它会经历两个阶段：

> - 等待数据准备就绪 (Waiting for the data to be ready)
> - 将数据从内核拷贝到进程中 (Copying the data from the kernel to the process)

正式因为这两个阶段，linux系统产生了下面五种网络模式的方案：

> - 阻塞式IO模型(blocking IO model)
> - 非阻塞式IO模型(noblocking IO model)
> - IO复用式IO模型(IO multiplexing model)
> - 信号驱动式IO模型(signal-driven IO model)
> - 异步IO式IO模型(asynchronous IO model)

对于这几种 IO 模型的详细说明，可以参考这篇文章：https://juejin.cn/post/6942686874301857800#heading-13

其中，IO多路复用模型指的是：使用单个进程同时处理多个网络连接IO，他的原理就是 select、poll、epoll 不断轮询所负责的所有 socket，当某个socket有数据到达了，就通知用户进程。该模型的优势并不是对于单个连接能处理得更快，而是在于能处理更多的连接。

30. select、 poll 和 epoll 之间的区别?

（1）select：时间复杂度 O(n)

select 仅仅知道有 I/O 事件发生，但并不知道是哪几个流，所以只能无差别轮询所有流，找出能读出数据或者写入数据的流，并对其进行操作。所以 select 具有 O(n) 的无差别轮询复杂度，同时处理的流越多，无差别轮询时间就越长。

（2）poll：时间复杂度 O(n)

poll 本质上和 select 没有区别，它将用户传入的数组拷贝到内核空间，然后查询每个 fd 对应的设备状态， 但是它没有最大连接数的限制，原因是它是基于链表来存储的。

（3）epoll：时间复杂度 O(1)

epoll 可以理解为 event poll，不同于忙轮询和无差别轮询，epoll 会把哪个流发生了怎样的 I/O 事件通知我们。所以说 epoll 实际上是事件驱动（每个事件关联上 fd）的。

> select，poll，epoll 都是 IO 多路复用的机制。I/O 多路复用就是通过一种机制监视多个描述符，一旦某个描述符就绪（一般是读就绪或者写就绪），就通知程序进行相应的读写操作。但 select，poll，epoll 本质上都是同步 I/O，因为他们都需要在读写事件就绪后自己负责进行读写，也就是说这个读写过程是阻塞的，而异步 I/O 则无需自己负责进行读写，异步 I/O 的实现会负责把数据从内核拷贝到用户空间。

End

以上就是操作系统-基础的面试题了，操作系统其它部分面试题正在整理中，敬请期待！

若有更新，将第一时间同步公众号【程序员库森】，扫描下方左侧二维码关注！

此外，我还创建了求职交流群，群内分享内推、面试等资源，一起来抱团取暖啊。扫描下方右侧二维码，备注【加群】，拉你进群~



十二、 Dubbo

1.Dubbo是什么?

Dubbo是阿里巴巴开源的基于 Java 的高性能 RPC 分布式服务框架，现已成为 Apache 基金会孵化项目。

其核心部分包含：

- 集群容错：提供基于接口方法的透明远程过程调用，包括多协议支持，以及软负载均衡，失败容错，地址路由，动态配置等集群支持。
- 远程通讯：提供对多种基于长连接的NIO框架抽象封装，包括多种线程模型，序列化，以及“请求-响应”模式的信息交换方式。
- 自动发现：基于注册中心目录服务，使服务消费方能动态的查找服务提供方，使地址透明，使服务提供方可以平滑增加或减少机器。

2. Dubbo和 Spring Cloud 有什么区别?

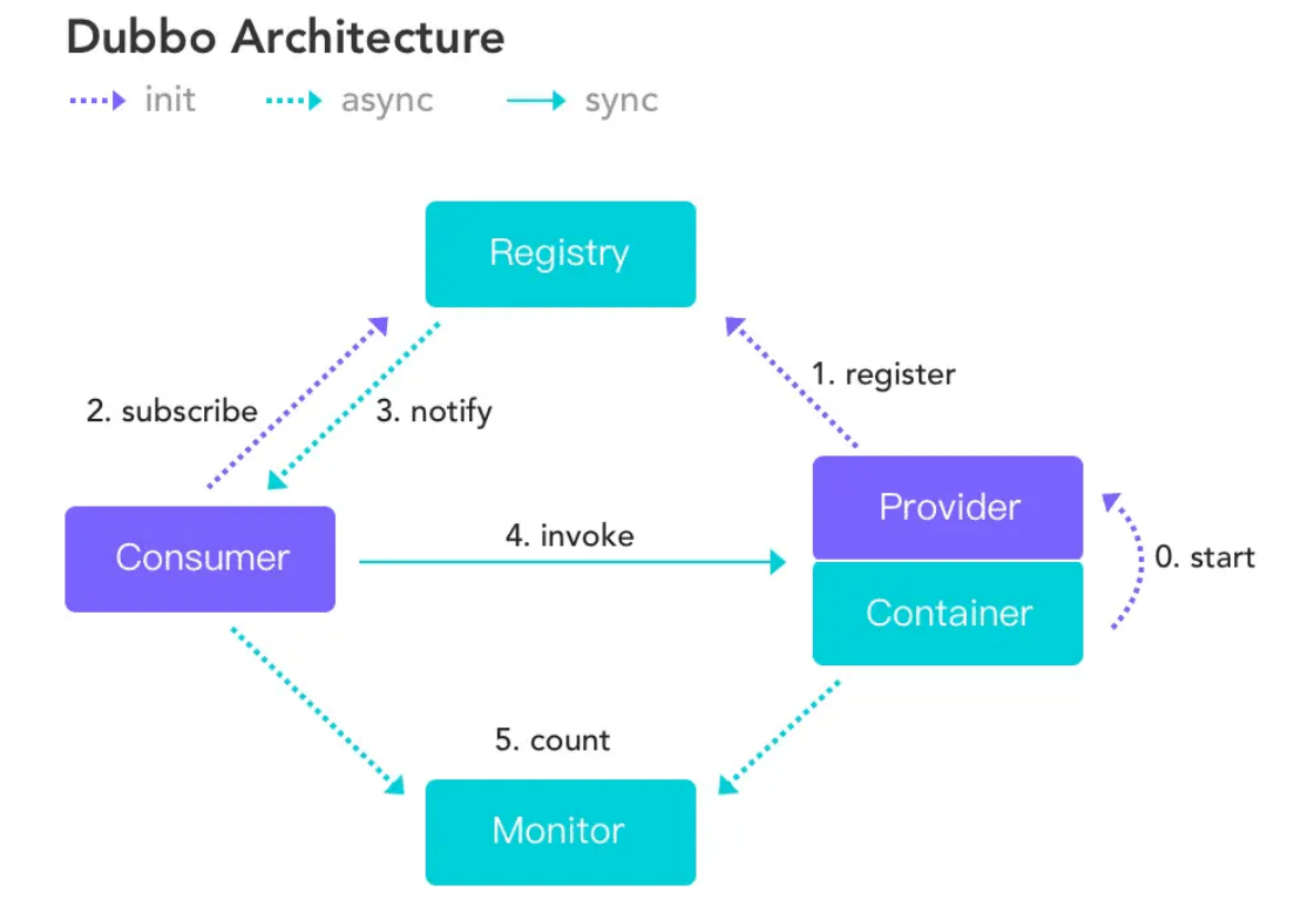
最大的区别：

- Dubbo底层是使用Netty这样的NIO框架，是基于TCP协议传输的，配合以Hession序列化完成RPC通信;
- 而SpringCloud是基于Http协议+rest接口调用远程过程的通信，相对来说，Http请求会有更大的报文，占的带宽也会更多。但是REST相比RPC更为灵活，服务提供方和调用方的依赖只依靠一纸契约，不存在代码级别的强依赖，这在强调快速演化的微服务环境下，显得更为合适，至于注重通信速度还是方便灵活性，具体情况具体考虑。

模块区别：

- Dubbo主要分为服务注册中心，服务提供者，服务消费者，还有管控中心;
- 相比起Dubbo简单的四个模块，SpringCloud则是一个完整的分布式一站式框架，他有着一样的服务注册中心，服务提供者，服务消费者，管控台，断路器，分布式配置服务，消息总线，以及服务追踪等;

3. Dubbo核心组件有哪些?



- Provider：暴露服务的服务提供方
- Consumer：调用远程服务消费方
- Registry：服务注册与发现注册中心
- Monitor：监控中心和访问调用统计
- Container：服务运行容器

4. Dubbo都支持什么协议，推荐用哪种?

1、 Dubbo协议：Dubbo默认使用Dubbo协议。

- 适合大并发小数据量的服务调用，以及服务消费者远大于提供者的情况
- Hessian二进制序列化。
- 缺点是不适合传送大数据包的服务。

2、rmi协议：采用JDK标准的rmi协议实现，传输参数和返回参数对象需要实现Serializable接口。使用java标准序列化机制，使用阻塞式短连接，传输数据包不限，消费者和提供者个数相当。

- 多个短连接，TCP协议传输，同步传输，适用常规的远程服务调用和rmi互操作

- 缺点：在依赖低版本的Common-Collections包，java反序列化存在安全漏洞，需升级commons-collections3 到3.2.2版本或commons-collections4到4.1版本。

3、 webservice协议：基于WebService的远程调用协议(Apache CXF的frontend-simple和transports-http)实现，提供和原生WebService的互操作多个短连接，基于HTTP传输，同步传输，适用系统集成和跨语言调用。

4、http协议：基于Http表单提交的远程调用协议，使用Spring的HttpInvoke实现。对传输数据包不限，传入参数大小混合，提供者个数多于消费者

- 缺点是不支持传文件，只适用于同时给应用程序和浏览器JS调用

5、hessian：集成Hessian服务，基于底层Http通讯，采用Servlet暴露服务，Dubbo内嵌Jetty作为服务器实现,可与Hession服务互操作
通讯效率高于WebService和Java自带的序列化

- 适用于传输大数据包(可传文件)，提供者比消费者个数多，提供者压力较大
- 缺点是参数及返回值需实现Serializable接口，自定义实现List、Map、Number、Date、Calendar等接口

6、thrift协议：对thrift原生协议的扩展添加了额外的头信息。使用较少，不支持传null值

7、memcache：基于memcached实现的RPC协议

8、redis：基于redis实现的RPC协议

5. Dubbo服务器注册与发现的流程?

- 服务容器Container负责启动，加载，运行服务提供者。
- 服务提供者Provider在启动时，向注册中心注册自己提供的服务。
- 服务消费者Consumer在启动时，向注册中心订阅自己所需的服务。
- 注册中心Registry返回服务提供者地址列表给消费者，如果有变更，注册中心将基于长连接推送变更数据给消费者。
- 服务消费者Consumer，从提供者地址列表中，基于软负载均衡算法，选一台提供者进行调用，如果调用失败，再选另一台调用。
- 服务消费者Consumer和提供者Provider，在内存中累计调用次数和调用时间，定时每分钟发送一次统计数据到监控中心Monitor。

6. Dubbo内置了哪几种服务容器?

三种服务容器：

- Spring Container
- Jetty Container
- Log4j Container

Dubbo的服务容器只是一个简单的 Main 方法，并加载一个简单的 Spring 容器，用于暴露服务。

7. Dubbo负载均衡的作用?

　将负载均衡功能实现在rpc客户端侧，以便能够随时适应外部的环境变化，更好地发挥硬件作用。而且客户端的负载均衡天然地就避免了单点问题。定制化的自有定制化的优势和劣势。

它可以从配置文件中指定，也可以在管理后台进行配置修改。

事实上，它支持 服务端服务/方法级别、客户端服务/方法级别 的负载均衡配置。

8. Dubbo有哪几种负载均衡策略，默认是哪种?

Dubbo提供了4种负载均衡实现：

1. RandomLoadBalance:随机负载均衡。随机的选择一个。是Dubbo的默认负载均衡策略。
2. RoundRobinLoadBalance:轮询负载均衡。轮询选择一个。
3. LeastActiveLoadBalance:最少活跃调用数，相同活跃数的随机。活跃数指调用前后计数差。使慢的 Provider 收到更少请求，因为越慢的 Provider 的调用前后计数差会越大。
4. ConsistentHashLoadBalance:一致性哈希负载均衡。相同参数的请求总是落在同一台机器上。

9. Dubbo服务之间的调用是阻塞的吗?

默认是同步等待结果阻塞的，支持异步调用。

Dubbo是基于 NIO 的非阻塞实现并行调用，客户端不需要启动多线程即可完成并行调用多个远程服务，相对多线程开销较小，异步调用会返回一个 Future 对象。

10. DubboMonitor 实现原理?

Consumer 端在发起调用之前会先走 filter 链；provider 端在接收到请求时也是先走 filter 链，然后才进行真正的业务逻辑处理。默认情况下，在 consumer 和 provider 的 filter 链中都会有 Monitorfilter。

1. MonitorFilter 向 DubboMonitor 发送数据
2. DubboMonitor 将数据进行聚合后（默认聚合 1min 中的统计数据）暂存到 ConcurrentMap<Statistics, AtomicReference> statisticsMap，然后使用一个含有 3 个线程（线程名字：DubboMonitorSendTimer）的线程池每隔 1min 钟，调用 SimpleMonitorService 遍历发送 statisticsMap 中的统计数据，每发送完毕一个，就重置当前的 Statistics 的 AtomicReference
3. SimpleMonitorService 将这些聚合数据塞入 BlockingQueue queue 中（队列大写为 100000）
4. SimpleMonitorService 使用一个后台线程（线程名为：DubboMonitorAsyncWriteLogThread）将 queue 中的数据写入文件（该线程以死循环的形式来写）
5. SimpleMonitorService 还会使用一个含有 1 个线程（线程名字：DubboMonitorTimer）的线程池每隔 5min 钟，将文件中的统计数据画成图表

11. Dubbo有哪些注册中心?

- Multicast 注册中心：Multicast 注册中心不需要任何中心节点，只要广播地址，就能进行服务注册和发现,基于网络中组播传输实现。
- Zookeeper 注册中心：基于分布式协调系统 Zookeeper 实现，采用 Zookeeper 的 watch 机制实现数据变更。
- Redis 注册中心：基于 Redis 实现，采用 key/map 存储，key 存储服务名和类型，map 中 key 存储服务 url，value 服务过期时间。基于 Redis 的发布/订阅模式通知数据变更。
- Simple 注册中心。
- 推荐使用 Zookeeper 作为注册中心

12. Dubbo的集群容错方案有哪些?

- Failover Cluster：失败自动切换，当出现失败，重试其它服务器。通常用于读操作，但重试会带来更长延迟。
- Failfast Cluster：快速失败，只发起一次调用，失败立即报错。通常用于非幂等性的写操作，比如新增记录。
- Failsafe Cluster：失败安全，出现异常时，直接忽略。通常用于写入审计日志等操作。
- Failback Cluster：失败自动恢复，后台记录失败请求，定时重发。通常用于消息通知操作。
- Forking Cluster：并行调用多个服务器，只要一个成功即返回。通常用于实时性要求较高的读操作，但需要浪费更多服务资源。可通过 forks="2″ 来设置最大并行数。

- Broadcast Cluster：广播调用所有提供者，逐个调用，任意一台报错则报错 。通常用于通知所有提供者更新缓存或日志等本地资源信息。

13. Dubbo超时设置有哪些方式?

Dubbo超时设置有两种方式：

- 服务提供者端设置超时时间，在Dubbo的用户文档中，推荐如果能在服务端多配置就尽量多配置，因为服务提供者比消费者更清楚自己提供的服务特性。
- 服务消费者端设置超时时间，如果在消费者端设置了超时时间，以消费者端为主，即优先级更高。因为服务调用方设置超时时间控制性更灵活。如果消费方超时，服务端线程不会定制，会产生警告。

14. Dubbo用到哪些设计模式?

1、**工厂模式**

Provider 在 export 服务时，会调用 ServiceConfig 的 export 方法。ServiceConfig中有个字段：

```
private static final Protocol protocol =
ExtensionLoader.getExtensionLoader(Protocol.class).getAdaptiveExtensi
on();
复制代码
```

Dubbo里有很多这种代码。这也是一种工厂模式，只是实现类的获取采用了 JDKSPI 的机制。这么实现的优点是可扩展性强，想要扩展实现，只需要在 classpath下增加个文件就可以了，代码零侵入。另外，像上面的 Adaptive 实现，可以做到调用时动态决定调用哪个实现，但是由于这种实现采用了动态代理，会造成代码调试比较麻烦，需要分析出实际调用的实现类。

2、**装饰器模式**

Dubbo在启动和调用阶段都大量使用了装饰器模式。以 Provider 提供的调用链为例，具体的调用链代码是在 ProtocolFilterWrapper 的 buildInvokerChain 完成的，具体是将注解中含有 group=provider 的 Filter 实现，按照 order 排序，最后的调用顺序是：

```
EchoFilter -> ClassLoaderFilter -> GenericFilter -> ContextFilter ->
ExecuteLimitFilter -> TraceFilter -> TimeoutFilter -> MonitorFilter ->
ExceptionFilter
复制代码
```

更确切地说，这里是装饰器和责任链模式的混合使用。例如，EchoFilter 的作用是判断是否是回声测试请求，是的话直接返回内容，这是一种责任链的体现。而像ClassLoaderFilter 则只是在主功能上添加了功能，更改当前线程的 ClassLoader，这是典型的装饰器模式。

3、**观察者模式**

Dubbo的 Provider 启动时，需要与注册中心交互，先注册自己的服务，再订阅自己的服务，订阅时，采用了观察者模式，开启一个 listener。注册中心会每 5 秒定时检查是否有服务更新，如果有更新，向该服务的提供者发送一个 notify 消息，provider 接受到 notify 消息后，运行 NotifyListener 的 notify 方法，执行监听器方法。

4、**动态代理模式**

Dubbo扩展 JDK SPI 的类 ExtensionLoader 的 Adaptive 实现是典型的动态代理实现。Dubbo 需要灵活地控制实现类，即在调用阶段动态地根据参数决定调用哪个实现类，所以采用先生成代理类的方法，能够做到灵活的调用。生成代理类的代码是 ExtensionLoader 的 createAdaptiveExtensionClassCode 方法。代理类主要逻辑是，获取 URL 参数中指定参数的值作为获取实现类的 key。

附：校招干货

1、简历如何写

简历是求职流程的第一步，是很关键的一环。

库森在校招时，投递了100来家企业，简历也修改了10多次，最近也帮10多个求职群的小伙伴修改了简历，积累了一些的经验（排坑经验）。

这篇文章就来谈谈如何写好一份简历，附简历模板~

首先来看，为什么说简历很重要?

- 能在初筛环节能让你脱颖而出，获得笔试、面试机会。在招聘高峰时，HR一天会看几百份简历，过一份简历的时间平均就30秒，而糟糕的简历可能不到10秒就被丢入了垃圾桶。
- 在面试环节，能为面试官引导提问方向，趋利避害。比如，在简历上写了”熟悉MySQL的索引、事务和锁的使用和原理“，面试官就很大概率去问MySQL方面的内容。

接下来我会逐个点分析下。

简历模板

简历模板不用多么花哨华丽，但要保证简洁美观，字体和样式要统一。常用的有word版、在线版、Markdown版和LaTeX版，推荐程序员小伙伴使用Markdown书写，比较干净，易于阅读，转成pdf投递即可。

如果想省事，直接使用在线版或者word版即可。

这里分享些我收集的优质简历模板。

- Markdown+css 自定义简历模板： https://github.com/CyC2018/Markdown-Resume
- Markdown 在线版简历排版工：https://resume.mdnice.com
- 冷熊简历：http://cv.ftqq.com/
- 超级简历： https://www.wondercv.com

下图是库森校招时使用的简历模板，是使用Markdown 书写的。

库森

电话：XXX-XXXX-XXXX | 邮箱：cosen1024@163.com | 博客：https://cosen1024.github.io
年龄：24 | 性别：男
求职意向：后端开发工程师（北京）

教育背景

2018.09 - 2021.06	XXX大学 XXX专业	硕士
2014.09 - 2018.06	XXX大学 XXX专业	本科

专业技能

- 熟练掌握 Java语言，熟悉多线程、集合框架、JVM等相关知识；
- 熟练使用Spring Cloud、Spring Boot、MyBatis等常用框架，熟悉Spring IOC、AOP原理，了解Nacos、Spring Cloud Gateway等组件；
- 熟练使用MySQL，熟悉MySQL索引、事务、锁机制、日志系统、数据备份与恢复；
- 深入理解Redis数据类型使用场景和内部实现，熟悉持久化和淘汰策略，熟悉缓存高并发场景；
- 了解Kafka、Elasticsearch的使用，熟悉Kafka基本结构、消费者组、备份机制的原理，熟悉Elasticsearch倒排索引的原理。

实习经历

2020.02 - 至今 **字节跳动（XXX部门）** **后端开发实习生**

项目经历

2019.10 - 2020.01 **社区论坛** **全栈开发工程师**

项目简介： 该论坛是一个互动交流平台，实现了注册登录、发帖评论、回复点赞、消息提醒、内容搜索和网站数据统计的功能，并将用户头像等信息存于七牛云。

项目地址： 演示地址: www.coolsen.cn Github: https://github.com/cosen1024/community

技术栈： Spring Boot + MyBatis + MySQL + Redis + Kafka + Elasticsearch

工作内容：

- 使用Redis存储登录ticket和验证码，解决分布式session问题；
- 使用Redis的set实现点赞，zset实现关注，HyperLogLog统计UV，Bitmap统计DAU；
- 使用Kafka处理发送评论、点赞和关注等系统通知，起到解耦和异步调用的作用；
- 使用Elasticsearch对帖子搜索功能进行重构，通过IK中文分词器增加增量索引和全局索引，实现搜索关键词高亮显示等功能；
- 对热帖排行模块，使用分布式缓存Redis和本地缓存Caffeine作为多级缓存，将QPS提升了20倍（10-200），大大提升了网站访问速度，并使用Quartz定时更新热帖排行榜。

个人荣誉

- 2020.05 2020年华为软件精英挑战赛总决赛 X等奖

这份简历模板，**点击下方公众号名片，后台回复「简历**」即可获取**。

下面详细说下每一部分该怎么写~

基本信息

最基本的信息：姓名、性别、年龄、电话、邮箱、求职意向等。

对于互联网技术岗，身高、体重、三围等对工作无效信息就不用写了~

求职意向这一栏一定要写，要不然面试官都不知道你要面试什么岗位。对城市有要求的，也要附上城市信息。

> Tips：求职意向跟招聘岗位保持一致，比如有公司招聘后端开发工程师，求职意向就不要继续写Java开发工程师了，尽量跟公司的招聘岗位保持一致，不然会给HR留下缺乏诚意的感觉。

教育背景

教育背景是必备的一环，尤其是应届生，千万不要觉得自己的学历低、学校差就空着不写。

示例：

> 2018.09 - 2021.06　　XXX大学 XXX专业
> 硕士
>
> 2014.09 - 2018.06　　XXX大学 XXX专业
> 本科

如果GPA很高，或者年级排行很靠前，可以附上这些信息，这也是自己的亮点。

专业技能

把握好专业技能的描述，专业技能的熟悉一般分为三个层次：

- 精通。慎用精通，尤其是应届生了，写了精通被会面试官怼到...
- 熟悉/熟练使用/掌握。对于经常使用到的技术，在很长一段时间内以它为主做过开发，可以使用这样的描述。比如使用Spring Cloud、MyBatis框架做过项目，在线上部署投入使用，算是“熟悉”。
- 了解。用这个技术自己做过一点东西，平时关心相关的发展，比如用Spring Boot做过小demo。

> TIps：对于专业技能板块，建议写真正玩明白的技术栈，千万不要写只是听说过，一问三不知的技术栈。

加分项：专业技能建议尽量写详细些。

比如对Redis很了解，最初写成“**熟悉Redis的基本使用和内部原理**”，可以修改为“**深入理解Redis核心数据类型使用场景和内部实现，熟悉持久化和过期淘汰策略，熟悉分布式锁，熟悉缓存高并发场景，比如缓存穿透、缓存击穿、缓存雪崩**”。

为什么这样做？对面试官来讲，面试官刚拿到你的简历，他可能没有想好具体的问题。

如果我们把专业技能写的很细，面试官有很大可能去问这些专业技能，面试官也正好通过此问题的沟通，来判断专业技能的真实性，甚至是整个简历的真实性。

并且，对我们面试者来讲，我们熟悉这些专业技能，如果回答的很漂亮，将是很大的加分项。

面试是有时间限制的，这方面聊的多了，那么其他方面就聊的少了，遇到不熟悉的概率就更小了。当然，前提是简历上写的要会，否则就是自己给自己“挖坑”。

这里附上一些技术描述，供小伙伴们参考

> 1、Java基础扎实、掌握JVM原理、多线程、网络原理、设计模式、常用数据结构和算法；
>
> 2、深入理解Spring、 Spring mvc 、MyBatis等开源框架设计原理及底层架构，研究过核心功能源码，具备一定的框架定制开发能力；
>
> 3、深入理解Redis线程模型，熟练掌握Redis的核心数据结构的使用场景，熟悉多级缓存架构，熟悉各种缓存高并发的使用场景，比如缓存雪崩，缓存穿透，缓存失效，热点缓存重建等；
>
> 4、熟悉常见消息中间件的使用，解决过各种消息通信场景的疑难问题，比如消息丢失、消息重复消费，消息顺序性，大规模消息积压问题；
>
> 5、对于高性能IO通信模型以及相关开源组件Netty等源码有过深度研究，熟悉Netty线程模型，熟悉百万级并发服务器架构设计；
>
> 6、深入理解JVM底层原理，熟悉JVM各种垃圾收集器的使用以及核心参数的调优，有过一定的JVM线上调优经验，对JVM调优有自己独到见解；

> 7、深入理解Spring Boot，Spring Cloud，Dubbo等微服务框架的设计原理及底层架构，研究过核心源码，熟悉各种微服务架构场景设计，比如服务注册与发现，服务限流、降级、熔断，服务网关路由设计，服务安全认证架构；
>
> 8、在项目中解决过各种分布式场景的技术难题，比如分布式锁，分布式事务，分布式session，分布式任务，海量数据的分库分表。任务，海量数据的分库分表。

项目/实习/工作经历

最基本的要点：项目经历的时间、项目名称、项目角色；实习的时间、实习的公司部门、工作职位。

此外，**简历上的项目、实习、工作经历，要体现出经历的价值**。善用数据的量化表达，比如“xx使QPS从100提升到2000”。善于STAR法则描述经历，会让简历更出彩，更有说服力。

项目经历可以包括以下几点：

- 描述清楚项目的背景、项目的整体设计；
- 自己在项目中角色是什么，负责哪些工作，项目是如何协作，遇到难题是怎么解决的；
- 用了什么样的技术方法，解决了什么问题；
- 项目取得了怎样的成果。

此外，项目描述不要给自己挖坑，简历上写的一定要熟悉，比如是业务类项目，要熟悉各种功能的场景和实现，系统的组件和架构等。

项目要有自己深入的思考，要熟悉项目难点的解决方案，以分布式锁为例，要清楚项目中为什么用分布式锁？目前这种使用的Redis分布式锁方案有什么问题？应该怎么改进？

项目经历并非多多益善，要选有亮点、成就感高的项目，一般应届生写上1-3个项目即可，要注意突出项目之间的区别、项目的亮点、个人的进步等。

如果能将项目部署，在简历上附上访问链接是更好不过的。便于在面试时根据项目功能与面试官沟通，面试效果会更好。同样，如果条件允许，在GitHub开放源码也是很加分的。

个人荣誉

如果有获奖经历，尤其是计算机领域含金量比较高的奖项一定要写下，如ACM、Kaggle、阿里天池、华为软件精英挑战赛等。

国家奖学金、全国大学生数学竞赛等含金量较高的校园荣誉也可以写上，而优秀学生会干部等对工作无关的荣誉不必写上。

自我评价

自我评价可写可不写，如果要写的话，一定不要写比较虚的废话，如个人性格乐观向上、活泼开朗。

如果要写，写比较实在、切实可见的方面，并简单举例描述，如团队合作经验、项目管理经验、架构设计经验等。

注意事项

- 投递简历一定要用PDF版本，一定要用PDF版本，一定要用PDF版本！重要的事情说三遍，这也是业内的规范，因为 PDF 不可编辑，格式也不会错乱，打开方便；
- 专业名词一定要拼写正确，不要写错大小写，如Java不要写成java，Redis不要写成redis，这也是技术人最基本的规范和态度问题；
- 简历命名，尤其是校招，建议采用：**学校、专业、学历、姓名+应聘岗位** 的格式，方便公司的HR一眼就能识别你的主要信息。如果招聘单位另有格式要求，按照招聘信息上的要求更改；
- 如果简历内容较多，没有必要非得费劲心思把简历压缩到一页，只要简历上没有废话、排版清晰简洁就行。
- 简历一定要真实，不要弄虚作假；
- 对于技术岗位，照片可贴可不贴，毕竟公司看重的是工作能力和专业性，又不是看长相，没有照片也不是什么问题。

End

需要帮忙看简历的小伙伴，可扫码加我微信，帮给建议~



2、大厂面试流程

库森在去年秋招时面试了50余家，加上暑期实习面试了20余家，加起来也面试了70余场。

基本把国内有名的互联网公司都面了一遍，不敢说自己的面试经验很丰富，但也是不差的。

这次库森专门把大厂的面试做了个总结，希望帮到大家拿到理想offer。



互联网拿到offer的完整流程一般有：

- 笔试、测评。
- 面试。

- offer沟通、三方。

限于篇幅，这篇文章主要讲面试中的那些事。

面试又分为技术面和hr面试，大厂的技术面一般有三面，中小厂通常只有二面。

每轮面试的整个流程一般分三部分，第一部让你做个自我介绍，第二部分面试官考察专业能力环节，最后一部分是反问面试官环节。

第二部分和第三部分，在一面、二面、三面等都有不同的侧重点，下文会详细讲到。

自我介绍部分是非常关键的，用1-2分钟介绍下自己，并突出下自己的相关亮点，有几点要注意：

- 面试官在面试前可能没看过你的简历，通过自我介绍的时候了解你的情况
- 可以用最擅长的来收尾，引导面试官的提问
- 别扯没用的经历，比如面试互联网大厂的技术岗，却在讲社团经历
- 人菜没关系，学校差没关系，项目low没关系，一定要自信

附一份自我介绍样例：

> 面试官您好，我叫XXX，是XXX大学2022届的学生，专业是XXX。在校期间，自己的成绩XXX，获得过XXX荣誉；做过XXX项目，项目使用了XXX，完成XXX；参加过XXX比赛，获得过XXX奖；在XXX公司实习，工作是XXX，为了完成这个工作，我做了xxx努力，最后取得xxx成果。

面试经验少的小伙伴，可以在面试前对着镜子模拟面试情景，反复刻意练习自我介绍，这样才能保证面试从容、不紧张。

一面

一面面试官通常由公司技术骨干担任，对于校招来说，一面比较看重基础。

一面通常考察：计算机基础八股文 + 技术栈八股文 + 手撕算法题 + 项目（实习）+ 开放性问题。

计算机基础八股文包括计算机网络、操作系统、数据结构与算法，这些课程在大学都会学到，某些大厂是很看重这些基础的，比如字节。

技术栈八股文指Java、C++、Python、Go等编程语言，比如Java，面试会问 Java基础、Java并发、JVM、MySQL、Redis、Spring、MyBatis、MQ等知识。

手撕算法题，就是写代码，分为线上和线下，因为疫情原因，现在基本都在线上。

在线写代码通常有两种形式：

1. 通过一些在线OJ平台，比如牛客网、赛码网，这些OJ平台支持视频面试，OJ平台都有自己的题库，在leetcode上基本都能找到。
2. 通过腾讯会议、zoom等在线视频会议系统，面试官会开共享屏幕，在本地IDE写代码。比如写个快排、二叉树的左视图。

手撕算法题现在面试都需要，至少一道起步，像字节这类要考算法题的公司，一般2-3道。

手撕算法题考的题目在leetcode上都可以找到，大多考察easy和medium类型的题目，刷300道高频leetcode题，面试的手撕算法题基本没问题。

> Tip：
>
> 1. 对于自己会的题目，不要高兴过头就立马写代码。相反，留下两分钟思考，组织语言讲清楚思路，再开始写代码，会显得自己头脑清晰。
> 2. 对于不会的题目、没有思路的题目，不要直接说一句“这个不会”，而是跟面试官交流，让面试官给点提示，一般面试官都会给提示的。比较好的公司都更会更看重于你解题过程中展现出来的思维能力和思考过程。
>
>   有时候你可能并没有得出最优解，但是如果你思路清晰，能跟上考官的提示，与面试官展开一个共同探讨的过程，往往比直接得出最优解更能获得考官的芳心。

项目或者实习在通常在一面中通常问的不会很深，只是简单聊聊项目的技术栈、项目的基本功能、框架的使用等。

最后，面试官会问一句“你还有什么问题要问我的吗？“，这个问题相当于把主动权交给你了，但也要注意，这时候就算不加分，至少也不要扣分。

最好不要问的，比如薪资水平、上下班时间、交通食宿补助、加班是否有加班费。因为这些问题是问hr的，现在问不合适，反而会造成坏印象。

可以问什么？对于一面的面试官，通常是技术骨干或者主管，可以问以下一些问题：

- 部门的主营业务是什么？表达下自己对业务的好奇

- 部门使用的技术栈、编程语言是什么、使用哪些框架、中间件？表达下自己对技术的好奇
- 您觉得我的专业能力有哪些需要提升的，您有什么建议？
- 您刚才那个问题，我想问下您的思路。体现自己好学

最重要的一点，某些同学可能想知道自己此轮面试是否通过，就直接问面试官“我这轮面试通过了没有”，

切记，这个问题不能直接这么问！大公司都有规定，面试官是不能现在跟你说的，会造成很尴尬的场面。

迫切想知道，自己面试的结果，可以问下面试官，一般多久会有结果，多久有下一轮的面试通知即可。

二面

对于中小厂来说，二面就是技术终面了。

二面的面试官一般是技术leader，更看重解决问题的能力。

二面中，八股文和算法题比一面考察的少些，会更多问项目和实习的问题，还会问些开放性问题。

二面官会以项目为切入点，先问项目本身的背景和基础功能等，在一步步深入，问到项目相关扩展，直到你不会为止。

项目相关的问题可分为两方面，一是项目本身，二是项目扩展。

我汇总了常见的项目问法，可以对照看下，能答上百分之90，项目就能成为加分项。

项目本身

- 项目的背景是什么，解决一个什么样的问题?
- 项目中你的职责是什么?
- 项目的基础功能有哪些?
- 项目使用的技术栈是什么，技术架构是怎么样的？使用微服务了吗?
- 项目是怎么搭建的，机器配置是什么样的，有做分布式吗?
- 项目的具体功能细节，比如论坛项目中评论是如何存储的？怎么展示所有的评论?
- 项目中框架或者中间件的使用细节。项目里怎么用ES的，ES怎么支持搜索的？缓存和DB是如何结合使用的?

项目扩展

- 项目存在哪些问题，你准备怎么解决?
- 项目的具体功能点如何优化？如论坛项目，查询评论是在DB里扫表查询吗？想要查询更快可以做哪些优化?
- 项目中最有挑战的模块是哪个，你是怎么解决的?
- 项目中使用某种框架的原因，比如使用了本地缓存Caffeine，为什么使用这个Caffeine，不使用Guava?
- 项目要增大10倍的qps，你会怎么设计?
- 项目上线后出现线上问题怎么解决？如频繁fullGc，定时任务失败怎么办?

开放性问题就比较灵活了，不要以为开放性问题就放松了下来，也是要认真回答的。

常见的开放性问题有:

- 最近在看什么书？面试官通过这个问题考察你对前言领域、对专业领域是否关注，是新事物理解如何。
- 你了解过XXX技术吗，如了解过Service Mesh吗？还知道有关微服务的吗？看好奇心，是否会主动探索。
- 你平时是怎么学习的？看学习的方法、学习的习惯，学习有没有体系?
- 人生发展、最近五年的职业规划?
- 你实习时或者做项目时，是如何协作的？看团队协作能力。
- 聊优缺点，你最大的亮点是什么?
- 聊在校经历，在校成绩
- 最有成就感的事情，最丧的事情

二面都是主管级别的面试官了，所以二面的反问环节可以比一面更宏观些了，可以问:

- 您面试后候选人，尤其是校招生，最看重什么?
- 如果我有幸进入您的团队，您对我的期望是什么?
- 您觉得胜任这个职位的人什么品质最重要?
- 您对于候选人，尤其是校招生的职业规划有什么建议?
- 这个职位的晋升机制、发展路径是怎么样的?
- 我将要进入的团队有多少人，业务发展方向是什么?
- 公司、部门对于新人的培养是什么样的?
- 您认为我对于这个岗位的差距在哪里，哪些方面需要提升?

三面

进入二面，拿offer的概率就很大了。

三面官一般都是大leader，技术总监这类级别的。

三面通常就不会再问很多基础相关的知识了，三面更看重个人素质，毕竟能通过前面两轮面试的，技术能力肯定是过关的。‘

三面主要考察项目、开放性问题、智力问题，有些公司还会在考两道算法题，问些八股文基础。

智力问题是想考察候选人是不是聪明，这些题目其实就是看过就会秒解，不会一小时都想不出来。

常考的智力题有很多，这里简单列上几个，后面会出个智力题汇总的文章

> 1. 通过烧绳子判断时间
> 2. 分蛋糕问题
> 3. 找出坏球
> 4. 最快时间过桥问题
> 5. 家庭聚会握手问题
> 6. 天秤找出变质的那瓶药
> 7. 猴子香蕉问题
>
>   ...
>
> 8. ...

HR面

进入HR面试，基本问题不大了。

HR面试主要考察的是候选人的**性格、上进心、价值观、工作的态度等**

HR面试也要提前准备下，尤其是涉及到对方公司的企业文化、有什么代表性产品、历年的薪资水平等。

HR面试的回答没有什么固定答案，根据实际情况回答就好。

这里有个注意点，HR问到“**你最大的缺点是什么？**”

千万不要回答没有缺点，我的缺点就是追求完美。只要是人就有缺点，说没有缺点一定会令人反感！

可以采用**避重就轻的描述**，并且自己还在不断改正，比如说自己性子急、对待效率低下的人，但是我平时和别人聊天的时候会控制自己语速和讲话，慢慢培养自己耐心。

参考

Java基础、JVM部分：

https://tech.meituan.com/2019/09/05/java-bytecode-enhancement.html

http://c.biancheng.net/view/769.html

http://www.51gjie.com/java/81.html

http://www.justdojava.com/2019/03/21/Java-and-equals/

https://blog.csdn.net/qq_28051453/article/details/52701171

https://www.cnblogs.com/wkfvawl/p/11693260.html

https://henleylee.github.io/posts/2019/cc1a57ba.html

https://juejin.cn/post/6844903741032759310

https://blog.csdn.net/chenliguan/article/details/53888018

https://segmentfault.com/a/1190000010162647

https://juejin.cn/post/6856664924203663367

http://www.fanyilun.me/2015/10/29/Java%E5%8F%8D%E5%B0%84%E5%8E%9F%E7%90%86/

https://www.cnblogs.com/lixuwu/p/10829368.html

https://juejin.cn/post/6844903848167866375

https://blog.csdn.net/lanzhupi/article/details/109809836

https://juejin.cn/post/6844903746682486791

https://blog.csdn.net/ThinkWon/article/details/101681073

https://simplesnippets.tech/exception-handling-in-java-part-1/

https://www.cnblogs.com/xingyunblog/p/8688859.html

https://mp.weixin.qq.com/s/p5qM2UJ1uIWyongfVpRbCg

https://juejin.cn/post/6844903520856965128

https://jishuin.proginn.com/p/763bfbd35094

https://www.javanav.com/val/93550f179edb4a77bbf4d35faa6d560c.html

https://juejin.cn/post/6844903941805703181

https://www.cnblogs.com/chiangchou/p/jvm-2.html

https://juejin.cn/post/6844903887866953735

https://segmentfault.com/a/1190000023182342

https://blog.csdn.net/ThinkWon/article/details/103522351

https://www.codenong.com/cs106843764/

https://segmentfault.com/a/1190000039223696

https://www.jianshu.com/p/e94fa7340923

https://segmentfault.com/a/1190000016188385

https://www.cnblogs.com/ysocean/p/12422635.html

https://juejin.cn/post/6844904166624624654

https://segmentfault.com/a/1190000021815671

https://juejin.cn/post/6844904005315854343

https://www.nowcoder.com/discuss/568071

https://blog.csdn.net/yrx420909/article/details/104483455

https://www.cnblogs.com/xiaolincoding/p/12638546.html

https://imageslr.com/2020/07/07/tcp-shake-wave.html

https://cloud.tencent.com/developer/article/1537628

https://juejin.cn/post/6844903890840715271

https://www.justdojava.com/2019/11/03/Network_interview_question/

https://juejin.cn/post/6844903489596833800

https://segmentfault.com/a/1190000021494676

https://jiangren.work/2020/02/16/

https://www.cnblogs.com/ityouknow/p/10856177.html

https://juejin.cn/post/6844903575684907016

https://segmentfault.com/a/1190000011028633

https://god-jiang.github.io

https://segmentfault.com/a/1190000019750164

https://www.jianshu.com/p/c1015f5ffa74

MySQL：

https://blog.csdn.net/ThinkWon/article/details/104778621

https://haicoder.net/note/mysql-interview/mysql-interview-mysql-binlog.html

https://www.modb.pro/db/40241

https://www.jianshu.com/p/05da0fc0950e

https://blog.csdn.net/ThinkWon/article/details/104778621

redis：

https://blog.csdn.net/zt_star/article/details/98054284

https://blog.csdn.net/ThinkWon/article/details/103522351

https://www.codenong.com/cs106843764/

https://segmentfault.com/a/1190000039223696

https://www.jianshu.com/p/e94fa7340923

https://segmentfault.com/a/1190000016188385

https://www.cnblogs.com/ysocean/p/12422635.html

https://juejin.cn/post/6844904166624624654

https://www.cmsblogs.com/?p=18337

https://www.cnblogs.com/mr-wuxiansheng/p/12884356.html

https://www.cnblogs.com/yaopengfei/p/13946966.html

https://developer.aliyun.com/article/626532

https://mp.weixin.qq.com/s?__biz=MzUxNTQyOTIxNA==&mid=2247484002&idx=1&sn=f0ae222a1f59e47e2c17aaa6778e2085&chksm=f9b78453cec00d456fd1e922a138685c22838da6c8ef165d943be53e73b83b282bd83672c759&token=1568003997&lang=zh_CN#rd

https://www.cnblogs.com/ibigboy/p/12213532.html

https://www.cnblogs.com/rjzheng/p/9041659.html

https://juejin.cn/post/6850418121754050567#heading-0

https://blog.csdn.net/a745233700/article/details/88088669

https://xiaomi-info.github.io/2019/12/17/redis-distributed-lock/

https://cloud.tencent.com/developer/article/1415100

spring：

https://juejin.cn/post/6844903860658503693

https://www.cnblogs.com/jingmoxukong/p/9408037.html

http://www.ityouknow.com/springboot/2019/07/24/springboot-interview.html

MyBatis：

https://blog.csdn.net/ThinkWon/article/details/101292950

https://blog.csdn.net/a745233700/article/details/80977133

https://github.com/Homiss/Java-interview-questions/blob/master

MQ：

http://dockone.io/article/10853

https://segmentfault.com/a/1190000023716306

https://dongzl.github.io/2020/03/16/13-Solve-MQ-Problem-With-Kafka/index.html

https://www.cnblogs.com/helios-fz/p/12119727.html

https://juejin.cn/post/6844904192146931720

https://www.cnblogs.com/javazhiyin/p/13327925.html

https://juejin.cn/post/6844904068771479559

多线程：

https://www.cnblogs.com/java1024/p/13390538.html

https://segmentfault.com/a/1190000039258680

https://github.com/pengMaster/BestNote

http://static.kancloud.cn/alex_wsc/java_source_interview/1875015

https://blog.csdn.net/zycxnanwang/article/details/105321401

https://blog.csdn.net/weixin_45124488/article/details/115200512

dubbo

https://www.cnblogs.com/chenyanbin/p/13193539.html

https://blog.csdn.net/weixin_39309402/article/details/104745622

https://juejin.cn/post/6844904127076499463